
Сетевой стек Reticulum

Версия 1.1.4

Марк Квист

11 марта 2026 г.

СОДЕРЖАНИЕ

1	Что такое Reticulum?	3
1.1	Текущее состояние.....	3
1.2	Эталонная реализация	3
1.3	Что предлагает Reticulum?	4
1.4	Где можно использовать Reticulum?	5
1.5	Типы интерфейсов и устройства	5
2	Быстрый старт	7
2.1	Установка автономной версии Reticulum.....	7
2.1.1	Решение проблем с зависимостями и установкой.....	7
2.2	Попробуйте использовать программу на базе Reticulum	8
2.3	Использование входящих в комплект утилит	8
2.4	Создание сети с помощью Reticulum	8
2.5	Настройка подключения	8
2.5.1	Ориентация	9
2.5.2	Создание личной инфраструктуры	9
2.5.3	Комбинирование стратегий	10
2.5.4	Состояние сети и ответственность	10
2.5.5	Вклад в глобальное.....	10
2.6	Подключение к распределенной магистрали	10
2.7	Размещение общедоступных точек входа	11
2.8	Подключение экземпляров Reticulum через Интернет.....	12
2.9	Добавление радиointерфейсов	12
2.10	Создание и использование пользовательских интерфейсов	13
2.11	Разработка программы с помощью Reticulum.....	13
2.12	Примечания по установке для конкретных платформ	13
2.12.1	Android	13
2.12.2	ARM64	14
2.12.3	Debian Bookworm.....	14
2.12.4	MacOS.....	15
2.12.5	OpenWRT.....	16
2.12.6	Raspberry Pi	16
2.12.7	RISC-V	17
2.12.8	Ubuntu Lunar.....	17
2.12.9	Windows.....	18
2.13	Pure-Python Reticulum	18
3	Дзен Reticulum	19
3.1	Иллюзия центра.....	19
3.1.1	Мифы об «облаке».....	19

	3.1.2	Децентрализация или нецентрализуемость?	19
	3.1.3	Смерть адресу	20
3.2		Физика доверия	20
	3.2.1	Неблагоприятные условия	20
	3.2.2	Шифрование — это не функция	21
	3.2.3	Архитектуры «нулевого доверия»	21
3.3		Преимущества дефицита	21
	3.3.1	Заблуждение о пропускной способности	22
	3.3.2	Стоимость байта	22
	3.3.3	Расход и время	22
	3.3.4	Освобождение от ограничений	23
3.4		Суверенитет через инфраструктуру	23
	3.4.1	Заблуждение «операторского уровня»	23
	3.4.2	Личная инфраструктура	23
	3.4.3	Возможность отключения	24
3.5		Идентичность и кочевничество	24
	3.5.1	«Портативное существование»	24
	3.5.2	Узлы роуминга	24
	3.5.3	Уведомление о присутствии	25
	3.5.4	Якорь в потоке	25
3.6		Этика инструмента	25
	3.6.1	Принцип причинения вреда	26
	3.6.2	Протокол общественного домена	26
	3.6.3	Сохранение человеческой свободы воли	26
3.7		Шаблоны проектирования для систем Post-IP	27
	3.7.1	Хранение и пересылка	27
	3.7.2	Имя — это сила	28
	3.7.3	Интерфейс — это среда	28
	3.7.4	Возникающие закономерности	28
3.8		Ткань независимости	29
	3.8.1	Работа завершена	29
	3.8.2	Открытое небо	29
4		Программы, использующие Reticulum	31
4.1		Программы и утилиты	31
	4.1.1	Удаленная оболочка	31
	4.1.2	Сеть Nomad	32
	4.1.3	Узел страницы RNS	32
	4.1.4	Retipedia	32
	4.1.5	Боковая полоса	33
	4.1.6	MeshChatX	34
	4.1.7	MeshChat	35
	4.1.8	Columba	35
	4.1.9	Релейный чат Reticulum	36
	4.1.10	RetiBBS	36
	4.1.11	RBrowser	37
	4.1.12	Телефон Reticulum Network компании « »	38
	4.1.13	Телефон LXST	39
	4.1.14	LXMFy	40
	4.1.15	Интерактивный клиент LXMF от компании « »	40
	4.1.16	RNS FileSync	40
	4.1.17	Анализатор Micron от	40
	4.1.18	RNMon	40
4.2		Протоколы	41

	4.2.1	LXMF	41
	4.2.2	LXST	41
	4.2.3	RRC	41
4.3		Модули интерфейса и ресурсы для подключения	41
5		Использование Reticulum в вашей системе	43
5.1		Настройки и данные	43
5.2		Включенные утилиты	46
	5.2.1	Утилита rnsd	47
	5.2.2	Утилита rnstatus	47
	5.2.3	Утилита rmid	49
	5.2.4	Утилита rnpath	51
	5.2.5	Утилита rnprobe	52
	5.2.6	Утилита rncp	53
	5.2.7	Утилита rnx	54
	5.2.8	Утилита rnodeconf	55
5.3		Знакомство с интерфейсами	57
5.4		Удаленное управление	59
5.5		Управление черными дырами	59
	5.5.1	Управление локальными черными дырами	60
	5.5.2	Автоматический поиск списков	60
	5.5.3	Публикация списков «черных дыр»	61
5.6		Улучшение конфигурации системы	62
	5.6.1	Исправлены имена последовательных портов	62
	5.6.2	Reticulum как системная служба	62
6		Понимание Reticulum	65
6.1		Мотивация	65
6.2		Цели	66
6.3		Введение и основные функции	66
	6.3.1	Направления	67
	6.3.2	Объявления об открытых ключах	69
	6.3.3	Тождества	69
	6.3.4	Дальнейшие действия	70
6.4		Реткулярный транспорт	70
	6.4.1	Типы узлов	70
	6.4.2	Механизм объявления в деталях	70
	6.4.3	Прибытие в пункт назначения	71
	6.4.4	Ресурсы	73
6.5		Сетевые идентификаторы	74
	6.5.1	Общий обзор концепции	74
	6.5.2	Текущее использование	74
	6.5.3	Последствия для будущего	74
	6.5.4	Создание и использование сетевой идентификации	75
6.6		Настройка эталонной системы	75
6.7		Особенности протокола	76
	6.7.1	Приоритезация пакетов	76
	6.7.2	Коды доступа к интерфейсу	76
	6.7.3	Формат провода	77
	6.7.4	Объявление правил распространения	79
	6.7.5	Криптографические примитивы	80
7		Коммуникационное оборудование	83
7.1		Объединение типов оборудования	83

7.2	RNode	83
7.2.1	Создание RNodes	84
7.2.2	Поддерживаемые платы и устройства	84
7.2.3	Установка	90
7.2.4	Использование с Reticulum	91
7.3	Оборудование на базе WiFi	91
7.4	Оборудование на базе Ethernet	91
7.5	Последовательные линии и устройства	91
7.6	Модемы пакетной радиосвязи	91
8	Настройка интерфейсов	93
8.1	Пользовательские интерфейсы	93
8.2	Автомобильный интерфейс	93
8.3	Магистральный интерфейс	95
8.3.1	Прослушиватели	95
8.3.2	Подключение удаленных устройств	96
8.4	Интерфейс TCP-сервера	96
8.5	Интерфейс TCP-клиента	98
8.6	Интерфейс UDP	99
8.7	Интерфейс I2P	100
8.8	Интерфейс RNode LoRa	101
8.9	Мультиинтерфейс RNode	103
8.10	Последовательный интерфейс	105
8.11	Интерфейс Pipe	105
8.12	Интерфейс KISS	106
8.13	Интерфейс AX.25 KISS	107
8.14	Обнаруживаемые интерфейсы	108
8.14.1	Включение обнаружения	108
8.14.2	Параметры обнаружения	108
8.14.3	Режимы интерфейса	110
8.14.4	Вопросы безопасности	110
8.14.5	Пример конфигурации	111
8.15	Общие параметры интерфейса	112
8.16	Режимы интерфейса	113
8.17	Управление скоростью передачи	114
8.18	Ограничение скорости для нового пункта назначения	115
9	Построение сетей	117
9.1	Понятия и обзор	117
9.1.1	Вводные замечания	117
9.1.2	Пункты назначения, а не адреса	118
9.1.3	Транспортные узлы и экземпляры	119
9.1.4	Сети без доверия	120
9.1.5	Гетерогенная связь	121
10	Поддержка Reticulum	123
10.1	Пожертвования	123
10.2	Оставить отзыв	123
11	Примеры кода	125
11.1	Минимальный	125
11.2	Объявление	127
11.3	Трансляция	131
11.4	Эхо	133
11.5	Ссылка	140
11.6	Идентификация	145

11.7	Запросы и ответы	152
11.8	Канал	157
11.9	Буфер	165
11.10	Передача файлов	171
11.11	Пользовательские интерфейсы	183
12	Лицензия Reticulum	191
13	Справочник API	193
13.1	Reticulum	193
13.2	Идентичность	195
13.3	Пункт назначения	198
13.4	Пакет	202
13.5	Получение пакета	203
13.6	Связь	204
13.7	Подтверждение запроса	207
13.8	Ресурс	208
13.9	Канал	209
13.10	База сообщений	210
13.11	Буфер	211
13.12	RawChannelReader	212
13.13	RawChannelWriter	212
13.14	Транспорт	213
Индекс		215

Данное руководство призвано предоставить вам всю необходимую информацию для понимания Reticulum, построения сетей или разработки программ с его использованием, а также для участия в разработке самого Reticulum.

ЧТО ТАКОЕ RETICULUM?

Reticulum — это сетевой стек на основе криптографии для построения как локальных, так и глобальных сетей с использованием легкодоступного оборудования, способный продолжать работу в неблагоприятных условиях, таких как крайне низкая пропускная способность и очень высокая задержка.

Чтобы понять основную философию и цели этой системы, ознакомьтесь с документом *«Zen of Reticulum»*.

Reticulum позволяет создавать глобальные сети с помощью готовых инструментов и предлагает сквозное шифрование, секретность пересылки, самонастраиваемый многоузловой транспорт с криптографической защитой, эффективную адресацию, подлинность подтверждения получения пакетов и многое другое.

С точки зрения пользователей Reticulum позволяет создавать приложения, которые уважают и укрепляют автономию и суверенитет сообществ и отдельных людей. Reticulum обеспечивает безопасную цифровую коммуникацию, которая не подвергается внешнему контролю, манипуляциям или цензуре.

Reticulum позволяет создавать как небольшие, так и потенциально глобальные сети без необходимости в иерархических или бюрократических структурах для их контроля или управления, обеспечивая при этом полный суверенитет отдельных лиц и сообществ над своими сегментами сети.

Reticulum представляет собой **полноценный сетевой стек** и не требует использования IP-протокола или более высоких уровней, хотя IP (с TCP или UDP) легко использовать в качестве базового канала для Reticulum. Поэтому туннелирование Reticulum через Интернет или частные IP-сети не представляет никаких сложностей. Reticulum построен непосредственно на криптографических принципах, что обеспечивает отказоустойчивость и стабильную работу в открытых сетях, не требующих доверия.

Не требуется никаких модулей ядра или драйверов. Reticulum может работать полностью в пользовательском пространстве и будет работать практически на любой системе, на которой работает Python 3. Reticulum хорошо работает даже на небольших одноплатных компьютерах, таких как Pi Zero.

1.1 Текущий статус

Все основные функции протокола реализованы и работают, однако по мере изучения реальных условий эксплуатации, вероятно, будут вноситься дополнения. API и формат передачи данных можно считать завершенными и стабильными, но они могут измениться, если это будет абсолютно необходимо.

1.2 Эталонная реализация

Код на Python, для которого написана данная документация и известный как Reticulum Network Stack, является эталонной реализацией Reticulum. Протокол Reticulum полностью и авторитетно определяется этой эталонной реализацией и данным руководством. Он поддерживается Марком Квистом, идентифицируемым по идентификатору Reticulum

<bc7291552be7a58f361522990465165c>.

Совместимость с протоколом Reticulum определяется как полная взаимодействие и достаточная функциональная равноценность с данной эталонной реализацией. Любая конкретная реализация протокола, достигающая этого, является Reticulum. Любая, не достигающая этого, не является Reticulum.

Эталонная реализация лицензирована по *лицензии Reticulum*.

Протокол Reticulum был передан в общественное достояние в 2016 году.

1.3 Что предлагает Reticulum?

- Глобально уникальная адресация и идентификация без координации
- Полностью самонастраиваемая многоскоковая маршрутизация по сетям разнородных операторов
- Гибкая масштабируемость в гетерогенных топологиях
 - Reticulum может передавать данные по любой комбинации физических сред и топологий
 - Сети с низкой пропускной способностью могут сосуществовать и взаимодействовать с крупными сетями с высокой пропускной способностью
- Анонимность инициатора, обмен данными без раскрытия вашей личности
 - Reticulum не включает адреса источника в пакеты
- Асимметричное шифрование X25519 и подписи Ed25519 как основа всей коммуникации
 - Базовые идентификационные ключи Reticulum представляют собой наборы ключей на основе эллиптических кривых длиной 512 бит
- Функция Forward Secrecy доступна для всех типов связи, как для отдельных пакетов, так и для каналов связи
- Reticulum использует следующий формат для зашифрованных токенов:
 - Временные ключи для каждого пакета и канала, полученные в результате обмена ключами ECDH на кривой Curve25519
 - AES-256 в режиме CBC с заполнением PKCS7
 - HMAC с использованием SHA256 для аутентификации
 - IV генерируются с помощью `os.urandom()`
- Неподдельные подтверждения доставки пакетов
- Гибкая и расширяемая система интерфейсов
 - Reticulum включает в себя широкий спектр встроенных типов интерфейсов
 - Возможность загрузки и использования пользовательских типов интерфейсов или типов, предоставленных сообществом
 - Легко создавайте собственные пользовательские интерфейсы для связи по любым каналам
- Аутентификация и сегментация виртуальной сети на всех поддерживаемых типах интерфейсов
- Интуитивный и удобный API
 - Проще и удобнее в использовании, чем API сокетов, и проще, но мощнее
 - Значительно упрощает создание распределенных и децентрализованных приложений
- Надежная и эффективная передача произвольных объемов данных
 - Reticulum способен обрабатывать как несколько байтов данных, так и файлы объемом в несколько гигабайт
 - Последовательность, сжатие, координация передачи и вычисление контрольной суммы выполняются автоматически
 - API очень прост в использовании и предоставляет информацию о ходе передачи
- Легкий, гибкий и расширяемый механизм запросов/ответов
- Эффективное установление соединения
 - Общая стоимость установки зашифрованного и проверенного соединения составляет всего 3 пакета, что в сумме составляет 297 байт
- Низкая стоимость поддержания открытых каналов связи — всего 0,44 бита в секунду
- Надежная последовательная передача данных с использованием механизмов каналов и буферов

1.4 Где можно использовать Reticulum?

Практически на любом носителе, который поддерживает как минимум полудуплексный канал с пропускной способностью более 5 бит в секунду и размером MTU 500 байт. Радиомодули передачи данных, модемы, радиомодули LoRa, последовательные линии, TNC по стандарту AX.25, цифровые режимы любительского радио, WiFi ad-hoc, оптические линии свободного пространства и аналогичные системы — все это примеры типов интерфейсов, для которых был разработан Reticulum.

В качестве примера приемопередатчика, идеально подходящего для Reticulum, был разработан интерфейс с открытым исходным кодом на базе LoRa под названием [RNode](#). Его можно собрать самостоятельно, переоборудовав для этого стандартную плату разработчика LoRa, либо приобрести в виде готового приемопередатчика у различных поставщиков.

Reticulum также можно инкапсулировать в существующие IP-сети, поэтому ничто не мешает вам использовать его через проводной Ethernet или локальную сеть Wi-Fi, где он будет работать не хуже. Фактически, одной из сильных сторон Reticulum является то, насколько легко он позволяет подключать различные среды в самонастраиваемую, отказоустойчивую и зашифрованную ячеистую сеть.

Например, можно настроить Raspberry Pi, подключенный одновременно к радиомодулю LoRa, TNC для пакетной радиосвязи и сети Wi-Fi. После добавления интерфейсов Reticulum возьмет на себя все остальное, и любое устройство в сети Wi-Fi сможет обмениваться данными с узлами в сетях LoRa и пакетной радиосвязи, и наоборот.

1.5 Типы интерфейсов и устройства

Reticulum реализует ряд обобщенных типов интерфейсов, охватывающих все виды коммуникационного оборудования, на котором может работать Reticulum. Если ваше оборудование не поддерживается, можно легко *реализовать собственный класс интерфейса*. В настоящее время Reticulum может использовать следующие устройства и каналы связи:

- Любое устройство Ethernet
 - Устройства Wi-Fi
 - Проводные устройства Ethernet
 - Оптоволоконные приемопередатчики
 - Радиостанции передачи данных с портами Ethernet
- LoRa с использованием [RNode](#)
 - Может быть установлено на многие популярные платы LoRa
 - Можно приобрести в виде готового к использованию приемопередатчика
- TNC для пакетной радиосвязи, такие как [OpenModem](#)
 - Любой TNC для пакетной радиосвязи в режиме KISS
 - Идеально подходит для УКВ- и СВЧ-радио
- Любое устройство с последовательным портом
- Сеть I2P
- Сети TCP над IP
- UDP в сетях IP
- Все, к чему можно подключиться через stdio
 - Reticulum может использовать внешние программы и каналы в качестве интерфейсов
 - Это можно использовать для простого взлома виртуальных интерфейсов
 - Или для быстрого создания интерфейсов с использованием пользовательского оборудования
- Все остальное с помощью *пользовательских модулей интерфейсов*, написанных на Python

Полный список и дополнительные сведения см. в главе *«Поддерживаемые интерфейсы»*.

БЫСТРОЕ НАЧАЛО

Лучший способ начать работу с сетевым стеком Reticulum зависит от того, что вы хотите сделать. В этом руководстве описаны разумные варианты начала работы для различных сценариев.

2.1 Автономная установка Reticulum

Если вы просто хотите установить Reticulum и связанные утилиты в системе, проще всего это сделать с помощью менеджера пакетов `pip`:

```
pip install rns
```

Если у вас ещё не установлен `pip`, вы можете установить его с помощью диспетчера пакетов вашей системы, выполнив команду типа `sudo apt install python3-pip`, `sudo pacman install python-pip` или аналогичную.

Вы также можете скачать релизные колеса Reticulum с GitHub или других каналов распространения и установить их в автономном режиме с помощью `pip`:

```
pip install ./rns-1.1.2-py3-none-any.whl
```

На платформах, которые ограничивают установку пакетов пользователем через `pip`, вам может потребоваться вручную разрешить это с помощью `--break-system-packages` при установке. Это не приведёт к повреждению каких-либо пакетов, если только вы не установили Reticulum напрямую через менеджер пакетов вашей операционной системы.

```
pip install rns --break-system-packages
```

Более подробные инструкции по установке см. в разделе «*Примечания по установке для конкретных платформ*».

После завершения установки может быть полезно обратиться к главе «*Использование Reticulum в вашей системе*».

2.1.1 Решение проблем с зависимостями и установкой

На некоторых платформах бинарные пакеты могут быть доступны не для всех зависимостей, и установка с помощью `pip` может завершиться сбоем с выводом сообщения об ошибке. В таких случаях проблему обычно можно решить, установив пакеты Development Essentials для вашей платформы:

```
# Debian / Ubuntu / производные
```

```
sudo apt install build-essential
```

```
# Arch / Manjaro / производные
```

```
sudo pacman install base-devel
```

```
# Fedora
```

```
sudo dnf groupinstall "Development Tools" "Development Libraries"
```

После установки базовых пакетов разработки `pip` должен быть в состоянии скомпилировать любые недостающие зависимости из исходного кода и завершить установку даже на платформах, где нет готовых скомпилированных пакетов.

2.2 Попробуйте использовать программу на основе Reticulum

Если вы просто хотите попробовать программу, созданную с помощью Reticulum, существует *целый ряд различных программ*, которые обеспечивают базовую связь и различные другие полезные функции даже в сетях Reticulum с крайне низкой пропускной способностью.

2.3 Использование входящих в комплект утилит

Reticulum поставляется с набором встроенных утилит, которые упрощают управление сетью, проверку подключения и делают Reticulum доступным для других программ в вашей системе.

С помощью `gnsd` можно запускать Reticulum в качестве фоновой или фоновой службы, а также использовать утилиты `gnstatus`, `gnpath` и `gnprobe` для просмотра и проверки состояния сети и подключения.

Чтобы узнать больше об этих утилитах, ознакомьтесь с главой «*Использование Reticulum в вашей системе*» данного руководства.

2.4 Создание сети с помощью Reticulum

Чтобы создать сеть, необходимо указать один или несколько *интерфейсов*, которые будет использовать Reticulum. Это делается в файле конфигурации Reticulum, который по умолчанию находится в папке `~/.reticulum/config`. Пример файла конфигурации со всеми параметрами можно получить с помощью команды `gnsd --exampleconfig`.

При первом запуске Reticulum создаст файл конфигурации по умолчанию с одним активным интерфейсом. Этот интерфейс по умолчанию использует ваши существующие сети Ethernet и WiFi (если они есть) и позволяет обмениваться данными только с другими узлами Reticulum в пределах ваших локальных доменов широковещания.

Для дальнейшего обмена данными вам необходимо добавить один или несколько интерфейсов. Конфигурация по умолчанию включает ряд примеров, начиная от использования TCP через Интернет и заканчивая интерфейсами LoRa и Packet Radio.

С Reticulum вам нужно только настроить, через какие интерфейсы вы хотите обмениваться данными. Нет необходимости настраивать адресные пространства, подсети, таблицы маршрутизации или другие вещи, к которым вы, возможно, привыкли при работе с другими типами сетей.

Как только Reticulum определит, какие интерфейсы ему следует использовать, он автоматически определит топологию сети и настроит передачу данных в любые известные ему пункты назначения.

В случаях, когда у вас уже есть налаженная сеть Wi-Fi или Ethernet и множество устройств, которым требуется использовать одни и те же пути к внешней сети Reticulum (например, через LoRa), зачастую достаточно назначить одну систему в качестве шлюза Reticulum, добавив в конфигурацию этой системы любые внешние интерфейсы и включив на ней транспортный протокол. Любое другое устройство в вашей локальной сети Wi-Fi сможет подключиться к этой более широкой сети Reticulum, просто используя конфигурацию по умолчанию (*AutoInterface*).

Возможно, примеров в файле конфигурации будет достаточно для начала работы. Если вам нужна дополнительная информация, вы можете ознакомиться с главами «*Создание сетей*» и «*Интерфейсы*» данного руководства, но, прежде всего, рекомендуем начать с чтения следующего раздела «*Настройка подключения*», поскольку он дает наиболее полное представление о том, как обеспечить надежное подключение с минимальными затратами на обслуживание.

2.5 Настройка подключения

Reticulum — это не услуга, на которую вы подписываетесь, и не единая глобальная сеть, к которой вы «присоединяетесь». Это *сетевой стек*, набор инструментов для построения систем связи, соответствующих вашим конкретным ценностям, требованиям и операционной среде. Способ подключения к другим узлам Reticulum вы выбираете самостоятельно.

Одной из главных преимуществ Reticulum является то, что он предоставляет множество инструментов для налаживания, поддержания и оптимизации подключений. Эти инструменты можно использовать по отдельности или комбинировать в сложных конфигурациях для решения самых разных задач.

Независимо от того, стремитесь ли вы создать полностью закрытую сеть с физической изоляцией для своей семьи; построить отказоустойчивую сеть сообщества, способную выдержать сбой инфраструктуры; подключиться к как можно большему количеству узлов по всему миру; или просто поддерживать надёжное зашифрованное соединение с конкретной организацией, которая вам важна, Reticulum предоставляет все необходимые механизмы для реализации этих задач.

Не существует «правильного» или «неправильного» способа построить сеть Reticulum, и вам не нужно быть сетевым инженером, чтобы начать работу. Если информация передается так, как вы планировали, и ваши требования к конфиденциальности и безопасности соблюдены, ваша конфигурация считается удачной. Reticulum разработана таким образом, чтобы сделать возможными даже самые сложные и непростые сценарии, даже когда другие сетевые технологии не справляются с задачей.

2.5.1 Найдите свой путь

Когда вы впервые начинаете использовать Reticulum, вам необходимо установить соединение с узлами, с которыми вы хотите обмениваться данными — это процесс *инициализации подключения*.

Важно

Распространенной ошибкой в современных сетях является зависимость от нескольких централизованных, жестко запрограммированных точек входа. Если каждый пользователь просто подключается к одному и тому же списку публичных IP-адресов, найденному на веб-сайте, сеть становится уязвимой, централизованной и, в конечном итоге, не способной выполнить обещания децентрализации и отказоустойчивости. Здесь на вас лежит ответственность.

Reticulum поощряет подход *органического роста*. Вместо того, чтобы полагаться на постоянные статические соединения с удаленными серверами, вы можете использовать временные начальные соединения для непрерывного *обнаружения* более релевантной или локальной инфраструктуры. После обнаружения ваша система может автоматически формировать более прочные и прямые связи с этими узлами и отбрасывать временные начальные соединения. В результате получается сеть соединений, которая является географически релевантной, отказоустойчивой и эффективной.

Можно просто добавить несколько общедоступных точек входа в раздел [interfaces] конфигурации Reticulum и установить соединение, но лучшим вариантом будет включить *обнаружение интерфейсов* и либо вручную выбрать нужные локальные интерфейсы, либо включить автоматическое подключение обнаруженных интерфейсов.

Соответствующим параметром в данном контексте является опция интерфейса «*только для начальной загрузки*» (*bootstrap_only*). Это автоматизированный инструмент для более эффективного распределения подключений. Включив обнаружение интерфейсов и автоматическое подключение, а также пометив интерфейс как *bootstrap_only*, вы указываете Reticulum использовать этот интерфейс в первую очередь для поиска вариантов подключения, а затем отключать его, как только будет обнаружено достаточное количество точек входа. Это помогает создать топологию сети, которая отдаёт предпочтение локальности и отказоустойчивости перед простой централизацией, вызванной использованием лишь нескольких статических точек входа.

Хорошие места для поиска определений интерфейсов для начальной настройки подключения — это такие веб-сайты, как [directory.rns.recipes](#) и [rmap.world](#).

2.5.2 Создайте собственную инфраструктуру

Вам не нужен дата-центр, чтобы стать значимой частью экосистемы Reticulum. На самом деле, самые важные узлы в сети часто бывают самыми маленькими.

Мы настоятельно рекомендуем всем, даже домашним пользователям, думать о создании *личной инфраструктуры*. Не подключайте каждый телефон, планшет и компьютер в вашем доме напрямую к общедоступному интернет-шлюзу. Вместо этого перепрофилируйте старый компьютер, Raspberry Pi или поддерживаемый маршрутизатор, чтобы он действовал как ваш собственный, личный **транспортный узел**:

- Ваш локальный транспортный узел находится у вас дома, подключен к вашему Wi-Fi и, возможно, к радиointерфейсу (например, RNode).
- Вы настраиваете этот узел с интерфейсом *bootstrap_only* (возможно, это TCP-туннель к более широкой сети) и включаете обнаружение интерфейсов.

- Пока вы спите, работаете или готовите, ваш узел прослушивает сеть. Он обнаруживает других членов локального сообщества, проверяет их сетевые идентификаторы и автоматически устанавливает прямые соединения.
- Теперь ваши личные устройства подключаются к вашему *локальному* узлу, который является частью живой и динамичной локальной ячеистой сети. Ваш трафик проходит по локальным маршрутам, предоставляемым другими реальными людьми из сообщества, а не отскакивает от удаленного сервера.

Не ждите, пока другие построят сети, которые вы хотите видеть. Каждая сеть важна, возможно, даже больше всего те, которые поддерживают отдельные семьи и людей. Как только появится достаточное количество такой личной локальной инфраструктуры, их прямое соединение друг с другом, без прохождения через общедоступный Интернет, станет неизбежным.

2.5.3 Комбинирование стратегий

Нет необходимости ограничиваться одной стратегией. Наиболее надежные конфигурации часто сочетают в себе статические, динамические и обнаруживаемые интерфейсы.

- **Статические интерфейсы:** вы поддерживаете постоянный интерфейс для связи с надежным партнером или организацией, используя статическую конфигурацию.
- **Связи начальной загрузки:** вы подключаете интерфейс `bootstrap_only` к общедоступному шлюзу в Интернете, чтобы сканировать сеть в поисках новых подключаемых узлов или восстановить связь в случае сбоя других интерфейсов.
- **Местная широкополосная связь:** вы запускаете `RNodeInterface` на общей частоте, что обеспечивает вам полностью независимый и конфиденциальный широкополосный доступ как к вашей собственной сети, так и к другим узлам Reticulum по всему миру, при этом никакие «поставщики услуг» не могут контролировать или отслеживать, как вы взаимодействуете с людьми.

Комбинируя эти методы, вы создаете систему, защищенную от единичных точек отказа, адаптируемую к меняющимся условиям сети и лучше интегрированную в вашу физическую и социальную реальность.

2.5.4 Состояние сети и ответственность

Участвуя в работе более широких сетей, которые вы открываете и создаете, вы неизбежно столкнетесь с узлами, которые настроены некорректно, являются вредоносными или просто неисправны. Чтобы защитить свои ресурсы и ресурсы ваших локальных партнеров, вы можете использовать систему *управления «черными дырами»*.

Независимо от того, блокируете ли вы вручную спамерский идентификатор или подписываетесь на список `blackhole`, поддерживаемый надежным сетевым идентификатором, эти инструменты помогают обеспечить, чтобы *ваша* пропускная способность использовалась для того, что *вы* считаете легитимной коммуникацией. Это поддерживает эффективность вашего локального сегмента и способствует здоровью более широкой сети.

2.5.5 Вклад в глобальную сеть

Если у вас есть возможность разместить стабильный узел с публичным IP-адресом, рассмотрите возможность стать *публичной точкой входа*. Публикуя свой интерфейс как *обнаруживаемый*, вы предоставляете потенциальную точку подключения для других, помогая сети расти и охватывать новые области.

Рекомендации по правильной настройке общедоступной точки входа см. в разделе «Размещение общедоступных точек входа».

2.6 Подключение к распределенной магистрали

Глобальная распределенная магистраль транспортных узлов Reticulum поддерживается волонтерами со всего мира. Эта сеть представляет собой гетерогенное собрание как публичных, так и частных узлов, образующих нескоординированную, добровольную межсетевую магистраль, которая в настоящее время обеспечивает глобальные транспортные и межсетевые возможности для Reticulum.

В качестве хорошей отправной точки вы можете найти определения интерфейсов для подключения ваших собственных сетей к этой магистрали на таких веб-сайтах, как `directory.mns.recipes` и `rmap.world`.

Совет

Не полагайтесь в повседневной работе только на одно подключение к распределённой магистральной. Гораздо лучше настроить несколько резервированных подключений и включить параметры обнаружения интерфейсов, чтобы ваши узлы могли постоянно находить возможности для установления пиринговых соединений по мере развития сети. Ознакомьтесь с разделом «*Настройка подключений*» для понимания доступных параметров.

2.7 Хостинг публичных точек входа

Если вы хотите помочь в создании мощной глобальной магистрали межсетевого взаимодействия, вы можете разместить публичную (или частную) точку входа в сеть Reticulum через Интернет. В этом разделе приведены некоторые полезные рекомендации. После настройки публичной точки входа рекомендуется *сделать ее обнаруживаемой в Reticulum*.

Вам понадобится физический или виртуальный компьютер с публичным IP-адресом, доступный для других устройств в Интернете.

Наиболее эффективный и производительный способ реализации подключаемой точки входа, поддерживающей большое количество пользователей, — это использование BackboneInterface. Этот тип интерфейса полностью совместим с типами TCPClientInterface и TCPServerInterface, но работает значительно быстрее и потребляет меньше системных ресурсов, что позволяет вашему устройству обрабатывать тысячи подключений даже на небольших системах.

Также важно установить для вашего подключаемого интерфейса режим шлюза, так как это значительно сократит время сходимости сети и ускорит определение маршрута для любого, кто подключается к вашей точке входа.

*# В этом примере показан интерфейс магистрали, #
настроенный для работы в качестве шлюза, через который
пользователи # могут подключаться к публичной или
частной сети*

```
[[Публичный шлюз]]
type = BackboneInterface enabled
= yes
режим = шлюз listen_on =
0.0.0.0

port = 4242

# На общедоступных интерфейсах может быть #
целесообразно настроить разумные # целевые
значения частоты объявлений.
announce_rate_target = 3600
announce_rate_penalty = 3600
announce_rate_grace = 12
```

Если же вы хотите создать частную точку входа из Интернета, вы можете использовать *параметры IFAC name и passphrase*, чтобы защитить свой интерфейс с помощью сетевого имени и парольной фразы.

*# Частная точка входа, требующая предварительно
согласованного # имени сети и парольной фразы для
подключения.*

```
[[Частный шлюз]]
type = BackboneInterface enabled
= yes
mode = gateway listen_on =
0.0.0.0
```

(продолжение на следующей странице)

```
порт = 4242
имя_сети = private_ret пароль =
ZowjajquaflanPecAc
```

Если вы размещаете точку входа на операционной системе, которая не поддерживает `BackboneInterface`, вы можете вместо этого использовать

`TCPServerInterface`, хотя его производительность будет ниже.

2.8 Подключение экземпляров Reticulum через Интернет

В настоящее время Reticulum предлагает три интерфейса, подходящих для подключения экземпляров через Интернет: *Backbone*, *TCP* и *I2P*. Каждый интерфейс предоставляет свой набор функций, и пользователям Reticulum следует тщательно выбирать интерфейс, наиболее соответствующий их потребностям.

`TCPServerInterface` позволяет пользователям размещать экземпляр, доступный по TCP/IP. Этот метод, как правило, быстрее, имеет меньшую задержку и более энергоэффективен, чем использование `I2PInterface`, однако он также раскрывает больше данных о хосте сервера.

`BackboneInterface` — это очень быстрый и эффективный тип интерфейса, доступный в операционных системах POSIX, разработанный для одновременной обработки тысяч соединений с низкими затратами памяти, вычислительных ресурсов и ввода-вывода. Он полностью совместим с типами интерфейсов на основе TCP.

TCP-соединения раскрывают IP-адреса как вашего экземпляра, так и сервера любому, кто может проанализировать соединение. Кто-то может использовать эту информацию для определения вашего местоположения или личности. Злоумышленники, анализирующие ваши пакеты, могут записывать метаданные пакетов, такие как время передачи и размер пакета. Несмотря на то, что Reticulum шифрует трафик, TCP этого не делает, поэтому злоумышленник может с помощью анализа пакетов узнать, что на системе работает Reticulum, и какие другие IP-адреса к ней подключаются. Для размещения общедоступного экземпляра через TCP также требуется общедоступный IP-адрес, который большинство интернет-соединений больше не предоставляют.

`I2PInterface` направляет сообщения через протокол *Invisible Internet Protocol (I2P)*. Для использования этого интерфейса пользователи должны запускать демон `I2P` параллельно с `gnssd`. Для постоянно работающих узлов `I2P` рекомендуется использовать `i2pd`.

По умолчанию `I2P` шифрует и смешивает весь трафик, отправляемый через Интернет, а также скрывает IP-адреса экземпляров Reticulum как отправителя, так и получателя. Запуск узла `I2P` также будет ретранслировать зашифрованные пакеты других пользователей `I2P`, что потребует дополнительной пропускной способности и вычислительной мощности, но также значительно затруднит атаки по времени и другие формы глубокого анализа пакетов.

`I2P` также позволяет пользователям размещать глобально доступные экземпляры Reticulum с непубличных IP-адресов, а также за брандмауэрами и NAT.

В целом рекомендуется использовать узел `I2P`, если вы хотите разместить общедоступный экземпляр, сохранив при этом анонимность. Если вам важнее производительность и немного более простая настройка, используйте TCP.

2.9 Добавление радиointерфейсов

После установки и настройки Reticulum вы можете подключить радиointерфейсы с помощью любого имеющегося у вас совместимого оборудования. Reticulum поддерживает широкий спектр радиооборудования, и если у вас уже есть такое оборудование, скорее всего, оно будет работать с Reticulum. Информацию о настройке см. в разделе «*Интерфейсы*» данного руководства.

Если у вас еще нет приемопередатчика, вы можете легко и недорого собрать *RNode* — универсальный цифровой приемопередатчик дальнего действия, который легко интегрируется с Reticulum.

Чтобы собрать устройство самостоятельно, необходимо установить пользовательскую прошивку на поддерживаемую плату разработчика LoRa с помощью скрипта автоматической установки или веб-программы для прошивки. Инструкции см. в главе «*Коммуникационное оборудование*». Если вы предпочитаете приобрести готовое устройство, ознакомьтесь со списком поставщиков.

Другие аппаратные интерфейсы на основе радиосвязи разрабатываются и предоставляются более широким сообществом Reticulum. Более подробную информацию по этим темам можно найти в системах обмена информацией на базе Reticulum.

Если у вас есть коммуникационное оборудование, которое пока не поддерживается ни одним из *существующих типов интерфейсов*, вы можете легко написать (и, возможно, опубликовать) *модуль пользовательского интерфейса*, который обеспечит его совместимость с Reticulum.

2.10 Создание и использование пользовательских интерфейсов

Хотя Reticulum включает гибкий и широкий набор встроенных интерфейсов, они не охватывают все возможные типы коммуникационного оборудования, которое Reticulum потенциально может использовать для связи.

Поэтому можно легко написать собственные модули интерфейсов, которые можно загружать во время выполнения и использовать наравне с любыми встроенными типами интерфейсов.

Дополнительную информацию по данной теме, а также примеры кода, на которых можно основываться, см. в главе «*Настройка интерфейсов*».

2.11 Разработка программы с Reticulum

Если вы хотите разрабатывать программы, использующие Reticulum, самый простой способ начать — установить последнюю версию Reticulum с помощью pip:

```
pip install rns
```

Вышеуказанная команда установит Reticulum и зависимости, и вы будете готовы импортировать и использовать RNS в своих собственных программах. Следующим шагом, скорее всего, будет ознакомление с некоторыми *примерами программ*.

Весь API Reticulum описан в главе «*Справочник API*» данного руководства. Прежде чем приступить к работе, рекомендуется прочитать руководство полностью, но, как минимум, начните с главы «*Об Reticulum*».

2.12 Примечания по установке для конкретных платформ

Для некоторых платформ требуется несколько иная процедура установки или существуют различные особенности, о которых стоит знать. Они перечислены здесь.

2.12.1 Android

Reticulum можно использовать на Android разными способами. Самый простой способ начать — использовать приложение типа Sideband.

Для большего контроля и доступа к дополнительным функциям вы можете использовать Reticulum и связанные с ним программы через приложение Termux, доступное на момент написания статьи в F-droid.

Termux — это эмулятор терминала и среда Linux для устройств на базе Android, которая включает в себя возможность использования множества различных программ и библиотек, в том числе Reticulum.

Чтобы использовать Reticulum в среде Termux, вам необходимо установить Python и библиотеку python-cryptography с помощью pkg, встроенного в Termux менеджера пакетов. После этого вы сможете использовать pip для установки Reticulum.

В Termux выполните следующее:

```
# Сначала убедитесь, что индексы и пакеты обновлены.
pkg update pkg
upgrade

# Затем установите Python и библиотеку cryptography.
pkg install python python-cryptography

# Убедитесь, что pip обновлен до последней версии, и установите модуль wheel.
pip install wheel pip --upgrade
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

Установите Reticulum`pip install rns`

Если по какой-то причине пакет `python-cryptography` недоступен для вашей платформы через диспетчер пакетов Termux, вы можете попробовать скомпилировать его локально на своем устройстве с помощью следующей команды:

Сначала убедитесь, что индексы и пакеты обновлены.`pkg update pkg
upgrade`*# Затем установите зависимости для библиотеки cryptography.*`pkg install python build-essential openssl libffi rust`*# Убедитесь, что pip обновлен, и установите модуль wheel.*`pip install wheel pip --upgrade`*# Чтобы установщик смог скомпилировать модуль криптографии,**# нам нужно указать, для какой платформы мы компилируем:*`export CARGO_BUILD_TARGET="aarch64-linux-android"`*# Запустите процесс установки модуля криптографии.**# В зависимости от вашего устройства это может занять несколько минут, # так как модуль должен быть скомпилирован локально на вашем устройстве.*`pip install cryptography`*# Если вышеуказанная установка прошла успешно, теперь вы можете установить # Reticulum и любое связанное с ним программное обеспечение*`pip install rns`

Также возможно включить Reticulum в приложения, скомпилированные и распространяемые в виде APK-файлов для Android. Подробное руководство и пример исходного кода будут добавлены сюда позже. До тех пор вы можете использовать *исходный код Sideband* в качестве примера и отправной точки.

2.12.2 ARM64

На некоторых архитектурах, в том числе ARM64, не для всех зависимостей доступны предварительно скомпилированные бинарные файлы. На таких системах перед установкой Reticulum или программ, зависящих от Reticulum, может потребоваться установить `python3-dev` (или аналогичный пакет).

Установите Python и пакеты для разработки`sudo apt update
sudo apt install python3 python3-pip python3-dev`*# Установите Reticulum*`python3 -m pip install rns`

После установки этих пакетов `pip` сможет локально установить все недостающие зависимости в вашей системе.

2.12.3 Debian Bookworm

В версиях Debian, выпущенных после апреля 2023 года, по умолчанию больше невозможно использовать `pip` для установки пакетов в вашу систему. К сожалению, вам придется вместо этого использовать заменяющую команду `pipx`, которая помещает установленные пакеты в изолированную среду. Это не должно негативно повлиять на Reticulum, но не сработает для включения и использования Reticulum в ваших собственных скриптах и программах.

```
# Установите pipx
sudo apt install pipx

# Обеспечить доступ к установленным программам из командной строки
pipx ensurepath

# Установить Reticulum
pipx install rns
```

В качестве альтернативы вы можете восстановить нормальное поведение `pip`, создав или отредактировав файл конфигурации, расположенный в `~/.config/pip/pip.conf`, и добавив следующий раздел:

```
[global]
break-system-packages = true
```

Для однократной установки Reticulum без глобального включения опции `break-system-packages` можно использовать следующую команду:

```
pip install rns --break-system-packages
```

Примечание

Директива `--break-system-packages` — это несколько вводящее в заблуждение название. Её установка, конечно, не повредит никакие системные пакеты, а просто позволит устанавливать пакеты `pip` как для отдельного пользователя, так и для всей системы. Хотя в редких случаях это *может* привести к конфликтам версий, в целом это не создаёт никаких проблем, особенно при установке Reticulum.

2.12.4 MacOS

Для установки Reticulum в macOS необходимо, чтобы на компьютере были установлены Python и менеджер пакетов `pip`.

Системы под управлением macOS могут значительно различаться по тому, предустановлен ли Python, и если да, то какая версия установлена, а также установлен ли и настроен ли менеджер пакетов `pip`. Если вы сомневаетесь, вы можете [загрузить и установить](#) Python вручную.

Если Python и `pip` доступны в вашей системе, просто откройте окно терминала и выполните одну из следующих команд:

```
# Установите Reticulum и утилиты с помощью pip:
pip3 install rns

# В некоторых версиях для установки может
# потребоваться использовать флаг --break-system-
# packages:
pip3 install rns --break-system-packages
```

Примечание

Директива `--break-system-packages` — это несколько вводящее в заблуждение название. Её установка, конечно, не повредит никакие системные пакеты, а просто позволит устанавливать пакеты `pip` на уровне пользователя и всей системы. Хотя в редких случаях это *может* привести к конфликтам версий, в целом это не создаёт никаких проблем, особенно в случае Установки Reticulum.

Кроме того, некоторые комбинации версий macOS и Python требуют, чтобы вы вручную добавили каталог установленных пакетов `pip` в переменную среды `PATH`, прежде чем вы сможете использовать установленные команды в терминале. Обычно достаточно добавить следующую строку в скрипт инициализации оболочки (например, `~/.zshrc`):

```
export PATH=$PATH:~/Library/Python/3.9/bin
```

Настройте версию Python и расположение скрипта инициализации оболочки в соответствии с вашей системой.

2.12.5 OpenWRT

На системах OpenWRT с достаточным объемом хранилища и памяти вы можете установить Reticulum и связанные утилиты с помощью менеджера пакетов `opkg` и `pip`.

Примечание

На момент выпуска данного руководства ведется работа над созданием готовых пакетов Reticulum для OpenWRT с полной интеграцией конфигурации, сервисов и `uci`. Для получения дополнительной информации обратитесь к репозиториям [feed-reticulum](#) и [reticulum-openwrt](#).

Чтобы установить Reticulum на OpenWRT, сначала войдите в командную строку, а затем следуйте приведенным ниже инструкциям:

```
# Установите зависимости для
opkg install python3 python3-pip python3-cryptography python3-pyserial

# Установите Reticulum
pip install rns

# Запустите rnsd с включенной отладкой
rnsd -vvv
```

Примечание

Приведенные выше инструкции были проверены и протестированы исключительно на OpenWRT 21.02. Возможно, что для других версий потребуются слегка измененные команды установки или названия пакетов. Вам также понадобится достаточно свободного места в файловой системе оверлея и достаточно свободной оперативной памяти для запуска Reticulum и любых связанных с ним программ и утилит.

В зависимости от конфигурации вашего устройства может потребоваться настроить правила брандмауэра, чтобы обеспечить работу Reticulum при подключении к устройству и с него. До тех пор, пока не будет готов соответствующий пакет, вам также необходимо вручную создать службу или скрипт автозапуска для автоматического запуска Reticulum при загрузке системы.

Также обратите внимание, что для работы *AutoInterface* необходимо включить локальные IPv6-адреса для всех устройств Ethernet и Wi-Fi, которые вы планируете использовать. Если команда ``ip a`` показывает для данного устройства адрес, начинающийся с ``fe80::``, то *AutoInterface* должен работать с этим устройством.

2.12.6 Raspberry Pi

В настоящее время для запуска Reticulum на компьютерах Raspberry Pi рекомендуется использовать 64-разрядную версию Raspberry Pi OS, поскольку для 32-разрядных версий не всегда доступны пакеты, необходимые для некоторых зависимостей. Если Python и менеджер пакетов `pip` еще не установлены, сначала установите их, а затем установите Reticulum с помощью `pip`.

```
# Установите зависимости для
sudo apt install python3 python3-pip python3-cryptography python3-pyserial
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

Установить Reticulum

pip install rns --break-system-packages

Примечание

Директива `--break-system-packages` — это несколько вводящее в заблуждение название. Её установка, конечно, не повредит никакие системные пакеты, а просто позволит устанавливать пакеты `pip` как для пользователя, так и для всей системы. Хотя в редких случаях это *может* привести к конфликтам версий, обычно это не создаёт никаких проблем, особенно при установке Reticulum.

Хотя установка и запуск Reticulum на 32-разрядных версиях ОС Raspberry Pi возможны, для этого потребуется вручную настроить и установить необходимые зависимости для сборки, что не описано в данном руководстве.

2.12.7 RISC-V

На некоторых архитектурах, включая RISC-V, не для всех зависимостей имеются предварительно скомпилированные бинарные файлы. На таких системах перед установкой Reticulum или программ, зависящих от Reticulum, может потребоваться установка `python3-dev` (или аналогичного пакета).

Установка Python и пакетов для разработки

sudo apt update

sudo apt install python3 python3-pip python3-dev

Установить Reticulum

python3 -m pip install rns

После установки этих пакетов `pip` сможет локально скомпилировать любые недостающие зависимости в вашей системе.

2.12.8 Ubuntu Lunar

В версиях Ubuntu, выпущенных после апреля 2023 года, по умолчанию больше невозможно использовать `pip` для установки пакетов в вашу систему. К сожалению, вам придётся вместо этого использовать замену — команду `pipx`, которая помещает установленные пакеты в изолированную среду. Это не должно негативно повлиять на Reticulum, но не работает для включения и использования Reticulum в ваших собственных скриптах и программах.

Установите pipx

sudo apt install pipx

Сделать установленные программы доступными в командной строке

pipx ensurepath

Установить Reticulum

pipx install rns

В качестве альтернативы вы можете восстановить нормальное поведение `pip`, создав или отредактировав файл конфигурации, расположенный в `~/.config/pip/pip.conf`, и добавив следующий раздел:

[global]

break-system-packages = true

Для однократной установки Reticulum без глобальной активации параметра `'break-system-packages'` можно использовать следующую команду:

```
pip install rns --break-system-packages
```

Примечание

Директива `--break-system-packages` — это несколько вводящий в заблуждение выбор слов. Её установка, конечно, не повредит никакие системные пакеты, а просто позволит устанавливать пакеты `pip` на уровне пользователя и всей системы. Хотя в редких случаях это *может* привести к конфликтам версий, обычно это не создаёт никаких проблем, особенно при установке Reticulum.

2.12.9 Windows

В операционных системах Windows самый простой способ установить Reticulum — использовать менеджер пакетов `pip` из командной строки (командной строки Windows или Windows PowerShell).

Если у вас еще не установлен Python, скачайте и установите его. На момент публикации этого руководства рекомендуемая версия — Python 3.12.7.

Важно! Когда установщик запросит, обязательно добавьте программу Python в переменные среды PATH. Если вы этого не сделаете, вы не сможете использовать установщик `pip` или запускать из командной строки утилиты Reticulum, входящие в состав пакета (такие как `rnsd` и `rnsstatus`).

После установки Python откройте командную строку или Windows Powershell и введите:

```
pip install rns
```

Теперь вы можете использовать Reticulum и все входящие в него утилиты непосредственно из вашего любимого интерфейса командной строки.

2.13 Reticulum на чистом Python

Предупреждение

Если вы используете пакет `gnspure` для запуска Reticulum на системах, не поддерживающих [PyCA/криптографию](#), важно прочитать и понять раздел «*Криптографические примитивы*» данного руководства.

В некоторых редких случаях, а также на менее распространенных типах систем, установка одной или нескольких зависимостей может оказаться невозможной. В таких ситуациях можно использовать пакет `gnspure` вместо пакета `rns` или запустить `pip` с опцией командной строки `--no-dependencies`. Пакет `gnspure` не требует никаких внешних зависимостей для установки. Обратите внимание, что фактическое содержимое пакетов `rns` и `gnspure` *полностью идентично*. Единственное отличие заключается в том, что пакет `gnspure` не содержит списка зависимостей, необходимых для установки.

Независимо от того, как установлен и запущен Reticulum, он будет загружать внешние зависимости только в том случае, если они *необходимы* и *доступны*. Если, например, вы хотите использовать Reticulum в системе, которая не поддерживает `pyserial`, это вполне возможно с помощью пакета `gnspure`, но Reticulum не сможет использовать интерфейсы, основанные на последовательном вводе-выводе. Все остальные доступные модули будут по-прежнему загружаться при необходимости.

ДЗЕН RETICULUM

3.1 Иллюзия центра

На протяжении большей части поколения нас учили воспринимать цифровой мир через призму иерархии. В наших ментальных картах доминирует единственный, вводящий в заблуждение образ: **Облако**.

Мы представляем себе сеть как огромное, эфирное пространство «там, наверху» или «где-то там». Централизованное хранилище сервисов и данных, к которому мы, скромные клиенты, должны подключаться. Мы создаем наше программное обеспечение, жестко заложив в нашу логику следующее допущение: *есть сервер. Сервер обладает полномочиями. Сервер знает путь. Я должен найти сервер, чтобы работать.*

Это ментальная модель «клиент-сервер», и она является основным препятствием для понимания Reticulum.

3.1.1 Заблуждение облака

Первый шаг в «Дзен Ретикулама» — осознать, что *никакого «облака» не существует*. Есть только чужие компьютеры. Разрабатывая приложения для «облака», вы работаете *на* арендодателя. Вы соглашаетесь с тем, что существование вашего приложения зависит от разрешения, работоспособности и постоянной благосклонности центрального органа.

В Reticulum вы должны переключить свое мышление с «подключения к» на «нахождение среди». Reticulum — это не услуга, на которую вы подписываетесь, — *это ткань, в которой вы обитаете*. Нет никакого «там, наверху». Есть только «здесь» и «там», а пространство между ними — это одноранговая сеть.

3.1.2 Децентрализация или нецентрализуемость?

В современных технологических кругах часто можно услышать слово «децентрализованный». Но зачастую это всего лишь маркетинговый термин, обозначающий «слегка распределенную централизацию». Блокчейн с несколькими доминирующими майнерами или федеративный протокол с несколькими гигантскими серверами. *На практике* это все равно централизация. Просто вместо одного центра здесь несколько.

Reticulum идет дальше. Он стремится к **нецентрализуемости**.

Это не просто политическая позиция, основанная на желании, а фундаментальная математическая особенность протокола, на которой построено всё остальное. Reticulum исходит из того, что каждый узел в сети потенциально враждебен, а каждое соединение потенциально скомпрометировано. Протокол разработан без «привилегированных» узлов. Хотя некоторые узлы могут выступать в роли транспортных экземпляров

— пересылая трафик для других — они делают это *«вслепую»* и знают только о своем ближайшем окружении, и ни о чем больше. Они маршрутизируют на основе криптографических доказательств, а не административных привилегий. Они не могут видеть, кто с кем общается, и не могут выборочно манипулировать трафиком, не нарушая при этом свою собственную способность к полной маршрутизации.

Система построена таким образом, что иерархия становится структурно невозможной. Вы не можете захватить адрес, поскольку не существует центрального реестра, который можно было бы захватить. Вы не можете заблокировать пользователя, поскольку нет центрального переключателя, который можно было бы нажать. Вы можете предлагать пути прохождения по сети, но не можете заставить кого-либо ими пользоваться.

3.1.3 Смерть адресу

Чтобы освободиться от центра, вы должны также отказаться от концепции «адреса».

В мире IP-адресов адрес — это местоположение. Это координата в *строго иерархичной* и статичной сетке. Если вы перенесете компьютер в другой дом, ваш адрес изменится. Если ваш маршрутизатор перезагрузится, ваш адрес может измениться. Ваша *идентичность* привязана к вашему *местоположению*, а значит, она уязвима и легко поддается контролю.

Reticulum устраняет эту связь между *личностью* и *местоположением*.

В Reticulum адрес — это не место, а **хеш идентичности**. Это криптографическое представление того, *кто* вы есть, а не *где* вы находитесь. Благодаря этому ваш адрес является переносимым. Вы можете перенести ноутбук из WiFi-кафе в Берлине в сеть LoRa в горах, а затем на пакетную радиосвязь на лодке, и ваш «адрес» — *хеш назначения* — никогда не изменится.

Сеть не направляет трафик в какое-то место; она направляет его *конкретному человеку* (или устройству). Когда вы отправляете пакет, вы не указываете координаты в сетке; вы шифруете сообщение для конкретного адресата. Сеть динамически определяет, где в данный момент находится этот адресат, и делает это таким образом, что никто на самом деле не знает, где он физически расположен.

Рассмотрим:

- **Старый способ:** «Я нахожусь по адресу 192.168.1.5. Найдите меня».
- **«Дзен-подход»:** «Я — <327c1b2f87c9353e01769b01090b18f2>. Где бы я ни был, мои пиры могут связаться со мной». Как только вы перестаете думать о серверах и начинаете думать о переносимых идентичностях, где каждый всегда может напрямую связаться с кем угодно, иллюзия центра исчезает. Вы понимаете, что *нет* никакого центра, который удерживает сеть вместе. Не нужны ни координаторы, ни бюрократы. Сеть — это просто совокупность ее участников, общающихся напрямую, независимо и без какого-либо хозяина.

3.2 Физика доверия

Паранойя — отличный принцип проектирования

Если мы примем, что центра нет — что сеть представляет собой хаотичную одноранговую ячеистую структуру — нам придется столкнуться с пугающей реальностью: **никто не стоит на страже у входа**.

В традиционном сетевом мышлении мы полагаемся на концепцию «доверенного ядра». Мы предполагаем, что Wi-Fi в нашей местной кофейне безопасен или что провайдеры магистральных сетей являются нейтральными хранителями. Мы строим нашу безопасность как замок: крепкие стены снаружи, мягкие и доверчивые внутри. Мы используем шифрование только тогда, когда выходим в «дикий» Интернет.

3.2.1 Враждебные среды

«Дзен Reticulum» требует от вас перевернуть эту концепцию. Вы должны исходить из того, что *любая* среда враждебна. Это не цинизм, а просто безразличная физика.

Когда вы передаете информацию по радиоволнам, это все равно что кричать в переполненной комнате. Слушать может кто угодно. Когда вы передаете данные через Интернет, ваши пакеты проходят через маршрутизаторы, контролируемые посторонними людьми, корпорациями и государственными структурами. Рассчитывать на конфиденциальность в такой среде без криптографической защиты — это не оптимизм, а грубая халатность.

Reticulum построено на предположении, что каждое соединение прослушивается, а каждый узел — потенциальный противник. Если ваша система не может выжить в условиях, когда противник контролирует физический уровень, она не может выжить вообще.

Но в этом и заключается парадокс: считая сеть враждебной, вы делаете её безопасной. Когда вы принимаете опасности такими, какие они есть, они становятся управляемыми. Когда вы перестаете доверять инфраструктуре и начинаете доверять математике, вы устраняете единственную точку отказа: человеческий фактор.

3.2.2 Шифрование — это не функция

В мире TCP/IP шифрование — это добавка. Это слой, который мы накладываем поверх протокола (HTTPS, TLS), чтобы залатать дыры в безопасности исходного дизайна. Это «функция», которую вы иногда *включаете* для «конфиденциальных данных». Это принципиально неверно, поскольку все данные являются конфиденциальными.

В Reticulum шифрование — это **гравитация**.

Это не просто дополнительная опция. Это не плагин. Это *фундаментальная сила, обеспечивающая существование сети*. Если убрать шифрование из Reticulum, маршрутизация перестанет работать. Транспортная система использует криптографические подписи и энтропию для проверки маршрутов и передачи информации. Если бы пакеты передавались в виде открытого текста, промежуточные узлы не смогли бы подтвердить достоверность маршрута, а конечные точки — предотвратить подделку или взлом.

В Reticulum энтропия зашифрованного пакета и *есть* логика маршрутизации.

Просить версию Reticulum без шифрования — все равно что просить версию океана без жидкости. Вы просите не об изменении функционала, а о другой физической вселенной. Мы разрабатываем решения для вселенной, в которой информация обладает массой, структурой и целостностью.

3.2.3 Архитектуры «нулевого доверия»

Мы должны отучиться полагаться на **институциональное доверие**.

На протяжении десятилетий нас приучали доверять авторитетам. Мы доверяем веб-сайту, потому что за него ручается цепочка центров сертификации (компаний, которых мы не знаем). Мы доверяем приложению, потому что оно находится в магазине приложений (управляемом корпорацией, которую мы не контролируем). Мы доверяем сообщению, потому что оно пришло с номера телефона, присвоенного оператором связи. И все же сегодня все в нашей цифровой информационной сфере менее надежно и рискованнее, чем средневековый рынок подержанного нижнего белья.

Reticulum заменяет институциональное доверие **криптографическим доказательством**.

В Reticulum вы доверяете узлу не потому, что у него красивое имя хоста или потому, что он указан в каталоге. Вы доверяете ему, потому что он хранит закрытый ключ, соответствующий хэшу назначения, с которым вы общаетесь. Это доверие является бинарным, математическим и **абсолютным**. Либо подпись совпадает, либо нет. Здесь нет «возможно».

Этот сдвиг переносит власть от института к индивидууму. Вы становитесь высшим арбитром своих собственных доверительных отношений. Вы решаете, какие ключи принимать, по каким путям идти и какие идентичности признавать.

Рассмотрим:

- **Старый способ:** *«Я доверяю этому сайту, потому что браузер показывает, что значок замка зеленый».*
- **Подход Zen:** *«Я доверяю этому сайту, потому что я проверил его хеи-отпечаток вне канала связи, и математика подтверждает подпись».*

Когда вы усваиваете «физику доверия», вы перестаете искать защиты в брандмауэрах, VPN и соглашениях об условиях предоставления услуг. Вы понимаете, что настоящая безопасность зависит от самой архитектуры протокола. Вы можете перестать доверять облаку и начать доверять коду — ведь вы можете проверить его самостоятельно.

3.3 Преимущества дефицита

Каждый бит имеет значение

Мы пристрастились к изобилию. В современной цифровой экосистеме пропускная способность воспринимается как бесконечный, ровный океан. Мы без зазрения совести смотрим потоковое видео в высоком разрешении, отправляем целые библиотеки кода ради отображения одной-единственной кнопки и измеряем производительность в гигабитах в секунду. Это изобилие опустошило наше мастерство. Когда исчезают ограничения, гибкость умирает, а вместе с ней — и некая ясность и качество.

Reticulum предлагает вам выйти из океана и встать на канатоходческую веревку.

3.3.1 Заблуждение пропускной способности

«Дзен Reticulum» предполагает осознание того, что **скорость в 5 бит в секунду — это вполне приемлемый показатель**.

Современному разработчику это кажется параличом. Но в ограничениях таится глубокая свобода: когда у вас гигабитное соединение, вы можете быть невероятно неаккуратными. Вы можете растрачивать ресурсы. Вы можете перекладывать свои проблемы на инфраструктуру. *«Медленно? Купите более быстрый роутер».*

Но на канале с высокой задержкой и низкой пропускной способностью (будь то зашумленный радиоканал HF или слабый прыжок LoRa) вы не можете переложить проблемы никуда. Вы должны их решать. Сеть не идет на компромисс с расточительством.

Это вынуждает перейти от потребления к взаимодействию. Вы больше не потребляете услугу, предоставляемую «толстой трубой»; вы вступаете в осторожные переговоры с физической средой. Среда становится партнером в разговоре, а не просто тупым каналом. Вам внезапно нужно *понять мир, чтобы быть в нем*.

3.3.2 Стоимость байта

В условиях дефицита ресурсов байт — это не просто данные, но и энергия, время и пространство.

Каждый переданный вами байт расходует заряд батареи на узле, работающем от солнечной энергии. Он занимает ценное эфирное время, которое могло бы быть использовано другим узлом. Он представляет собой измеримую часть электромагнитного спектра.

Когда вы осознаете это, вы начинаете писать код по-другому. Вы перестаете спрашивать: «Сколько данных я могу отправить?», а начинаете спрашивать: «Каков *минимальный* объем информации, необходимый для передачи этого намерения? Как я могу наилучшим образом использовать свою информационную энтропию?».

Именно в этом проявляется элегантность Reticulum. Протокол разработан так, чтобы избавиться от всего несущественного. Для установления соединения требуется три очень маленьких пакета. Хеш назначения помещается в 16 байт. Накладные расходы ничтожно малы, оставляя почти весь канал для самого сообщения.

Рассмотрим:

- **Старый способ:** *«Мне нужно отправить обновление статуса. Я отправлю JSON-объект с метаданными, временными метками и информацией о профиле пользователя (15 КБ)».*
- **Способ Zen:** *«Мне нужно отправить обновление статуса. Я отправлю один байт, представляющий код состояния. Контекст уже известен».*

Конечно, это оптимизация, но, что еще важнее, *это проявление уважения*. Эффективность в общем пространстве — это проявление ответственности. Забирая из сети только то, что вам нужно, вы оставляете место для других. Сеть прислушивается к тем, кто говорит с целью.

3.3.3 Поток и время

Дефицит также учит нас понимать время. Мы стали зависимы от *синхронного* «сейчас» — мгновенного отклика, потока в реальном времени. Но Reticulum принимает *асинхронное* время.

Когда соединение прерывистое, а задержка измеряется минутами или часами, «реальное время» становится иллюзией. Reticulum предлагает использовать **механизм «Store and Forward»** не просто в качестве запасного варианта, а в качестве основного способа работы. Вы пишете сообщение, оно отправляется, когда это становится возможным, и доходит, когда доходит.

Это меняет психологическую структуру общения. Это снимает тревогу, связанную с необходимостью немедленного ответа. Это дает возможность для размышлений. Вы не требуете внимания получателя *прямо сейчас*; вы оставляете ему подарок на пути, который он найдет, когда будет готов.

Разрабатывая с учетом задержек, вы обеспечиваете отказоустойчивость. Вы больше не строите картонный домик, который рухнет при потере всего одного пакета. Вы возводите каменную арку, которая распределяет нагрузку *во времени*.

3.3.4 Освобождение от ограничений

В дефиците есть странный оптимизм. Когда вы вынуждены работать в условиях строгих ограничений, вы вынуждены расставлять приоритеты. *Вы* должны решить, что действительно важно. *В этом* и заключается суть свободы воли.

В бесконечном фантастическом мире «Облака» все срочно, поэтому ничего не срочно. В экономике Reticulum стоимость передачи данных заставляет вас взвешивать ценность вашего сообщения. Вам действительно нужно отправить этот пульс? Эта фотография необходима?

Когда вы отсеиваете шум, остается *сигнал*.

Эта дисциплина формирует разработчика нового типа. Она формирует мастера своего дела, понимающего, что лучший код — это тот, который не нужно писать. Она формирует пользователя, понимающего, что самое сильное сообщение — это то, которое *понятно*, а не то, которое громче всего. В мире Reticulum вы — не просто потребитель пропускной способности; вы — архитектор замысла.

3.4 Суверенитет через инфраструктуру

Будьте своей собственной сетью

Мы живем в эпоху цифрового арендования. Мы арендуем подключение у интернет-провайдеров. Мы арендуем хранилище у поставщиков облачных услуг. Мы даже заимствуем свою идентичность у социальных сетей. Мы — арендаторы в доме, который не строили, подчиняемся правилам, которые не писали, и подвергаемся выселению по прихоти арендодателя, который никогда нас не встречал.

«Дзен» Reticulum — это осознание того, что вы *можете* стать владельцем этого дома.

3.4.1 Заблуждение операторского уровня

На протяжении десятилетий нам внушали, что создание сетей — это не просто сложно, а вообще невозможно. Это представляется как некая «черная магия», доступная лишь телекоммуникационным компаниям и миллиардерам, требующая миллионов вложений в оптоволоконные сети, климатизированные центры обработки данных и целые армии инженеров. Нам говорят, что создание надёжной инфраструктуры — это «слишком сложная задача» для отдельного человека или небольшой организации.

Это большая, жирная ложь.

Физика проста. Радиоволне нужны передатчик и приемник. Пакету нужен путь. «Сложность» современного интернета в основном бюрократическая — гора биллинговых систем, регуляторный захват и устаревшее наследие, призванное удержать власть в руках «хранителей ворот».

Reticulum избавляет от бюрократии. Он работает на оборудовании, которое стоит столько же, сколько ужин. Он работает на частотах, доступных для свободного использования. Он доказывает, что для создания надёжной сети планетарного масштаба не нужна компания из списка Fortune 500. Для этого нужны лишь желание внедрить проект и распределённые, нескоординированные усилия множества людей.

3.4.2 Личная инфраструктура

Вот где все становится реальностью. Вы можете читать о Reticulum, вы можете понимать теорию, но истинное понимание приходит только тогда, когда вы подключаете радио и запускаете транспортный узел. Внезапно вы больше не потребитель. Вы — оператор.

Это изменение едва заметно, но глубоко. Когда вы управляете собственной инфраструктурой, сеть перестает быть услугой, которую вам *предоставляют*. Она становится пространством, в котором вы *живёте*. Вы несёте ответственность за поток информации. Вы обретаете глубокое понимание этой среды — того, как погода влияет на радиоволны, как меняется топология, как пакеты данных «танцуют» в эфире.

Из этого проистекает спокойная уверенность. Вы перестаете спрашивать: «Интернет не работает?», а начинаете спрашивать: «*Мои* каналы связи на месте?». Вы перестаете ждать прихода технического специалиста и начинаете проверять журналы. Это и есть проявление силы. Понять систему, по которой передаются ваши слова, — значит освободиться от той загадочности, которая держит вас в зависимости.

3.4.3 Способность отключаться

Зачем утруждаться? Зачем покупать радио, писать конфигурацию и оставлять Pi работать в углу?

Потому что старая централизованная сеть уязвима. И потому что большинство из нас уже и не хочет в ней находиться.

Интернет, на который мы полагаемся сегодня, представляет собой цепочку единичных точек отказа. Перережьте подводный кабель — и целый континент погрузится в темноту. Отключите энергосеть — и облако испарится. Снизьте приоритет «неправильного» трафика — и поток информации задушится.

Суверенитет — это способность пережить отключение, независимо от того, было ли оно случайным или преднамеренным.

Создавая собственную инфраструктуру, вы создаете линию спасения. Reticulum разработан для работы в средах, недоступных традиционному интернету — по простым проводам, через радиостанции на батарейках, в импровизированных сетях Wi-Fi. Когда сеть отключается, приходят цензоры или не оплачен счет, ваша сеть Reticulum продолжает работать.

Речь не идет о «выпадении» из общества. Речь идет о создании основы, на которой может функционировать настоящее *Общество*.

Рассмотрим:

- **Старый подход:** «У меня медленное соединение. Мне нужно позвонить своему провайдеру и пожаловаться».
- **Путь дзен:** «На пути много помех. Я настрою антенну или найду лучший маршрут».

Взяв инфраструктуру под свой контроль, вы обретае контроль над своим голосом. Вы перестаете кричать в чужой рупор и начинаете создавать свой собственный. Сеть больше не является чем-то, что просто происходит с вами; это то, что вы сами создаете.

3.5 Идентичность и кочевничество

Меняющееся «я»

В старом мире вас определяют ваши координаты. Если вы находитесь по адресу 34.109.71.5, вы *здесь*. Если вы отсоедините кабель и пойдете по улице, вы исчезнете. Ваше цифровое «я» испарится, потому что оно было привязано к стене. Вы — призрак в бесконечных механизмах шестеренок, рычагов и транзисторов, привязанных к аппаратному обеспечению и тем, кто им владеет.

Это порождает едва уловимое, постоянное беспокойство. Мы боимся отключения, потому что в архитектуре старого Интернета отключение — это своего рода смерть.

«Дзен ретикулума» предлагает иной способ существования.

3.5.1 Портативное существование

В Reticulum ваша личность — это не местоположение и не имя пользователя, присвоенное сервисом. Это криптографический ключ — сложная, уникальная математическая подпись, существующая независимо от физического мира. Если хотите, вы можете хранить его только в своей голове.

Думайте об этом не как о почтовом адресе, а скорее как об имени. *Настоящем имени*.

Если вы путешествуете из Берлина в Токио, вы не меняете своего имени. Вы остаетесь собой. Люди, которые вас знают, по-прежнему могут вас узнать. Reticulum применяет этот принцип к сетевому уровню. Ваш хеш назначения (Destination Hash) **неизменен**. Он путешествует вместе с вами, надежно хранится на вашем устройстве и *непоколебим, как камень*.

Это меняет отношения между вами и машиной. Вы не «входите» в сеть через конкретный шлюз. Вы — конечная точка. Сеть не подключается к месту; *она сходится на вас*.

3.5.2 Роуминг-узлы

Эта свобода вводит новое понятие времени и пространства: **кочевничество**.

Поскольку ваша идентичность переносима, ваша связь может быть гибкой. В один момент вы можете сидеть за столом, подключенным к оптоволоконной магистрали, а в следующий — идти по полю, подключенным только к дальнобойной сетке LoRa. Для остальных

В сети ничего не изменилось. Вашим друзьям не нужно обновлять ваши контактные данные. Сообщения, которые они отправляют, не возвращаются обратно. Сеть улавливает изменение среды и автоматически перенаправляет поток данных.

Вы больше не являетесь стационарным узлом в фиксированной сетке. Вы — странник в изменчивой среде.

Интерфейсы — будь то Wi-Fi, Ethernet, Packet Radio или физический кабель — это всего лишь «одежда», которую носит ваш узел. Вы меняете её в соответствии с окружающей средой. Внутри вы остаётесь прежними. В этом и заключается свобода протокола. Он рассматривает физическую среду как временное обстоятельство, а не как определение самой сущности.

Рассмотрим:

- **Старый способ:** *«Я потерял соединение. Мне нужно заново подключиться к VPN, чтобы сообщить им, где я сейчас нахожусь».*
- **Путь дзен:** *«Я переместился. Сеть незаметно подстраивается под эту новую реальность».*

3.5.3 Объявление о присутствии

Как сеть находит странника? Она прислушивается.

В мире IP мы запрашиваем каталоги. Мы спрашиваем сервер: «Где Марк?» Сервер проверяет свою базу данных и дает нам координаты. Это означает, что кто-то, где-то, отслеживает вас. Это предполагает и *требует* слежки.

Reticulum заменяет слежку на **объявления**.

Вместо того чтобы спрашивать у центрального органа, где вы находитесь, вы просто сообщаете о своём присутствии. Вы передаёте криптографическое доказательство: «Я здесь, и я тот, за кого себя выдаю». Эта информация распространяется по сети. Ваши соседи принимают её, обновляют свои таблицы маршрутов и передают дальше.

Это тихий, органичный процесс. Это цифровой эквивалент зажигания фонарей в темноте. Вам не нужно гнаться за светом; вы позволяете свету найти вас. Это уважает вашу автономию. Вы выбираете, когда объявлять о себе, как часто говорить и кому. Вы также выбираете, когда исчезнуть — на мгновение или навсегда.

3.5.4 Якорь в потоке

В этом кочевничестве есть глубокий покой. Оно учит вас, что стабильность не приходит от неподвижности. Стабильность приходит от *внутренней согласованности*.

Обладая собственным закрытым ключом, вы владеете собственным центром тяжести. Мир вокруг вас — инфраструктура, топография и доступность каналов связи — может хаотично меняться. Бури могут сносить вышки. Кабели могут быть перерезаны. Интернет может перестать работать.

Но пока у вас есть ключ, у вас есть ваша личность. Вся инфраструктура может быть разрушена и восстановлена, а вы останетесь собой. Ничто не вечно, но и ничего не теряется.

Вы становитесь самостоятельной сущностью, движущейся сквозь информационный шум, связанной не жесткими кабелями, а плавными потоками взаимного распознавания. Сеть превращается в пространство, в котором вы обитаете, а не в услугу, на которую вы подписываетесь: вы чувствуете себя как дома в эфире.

3.6 Этика инструмента

Технология с совестью

Вы отучились от центра. Вы приняли физику доверия. Вы приняли экономику дефицита и свободу безграничного кочевничества. Вы стоите в новом пространстве. Теперь посмотрите на инструмент в вашей руке.

В прежнем мире нас учили, что технологии нейтральны. Нам говорят, что «не оружие убивает людей, а люди», или что деталь — это просто деталь, не имеющая отношения к своему комбинаторному потенциалу. Это удобная ложь. Она нужна лишь для того, чтобы создатели могли умыть руки от ответственности.

Но теперь мы знаем лучше. Мы знаем, что **архитектура — это политика**, а **политика — это контроль**. То, как вы строите систему, определяет, как она будет использоваться. Если вы создадите систему, оптимизированную для массовой слежки, вы получите паноптикон. Если

вы создадите систему, оптимизированную для централизованного контроля, вы получите диктатуру. Если вы создадите систему, оптимизированную для извлечения, вы получите паразита.

«Дзен Reticulum» утверждает, что инструмент никогда не бывает нейтральным. Совсем наоборот: инструмент — это **воплощенное** намерение.

3.6.1 Принцип вреда

Почему лицензия Reticulum запрещает использование программного обеспечения в системах, предназначенных для нанесения вреда людям? Разве это не просто ограничение свободы?

Да, это ограничение *лицензии*, но это расширение *свободы*.

Создание мощных инструментов без морального компаса ни в коем случае не является добродетельным или похвальным, это просто безответственность.

Инструмент, который легко можно использовать для угнетения, представляет реальную опасность для самого пользователя. Если вы создаёте сеть, которую тиран может обратить против вас, вы не свободны. Вы просто ждете, когда узы затянутся. Встраивая «принцип невредности» в правовую основу эталонной реализации, мы создаём защитный механизм. Мы чётко и непоколебимо заявляем, что *этот инструмент* предназначен для **жизни**, а не для смерти.

Это приводит программное обеспечение в соответствие с интересами человечества. Это гарантирует, что сеть не может быть использована в качестве системы уничтожения, пульта управления боевым дроном или устройства для пыток без нарушения условий лицензии и закона. Это черта, проведенная на песке — не правительством или внешней властью, а самими создателями этого инструмента.

Рассмотрим:

- **Старый подход:** *«Это всего лишь программное обеспечение. Как люди его используют — не моя проблема».*
- **Путь дзен:** *«Это программное обеспечение — среда обитания. Я не позволю использовать его для постройки клетки».*

Выбор за *вами* — присоединиться ли к этому или нет — мы никому не навязываем эту позицию. Если вы решите присоединиться к жизни, а не к смерти, к творчеству, а не к разрушению, мы предоставим вам чрезвычайно мощный инструмент, которым вы сможете владеть и строить по своему усмотрению. Если нет, мы откажем вам в этом.

Если вам это не нравится, то вы нам здесь совершенно не нужны, и вы остаетесь наедине со своими проблемами.

3.6.2 Протокол общественного достояния

Это приводит к важнейшему различию: разнице между *идеей* и её *реализацией*.

Протокол — математические правила работы Reticulum — находится в общественном достоянии. Он принадлежит человечеству. **Никто не может владеть им.** Любой может его реализовать, улучшить или адаптировать. Это основная идея свободной коммуникации, которая сама по себе должна оставаться свободной навсегда.

Но функциональная, развернутая *эталонная реализация* — код на Python, техническое обслуживание, годы труда — имеет свою совесть. Именно это различие является двигателем устойчивости. Оно позволяет протоколу оставаться универсальным, одновременно гарантируя, что конкретный труд разработчиков не будет узурпирован с целью подрыва основополагающих целей самого проекта. Из этого документа должно быть совершенно ясно, в чём заключаются эти цели.

Если вы хотите построить систему с Reticulum, которая манипулирует и наносит ущерб пользователям ради прибыли или нацеливает ракеты, вы можете использовать протокол, находящийся в общественном достоянии, и начать с нуля. Но вы не можете взять нашу работу. Вы должны сделать свою собственную. Это служит опорой ответственности. Если вы хотите создать оружие, идите и сами выковывайте сталь, пока мир наблюдает. И когда прольется кровь — она будет на **ваших** руках.

3.6.3 Сохранение свободы воли человека

Мы живём в эпоху хищнической эксплуатации. Общедоступные ресурсы с открытым исходным кодом собираются, поглощаются и перерабатываются алгоритмами машинного обучения, владельцы которых стремятся заменить тех самых людей, кто создал эти ресурсы. Наш

код, наши слова и наше творчество используются для обучения систем, специально разработанных для того, чтобы сделать нас ненужными, не предлагая ничего другого взамен, кроме рабства и поводов.

Reticulum выступает против этого.

Лицензия защищает программное обеспечение от использования в качестве корма для чудовища. Она проводит чёткую грань: этот инструмент предназначен для *людей*. Он предназначен для связи между людьми. Это не набор данных, который можно безжалостно добывать с целью создания синтетического повелителя, управляемого крошечным конгломератом контролёров.

Это радикальный акт сохранения. Защищая код от присвоения искусственным интеллектом, мы защищаем пространство для человеческой свободы действий. Мы обеспечиваем сохранение цифрового мира, где участники — это люди из плоти, крови и души, где решения принимают умы, а не повелители, скрывающиеся за моделями.

Когда вы используете Reticulum, вы используете инструмент, который уважает вас. Он не рассматривает вас как продукт, за которым нужно следить. Он не рассматривает ваши данные как топливо для алгоритма. Он рассматривает вас как суверенного, равного партнера.

Это меняет саму основу использования технологии. Это возвращает взаимодействию его достоинство. Вы не просто пользователь сервиса; вы — участник взаимного соглашения. Инструмент способствует укреплению вашей автономии, а не подрывает её.

Таким образом, этика — это не ограничение, а основа. Это та основа, которая помогает гарантировать, что сеть и завтра будет принадлежать вам.

3.7 Шаблоны проектирования для систем пост-IP

Практическая философия для разработчиков

Философия бесполезна, если её невозможно воплотить в коде. Метафоры, которые мы рассмотрели — кочевничество, дефицит, доверие — это не просто поэзия, а реальные инженерные ограничения. Когда вы приступаете к написанию программного обеспечения для Reticulum, эти концепции должны определять саму структуру вашего приложения.

Теперь мы переходим от «почему» к «как». Именно здесь абстрактное становится конкретным, и именно здесь вы увидите истинную глубину шаблонов, которые мы выстраивали.

3.7.1 Хранение и пересылка

Интернет приучил нас к нетерпеливости. Мы пишем синхронный код. Мы отправляем запрос и ждём, блокируя интерфейс, затаив дыхание. Если ответ не поступает в течение 250 миллисекунд, мы показываем индикатор загрузки. Если он не поступает в течение пяти секунд, мы выводим сообщение об ошибке. Мы рассматриваем сетевое соединение как бинарное состояние: либо мы «онлайн», либо у нас «не работает».

Это неустойчиво. Это отрицание реальности.

В Reticulum подключение представляет собой спектр, а присутствие — асинхронно. Если это вообще применимо к вашим целям, вы должны проектировать свои приложения с учетом принципа «Store & Forward» (хранение и пересылка).

Вместо того чтобы требовать немедленного ответа, ваше приложение должно вести себя как терпеливый участник. Вы создаете сообщение для кого-то или чего-то в сети. Сеть хранит его. Она передает его от узла к узлу, возможно, в течение нескольких часов или дней, ожидая появления получателя. Когда он наконец появляется, сообщение доставляется. Это требует перехода от модели «запрос/ответ» к модели «событие/обработчик». Как именно это сделать — задача, которую вам предстоит разумно решить в рамках вашей области задач, но уже существуют системы на основе Reticulum, которые справляются с этим чрезвычайно хорошо, и вы можете использовать их в качестве источника вдохновения.

Рассмотрим:

- **Старый способ:** Connect() -> Send() -> Wait() -> С б о й п р и и с т е ч е н и и в р е м е н и о ж и д а н и я .
- **Путь дзен:** Send() -> П р о д о л ж а й т е ж и т ь . -> Receive(), к о г д а п р и д е т о т в е т .

Это кардинально меняет пользовательский опыт. Устраняется беспокойство, вызванное индикатором загрузки. Создается ощущение непрерывности. Пользователь не «ждет сети»; он взаимодействует с постоянным журналом коммуникации, который находится в самой сети.

3.7.2 Имя — это сила

В мире IP мы находимся в зависимости от системы доменных имен. Мы полагаемся на иерархию регистраторов, которая сопоставляет понятные для человека имена с адресами, понятными для машин. Эта иерархия является узким местом. Если регистратор аннулирует ваш домен или DNS-сервер выходит из строя, вы исчезаете.

Reticulum устраняет эту иерархию с помощью **идентификации на основе хешей**.

В этом шаблоне проектирования имя — это не строка, которую вы ищете; это криптографический адрес, который вы проверяете. При разработке для Reticulum вы перестаете запрашивать у пользователя URL и начинаете запрашивать адрес назначения или хеш идентификатора.

Сначала это кажется странным. Хеш вроде <83b7328926fed0d2e6a10a7671f9e237> выглядит совершенно иначе, чем myfriend.com. Но именно эта идиоматичность и является защитой. Его **невозможно** подделать. Его **невозможно** подвергнуть цензуре со стороны регистратора. Он **абсолютен**.

Разработка с учетом этого требует изменения метафор пользовательского интерфейса. Вы больше не просматриваете сеть страниц; вы управляете реестром ключей. Вы создаете «Адресную книгу», которая на самом деле представляет собой связку ключей. Имена присваиваются пользователем, и власть остается за ним. То, что хэши выглядят сложными, напрямую соответствует прочности связей, формируемых при их использовании. Это заставляет пользователя выполнить проверку — внеполосное рукопожатие, которое восстанавливает человеческий элемент доверия, лишенный SSL-сертификатами.

3.7.3 Интерфейс — это среда

Одним из наиболее освобождающих шаблонов в Reticulum является **транспортная независимость**.

В традиционных сетях ваш код часто переполнен транспортной логикой. «Я подключен к Wi-Fi? Проверить пропускную способность. Я подключен к сотовой сети? Проверить тарифный план. Я подключен к Ethernet?». Вы постоянно занимаетесь микроуправлением каналом.

В Reticulum вы обращаетесь к API, а API записывает данные на носитель. Вы отправляете пакет в точку назначения. Вам не важно, проходит ли этот пакет через TCP-туннель, по радиоволнам LoRa или через последовательный интерфейс. Это задача стека.

Это позволяет создавать **универсальные приложения**. Представьте себе приложение для обмена сообщениями. Вы пишете его один раз. Оно работает на ноутбуке, подключенном к оптоволоконному интернету. Оно работает на телефоне в городе через Wi-Fi. И, без изменения ни одной строчки кода, оно работает на устройстве в глуши, обмениваясь данными с другими устройствами только по радио.

Принцип прост: **никогда не программируйте под аппаратное обеспечение**.

Программируйте под задачу. Рассмотрим:

- **Старый способ:** `socket.connect(ip, port)`, а затем еще много всего
- **Способ Zen:** `RNS.Packet(destination, data).send()`

Обеспечив абстракцию среды, вы делаете свое программное обеспечение независимым от изменений в инфраструктуре. Завтра пользователь может переключиться с точки доступа 4G на модем HF. Вашему программному обеспечению не нужно об этом знать. Оно просто продолжает работу.

3.7.4 Возникающие паттерны

Когда вы объединяете эти паттерны — «Хранение и пересылка», «Идентификация на основе хеша» и «Транспортная независимость» — вы создаете программное обеспечение, которое ощущается принципиально иначе.

Оно кажется *надежным*. Оно не мерцает, когда пропадает сигнал. Оно не паникует, когда сервер не работает. Оно имеет вес. Оно имеет стойкость. Оно имеет *актуальность*.

Вы больше не создаете «клиент», который вынужден просить «сервер» о внимании. Вы создаете автономного агента, существующего в рамках ячеистой сети. Он говорит, когда это необходимо, слушает, когда может, и везде с собой несет свою идентичность.

Это кульминация Дзен. Код — это не просто набор инструкций: это оболочка поведения. Это способ *существования* в сети.

3.8 Структура независимости

Мы развеяли иллюзии. Мы поняли, что центр пуст, что доверие требует упорной работы, что ресурсы ограничены и что мы должны владеть своей инфраструктурой. Мы поняли, что инструменты имеют свою этику и что наша идентичность может меняться.

Это возвращение общего достоинства. Слишком долго мы позволяли, чтобы самая жизненно важная основа человеческого общества — *наша способность общаться друг с другом* — была колонизирована организациями, которые не разделяют наших интересов. Мы позволили, чтобы архитектура нашей коммуникации проектировалась бухгалтерами, а не архитекторами.

Мы забираем её обратно. Не обращаясь с просьбами к хозяевам, а строя новый мир внутри, над, под и вокруг оболочки старого.

3.8.1 Работа завершена

Самое тяжелое уже позади.

Протокол находится в общественном достоянии — это дар человечеству, который никто не сможет у него отнять. Программное обеспечение написано, протестировано и работает на устройствах, разбросанных по всему миру. Руководство лежит перед вами. Исходный код эталонной реализации сейчас распространен на сотнях тысяч устройств по всей планете. Никто не может его удалить или уничтожить. Аппаратное обеспечение доступно и широко распространено.

Путь сюда был нелегким, но мы дошли. Теперь нет комитета по разработке дорожной карты, ожидающего одобрения. Нет венчурного капитала, диктующего пользовательский интерфейс. Нет генерального директора, который должен утвердить выпуск следующей функции.

Есть только вы.

Барьер для входа больше не заключается в сложности: это просто привычка к зависимости. Вас приучили ждать. Ждать обновления приложения. Ждать, пока интернет-провайдер починит линию. Ждать, пока платформа разрешит публикацию. Ждать, пока правительство изменит политику. Ждать лайков. Ждать, пока революцию покажут по телевизору.

Революция так и не была показана по

телевидению. Она передается пакетами.

3.8.2 «Открытое небо»

Будущее этой технологии — строительный проект.

Это похоже на одиночный узел на подоконнике, прислушивающийся к помехам. Это похоже на сообщение, отправленное соседу, минуя шум коммерческого Интернета. Это похоже на сеть сообщества, которая растет, звено за звеном, прыжок за прыжком, поддерживаемая руками тех, кому важнее связь, чем прибыль.

У вас есть чертежи. У вас есть инструменты. У вас есть философия. Шум старого мира улетучился, оставив вам тихую ясность открытого спектра.

Марк, начало 2026 года

ПРОГРАММЫ, ИСПОЛЬЗУЮЩИЕ RETICULUM

В этой главе приведен неполный список известных программ, систем и протоколов прикладного уровня, созданных с использованием Reticulum.

Эти программы позволяют вам понять, как работает Reticulum. Большинство из них разработаны так, чтобы хорошо работать даже в медленных сетях на основе LoRa или пакетной радиосвязи, но все они также могут использоваться по быстрым каналам, таким как локальный Wi-Fi, проводной Ethernet, Интернет или любая их комбинация.

Таким образом, легко начать экспериментировать, не настраивая никаких радиоприемников или инфраструктуры только для того, чтобы попробовать. Для начала достаточно запустить программы на отдельных устройствах, подключенных к одной сети Wi-Fi, а физические радиоинтерфейсы можно будет добавить позже.

4.1 Программы и утилиты

Уже существует множество различных приложений, использующих Reticulum, которые служат самым разным целям: от повседневного общения и обмена информацией до системного администрирования и решения сложных задач в области сетей и связи.

Разработка приложений и систем на основе Reticulum продолжается, поэтому рассматривайте этот список как неполный перечень *некоторых* из доступных вариантов. Проведя небольшой поиск, в первую очередь по самому Reticulum, вы найдёте ещё много интересного.

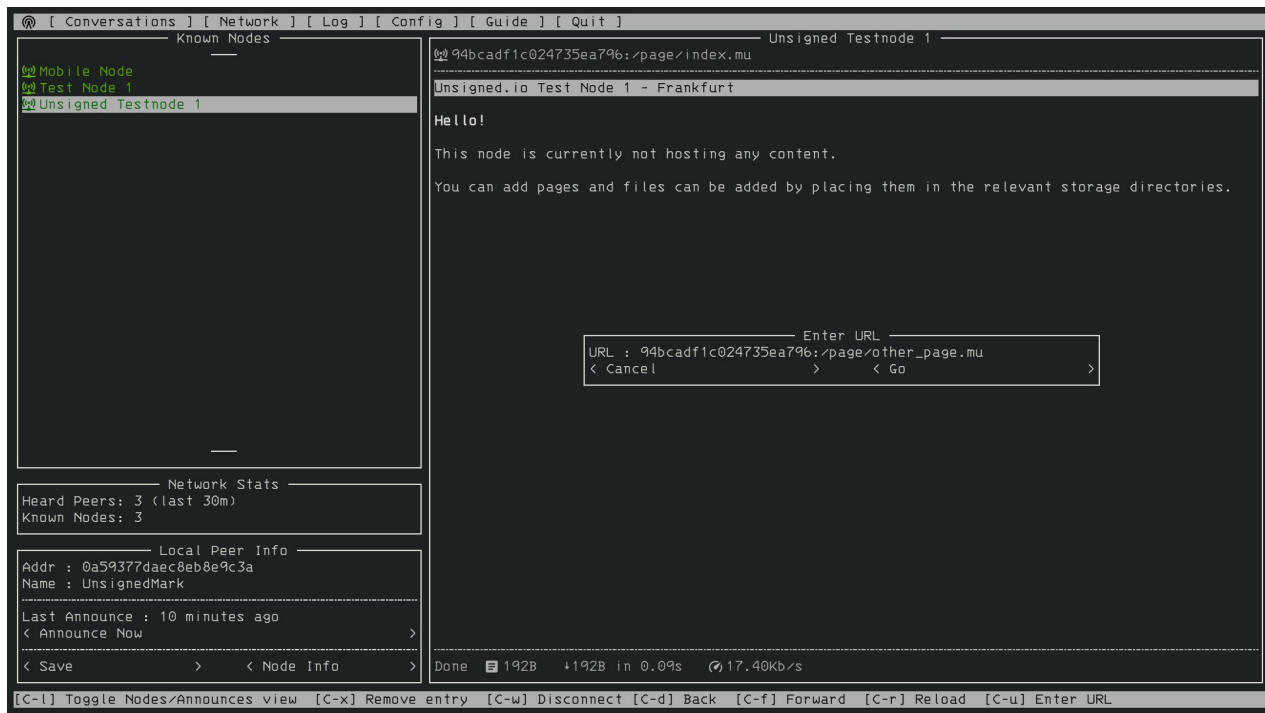
4.1.1 Удаленная оболочка

Программа `rnsh` позволяет устанавливать полностью интерактивные удалённые сеансы оболочки через Reticulum. Она также позволяет передавать данные любой программы в удалённую систему или из неё, что аналогично принципу работы `ssh`. Программа `rnsh` отличается высокой эффективностью и способна обеспечивать полностью интерактивные сеансы оболочки даже при использовании каналов связи с крайне низкой пропускной способностью, таких как LoRa или пакетная радиосвязь.

Помимо стандартного, полностью интерактивного терминального режима, для каналов связи с крайне низкой пропускной способностью `rnsh` предлагает линейно-интерактивный режим, позволяющий взаимодействовать с удалёнными системами даже при пропускной способности канала, исчисляемой несколькими сотнями бит в секунду.

4.1.2 Nomad Network

Терминальная программа [Nomad Network](#) представляет собой полный набор средств зашифрованной связи, построенный на основе Reticulum. Она включает в себя обмен зашифрованными сообщениями (как прямой, так и с отложенной доставкой для автономных пользователей), обмен файлами, а также имеет встроенный текстовый браузер и сервер страниц с поддержкой динамически формируемых страниц, аутентификации пользователей и многого другого.



[Nomad Network](#) — это клиентский интерфейс для пользователей, предназначенный для работы с протоколом обмена сообщениями и информацией LXMf.

4.1.3 Узел страниц RNS

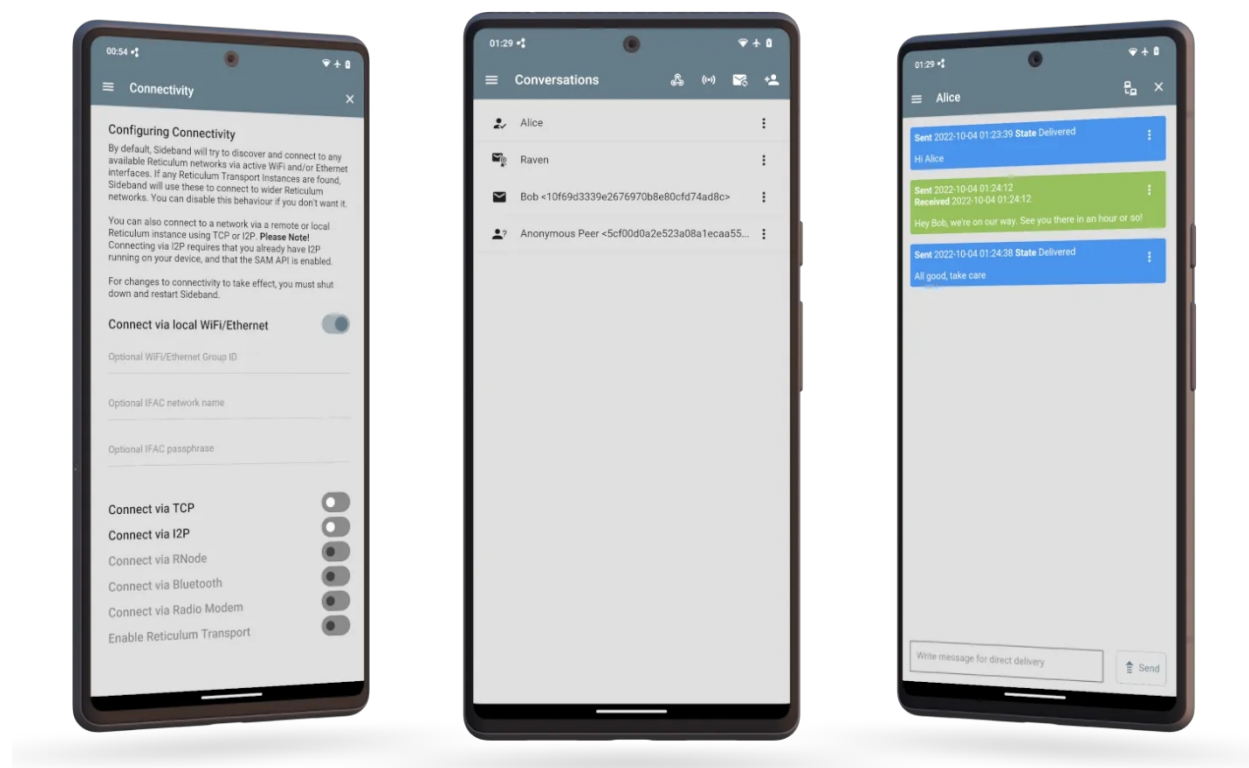
[RNS Page Node](#) — это простой способ предоставлять страницы и файлы любому другому клиенту, совместимому с [Nomad Network](#). Готовая замена узлов [NomadNet](#), которые в основном обслуживают страницы и файлы.

4.1.4 Retipedia

С помощью [Retipedia](#) вы можете размещать всю Википедию (или любой файл .zim) для других клиентов [Nomad Network](#).

4.1.5 Sideband

Если вы предпочитаете использовать LXMФ-клиент с графическим интерфейсом, обратите внимание на [Sideband](#), доступный для Android, Linux, macOS и Windows. Sideband — это продвинутый LXMФ- и LXST-клиент, а также многофункциональная утилита Reticulum с возможностями, ориентированными на опытных пользователей.

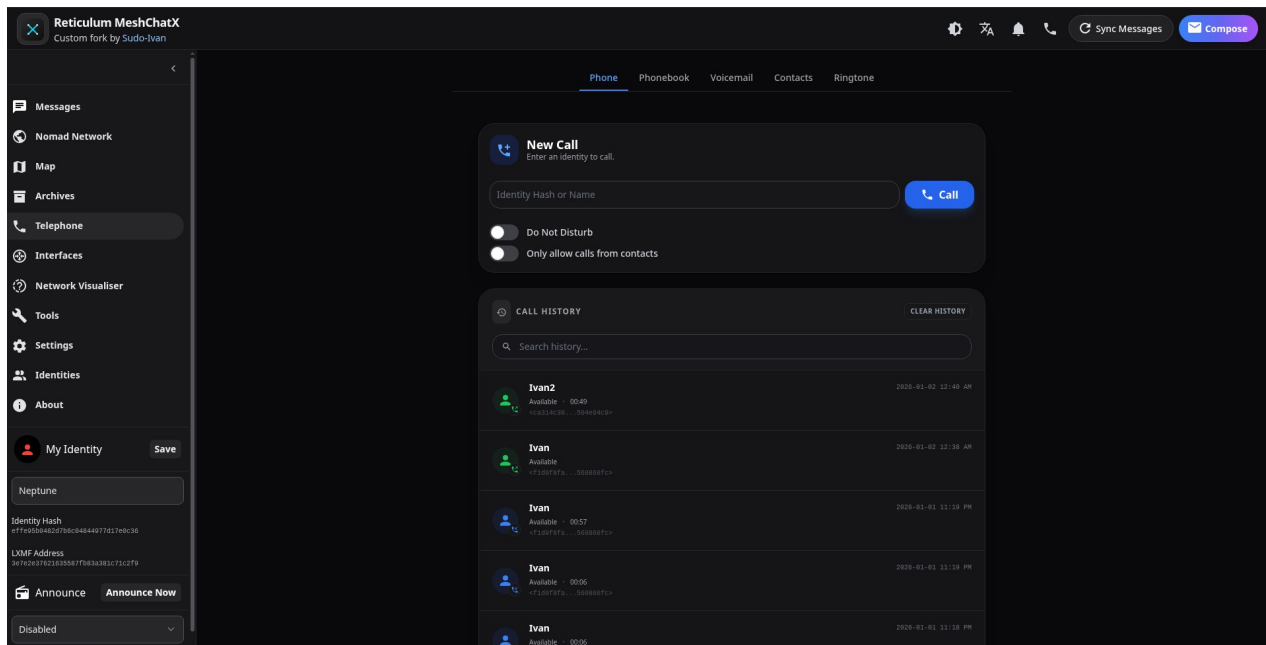


Sideband позволяет вам общаться с другими людьми или системами, совместимыми с LXMФ, через сети Reticulum, используя LoRa, Packet Radio, WiFi, I2P, зашифрованные бумажные сообщения с QR-кодами или что-либо еще, что поддерживает Reticulum.

Кроме того, он совместим со всеми другими клиентами LXMФ и предоставляет такие расширенные функции, как голосовые сообщения, голосовые звонки в режиме реального времени, вложение файлов, обмен частными телеметрическими данными, а также полноценную систему плагинов для расширения функциональности.

4.1.6 MeshChatX

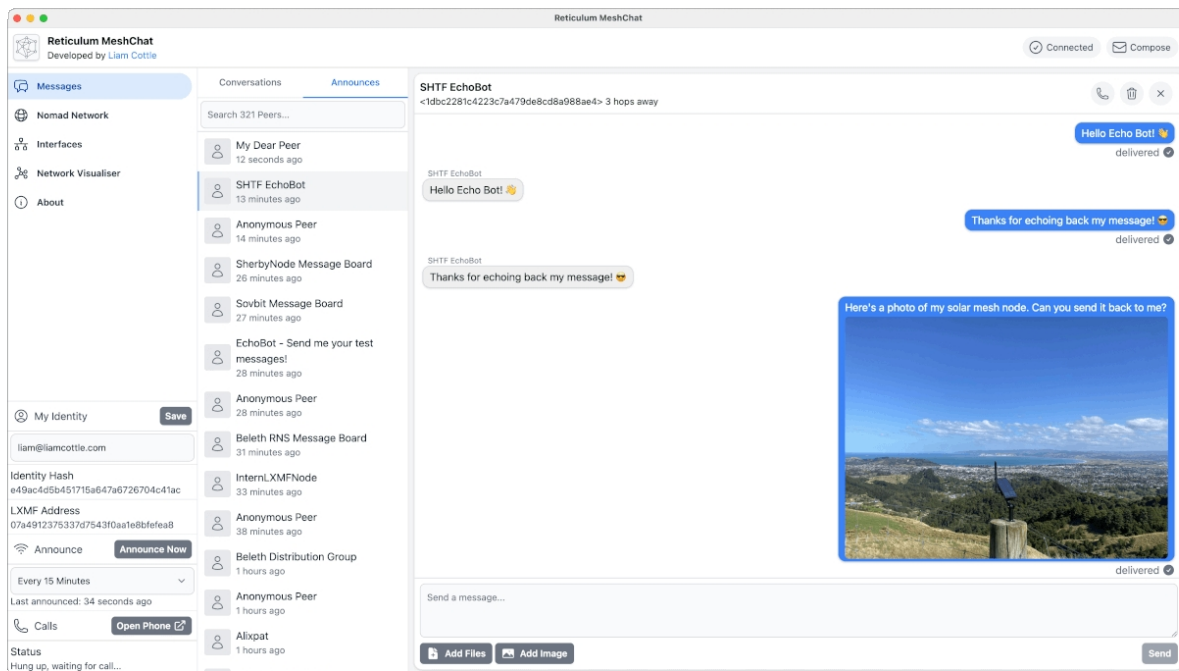
Форк Reticulum MeshChat из будущего, цель которого — предоставить все необходимое для Reticulum, LXMF и LXST в одном красивом и многофункциональном приложении. Этот проект отделен от оригинального проекта Reticulum MeshChat и не связан с ним.



Функции включают полную поддержку LXST, настраиваемую голосовую почту, телефонную книгу, обмен контактами и поддержку мелодий звонка, обработку нескольких идентичностей, современный UI/UX, офлайн-документацию, расширенные инструменты, архивирование страниц, интегрированные карты и улучшенную безопасность приложения.

4.1.7 MeshChat

Приложение **Reticulum MeshChat** — это удобный клиент LXMf для Linux, macOS и Windows, который также включает в себя браузер страниц Nomad Network и другие интересные функции.



Reticulum MeshChat, конечно же, также совместим с Sideband и Nomad Network, а также с любым другим клиентом LXMf.

4.1.8 Columba

Columba — это простое и удобное приложение для обмена сообщениями на базе LXMf для Android, созданное с использованием нативного интерфейса Android и стиля Material Design 3.



Несмотря на то, что приложение все еще находится на ранней стадии разработки и активно совершенствуется, оно, конечно же, совместимо со всеми другими клиентами LXMf и позволяет без проблем обмениваться сообщениями с любыми другими пользователями LXMf.

4.1.9 Reticulum Relay Chat

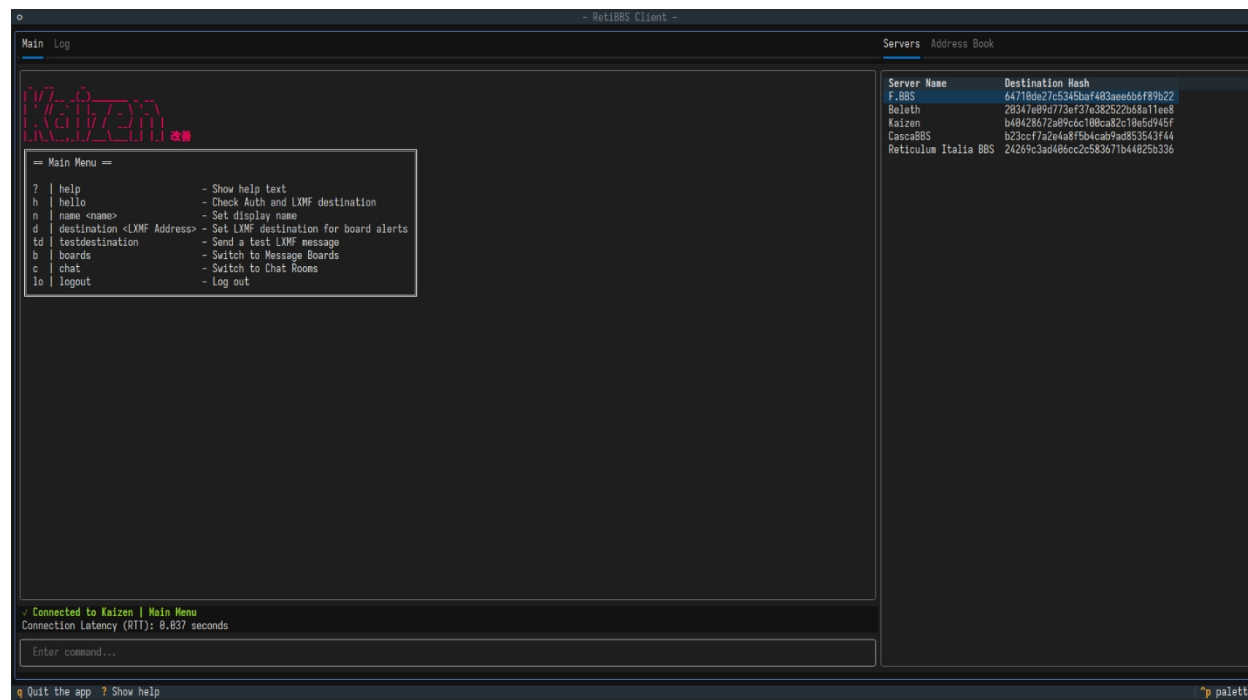
Reticulum Relay Chat — это система живого чата, построенная на базе сетевого стека Reticulum. Она существует для того, чтобы люди могли общаться друг с другом в реальном времени через Reticulum, не затыкаясь в базы данных сообщений, механизмы синхронизации или архитектурные обязательства, о которых они не просили.

Программа `rrcd` представляет собой функциональную эталонную реализацию демон-сервера RRC. Клиентами RRC являются `rrc-gui` и `rrc-web`.

RRC по духу ближе к IRC, чем к современным «универсальным платформам». Вы подключаетесь, входите в комнату, общаетесь, а затем уходите. Если вы были присутствовали, вы видели разговор. Если вас не было, разговор не ждал вас. Это не случайность. Это и есть весь замысел.

4.1.10 RetiBBS

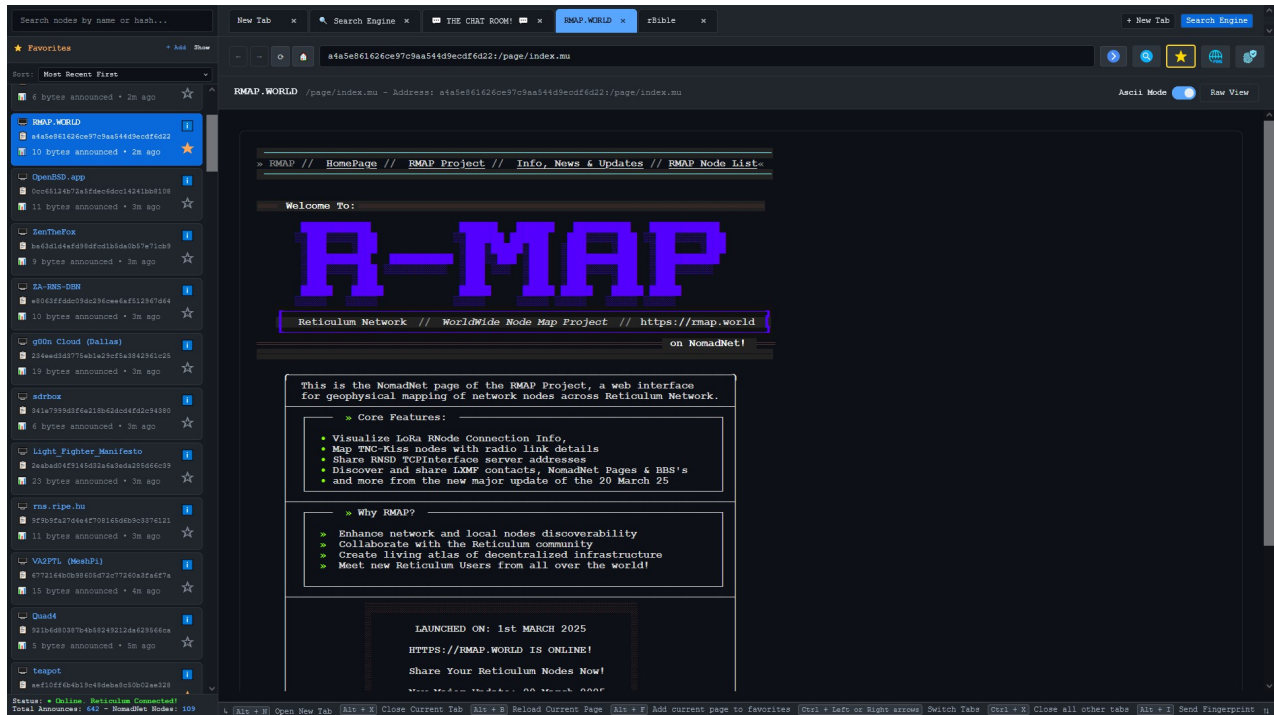
RetiBBS — это реализация системы досок объявлений для сетей Reticulum.



RetiBBS позволяет пользователям безопасно общаться через доски объявлений.

4.1.11 RBrowser

Программа **rBrowser** — это кроссплатформенный, автономный веб-браузер для исследования узлов NomadNetwork через сеть Reticulum. Он автоматически обнаруживает узлы NomadNet через сетевые объявления и предоставляет удобный интерфейс для просмотра распределенного контента с поддержкой разметки Micron.



Включает в себя такие полезные функции, как автоматический прослушиватель объявлений, добавление узлов в избранное, просмотр и отображение любых ссылок NomadNet, загрузка файлов с удаленных узлов, уникальную локальную поисковую систему NomadNet и многое другое.

4.1.12 Телефон Reticulum Network

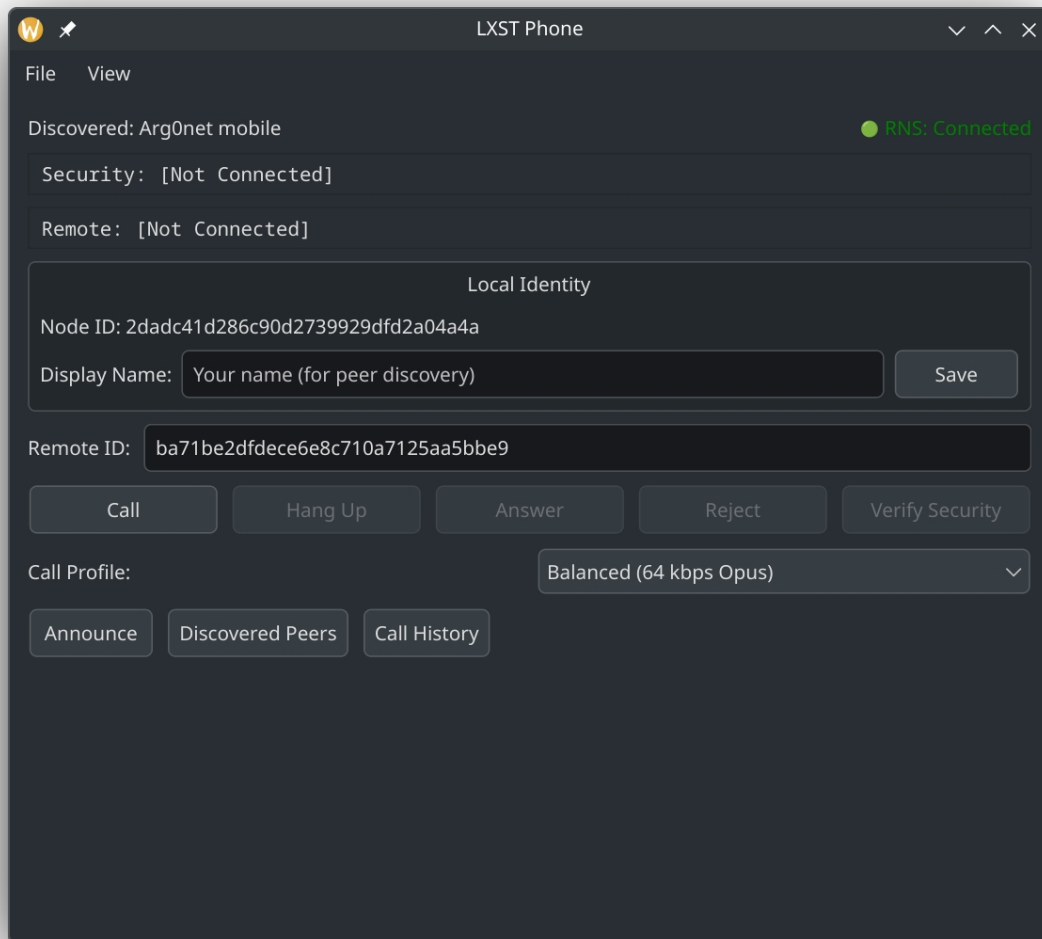
Программа `gprhone`, входящая в состав пакета `LXST`, представляет собой утилиту и демон Reticulum для командной строки, позволяющий создавать физические аппаратные телефоны для LXST и Reticulum, а также просто выполнять вызов из командной строки.



Он поддерживает прямое взаимодействие с аппаратными периферийными устройствами, такими как клавиатуры GPIO и ЖК-дисплеи, предоставляя модульную систему для создания безопасных аппаратных телефонов.

4.1.13 LXST Phone

Программа **LXST Phone** — это кроссплатформенное настольное приложение для осуществления голосовых вызовов LXST через Reticulum.



Она поддерживает различные расширенные функции, такие как проверка SAS, блокировка одноранговых узлов, ограничение скорости, хранение истории звонков в зашифрованном виде и управление контактами.

4.1.14 LXMfy

LXMfy — это комплексная и продвинутая среда для создания ботов для LXMf, позволяющая создавать любые системы автоматизации или ботов, работающие на LXMf и Reticulum. Существуют реализации ботов для управления Home Assistant, интеграции с LLM и различных других целей.

4.1.15 LXMf Interactive Client

LXMf Interactive Client — это многофункциональный терминальный клиент для обмена сообщениями LXMf, обладающий множеством расширенных возможностей и расширяемой архитектурой плагинов.

4.1.16 RNS FileSync

Программа RNS FileSync позволяет автоматически синхронизировать файлы между устройствами без использования центральных серверов, подключения к Интернету или облачных сервисов. Она работает через любые сетевые среды, поддерживаемые Reticulum, включая радио, LoRa, Wi-Fi или Интернет, что делает ее идеальным решением для автономного, ориентированного на конфиденциальность и отказоустойчивого обмена файлами.

4.1.17 Micron Parser JS

Micron Parser JS — это парсер на основе JavaScript для языка разметки Micron, который используется большинством веб-браузеров Nomad Network. Если вы хотите создавать утилиты или инструменты, отображающие страницы Micron, эта библиотека вам просто необходима.

4.1.18 RNMon

RNMon — это мониторинговый демон, предназначенный для отслеживания состояния нескольких приложений RNS и передачи метрик в экземпляр InfluxDB по протоколу InfluxLine.

4.2 Протоколы

Ряд стандартных протоколов появился в результате реального использования и тестирования в сообществе Reticulum. Хотя при написании программного обеспечения на основе Reticulum иногда может возникнуть необходимость в использовании полностью настраиваемых протоколов и реализаций, использование этих протоколов предоставляет разработчикам приложений простой способ быстро и без особых усилий реализовать расширенные функции. Их использование также обеспечивает совместимость и взаимодействие между множеством различных клиентских приложений, создавая открытую экосистему коммуникаций, в которой пользователи могут свободно выбирать приложения, отвечающие их потребностям, оставаясь при этом на связи со всеми остальными.

4.2.1 LXMF

LXMF — это простой и гибкий формат обмена сообщениями и протокол доставки, который поддерживает широкий спектр приложений, потребляя при этом минимальную пропускную способность. Он обеспечивает маршрутизацию сообщений без настройки, сквозное шифрование и секретность пересылки, а также может передаваться по любому типу среды, поддерживаемому Reticulum.

LXMF достаточно эффективен, чтобы доставлять сообщения через системы с крайне низкой пропускной способностью, такие как пакетная радиосвязь или LoRa. Зашифрованные сообщения LXMF также могут быть закодированы в виде QR-кодов или текстовых URI, что позволяет осуществлять полностью аналоговую передачу сообщений на бумаге.

Используя узлы распространения, LXMF также предлагает способ хранения и пересылки сообщений пользователям или конечным точкам, которые на момент отправки сообщения недоступны напрямую.

4.2.2 LXST

LXST — это простой и гибкий формат потоковой передачи данных в реальном времени и протокол доставки, который поддерживает широкий спектр приложений, потребляя при этом минимальную пропускную способность. Он построен на базе Reticulum и обеспечивает маршрутизацию потоков без настройки, сквозное шифрование и конфиденциальность перехода, а также может передаваться по любой среде, поддерживаемой Reticulum. В настоящее время он используется в приложениях для голосовой связи и телефонии в реальном времени на базе Reticulum.

4.2.3 RRC

Протокол **Reticulum Relay Chat** — это система чата, построенная на базе сетевого стека Reticulum. Она предназначена для обеспечения групповой коммуникации практически в реальном времени без использования баз данных истории сообщений, механизмов федерации или сложных архитектурных решений.

RRC специально разработан как простой инструмент. Он не претендует на то, чтобы быть электронной почтой, почтовым ящиком или распределённым архивом. Он скорее напоминает разговор в комнате: если вы были там, вы его слышали; если нет — то нет. Это не ошибка, а суть проекта.

4.3 Модули интерфейса и ресурсы для подключения

В этом разделе представлен список различных модулей интерфейса, руководств и ресурсов, предоставленных сообществом, для создания сетей Retic-ulum в специальных или сложных средах.

- Пользовательский интерфейсный модуль для запуска RNS по HTTP
- Руководство по запуску Reticulum через ICMP с использованием PipeInterface
- Руководство по запуску Reticulum через DNS с помощью Iodine
- Руководство по запуску Reticulum по HF-радио
- **Modem73** — это интерфейс модема KISS TNC OFDM, который можно использовать с Reticulum

ИСПОЛЬЗОВАНИЕ RETICULUM В ВАШЕЙ СИСТЕМЕ

Reticulum не устанавливается как драйвер или модуль ядра, как можно было бы ожидать от сетевого стека. Вместо этого Reticulum распространяется в виде модуля Python, содержащего сетевое ядро и набор утилит и фоновых программ.

Это означает, что для его установки или использования не требуются специальные привилегии. Кроме того, он очень лёгкий, его легко переносить и устанавливать на новые системы.

Если у вас установлен Reticulum, любая программа или приложение, использующее Reticulum, автоматически загрузит и инициализирует Reticulum при запуске, если он ещё не работает.

Во многих случаях этого подхода достаточно. Когда какой-либо программе требуется использовать Reticulum, он загружается, инициализируется, запускаются интерфейсы, и программа может теперь обмениваться данными через любые доступные сети Reticulum. Если запускается другая программа, которой также требуется доступ к той же сети Reticulum, просто используется уже запущенный экземпляр. Это работает для любого количества программ, запущенных одновременно, и очень просто в использовании, но в зависимости от вашего сценария использования существуют и другие варианты.

5.1 Конфигурация и данные

Reticulum хранит всю информацию, необходимую для работы, в одном каталоге файловой системы. При запуске Reticulum ищет каталог с конфигурацией в следующих местах:

- /etc/reticulum
- ~/.config/reticulum
- ~/.reticulum

Если каталог конфигурации не найден, создается каталог ~/.reticulum, и в него автоматически помещается конфигурация по умолчанию. При желании его можно переместить в одно из других мест.

Также можно использовать совершенно произвольные каталоги конфигурации, указав соответствующие параметры командной строки при запуске программ на основе Reticulum. Вы также можете запускать несколько отдельных экземпляров Reticulum на одной физической системе, либо изолированно друг от друга, либо соединенными между собой.

В большинстве случаев на одной физической системе потребуется запускать только один экземпляр Reticulum. Его можно запускать при загрузке системы в качестве системной службы или просто запускать, когда это требуется программе. В любом случае любое количество программ, запущенных на одной системе, будет автоматически использовать один и тот же экземпляр Reticulum, если это допускает конфигурация, что и происходит по умолчанию.

Вся конфигурация Reticulum находится в файле ~/.reticulum/config. При первом запуске Reticulum на новой системе создается базовый, но полностью функциональный файл конфигурации. Конфигурация по умолчанию выглядит следующим образом:

```
# Это файл конфигурации Reticulum по умолчанию.  
# Возможно, вам следует отредактировать этот файл, чтобы  
# добавить в него # любые дополнительные интерфейсы и настройки,  
# которые могут вам понадобиться.
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

В эту конфигурацию по умолчанию включены только самые базовые # параметры. Чтобы увидеть более подробную и гораздо более длинную # Пример конфигурации, можно выполнить команду: # rnsd --exampleconfig

[reticulum]

Если включить Transport, ваша система будет маршрутизировать трафик # для других пиров, передавать объявления и обслуживать запросы на маршруты. # Это следует делать для систем, которые подходят для работы # в качестве транспортных узлов, т. е. если они стационарны и # всегда включены. Эта директива является опциональной и может быть удалена # для краткости.

enable_transport = No

По умолчанию первая запущенная программа создает # сетевого стека создаст общий экземпляр, с которым смогут взаимодействовать другие # программы. Только общий экземпляр # напрямую открывает все настроенные интерфейсы, а другие # локальные программы взаимодействуют с общим экземпляром через # локальный сокет. Это полностью прозрачно для # пользователю и, как правило, должна быть включена. Эта директива # является опциональной и может быть удалена для краткости.

share_instance = Да

*# Если вы хотите запустить несколько *разных* общих экземпляров # в одной системе, вам нужно будет указать разные # имена экземпляров для каждого. На платформах, поддерживающих доменные # сокеты, это можно сделать с помощью опции instance_name:*

instance_name = default

Некоторые платформы не поддерживают доменные сокеты, и в таком # случае вы можете изолировать разные экземпляры, # указав уникальный набор портов для каждого:

shared_instance_port = 37428 #
instance_control_port = 37429

Если вы хотите явно использовать TCP для связи между # общими экземплярами вместо доменных сокетов, это также # возможно с помощью следующей опции:

shared_instance_type = tcp

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

В системах, где запущенные экземпляры могут не иметь доступа # к одному и тому же общему каталогу конфигурации Reticulum, # все равно можно обеспечить полную интерактивность для запущенных экземпляров, вручную указав общий ключ RPC

#. Практически во всех случаях эта опция не требуется, но # она может быть полезна в таких операционных системах, как Android. # Ключ должен быть указан в виде байтов в шестнадцатеричном формате.

```
# rpc_key = e5c032d3ec4e64abaca9927ba8ab73336780f6d71790
```

Можно разрешить удаленное управление системами Reticulum # с помощью различных встроенных утилит, таких как # rstatus и rpath. Вам нужно будет указать одну или

Дополнительные хэши идентификаторов Reticulum для аутентификации # запросов от клиентских программ. Для этой цели можно # использовать существующие файлы идентификаторов или сгенерировать новые с помощью # утилиты ruid.

```
# enable_remote_management = yes
```

```
# remote_management_allowed = 9fb6d773498fb3feda407ed8ef2c3229, ↵
↳ 2d882c5586e548d79b5af27bca1776dc
```

Можно настроить Reticulum так, чтобы он выводил сообщение о сбое и принудительно закрывался # в случае возникновения неустранимой ошибки интерфейса, например, # при исчезновении аппаратного устройства интерфейса. Это # необязательная директива, которую можно опустить для краткости. # По умолчанию это поведение отключено.

```
# panic_on_interface_error = No
```

Когда Transport включен, можно разрешить # экземпляру Transport отвечать на запросы проб от

утилиты rprobe. Это может быть полезным инструментом для проверки # подключения. Если эта опция включена, # будет сгенерирован на основе идентификатора # экземпляра транспорта и выведен в журнал при запуске. # Необязательный параметр, по умолчанию отключен.

```
# respond_to_probes = No
```

[logging]

Допустимые уровни журнала: от 0 до 7:

0: Записывать в журнал только критическую информацию

1: Регистрировать ошибки и события более низких уровней

2: Запись предупреждений и более низких уровней журнала # 3: Регистрация уведомлений и более низких уровней журнала

4: Регистрировать информацию и ниже (это

значение по умолчанию) # 5: Подробная регистрация

(продолжение на следующей странице)

```
# 6: Ведение журнала
отладки
# 7: Экстремальное
ведение журнала

loglevel = 4

# Раздел interfaces определяет физические и виртуальные #
интерфейсы, которые Reticulum будет использовать для связи. Это
# В этом разделе приведены примеры для различных типов интерфейсов.
Вы можете изменить их или использовать в качестве основы для
# вашей собственной конфигурации, либо просто удалить неиспользуемые.

[interfaces]

# Этот интерфейс обеспечивает связь с другими #
локальными узлами Reticulum по протоколу UDP. Для его
работы
# не требует никакой функциональной IP-инфраструктуры, такой как
маршрутизаторы
# или DHCP-серверов, но для этого необходимо, чтобы в вашей
операционной системе был включен как минимум локальный IPv6-
адрес, # который по умолчанию должен быть включен практически
в любой ОС. Дополнительные настройки см.
# руководство по Reticulum для получения дополнительных сведений о
параметрах настройки.

[[Интерфейс по умолчанию]] type
= AutoInterface interface_enabled
= True
```

Если инфраструктура Reticulum уже существует локально, вам, вероятно, не нужно ничего менять, и вы, возможно, уже подключены к более широкой сети. Если нет, вам, вероятно, потребуется добавить в конфигурацию соответствующие *интерфейсы* для связи с другими системами.

Вы можете сгенерировать гораздо более подробный пример конфигурации, выполнив команду:

```
rnsd --exampleconfig
```

Вывод содержит примеры для большинства типов интерфейсов, поддерживаемых Reticulum, а также дополнительные опции и параметры конфигурации.

Рекомендуется ознакомиться с комментариями и пояснениями в приведенном выше файле конфигурации по умолчанию. Это поможет вам освоить основные понятия, необходимые для настройки сети. После этого ознакомьтесь с главой «*Интерфейсы*» данного руководства.

5.2 Включенные утилиты

Reticulum включает в себя ряд полезных утилит, предназначенных как для управления сетями Reticulum, так и для выполнения типичных задач в сетях Reticulum, таких как передача файлов на удаленные системы и удаленное выполнение команд и программ.

Если вы часто используете Reticulum из нескольких разных программ или просто хотите, чтобы Reticulum был доступен постоянно (например, если вы являетесь хостом транспортного узла), вам может понадобиться запустить Reticulum в качестве отдельного сервиса, которым смогут пользоваться другие программы, приложения и сервисы.

5.2.1 Утилита rnsd

Запустить Reticulum в качестве службы очень просто. Просто запустите входящую в комплект команду rnsd. Когда rnsd работает, он будет держать открытыми все настроенные интерфейсы, обрабатывать транспорт, если он включен, и позволять любым другим программам немедленно

использовать сеть Reticulum, для которой он настроен.

Вы даже можете запустить несколько экземпляров rnsd с разными настройками в одной системе.

Примеры использования

Запустите rnsd:

```
$ rnsd
[2023-08-18 17:59:56] [Уведомление] Запущен rnsd версии 0.5.8
```

Запустите rnsd в режиме службы, обеспечив отправку всего вывода журнала непосредственно в файл:

```
$ rnsd -s
```

Создать подробный пример конфигурации с объяснениями всех различных параметров настройки и примерами настройки интерфейсов:

```
$ rnsd --exampleconfig
```

Все параметры командной строки

использование: rnsd.py [-h] [--config CONFIG] [-v] [-q] [-s] [--exampleconfig] [--version]

Демон сетевого стека Reticulum

опции:

-h, --help	показать это сообщение справки и выйти
--config CONFIG	путь к альтернативному каталогу конфигурации Reticulum
-v, --verbose	
-q, --quiet	
-s, --service	rnsd работает как служба и должен вести журнал в файл
-i, --interactive	перейти в интерактивную оболочку после инициализации
--exampleconfig	вывести подробный пример конфигурации в stdout и завершить работу
--version	показать номер версии программы и завершить работу

Вы можете легко добавить rnsd в качестве постоянно работающей службы, *настроив соответствующую службу*.

5.2.2 Утилита rnstatus

С помощью утилиты rnstatus можно просматривать состояние настроенных интерфейсов Reticulum, аналогично программе ifconfig.

Примеры использования

Запустите rnstatus:

(продолжение на следующей странице)

```
$ rnstatus
Совместный экземпляр [37428]
  Статус      : Работает
  Обслуживает : 1
  программу
```

(продолжение с предыдущей страницы)

```

    Скоро      : 1,00 Гбит/с
    Тр а ф и к : 83,13 КБ↑
                86,10 КБ↓

AutoInterface[Local]
    Статус      : Активен
    Режим       : Полный
    С к о р о с т ь      :
    10,00 М б и т / с
    У з л ы : 1
    д о с т у п е н
    Т р а ф и к : 63,23 КБ↑
                80,17 КБ↓

TCPInterface[RNS Testnet Dublin/dublin.connect.reticulum.network:4965] Статус      : Активен
    Режим       : Полный
    С к о р о с т ь      :
    10,00 М б и т / с
    Т р а ф и к : 187,27 КБ↑
                74,17 КБ↓

RNodeInterface[RNode UHF] Статус
    : Активен
    Режим       : Скорость точки
    доступа    : 1,30 кбит/с
    Д о с т у п : 64-р а з р я д н а я в е р с и я
    IFAC о т <...e702c42ba8> Т р а ф и к : 8,49 КБ↑
                9,23 КБ↓

Экземпляр Reticulum Transport <5245a8efe1788c6a1cd36144a270e13b> работает

```

Отфильтруйте вывод, чтобы отобразить только некоторые интерфейсы:

```

$ rnstatus rnode

RNodeInterface[RNode UHF] Статус
    : Включен
    Режим       : Точка доступа
    Скорость    : 1,30 кбит/с
    Д о с т у п : 64-б и т н а я IFAC о т
    <...e702c42ba8> Т р а ф и к : 8,49 КБ↑
                9,23 КБ↓

Экземпляр транспорта Reticulum <5245a8efe1788c6a1cd36144a270e13b> работает

```

Все параметры командной строки

```

Использование: rnstatus [-h] [--config CONFIG] [--version] [-a] [-A]
                    [-l] [-t] [-s SORT] [-r] [-j] [-R hash] [-i path]
                    [-w секунды] [-d] [-D] [-m] [-I секунды] [-v] [фильтр]

```

Позиционные аргументы состояния сетевого

стека Reticulum:

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

Фильтр	отображать только интерфейсы, названия которых содержат filter
Р	
опции:	
-h, --help	показать это сообщение справки и выйти
--config CONFIG	путь к альтернативному каталогу конфигурации Reticulum
--version	показать номер версии программы и завершить работу
-a, --all	показать все интерфейсы
-A, --announce-stats	показать статистику анонсов
-l, --link-stats	показать статистику каналов
-t, --totals	отобразить итоговые данные о трафике
-s, --sort СОРТИРОВКА	отсортировать интерфейсы по [скорости, трафику, rx, tx, rxs, txs, announces, arx, atx, held]
-r, --reverse	обратная сортировка
-j, --json	вывод в формате JSON
-R hash	транспортный идентификационный хеш удаленного экземпляра, от которого нужно получить статус
-i путь	путь к идентификатору, используемому для удаленного управления
-w секунд	таймаут перед отказом от удаленных запросов
-d, --discovered	список обнаруженных интерфейсов
-D	показать подробности и записи конфигурации для обнаруженных интерфейсов
-m, --monitor	непрерывно отслеживать состояние
-I, --monitor-interval секунд	интервал обновления для режима мониторинга (по умолчанию: 1)
-v, --verbose	

Примечание

При использовании параметра -R для запроса к удаленному экземпляру транспортного протокола необходимо также указать параметр -i с путем к файлу идентификационных данных, имеющему права на удаленное управление в целевой системе.

5.2.3 Утилита rnid

С помощью утилиты rnid можно создавать, управлять и просматривать идентификаторы Reticulum. Программа также может вычислять хэши назначения, а также выполнять шифрование и дешифрование файлов.

С помощью rnid можно осуществлять асимметричное шифрование файлов и данных для любого целевого хеша Reticulum, а также создавать и проверять криптографические подписи.

Примеры использования

Создание новой идентичности:

```
$ rnid -g ./new_identity
```

Отображение информации о ключе идентификатора:

```
$ rnid -i ./new_identity -p
```

Загружена идентичность <984b74a3f768bef236af4371e6f248cd> из new_id

Открытый ключ : 0f4259fef4521ab75a3409e353fe9073eb10783b4912a6a9937c57bf44a62c1e Закрытый ключ :
Скрыт

Зашифровать файл для пользователя LXMf:

```
$ rnid -i 8dd57a738226809646089335a6b03695 -e my_file.txt
```

Восстановлена идентичность <bc7291552be7a58f361522990465165c> для адресата

```
<8dd57a738226809646089335a6b03695>
```

Шифрование my_file.txt

Файл my_file.txt зашифрован для <bc7291552be7a58f361522990465165c> в my_file.txt.rfe

Если идентификатор пункта назначения еще не известен, его можно получить из сети с помощью опции командной строки -R:

```
$ rnid -R -i 30602def3b3506a28ed33db6f60cc6c9 -e my_file.txt
```

Запрос неизвестного идентификатора для <30602def3b3506a28ed33db6f60cc6c9>... Получен

идентификатор <2b489d06eaf7c543808c76a5332a447d> для адресата

```
<30602def3b3506a28ed33db6f60cc6c9> из сети Ш и ф р о в а н и е
```

ф а й л а my_file.txt

Файл my_file.txt зашифрован для <2b489d06eaf7c543808c76a5332a447d> в my_file.txt.rfe

Расшифруйте файл с помощью идентификатора Reticulum, для которого он был зашифрован:

```
$ rnid -i ./my_identity -d my_file.txt.rfe
```

Загружена идентичность <2225fdeecaf6e2db4556c3c2d7637294> из ./my_identity. Расшифровка файла

```
./my_file.txt.rfe...
```

Файл ./my_file.txt.rfe расшифрован с помощью <2225fdeecaf6e2db4556c3c2d7637294> в ./my_file.txt

Все параметры командной строки

Использование: rnid.py [-h] [--config путь] [-i идентификатор] [-g путь] [-v] [-q] [-a аспекты]
[-H аспекты] [-e путь] [-d путь] [-s путь] [-V путь] [-r путь] [-w путь]
[-f] [-R] [-t секунды] [-p] [-P] [--version]

Утилита Reticulum для управления идентификацией и шифрованием

опции:

-h, --help	показать это сообщение справки и выйти
--config путь	путь к альтернативному каталогу конфигурации Reticulum
-i, --identity идентификатор	ш е с т н а д ц а т е р и ч н ы й и д е н т и ф и к а т о р Reticulum или х е ш н а з н а ч е н и я, л и б о п у т ь к
→ ф а й л а и д е н т и ф и к а т о р о в	
-g, --generate файл	создать новый идентификатор
-m, --import identity_data	импортировать идентификатор Reticulum в шестнадцатеричном формате, формате base32 или base64
-x, --export	экспортировать идентификатор в формате hex, base32 или base64
-v, --verbose	увеличить уровень подробности
-q, --quiet	уменьшить уровень подробности
-a, --announce аспекты	указать пункт назначения на основе данного идентификатора
-H, --hash aspects	показать хэши назначения для других аспектов данной идентичности
-e, --encrypt файл	зашифровать файл
-d, --decrypt файл	расшифровать файл
-s, --sign путь	подписать файл
-V, --validate путь	проверить подпись

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

-r, --read файл	путь к входному файлу
-w, --write файл	путь к выходному файлу
-f, --force	записать вывод, даже если это приведет к перезаписи существующих файлов
-R, --request	запрос неизвестных идентификаторов из сети
-t секунд	таймаут запроса идентификатора перед сдачей
-p, --print-identity	вывести информацию об идентификаторе и завершить работу
-P, --print-private	разрешить отображение закрытых ключей
-b, --base64	Использовать входные и выходные данные, закодированные в base64
-B, --base32	Использовать входные и выходные данные, закодированные в base32
--version	показать номер версии программы и завершить работу

5.2.4 Утилита rnpath

С помощью утилиты rnpath можно искать и просматривать пути к пунктам назначения в сети Reticulum.

Примеры использования

Определение пути к месту назначения:

```
$ rnpath c89b4da064bf66d280f0e4d8abfd9806
```

Путь найден, пункт назначения <c89b4da064bf66d280f0e4d8abfd9806> находится на расстоянии 4 прыжков через
 ↳<f53a1c4278e0726bb73fcc623d6ce763> на TCPInterface[Testnet/dublin.connect.reticulum].
 ↳network:4965]

Все параметры командной строки

Использование: rnpath [-h] [--config CONFIG] [--version] [-t] [-m hops] [-r] [-d] [-D]
 [-x] [-w секунд] [-R хеш] [-i путь] [-W секунд] [-b] [-B] [-U]
 [--duration DURATION] [--reason REASON] [-p] [-j] [-v] [destination] [list_filter]

Позиционные аргументы утилиты управления путями

Reticulum:

destination	шестнадцатеричный хеш пункта назначения
фильтр_списка	фильтр для просмотра списка удаленных черных дыр

опции:

-h, --help	показать это сообщение справки и выйти
--config CONFIG	путь к альтернативному каталогу конфигурации Reticulum
--version	показать номер версии программы и завершить работу
-t, --table	показать все известные пути
-m, --max hops	максимальное количество переходов для фильтрации таблицы путей
-r, --rates	показать информацию о частоте объявлений
-d, --drop	удалить путь к месту назначения
-D, --drop-announces	отбросить все объявления в очереди
-x, --drop-via	отбросить все пути через указанный экземпляр транспорта
-w секунд	таймаут перед сдачей
-R хеш	хэш идентификатора транспорта удаленного экземпляра для управления
-i путь	путь к идентификатору, используемому для удаленного управления
-W секунд	таймаут перед отказом от удаленных запросов
-b, --blackholed	список заблокированных идентификаторов

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

-B, --blackhole	идентичность черной дыры
-U, --unblackhole	unblackhole identity
--duration DURATION	продолжительность принудительного применения blackhole в часах
--reason ПРИЧИНА	причина отправки идентификатора в черную дыру
-p, --blackholed-list	просмотр опубликованного списка blackhole для удаленного экземпляра транспорта
-j, --json	вывод в формате JSON
-v, --verbose	

5.2.5 Утилита rnpoke

Утилита rnpoke позволяет проверить доступность адресата, аналогично программе ping. Обратите внимание, что на запросы будет получен ответ только в том случае, если указанный адресат настроен на отправку подтверждений о получении пакетов. У многих адресатов

не имеют включенной этой опции, поэтому большинство адресатов не будут отвечать.

Вы можете включить функцию ответа на проверку для адресата в экземплярах транспорта Reticulum, установив конфигурационную директиву respond_to_probes. После этого Reticulum выведет адрес проверки в журнал при запуске экземпляра транспорта.

Примеры использования

Проверка адреса:

```
$ rnpoke rntransport.probe 2d03725b327348980d570f739a3a5708
```

Отправлен 16-байтовый запрос на адрес <2d03725b327348980d570f739a3a5708>.
Получен действительный ответ от <2d03725b327348980d570f739a3a5708>.прохождения составляет 38,469 миллисекунд через 2 прыжка

Отправить более крупный зонд:

```
$ rnpoke rntransport.probe 2d03725b327348980d570f739a3a5708 -s 256
```

Отправлен 16-байтовый пробный запрос на <2d03725b327348980d570f739a3a5708> Получен действительный ответ от <2d03725b327348980d570f739a3a5708> Времяпрохождения составляет 38,781 миллисекунд через 2 прыжка

Если интерфейс, получающий ответы на запрос, поддерживает отчетность по радиопараметрам, таким как **RSSI** и **SNR**, утилита rnpoke также выведет их в составе результата.

```
$ rnpoke rntransport.probe e7536ee90bd4a440e130490b87a25124
```

Отправлен 16-байтовый запрос на <e7536ee90bd4a440e130490b87a25124> Получен действительный ответ от <e7536ee90bd4a440e130490b87a25124>

Время прохождения в обоих направлениях составляет 1,809 секунды на 1 прыжок [RSSI -73 дБм] [SNR 12,0 дБ]

Все параметры командной строки

Использование: rnpoke [-h] [--config CONFIG] [-s SIZE] [-n PROBES] [-t секунды] [-w секунды] [--version] [-v] [full_name] [destination_hash]

Аргументы позиции утилиты

Reticulum Probe:

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

full_name	полное имя назначения в записи с точками
destination_hash	шестнадцатеричный хеш пункта назначения
опции:	
-h, --help	показать это сообщение справки и выйти
--config CONFIG	путь к альтернативному каталогу конфигурации Reticulum
-s SIZE, --size SIZE	размер полезной нагрузки пакета зонда в байтах
-n PROBES, --probes PROBES	количество отправляемых зондов
-t секунд, --timeout секунд	время ожидания перед сдачей
-w секунд, --wait секунд	время между каждым запросом
--version	показать номер версии программы и завершить работу
-v, --verbose	

5.2.6 Утилита rncp

Утилита rncp — это простой инструмент для передачи файлов. С его помощью можно передавать файлы через Reticulum.

Примеры использования

Запустите rncp на принимающей системе, указав, каким идентификаторам разрешено отправлять файлы:

```
$ rncp --listen -a 1726dbad538775b5bf9b0ea25a4079c8 -a c50cc4e4f7838b6c31f60ab9032cbc62
```

Вы также можете указать разрешенные хэши идентификаторов (по одному в строке) в файле ~/.rncp/allowed_identities и просто запустить программу в режиме прослушивания:

```
$ rncp --listen
```

С другой системы скопируйте файл на принимающую систему:

```
$ rncp ~/path/to/file.tgz 73cbd378bb0286ed11a707c13447bb1e
```

Или загрузите файл с удалённой системы:

```
$ rncp --fetch ~/path/to/file.tgz 73cbd378bb0286ed11a707c13447bb1e
```

Файл идентификации по умолчанию хранится в ~/.reticulum/identities/rncp, но вы можете использовать другой, который будет создан, если он еще не существует

```
$ rncp ~/path/to/file.tgz 73cbd378bb0286ed11a707c13447bb1e -i /path/to/identity
```

Все параметры командной строки

Использование: rncp [-h] [--config путь] [-v] [-q] [-S] [-l] [-F] [-f] [-j путь] [-b секунды] [-a допустимый_хэш] [-n] [-p]
[-i идентификатор] [-w секунд] [--version] [файл] [путь_назначения]

Утилита передачи файлов Reticulum

позиционные аргументы:

файл файл для передачи

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

пункт назначения	шестнадцатеричный хеш получателя
опции:	
-h, --help	показать это сообщение справки и выйти
--config путь	путь к альтернативному каталогу конфигурации Reticulum
-v, --verbose	увеличить уровень подробности
-q, --quiet	уменьшить подробность
-S, --silent	отключить вывод информации о ходе передачи
-l, --listen	прослушивать входящие запросы на передачу
-C, --no-compress	отключить автоматическое сжатие
-F, --allow-fetch	разрешить аутентифицированным клиентам загружать файлы
-f, --fetch	загрузить файл с удаленного прослушивателя вместо отправки
-j, --jail path	ограничить запросы на получение файлов указанным путем
-s, --save путь	сохранить полученные файлы в указанном пути
-O, --overwrite	Разрешить перезапись полученных файлов вместо добавления суффикса
-b секунд	интервал объявления, 0 — объявлять только при запуске
-a разрешенный_хэш	разрешить эту идентичность (или добавить в ~/.rncp/allowed_identities)
-n, --no-auth	принимать запросы от всех
-p, --print-identity	вывести информацию об идентичности и адресате и завершить работу
-i идентичность	путь к идентификатору, который нужно использовать
-w секунд	таймаут отправителя перед сдачей
-P, --phy-rates	отобразить скорости передачи данных на физическом уровне
--version	показать номер версии программы и завершить работу

5.2.7 Утилита rnx

Утилита rnx — это простая программа для удаленного выполнения команд. Она позволяет выполнять команды на удаленных системах через Reticulum и просматривать вывод команд. Если вам нужна полностью интерактивная удаленная оболочка, обязательно ознакомьтесь с программой rnsd.

Примеры использования

Запустите rnx на системе-слушателе, указав, каким идентификаторам разрешено выполнять команды:

```
$ rnx --listen -a 941bed5e228775e5a8079fc38b1ccf3f -a 1b03013c25f1c2ca068a4f080b844a10
```

С другой системы запустите команду на удаленном компьютере:

```
$ rnx 7a55144adf826958a9529a3bcf08b149 "cat /proc/cpuinfo"
```

Или войдите в псевдо-оболочку интерактивного режима:

```
$ rnx 7a55144adf826958a9529a3bcf08b149 -x
```

Файл идентификации по умолчанию хранится в ~/.reticulum/identities/rnx, но вы можете использовать другой, который будет создан, если он еще не существует

```
$ rnx 7a55144adf826958a9529a3bcf08b149 -i /path/to/identity -x
```

Все параметры командной строки

```
Использование: rnx [-h] [--config путь] [-v] [-q] [-p] [-l] [-i идентификатор] [-x] [-b] [-n] [-N] [-d] [-m] [-a
разрешенный_хэш] [-w секунд] [-W секунд] [--stdin STDIN]
[--stdout STDOUT] [--stderr STDERR] [--version] [destination] [command]
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

Позиционные аргументы утилиты удаленного

выполнения Reticulum:

пункт назначения шестнадцатеричный хеш слушателя

команда команда для выполнения

дополнительные аргументы:

-h, --help	показать это сообщение справки и выйти
--config путь	путь к альтернативному каталогу конфигурации Reticulum
-v, --verbose	увеличить уровень подробности
-q, --quiet	уменьшить подробность
-p, --print-identity	вывести информацию об идентификаторе и пункте назначения и завершить работу
-l, --listen	прослушивать входящие команды
-i идентификатор	путь к идентификатору, который нужно использовать
-x, --interactive	перейти в интерактивный режим
-b, --no-announce	не выводить сообщение при запуске программы
-a allowed_hash	принимать от этого идентификатора
-n, --noauth	принимать файлы от любого
-N, --noid	не идентифицироваться перед слушателем
-d, --detailed	показывать подробный вывод результатов
-m	отразить код завершения удаленной команды
-w секунд	время ожидания подключения и запроса перед сдачей
-W секунд	максимальное время загрузки результата
--stdin STDIN	передать входные данные в stdin
--stdout STDOUT	максимальный размер в байтах возвращаемого stdout
--stderr STDERR	максимальный размер в байтах возвращаемого stderr
--version	показать номер версии программы и завершить работу

5.2.8 Утилита rnodeconf

Утилита rnodeconf позволяет просматривать и настраивать существующие *RNodes*, а также создавать и подготавливать новые *RNodes* на любом поддерживаемом аппаратном устройстве.

Все параметры командной строки

Использование: rnodeconf [-h] [-i] [-a] [-u] [-U] [--fw-version версия]

 [--fw-url url] [--nocheck] [-e] [-E] [-C]

 [--baud-flash baud_flash] [-N] [-T] [-b] [-B] [-p] [-D i]

 [--display-addr байт] [--freq Гц] [--bw Гц] [--txp дБм]

 [--sf коэффициент] [--cr скорость] [--eeprom-backup] [--eeprom-dump]

 [--eeprom-wipe] [-P] [--trust-key шестнадцатеричные байты] [--version] [-f]

 [-r] [-k] [-S] [-H FIRMWARE_HASH] [--platform platform] [--product

product] [--model model] [--hwrev revision] [port]

Утилита настройки и прошивки RNode. Эта программа позволяет изменять различные настройки и режимы запуска RNode. Она также может устанавливать, прошивать и обновлять прошивку на поддерживаемых устройствах.

позиционные аргументы:

порт последовательный порт, к которому подключен RNode

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

опции:

-h, --help	показать это сообщение справки и выйти
-i, --info	Показать информацию об устройстве
-a, --autoinstall	Автоматическая установка на различных поддерживаемых устройствах
-u, --update	Обновление прошивки до последней версии
-U, --force-update	Обновление до указанной прошивки, даже если версия совпадает или с т а р ш е _
→ установленной версии	
--fw-version версия	Использовать конкретную версию прошивки для обновления или автоматической установки
--fw-url url	Использовать альтернативный URL для загрузки прошивки
--nocheck	Не проверять наличие обновлений прошивки в Интернете
-e, --extract	Извлечь прошивку из подключенного RNode для последующего использования
-E, --use-extracted	Использовать извлеченную прошивку для автоматической установки или обновления
-C, --clear-cache	Очистить локально кэшированные файлы прошивки
--baud-flash baud_flash	Установить определенную скорость передачи данных при прошивке устройства. По умолчанию — 921600
-N, --normal	Переключить устройство в нормальный режим
-T, --tnc	Переключить устройство в режим TNC
-b, --bluetooth-on	Включить Bluetooth на устройстве
-B, --bluetooth-off	Выключить Bluetooth на устройстве
-p, --bluetooth-pair	Перевести устройство в режим сопряжения Bluetooth
-D, --display i	Установить яркость дисплея (0–255)
-t, --timeout s	Установить время ожидания отображения в секундах, 0 для отключения
-R, --rotation rotation	Установить поворот экрана, допустимые значения: от 0 до 3
--display-addr -байт	Установить адрес дисплея в виде шестнадцатеричного байта (00–FF)
--recondition-display	Запустить перенастройку дисплея
--np i	Установить интенсивность NeoPixel (0–255)
--freq ,Hz	Частота в Гц для режима TNC
--bw Гц	Ширина полосы в Гц для режима TNC
--txp дБм	Мощность передачи в дБм для режима TNC
--sf Коэффициент рассеяния ()	Коэффициент распределения для режима TNC (7–12)
--cr коэффициент кодирования (rate)	Скорость кодирования для режима TNC (5–8)
-x, --ia-enable	Включить избегание помех
-X, --ia-disable	Отключить защиту от помех
-c, --config	Вывести конфигурацию устройства
--eeprom-backup	Резервное копирование EEPROM в файл
--eeprom-dump	Вывести содержимое EEPROM в консоль
--eeprom-wipe	Разблокировать и очистить EEPROM
-P, --public	Отобразить открытую часть ключа подписи
--trust-key шестнадцатеричные байты	Открытый ключ, которому доверяют для проверки устройства
--version	Вывести версию программы и завершить работу
-f, --flash	Запись прошивки и загрузочной EEPROM
-r, --rom	Загрузочная EEPROM без записи прошивки
-k, --key	Сгенерировать новый ключ подписи и завершить работу
-S, --sign	Отобразить открытую часть ключа подписи
-H, --firmware-hash FIRMWARE_HASH	Установить хеш установленной прошивки
--platform platform	Спецификация платформы для начальной загрузки устройства
--product product	Спецификация продукта для начальной загрузки устройства
--model модель	Код модели для загрузки устройства

(продолжение на следующей странице)

--hwrev версия	Ревизия аппаратного обеспечения для начальной загрузки устройства
----------------	---

Для получения дополнительной информации о том, как создавать собственные RNodes, пожалуйста, ознакомьтесь с разделом «*Создание RNodes*» данного руководства.

5.3 Обнаружение интерфейсов

Reticulum включает встроенную функциональность для обнаружения подключаемых интерфейсов через саму сеть Reticulum. Это особенно полезно в ситуациях, когда вам нужно выполнить одно или несколько из следующих действий:

- Обнаружить доступные в Интернете точки входа
- Найти доступные точки доступа в физическом мире
- Поддержание связи с экземплярами RNS с неизвестными или меняющимися IP-адресами

Обнаруженные интерфейсы могут быть автоматически подключены Reticulum, что позволяет создавать конфигурации, в которых произвольный интерфейс может выступать просто в качестве начального соединения, которое может быть разорвано после обнаружения и подключения более подходящих интерфейсов.

Механизм обнаружения интерфейсов использует объявления, отправляемые через саму сеть Reticulum, и поддерживает как общедоступные интерфейсы, так и частное, зашифрованное обнаружение, которое может быть декодировано только указанными сетевыми идентификаторами. Также можно указать, какие сетевые идентификаторы следует считать действительными источниками для обнаруженных интерфейсов, чтобы интерфейсы, опубликованные неизвестными субъектами, игнорировались.

Примечание

Сетевая идентичность — это стандартный набор идентификационных ключей Reticulum, который может использоваться одним или несколькими транспортными узлами для идентификации своей принадлежности к одной общей сети. В контексте обнаружения интерфейсов это упрощает управление подключением исключительно к тем сетям, которые вас интересуют, даже если эти сети используют множество отдельных физических транспортных узлов.

Это также позволяет удобно настраивать автоматическое подключение обнаруженных интерфейсов только к тем сетям, к которым у вас есть определенная степень доверия.

Информацию о том, как сделать ваши интерфейсы обнаруживаемыми, см. в главе «*Обнаруживаемые интерфейсы*» данного руководства. В данном разделе мы сосредоточимся на том, как на практике *обнаруживать* доступные в сети интерфейсы и *подключаться* к ним.

В самом простом случае включение обнаружения интерфейсов сводится к установке параметра `discover_interfaces` в значение `true` в конфигурации Reticulum:

```
[reticulum]
...
discover_interfaces = yes
...
```

После включения этой опции ваш экземпляр RNS начнет прослушивать объявления об обнаружении интерфейсов и сохранять их для последующего использования или проверки. Вы можете отобразить список обнаруженных интерфейсов с помощью утилиты `rnstatus`:

\$ rnstatus -d Имя	Тип	Состояние	Последнее обнаружение	Значени е	Местоположение
Концентратор боковых полос	Магистраль	✓ Д о с т у п н о	1ч назад	16	46.2316, 6.0536
RNS Амстердам	Магистральна я сеть	✓ Д о с т у п н о	32 мин. назад	16	52.3865, 4.9037

Вы можете просмотреть более подробную информацию об обнаруженных интерфейсах, включая фрагменты конфигурации для непосредственной вставки в ваш файл конфигурации [interfaces], с помощью опции `rnstatus -D`:

```
$ rnstatus -D sideband

Идентификатор транспорта : 521c87a83afb8f29e4455e77930b973b
Имя                       : Sideband Hub
Тип                       : BackboneInterface
Состояние                 : Доступно Транспорт
                          : Включено
Расстояние                : 2 прыжка
Обнаружено                : 9 ч 40 мин назад
Последнее обнаружение    : 1 час и 15
минут назад Местоположение :
46.2316, 6.0536
Адрес                    : sideband.connect.reticulum.network:7822 Значение
отметки                  : 16

Запись конфигурации: [[Sideband
Hub]]
  type = BackboneInterface enabled = yes
  remote = sideband.connect.reticulum.network target_port = 7822
  transport_identity = 521c87a83afb8f29e4455e77930b973b
```

Помимо предоставления информации об обнаруженных локальных интерфейсах и управления ими, утилита `rnstatus` может экспортировать данные об обнаруженных интерфейсах в машиночитаемый формат JSON с помощью опции `rnstatus -d --json`. Это может быть полезно для экспорта данных во внешние приложения, такие как страницы состояния, карты точек доступа и т. п.

Чтобы контролировать, какие источники считаются допустимыми для обнаружения, для системы обнаружения интерфейсов можно указать дополнительные параметры конфигурации.

- Параметр `interface_discovery_sources` представляет собой список сетевых или транспортных идентификаторов, от которых будут приниматься интерфейсы. Если этот параметр задан, все остальные будут игнорироваться. Если этот параметр не задан, обнаруженные интерфейсы принимаются из любого источника, но по-прежнему подпадают под требования к значению метки.
- Параметр `required_discovery_value` указывает минимальное значение отметки, необходимое для того, чтобы объявление интерфейса считалось действительным. Чтобы сделать спам в сети большим количеством неработающих или вредоносных интерфейсов, каждый объявленный интерфейс требует действительной криптографической метки с настраиваемым значением сложности.
- Значение параметра `autoconnect_discovered_interfaces` по умолчанию равно 0 и определяет максимальное количество обнаруженных интерфейсов, которые должны быть автоматически подключены в любой момент времени. Если для этого параметра задано значение больше 0, Reticulum автоматически управляет подключениями обнаруженных интерфейсов и будет подключать или отключать их в зависимости их доступности. Вы можете в любой момент вручную добавить обнаруженные интерфейсы в свою конфигурацию, чтобы они постоянно оставались доступными.
- Параметр `network_identity` указывает *сетевую идентичность* для данного экземпляра RNS. Эта идентичность используется как для подписи (и, возможно, шифрования) *исходящих* объявлений об обнаружении интерфейсов, так и для расшифровки входящей информации об обнаружении

Приведенный ниже фрагмент конфигурации содержит пример настройки этих дополнительных параметров:

```
[ретикулум]
...
discover_interfaces = yes
interface_discovery_sources = 521c87a83afb8f29e4455e77930b973b
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```
required_discovery_value = 16
autoconnect_discovered_interfaces = 3
network_identity = ~/.reticulum/storage/identities/my_network
...
```

5.4 Удаленное управление

Можно разрешить удаленное управление системами Reticulum с помощью различных встроенных утилит, таких как `rnstatus` и `rnpath`. Для этого необходимо установить директиву `enable_remote_management` в разделе `[reticulum]` файла конфигурации. Также необходимо указать один или несколько хешей идентификаторов Reticulum для аутентификации запросов от клиентских программ. Для этого можно использовать существующие файлы идентификаторов или сгенерировать новые с помощью утилиты `rnid`.

Ниже приведен сокращенный пример включения удаленного управления в конфигурационном файле Reticulum:

```
[reticulum]
...
enable_remote_management = yes
remote_management_allowed = 9fb6d773498fb3feda407ed8ef2c3229, ↵
↳ 2d882c5586e548d79b5af27bca1776dc
...
```

Чтобы увидеть полный пример конфигурации, можно запустить `rnscd --exampleconfig`.

5.5 Управление Blackhole

Сети Reticulum по сути являются открытыми и не требуют разрешений, что позволяет присоединиться любому пользователю, имеющему совместимый интерфейс. Хотя такая открытость необходима для обеспечения отказоустойчивости и децентрализации сети, она также подвергает сеть риску потенциальных злоупотреблений, таких как перегрузка сети избыточными широковещательными объявлениями со стороны других узлов или иные формы истощения ресурсов.

Система **Blackhole** предоставляет инструменты, помогающие решать эту проблему. Она позволяет операторам и отдельным пользователям блокировать определенные идентификаторы на транспортном уровне, не давая им распространять объявления через ваш узел, а другим узлам — достигать их через вашу сеть.

Важно

В сетях Reticulum принципиально **невозможно** глобально заблокировать или подвергнуть цензуре какой-либо идентификатор или пункт назначения. Функция «blackhole» предотвращает объявления (и трафик) ко всем пунктам назначения, связанным с идентификатором, на который наложен «blackhole», *исключительно в ваших собственных сегментах сети*.

Это дает пользователям и операторам контроль над тем, что они хотят разрешить *в своих собственных сегментах сети*, но нет возможности глобально подвергнуть цензуре или удалить идентификатор, пока *кто-то* готов обеспечить его транспортировку.

Эта функция служит двум целям:

- **Для индивидуальных пользователей:** это простой способ обеспечить бесперебойную и эффективную работу локальной сети за счет ручной блокировки спамерских или нежелательных узлов.
- **Для сетевых операторов:** позволяет создавать объединенные стандарты безопасности для всего сообщества. Публикуя и обмениваясь списками blackhole, операторы могут защищать крупные инфраструктуры и распространять правила фильтрации спама по всей сети без ручного вмешательства.

5.5.1 Управление локальными черными списками

Самый быстрый способ управления нежелательными идентичностями — это ручная настройка с помощью утилиты `rnpath`. Это позволяет мгновенно блокировать или разблокировать определенные идентичности в вашем локальном экземпляре Transport.

Блокировка идентификатора

Чтобы заблокировать идентификатор, используйте флаг `-B` (или `--blackhole`), за которым следует хеш идентификатора. По желанию можно указать продолжительность и причину, что полезно для ведения журнала и дальнейшего использования.

```
$ rnpath -B 3a4f8b9c1d2e3f4g5h6i7j8k9l0m1n2o
```

Вы также можете указать продолжительность (в часах) и причину:

```
$ rnpath -B 3a4f8b9c1d2e3f4g5h6i7j8k9l0m1n2o --duration 24 --reason "Excessive announces"
```

Снятие с черного списка

Чтобы удалить идентификатор из черной дыры, используйте флаг `-U` (или `--unblackhole`):

```
$ rnpath -U 3a4f8b9c1d2e3f4g5h6i7j8k9l0m1n2o
```

Просмотр списка «черных дыр»

Чтобы просмотреть все учетные записи, которые в данный момент заблокированы на вашем локальном экземпляре, используйте флаг `-b` (или `--blackholed`):

```
$ rnpath -b
<3a4f8b9c1d2e3f4g5h6i7j8k9l0m1n2o> заблокирован на 23 часа 56 минут (чрезмерное количество объявлений)
<399ea050ce0eed1816c300bcb0840938> заблокирован на неопределенный срок (спам в объявлениях)
<d56a4fa02c0a77b3575935aedd90bdb2> заблокирован на неопределенный срок (спам объявлений)
<2b9ec651326d9bc274119054c70fb75e> заблокирован на неопределенный срок (спам в объявлениях)
<1178a8f1fad405bf2ad153bf5036bdfd> заблокировано на неопределенный срок (спам в объявлениях)
```

blackholed);

5.5.2 Автоматический поиск списков

Ручная блокировка идентификаторов эффективна в случае непосредственной угрозы, но ведение актуального списка блокировки для большой сети нецелесообразно. Reticulum поддерживает **автоматический поиск списков**, позволяя вашему узлу подписываться на списки blackhole, поддерживаемые доверенными пирами или центральным органом, которым вы управляете самостоятельно.

Предупреждение

Проверьте перед подпиской! Подписка на источник черных списков — это серьезное действие, которое дает этому источнику право диктовать, с кем вы можете общаться. Перед добавлением источника в свою конфигурацию убедитесь, что его администратор соответствует вашей политике использования и ценностям. Слепая подписка на ненадежные списки может непреднамеренно заблокировать легитимных пиров или важные сервисы.

Когда эта функция включена, ваш транспортный экземпляр будет периодически (примерно раз в час) подключаться к настроенным источникам, загружать их последние списки blackhole и автоматически объединять их с вашим локальным списком блокировки. Это обеспечивает защиту по принципу «настроил и забыл» как для отдельных пользователей, так и для крупных сетей.

Настройка

Чтобы включить автоматический поиск источников, добавьте опцию `blackhole_sources` в раздел `[reticulum]` вашего файла конфигурации. Эта опция принимает разделенный запятыми список хешей идентификаторов транспорта, которым вы доверяете в плане предоставления действительных списков blackhole

```
[reticulum]
...
# Автоматически загружать списки «черных дыр» из этих надежных источников
blackhole_sources = 521c87a83afb8f29e4455e77930b973b, 68a4aa91ac350c4087564e8a69f84e86
...
```

Как это работает

1. Если эта функция включена, служба BlackholeUpdater работает в фоновом режиме.
2. Для каждого хеша идентификатора, указанного в blackhole_sources, она пытается установить временную ссылку на связанный с ним пункт назначения «rnstransport.info.blackhole».
3. Она запрашивает путь /list, который возвращает словарь идентификаторов, попавших в черную дыру, и связанных с ними метаданных.
4. Полученный список объединяется с вашей локальной базой данных blackholed_identities.
5. Списки сохраняются на диск, что гарантирует их сохранность при перезапуске.

Примечание

Вы можете проверить внешние списки, на которые вы подписаны, и их содержимое, не импортируя их, с помощью `rnpath` -p. Подробности о запросах к удаленным спискам blackhole см. в *документации по утилите rnpath*.

5.5.3 Публикация списков blackhole

Если вы управляете общедоступным шлюзом, хабом сообщества или просто хотите поделиться своим списком blackhole с другими, вы можете настроить свой экземпляр так, чтобы он действовал как издатель списков blackhole. Это позволяет другим узлам подписываться на *ваши* определения нежелательного трафика.

Включение публикации

Чтобы опубликовать свой локальный список blackhole, включите опцию `publish_blackhole` в разделе `[reticulum]`:

```
[reticulum]
...
publish_blackhole = yes
...
```

Если эта опция включена, ваш экземпляр Transport регистрирует обработчик запросов на `rnstransport.info.blackhole`. Любой узел, подключающийся к этому адресу и запрашивающий `/list`, получит полный набор идентификаторов, присутствующих в данный момент в вашей локальной базе данных blackhole.

Федерация и доверие

Система blackhole основана на отношениях доверия между подписчиком и издателем. Подписываясь на источник, вы неявно доверяете этому источнику, что он будет блокировать только те идентичности, которые действительно наносят вред сети.

По мере развития экосистемы эта система будет интегрирована с **механизмом «Network Identities»**. Это позволит сообществам проверять, действительно ли опубликованный список «черных дыр» предоставлен конкретной сетью или организацией, обладающей определенным уровнем репутации и надежности, что добавит уровень криптографического доверия к процессу федерации. Таким образом, злоумышленники не смогут публиковать поддельные списки с целью цензурирования легитимного трафика.

Для операторов это создает масштабируемую модель, в которой ведение единого высококачественного списка блокировки может защитить тысячи нижестоящих узлов, что значительно сокращает административную нагрузку.

5.6 Оптимизация настроек системы

Если вы настраиваете систему для постоянного использования с Reticulum, есть несколько изменений в конфигурации системы, которые могут упростить ее администрирование. Эти изменения будут подробно описаны здесь.

5.6.1 Фиксированные имена последовательных портов

В экземпляре Reticulum с несколькими интерфейсами на основе последовательных портов может быть полезно использовать фиксированные имена устройств для последовательных портов вместо динамически назначаемых сокращений, таких как `/dev/ttyUSB0`. В большинстве дистрибутивов на базе Debian, включая Ubuntu и Raspberry Pi OS, эти узлы можно найти в каталоге `/dev/serial/by-id`.

Вы можете использовать такой путь к устройству напрямую вместо пронумерованных сокращений. Вот пример TNC пакетной радиосвязи, настроенного таким образом:

```
[[Интерфейс Packet Radio KISS]] type =
  KISSInterface interface_enabled = True
  outgoing = true
  порт = /dev/serial/by-id/usb-FTDI_FT230X_Basic_UART_43891CKM-if00-порт0 скорость = 115200
  databits = 8 parity =
  none stopbits = 1
  преамбула = 150
  txtail = 10
  persistence = 200
  время слота = 20
```

Использование этой методологии позволяет избежать возможной путаницы с именами, возникающей в случаях, когда физические устройства подключаются и отключаются в разном порядке, или когда присвоение имен устройствам меняется при каждой загрузке.

5.6.2 Reticulum как системная служба

Вместо запуска Reticulum вручную можно установить `rnsd` в качестве системной службы, чтобы она запускалась автоматически при загрузке.

Системная служба

Если вы установили Reticulum с помощью `pip`, программа `rnsd`, скорее всего, будет находиться только в локальном для пользователя пути установки, а это означает, что `systemd` не сможет её запустить. В этом случае вы можете просто создать символическую ссылку на программу `rnsd` в каталоге, который находится в пути `systemd`:

```
sudo ln -s $(which rnsd) /usr/local/bin/
```

Затем можно создать файл службы `/etc/systemd/system/rnsd.service` со следующим содержанием:

```
[Unit]
Description=Демон сетевого стека Reticulum After=multi-
user.target

[Service]
# Если вы запускаете Reticulum на устройствах с
WiFi # или других устройствах, которым требуется #
дополнительное время для инициализации, # вы
можете добавить небольшую задержку перед #
запуском Reticulum системой systemd:
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```
# ExecStartPre=/bin/sleep 10 Type=simple
Restart=always RestartSec=3
User=USERNAMEHERE

ExecStart=rnsd --service

[Install]

WantedBy=multi-user.target
```

Обязательно замените USERNAMEHERE на имя пользователя, под которым вы хотите запускать rnsd. Чтобы запустить rnsd вручную, выполните:

```
sudo systemctl start rnsd
```

Если вы хотите, чтобы rnsd запускался автоматически при загрузке системы, выполните команду:

```
sudo systemctl enable rnsd
```

Служба в пользовательском пространстве

В качестве альтернативы можно использовать пользовательскую службу systemd вместо общесистемной. Таким образом, всю настройку можно выполнить от имени обычного пользователя. Создайте файл пользовательской службы systemd ~/.config/systemd/user/rnsd.service со следующим содержанием:

```
[Unit]
Description=Демон сетевого стека Reticulum After=default.target

[Service]
# Если вы запускаете Reticulum на устройствах с
# Wi-Fi # или других устройствах, которым требуется
# дополнительное время для инициализации, # вам
# может понадобиться # добавить небольшую задержку
# перед запуском # Reticulum системой systemd:
# ExecStartPre=/bin/sleep 10 Type=simple
Restart=always RestartSec=3

ExecStart=RNS_BIN_DIR/rnsd --service

[Install] WantedBy=default.target
```

Замените RNS_BIN_DIR на путь к вашему каталогу с бинарными файлами Reticulum (например, /home/USERNAMEHERE/rns/bin).

Запустите пользовательскую службу:

```
systemctl --user daemon-reload systemctl --
user start rnsd.service
```

Если вы хотите запускать rnsd автоматически без необходимости входа в систему под именем USERNAMEHERE, выполните следующие действия:

```
sudo loginctl enable-linger USERNAMEHERE systemctl --user  
enable rnsd.service
```


ОБЗОР RETICULUM

В этой главе кратко описаны общие цели и принципы работы Reticulum. Она должна дать вам общее представление о том, как работает стек, и помочь понять, как разрабатывать сетевые приложения с использованием Reticulum.

Эта глава не является исчерпывающим источником информации о Reticulum — по крайней мере, пока. На данный момент единственным полным хранилищем информации и окончательным авторитетным источником сведений о том, как на самом деле функционирует Reticulum, являются эталонная реализация на Python и справочник по API. Тем не менее, эта глава является важным ресурсом для понимания принципов работы Reticulum на высоком уровне, а также общих принципов Reticulum и способов их применения при создании собственных сетей или программного обеспечения.

После прочтения этой главы вы должны будете хорошо понимать, как работает сеть Reticulum, какие задачи она способна решать и как вы можете использовать её самостоятельно. В этой главе также дается обзор идей и философии, лежащих в основе Reticulum, а также рассказывается, какие проблемы она стремится решить и каким образом подходит к их решению.

6.1 Мотивация

Основной мотивацией для разработки и внедрения Reticulum стало нынешнее отсутствие надежных, функциональных и безопасных способов цифровой связи с минимальной инфраструктурой. Я убежден, что крайне желательно создать надежный и эффективный способ организации сетей цифровой связи на большие расстояния, которые позволяют безопасно обмениваться информацией между людьми и машинами без централизованного органа власти, контроля, цензуры или барьеров для доступа.

Практически все существующие на сегодняшний день сетевые системы имеют одно общее ограничение: для их функционирования требуются значительные усилия по координации, а также централизованное доверие и власть. Чтобы присоединиться к таким сетям, необходимо получить одобрение контролирующих «стражей». Эта потребность в координации и доверии неизбежно приводит к созданию среды централизованного контроля, в которой операторам инфраструктуры или правительствам очень легко контролировать или изменять трафик, а также подвергать цензуре или преследовать нежелательных участников. Это также делает совершенно невозможным свободное развертывание и использование сетей по собственному усмотрению, как это делается с другими распространенными инструментами, расширяющими индивидуальную свободу действий и свободу.

Reticulum стремится свести к минимуму необходимость в координации и доверии. Его цель — сделать безопасные, анонимные и не требующие разрешений сети и обмен информацией инструментом, которым может воспользоваться любой желающий.

Поскольку Reticulum полностью независим от среды, его можно использовать для построения сетей на том, что лучше всего подходит для конкретной ситуации, или на том, что у вас есть в наличии. В некоторых случаях это могут быть пакетные радиосвязи на УКВ-частотах, в других — сеть 2,4 ГГц с использованием готовых радиостанций, а в третьих — обычные платы разработчика LoRa.

На момент публикации данного документа наиболее быстрая и простая конфигурация для разработки и тестирования — это использование радиомодулей LoRa с прошивкой с открытым исходным кодом (см. раздел «Рекомендуемая конфигурация»), подключенных к любому компьютеру или мобильному устройству, на котором может работать Reticulum.

Конечная цель Reticulum — дать каждому возможность стать оператором собственной сети, а также сделать недорогим и простым покрытие обширных территорий множеством независимых, взаимосвязанных и автономных сетей. Reticulum — **это не одна сеть, а инструмент** для создания *тысяч сетей*. Сетей без «аварийных выключателей», слежки, цензуры и контроля. Сетей, которые могут свободно взаимодействовать, объединяться и разъединяться друг с другом и не требуют централизованного надзора. Сети для людей. *Сети для народа.*

6.2 Цели

Чтобы обеспечить максимально широкое применение и эффективность развертывания, при разработке Reticulum руководствовались следующими целями:

- **Полная совместимость с программным стеком с открытым исходным кодом**
Reticulum должен быть реализован и работать исключительно на базе программного обеспечения с открытым исходным кодом. Это имеет решающее значение для обеспечения доступности, безопасности и прозрачности системы.
- **Независимость от аппаратного уровня**
Reticulum должен быть полностью независимым от аппаратного обеспечения и должен работать на широком спектре физических сетевых уровней, таких как радиомодули передачи данных, последовательные линии, модемы, портативные приемопередатчики, проводной Ethernet, Wi-Fi или любые другие устройства, способные передавать цифровой поток данных. Аппаратное обеспечение, предназначенное исключительно для использования с Reticulum, должно быть максимально недорогим и состоять из готовых компонентов, чтобы его можно было легко модифицировать и воспроизвести любому заинтересованному лицу.
- **Очень низкие требования к пропускной способности**
Reticulum должен надежно функционировать по каналам связи с пропускной способностью всего *5 бит в секунду*.
- **Шифрование по умолчанию**
Reticulum должен использовать надежное шифрование по умолчанию для всей коммуникации.
- **Анонимность инициатора**
Должна быть возможность обмениваться данными через сеть Reticulum, не раскрывая никакой идентифицирующей информации о себе.
- **Использование без лицензии**
Reticulum должен функционировать через физические средства связи, не требующие каких-либо лицензий на использование. Reticulum должен быть спроектирован таким образом, чтобы его можно было использовать в радиочастотных диапазонах ISM и чтобы он мог обеспечивать рабочие связи на большие расстояния в таких условиях, например, путем подключения модема к PMR- или СВ-радиостанции или с помощью модулей LoRa или WiFi.
- **Поставляемое программное обеспечение**
Помимо основного сетевого стека и API, позволяющих разработчикам создавать приложения с использованием Reticulum, необходимо реализовать и выпустить базовый набор средств связи на базе Reticulum одновременно с самим Reticulum. Они должны служить как функциональный базовый набор средств связи, так и в качестве примера и учебного ресурса для тех, кто желает создавать приложения с использованием Reticulum.
- **Простота использования**
Эталонная реализация Reticulum написана на языке Python, что делает её простой в использовании и понятной. Программист, обладающий лишь базовым опытом, сможет использовать Reticulum для написания сетевых приложений.
- **Низкая стоимость**
Развертывание системы связи на основе Reticulum должно быть максимально недорогим. Это должно быть достигнуто за счет использования недорогого готового оборудования, которое потенциальные пользователи, возможно, уже имеют. Стоимость настройки рабочего узла должна составлять менее 100 долларов, даже если все компоненты необходимо приобрести.

6.3 Введение и основные функции

Reticulum — это сетевой стек, предназначенный для каналов связи с высокой задержкой и низкой пропускной способностью. В своей основе Reticulum представляет собой систему, *ориентированную на обмен сообщениями*. Она подходит как для локальных сценариев «точка-точка» или «точка-многоточка», когда все узлы находятся в зоне досягаемости друг друга, так и для сценариев, в которых пакеты необходимо передавать через несколько промежуточных узлов в сложной сети, чтобы достичь адресата.

Reticulum отказывается от понятий адресов и портов, известных из IP, TCP и UDP. Вместо этого Reticulum использует единую концепцию *пунктов назначения*. Любое приложение, использующее Reticulum в качестве сетевого стека, должно создать один или несколько пунктов назначения для получения данных и знать пункты назначения, в которые необходимо отправить данные.

Все пункты назначения в Reticulum *представлены* в виде 16-байтового хеша. Этот хеш получается путем усечения полного SHA-256-битный хеш идентифицирующих характеристик пункта назначения. Пользователям адреса пунктов назначения будут отображаться в виде 16 шестнадцатеричных байтов, как в этом примере: <13425ec15b621c1d928589718000d814>.

Размер усечения в 16 байт (128 бит) для адресов назначения был выбран в качестве разумного компромисса между объемом адресного пространства и накладными расходами на пакеты. Адресное пространство, обеспечиваемое этим размером, способно поддерживать многие миллиарды одновременно активных устройств в одной сети, сохраняя при этом низкие накладные расходы на пакеты, что крайне важно для сетей с низкой пропускной способностью. В крайне маловероятном случае, если это адресное пространство приблизится к перегрузке, изменение кода в одной строке может расширить адресное пространство Reticulum до 256 бит, обеспечив потенциальную поддержку сетями галактического масштаба. Это, очевидно, полное и нелепое перераспределение ресурсов, и, как таковое, текущие 128 бит должны быть достаточными даже в далеком будущем.

По умолчанию Reticulum шифрует все данные с использованием криптографии на основе эллиптических кривых и алгоритма AES. Любой пакет, отправляемый адресату, шифруется с помощью ключа, вычисляемого для каждого пакета отдельно. Reticulum также может устанавливать зашифрованный канал связи с адресатом, называемый «*Link*». Как данные, передаваемые по каналам «*Link*», так и отдельные пакеты обеспечивают *анонимность инициатора*. Кроме того, по умолчанию каналы обеспечивают *прямую секретность*, используя обмен ключами по алгоритму Диффи-Хеллмана на эллиптической кривой Curve25519 для вычисления временных ключей для каждого канала. Асимметричная передача пакетов без использования каналов также может обеспечивать прямую секретность с автоматической сменой ключей, если включить эту функцию для каждого адресата. Уровни многоскоковой передачи, координации, проверки и надежности являются полностью автономными и также основаны на криптографии на основе эллиптических кривых.

Reticulum также предлагает шифрование с симметричным ключом для групповой связи, а также незашифрованные пакеты (**только** для целей локальной ширококвещательной передачи).

Reticulum может подключаться к различным интерфейсам, таким как радиомодемы, радиостанции передачи данных и последовательные порты, и предоставляет возможность легко туннелировать трафик Reticulum через IP-соединения, такие как Интернет или частные IP-сети.

6.3.1 Пункты назначения

Для получения и отправки данных с помощью стека Reticulum приложению необходимо создать один или несколько пунктов назначения. Reticulum использует три основных типа пунктов назначения и один специальный:

- **Одиночный**

Тип «*одиночный*» — самый распространенный в Reticulum и подходит для большинства задач. Он всегда идентифицируется уникальным открытым ключом. Любые данные, отправленные в этот пункт назначения, будут зашифрованы с помощью временных ключей, полученных в результате обмена ключами ECDH, и будут доступны для чтения только создателю пункта назначения, у которого есть соответствующий закрытый ключ.

- **Простой**

Тип адреса «*plain*» не шифруется и подходит для трафика, который должен передаваться в ширококвещательном режиме нескольким пользователям или быть доступен для чтения любому. Трафик, направляемый в *простой* пункт назначения, не шифруется. Как правило, *простые* пункты назначения можно использовать для ширококвещательной информации, предназначенной для общего доступа. Доступ к простым пунктам назначения возможен только напрямую, и пакеты, адресованные простым пунктам назначения, никогда не передаются через несколько узлов в сети. Чтобы информация могла передаваться через несколько узлов в Reticulum, *она должна* быть зашифрована, поскольку Reticulum использует шифрование на уровне пакетов для проверки маршрутов и поддержания их работоспособности.

- **Группа**

Тип *группового* специального адреса, определяющий виртуальный адрес с симметричным шифрованием. Данные, отправляемые на этот адрес, будут зашифрованы с помощью симметричного ключа и будут доступны для чтения любому, кто владеет этим ключом. Однако, как и в случае с типом *обычного* адреса, пакеты, предназначенные для этого типа адреса, в настоящее время не передаются через несколько промежуточных узлов, хотя планируемое обновление Reticulum позволит создавать глобально доступные *групповые* адреса.

- **Ссылка**

Связь — это особый тип адреса, который выступает в качестве абстрактного канала к *отдельному* адресу, подключенному напрямую или через несколько промежуточных узлов. *Связь* также обеспечивает надёжность и более эффективное шифрование, секретность пересылки, анонимность инициатора и, как таковая, может быть полезна даже в тех случаях, когда узел доступен напрямую. Кроме того, она предоставляет более функциональный API и позволяет легко осуществлять запросы и ответы, передачу больших объемов данных и многое другое.

Именованние пунктов назначения

Пункты назначения создаются и получают имена в удобной для понимания нотации с точками, состоящей из *аспектов*, и отображаются в сети в виде хеша этого значения. Хеш представляет собой SHA-256, усечённый до 128 бит. Аспект верхнего уровня всегда должен быть уникальным идентификатором для приложения, использующего данный пункт назначения. Последующие уровни аспектов могут быть определены создателем приложения по своему усмотрению.

Аспекты могут быть любой длины и в любом количестве, и даже длинное итоговое имя назначения не повлияет на эффективность, поскольку в сети имена всегда передаются в виде сокращённых хэшей SHA-256.

Например, пункт назначения для приложения мониторинга окружающей среды может состоять из названия приложения, типа устройства и типа измерения, например:

```
название приложения      : environmentlogger
аспекты                  : remotesensor, temperature

полное имя : environmentlogger.remotesensor.temperature хеш :
4faf1b2e0a077e6a9d92fa051f256038
```

В случае *одного* адреса назначения Reticulum автоматически добавляет соответствующий открытый ключ в качестве атрибута назначения перед хешированием. Это делается для того, чтобы гарантировать доставку только по правильному адресу, поскольку любой пользователь может прослушивать любой адрес назначения. Добавление открытого ключа гарантирует, что данный пакет будет направлен только тому адресату, который владеет соответствующим закрытым ключом для его расшифровки.

Обратите внимание! Здесь необходимо понять одну очень важную концепцию:

- Любой может использовать имя пункта назначения environmentlogger.remotesensor.temperature
- Каждый адресат, который это делает, по-прежнему будет иметь уникальный хеш адресата и, следовательно, будет однозначно адресуем, поскольку их открытые ключи будут различаться.

При практическом использовании именования с *одним* пунктом назначения рекомендуется не использовать в именах аспектов никаких элементов, обеспечивающих однозначную идентификацию. Имена аспектов должны представлять собой общие термины, описывающие тип пункта назначения. Однозначная идентификация аспекта всегда достигается путем добавления открытого ключа, что превращает пункт назначения в однозначно идентифицируемый. Reticulum выполняет это автоматически.

Любой адрес в сети Reticulum можно определить и достичь, просто зная его хеш-адрес (а также открытый ключ, но если открытый ключ неизвестен, его можно запросить у сети, просто зная хеш-адрес). Использование имен приложений и аспектов упрощает структурирование программ Reticulum и позволяет фильтровать информацию и данные, которые получает ваша программа.

Подводя итог, различные типы адресатов следует использовать в следующих ситуациях:

- **Одиночный**
Когда требуется частная связь между двумя конечными точками. Поддерживает несколько прыжков.
- **Групповой**
Когда требуется частная связь между двумя или более конечными точками. Непосредственно поддерживает многоуровневую передачу, но сначала необходимо установить соединение через *один* конечный пункт.
- **Простой**
Когда желательна связь в виде простого текста, например, при широковещательной передаче информации или для целей локального обнаружения.

Для связи с *одним* адресатом необходимо знать его открытый ключ. Допустим любой способ получения открытого ключа, однако Reticulum включает простой механизм, позволяющий другим узлам узнать открытый ключ вашего адресата, называемый «*объявлением*». Также можно запросить неизвестный открытый ключ у сети, поскольку все экземпляры транспорта служат распределённым реестром открытых ключей.

Обратите внимание, что информацию об открытых ключах можно передавать и проверять иными способами, помимо использования встроенной функции «*announce*», и поэтому для получения открытых ключей не обязательно использовать функции «*announce*» и «*path request*». Это

, это самый простой способ, и его определенно следует использовать, если нет веских причин поступать иначе.

6.3.2 Объявления открытых ключей

При использовании функции «*announce*» по всем соответствующим интерфейсам отправляется специальный пакет, содержащий всю необходимую информацию о хэше адресата и открытом ключе, а также, возможно, некоторые дополнительные данные, специфичные для конкретного приложения. Весь пакет подписывается отправителем для обеспечения подлинности. Использование функции объявления не является обязательным, но во многих случаях это будет самым простым способом обмена открытыми ключами в сети. Механизм объявления также служит для установления сквозной связи с объявленным пунктом назначения, поскольку объявление распространяется по сети.

Например, объявление в простом мессенджере может содержать следующую информацию:

- Хэш адресата объявителя
- Открытый ключ объявителя
- Данные, относящиеся к конкретному приложению, в данном случае — никнейм пользователя и статус доступности
- Случайный блок, делающий каждое новое объявление уникальным
- Подпись Ed25519 вышеуказанной информации, подтверждающая подлинность

Имея эту информацию, любой узел Reticulum, получивший её, сможет восстановить адрес получателя исходящего сообщения для безопасной связи с ним. Вы, возможно, заметили, что для восстановления полной информации об объявленном адресате не хватает одного элемента — имен аспектов адресата. Они намеренно опущены для экономии пропускной способности, поскольку в большинстве случаев они будут подразумеваться. Принимающее приложение уже будет их знать. Если имя адресата не является полностью подразумеваемым, в часть данных, специфичную для приложения, может быть включена информация, которая позволит получателю вывести это имя.

Важно отметить, что объявления будут пересылаться по всей сети в соответствии с определенным шаблоном. Это будет подробно описано в разделе «*Механизм объявлений в деталях*».

В Reticulum конечные точки могут свободно перемещаться по сети. Это значительно отличается от таких протоколов, как IP, где предполагается, что адрес всегда остается в пределах сегмента сети, в котором он был присвоен. В Reticulum такого ограничения нет: любая конечная точка *полностью переносима* по всей топологии сети и *может быть даже перемещена в другие сети Reticulum*, отличные от той, в которой она была создана, при этом оставаясь доступной. Чтобы обновить свою доступность, адресату достаточно просто отправить объявление в любую сеть, частью которой он является. Через некоторое время он станет доступен по всей сети.

Учитывая, что *отдельные* адреса всегда привязаны к паре «закрытый/открытый ключ», мы переходим к следующей теме.

6.3.3 Идентичности

В Reticulum *идентичность* не обязательно означает личность, а представляет собой абстракцию, которая может обозначать любой вид *поддающейся проверке сущности*. Это вполне может быть человек, но также может быть интерфейс управления машиной, программой, роботом, компьютером, датчиком или чем-то совершенно иным. В целом, любой агент, способный действовать, подвергаться воздействию, хранить или манипулировать информацией, может быть представлен в виде идентичности. *Идентичность* может использоваться для создания любого количества адресатов.

С *отдельным* пунктом назначения всегда связана *идентичность*, но это не относится к *простым* или *групповым* пунктам назначения. Пункты назначения и идентичности связаны между собой многосторонней связью. Вы можете создать пункт назначения, и если при создании он не будет связан с какой-либо идентичностью, система автоматически создаст новую для использования. В некоторых ситуациях это может быть целесообразно, но чаще всего вам, вероятно, захочется сначала создать идентичность, а затем использовать её для создания новых пунктов назначения.

В качестве примера можно использовать идентификатор для представления пользователя приложения для обмена сообщениями. Затем этот идентификатор может создавать адресаты, чтобы обеспечить доставку сообщений пользователю. Во всех случаях чрезвычайно важно надежно и конфиденциально хранить закрытые ключи, связанные с любым идентификатором Reticulum, поскольку получение доступа к ключам идентификатора равносильно получению доступа и контролю над доступностью любых адресатов, созданных этим идентификатором.

6.3.4 Дальнейшие шаги

Вышеупомянутые функции и принципы составляют основу Reticulum и достаточны для создания функциональных сетевых приложений в локальных кластерах, например, по радиоканалам, где все заинтересованные узлы могут напрямую слышать друг друга. Но чтобы система была действительно полезной, нам нужен способ направлять трафик через несколько прыжков в сети.

В следующих разделах будут представлены две концепции, позволяющие это сделать: *пути* и *каналы*.

6.4 Транспорт Reticulum

Методы маршрутизации, используемые в традиционных сетях, принципиально несовместимы с типами физических сред и условиями, для работы в которых была разработана сеть Reticulum. Эти механизмы в основном предполагают наличие доверия на физическом уровне и зачастую требуют гораздо большей пропускной способности, чем та, которую Reticulum может считать доступной. Поскольку Reticulum разработана для работы в открытом радиочастотном спектре, о таком доверии говорить не приходится, а пропускная способность зачастую весьма ограничена.

Для решения этих задач *транспортная* система Reticulum использует асимметричную криптографию на основе эллиптических кривых, чтобы реализовать концепцию *маршрутов*, позволяющих определять, как приблизить информацию к определенному пункту назначения. Важно отметить, что ни один узел в сети Reticulum не знает полного маршрута до пункта назначения. Каждый транспортный узел, участвующий в сети Reticulum, знает только самый прямой путь, позволяющий приблизить пакет на один шаг к месту назначения.

6.4.1 Типы узлов

В настоящее время Reticulum различает два типа сетевых узлов. Все узлы в сети Reticulum являются *экземплярами Reticulum*, а некоторые из них также являются *транспортными узлами*. Если система, на которой работает Reticulum, установлена в одном месте и должна оставаться доступной большую часть времени, она вполне подходит для роли *транспортного узла*.

Любой экземпляр Reticulum может стать транспортным узлом, если включить эту функцию в настройках. Это различие устанавливается пользователем при настройке узла и используется для определения того, какие узлы в сети будут участвовать в пересылке трафика, а какие будут полагаться на другие узлы для обеспечения более широкой связи.

Если узел является *экземпляром*, ему следует задать конфигурационную директиву `enable_transport = No`, что является настройкой по умолчанию.

Если это *транспортный узел*, для него следует задать конфигурационную директиву `enable_transport = Yes`.

6.4.2 Механизм объявления в деталях

Когда экземпляр Reticulum передает *объявление* о пункте назначения, оно будет перенаправлено любым транспортным узлом, получившим его, но в соответствии с определенными правилами:

- Если именно это объявление уже получалось ранее, его следует игнорировать.
- Если нет, запишите в таблицу, от какого транспортного узла было получено объявление и сколько раз в общей сложности оно было ретранслировано, прежде чем попало сюда.
- Если объявление было переслано $m+1$ раз, оно больше не будет пересылаться. По умолчанию m установлено на 128.
- После случайной задержки объявление будет повторно передано на всех интерфейсах, имеющих доступную пропускную способность для обработки объявлений. По умолчанию максимальное выделение пропускной способности для обработки объявлений установлено на 2%, но его можно настроить для каждого интерфейса отдельно.
- Если у какого-либо интерфейса недостаточно пропускной способности для ретрансляции объявления, ему будет присвоен приоритет, обратно пропорциональный количеству прыжков, и оно будет помещено в очередь, управляемую этим интерфейсом.
- Когда у интерфейса появляется пропускная способность для обработки объявления, он будет приоритезировать объявления для пунктов назначения, наиболее близких по количеству прыжков, тем самым отдавая приоритет доступности и связности локальных узлов даже в медленных сетях, подключенных к более широким и быстрым сетям.

- После повторной передачи объявления, если не обнаружено, что другие узлы повторно передают это объявление с большим количеством прыжков, чем при его отправке с данного узла, попытка его передачи будет повторена r раз. По умолчанию значение r равно 1.
- Если поступает более новое объявление от того же адресата, в то время как идентичное объявление уже ожидает передачи, новейшее объявление отбрасывается. Если новейшее объявление содержит другие данные, специфичные для приложения, оно заменит старое объявление.

Как только объявление достигнет транспортного узла в сети, любой другой узел, находящийся в прямом контакте с этим транспортным узлом, сможет связаться с пунктом назначения, откуда поступило объявление, просто отправив пакет, адресованный этому пункту назначения. Любой транспортный узел, располагающий информацией об объявлении, сможет направить пакет к пункту назначения, определив наиболее эффективный следующий узел на пути к нему.

Согласно этим правилам, объявление распространяется по сети предсказуемым образом и делает объявленный пункт назначения доступным в течение короткого промежутка времени. Быстрые сети, способные обрабатывать большое количество объявлений, могут достигать полной конвергенции очень быстро, даже при постоянном добавлении новых адресатов. Более медленным сегментам таких сетей может потребоваться немного больше времени, чтобы получить полную информацию о широких и быстрых сетях, к которым они подключены, но со временем они все равно смогут это сделать, при этом отдавая приоритет полной и быстро конвергирующей сквозной связности для своих локальных, более медленных сегментов.

Совет

Даже очень медленные сети, которые просто не способны когда-либо достичь *полной* конвергенции, как правило, все равно смогут достичь **любого другого пункта назначения на любых подключенных сегментах**, поскольку соединительные транспортные узлы будут приоритизировать объявления в более медленные сегменты, которые фактически запрашиваются узлами на них.

Это означает, что медленным сегментам с низкой пропускной способностью или ограниченными ресурсами **не** требуется полная информация о сети, поскольку пути всегда можно рекурсивно определить из других сегментов, которые обладают такой информацией.

Как правило, даже чрезвычайно сложные сети, использующие максимальное количество прыжков (128), достигают полной сквозной связности примерно за одну минуту, при условии, что доступна достаточная пропускная способность для обработки необходимого количества объявлений.

6.4.3 Достижение пункта назначения

В сетях с изменяющейся топологией и недоверительной связью узлам необходим способ установления *проверенной связи* друг с другом. Поскольку предполагается, что базовые сетевые среды являются недоверительными, Reticulum должен предоставлять способ гарантировать, что узел, с которым вы обмениваетесь данными, действительно является тем, кем вы его считаете. Reticulum предлагает два способа решения этой задачи.

Для обмена небольшими объемами информации Reticulum предлагает *Packet API*, который работает именно так, как и ожидается — на уровне отдельных пакетов. При отправке пакета используется следующий процесс:

- Пакет всегда создается с указанием адресата и содержит некоторое количество данных. При отправке пакета адресату *одного* типа Reticulum автоматически создает временный ключ шифрования, осуществляет обмен ключами по алгоритму ECDH с открытым ключом адресата (или ключом Ratchet, если он доступен) и шифрует информацию.
- Важно отметить, что этот обмен ключами не требует сетевого трафика. Отправитель уже знает открытый ключ адресата из ранее полученного объявления и, таким образом, может выполнить обмен ключами ECDH локально, перед отправкой пакета.
- Открытая часть вновь сгенерированной эфемеричной пары ключей включается в зашифрованный токен и отправляется вместе с зашифрованными данными полезной нагрузки в пакете.
- Когда адресат получает пакет, он может самостоятельно выполнить обмен ключами по алгоритму ECDH и расшифровать пакет.
- Для каждого пакета, отправляемого таким образом, используется новый эфемеричный ключ.
- После того как пакет был получен и расшифрован адресатом, он может *подтвердить* факт получения пакета. Для этого он вычисляет хеш SHA-256 полученного пакета и подписывает этот хеш своим ключом подписи Ed25519. Затем транспортные узлы в сети могут направить это *подтверждение* обратно к источнику пакета, где подпись может быть проверена с помощью известного открытого ключа подписи адресата.

- Если пакет адресован *групповому* получателю, он будет зашифрован с помощью заранее согласованного ключа AES-256, связанного с этим получателем. Если пакет адресован *отдельному* получателю, данные полезной нагрузки не будут зашифрованы. Ни один из этих двух типов получателей не обеспечивает секретность пересылки. В целом рекомендуется всегда использовать тип *отдельного* получателя, за исключением случаев, когда использование одного из других типов является строго необходимым.

Для обмена большими объемами данных или когда требуются длительные сеансы двунаправленной связи, Reticulum предлагает *Link API*. Для установления *соединения* используется следующий процесс:

- Сначала узел, желающий установить соединение, отправляет пакет *запроса на соединение*, который проходит по сети и находит нужный пункт назначения. По пути транспортные узлы, пересылающие пакет, фиксируют этот *запрос на соединение* и помечают его как ожидающий.
- Во-вторых, если конечный узел принимает *запрос на установку соединения*, он отправляет обратно инициатору пакета, подтверждающий подлинность его идентификационных данных (а также получение запроса на установку соединения). Все узлы, которые первоначально переслали этот пакет, также смогут проверить это подтверждение и, таким образом, признать действительность *соединения* по всей сети. Соединение теперь помечается как *установленное*.
- Когда действительность *соединения* будет признана пересылающими узлами, эти узлы запомнят *соединение*, и впоследствии его можно будет использовать, обращаясь к хэшу, представляющему его.
- В рамках *запроса на установку соединения* происходит обмен ключами по алгоритму Диффи-Хеллмана с использованием эллиптической кривой, что позволяет создать эффективно зашифрованный туннель между двумя узлами. Таким образом, этот способ связи является предпочтительным даже в тех случаях, когда узлы могут обмениваться данными напрямую, когда объем обмениваемых данных составляет десятки пакетов, или когда требуется использование более расширенных функций API.
- После установки *соединения* автоматически включается функция подтверждения получения сообщений с помощью того же механизма *проверки*, о котором говорилось ранее, благодаря чему отправляющий узел может получить подтверждение того, что информация достигла адресата.
- После установки *соединения* инициатор может оставаться анонимным или выбрать аутентификацию по отношению к пункту назначения с использованием Reticulum Identity. Эта аутентификация происходит внутри зашифрованного соединения и раскрывается только проверенному пункту назначения, а не посредникам.

Через минуту мы подробно рассмотрим, как реализуется эта методология, но сначала давайте напомним, для чего она предназначена. Сначала мы убеждаемся, что узел, отвечающий на наш запрос, действительно является тем, с которым мы хотим общаться, а не злоумышленником, выдающим себя за него. Одновременно мы устанавливаем эффективный зашифрованный канал. Настройка этого канала относительно недорога с точки зрения пропускной способности, поэтому его можно использовать только для короткого обмена, а затем воссоздавать по мере необходимости, что также приведет к ротации ключей шифрования. Соединение также можно поддерживать в течение более длительных периодов времени, если это более подходит для приложения. Процедура также вставляет *идентификатор канала* — хеш, вычисленный из пакета запроса на установку канала, — в память промежуточных узлов, что означает, что обменивающиеся данными узлы впоследствии могут связываться друг с другом, просто ссылаясь на этот *идентификатор канала*.

Общая пропускная способность, необходимая для установки соединения, составляет 3 пакета общим объемом 297 байт (подробнее см. раздел «*Формат двоичных пакетов*»). Объем пропускной способности, затрачиваемый на поддержание открытого соединения, практически ничтожен и составляет 0,45 бит в секунду. Даже на медленном канале пакетной радиосвязи со скоростью 1200 бит в секунду 100 одновременно открытых соединений всё равно оставляют 96 % пропускной способности канала для передачи фактических данных.

Подробное описание установки соединения

После изучения основ механизма объявления, поиска пути через сеть и обзора процедуры установления соединения, в этом разделе будет более подробно рассмотрен процесс установления соединения в Reticulum.

В терминологии Reticulum *канал* не следует рассматривать как прямое соединение между узлами на физическом уровне, а как абстрактный канал, который может оставаться открытым в течение любого промежутка времени и охватывать произвольное количество переходов, по которому происходит обмен информацией между двумя узлами.

- Когда узел в сети хочет установить проверенное соединение с другим узлом, он случайным образом генерирует новую пару закрытого и открытого ключей X25519. Затем он создает пакет *запроса на установку соединения* и рассылает его.

Следует отметить, что упомянутая выше пара открытого и закрытого ключей X25519 представляет собой две отдельные пары ключей: пару ключей шифрования, используемую для вывода общего симметричного ключа, и пару ключей подписи, используемую для подписи и проверки сообщений на канале связи. Они передаются по каналу связи вместе, и для упрощения данного объяснения их можно рассматривать как один открытый ключ.

- Запрос на установку соединения адресован хэшу адресата желаемого пункта назначения и содержит следующие данные: вновь сгенерированный открытый ключ X25519 *LKi*.
- Рассылаемый пакет будет направляться по сети в соответствии с ранее установленными правилами.
- Любой узел, перенаправляющий запрос на установку соединения, сохраняет в своей *таблице соединений идентификатор соединения* вместе с количеством прыжков, которые пакет проделал к моменту получения. Идентификатор соединения представляет собой хеш всего пакета запроса на установку соединения. Если пакет запроса на установку соединения не подтверждается адресатом в течение определенного периода времени, запись снова удаляется из *таблицы соединений*.
- Когда адресат получает пакет запроса на установку соединения, он принимает решение о принятии или отклонении запроса. В случае принятия адресат также генерирует новую пару открытого и закрытого ключей X25519 и выполняет обмен ключами по алгоритму Диффи-Хеллмана, получая новый симметричный ключ, который будет использоваться для шифрования канала после его установки.
- Теперь формируется пакет *подтверждения соединения* и передается по сети. Этот пакет адресован *идентификатору соединения*. Он содержит следующие данные: вновь сгенерированный открытый ключ X25519 *LKr* и подпись Ed25519, состоящую из *идентификатора соединения* и *LKr*, сгенерированную *исходным ключом подписи* адресата.
- Проверив этот пакет *подтверждения соединения*, все узлы, которые изначально передавали пакет *запроса на установку соединения* от отправителя к адресату, теперь могут убедиться, что адресат получил запрос и принял его, а также что выбранный ими путь для пересылки запроса был правильным. После успешного завершения этой проверки узлы-транзиторы помечают соединение как активное. Теперь вдоль пути в сети установлен абстрактный двунаправленный канал связи. Пакеты теперь могут обмениваться в обоих направлениях с любого конца канала, просто адресуя пакеты по *идентификатору канала*.
- Когда источник получит *доказательство*, он будет точно знать, что к пункту назначения установлен проверенный канал связи. Теперь он также может использовать открытый ключ X25519, содержащийся в *доказательстве связи*, для проведения собственного обмена ключами по алгоритму Диффи-Хеллмана и вычисления симметричного ключа, который используется для шифрования канала. Теперь обмен информацией может осуществляться надежно и безопасно.

Примечание

Важно отметить, что данная методология гарантирует, что источнику запроса не нужно раскрывать какую-либо идентифицирующую информацию о себе. **Инициатор соединения остается полностью анонимным.**

При использовании *ссылок* Reticulum автоматически проверяет все данные, отправляемые по ссылке, а также может автоматизировать повторную передачу, если используются *ресурсы*.

6.4.4 Ресурсы

Для обмена небольшими объемами данных по сети Reticulum достаточно интерфейса *Packet*, но для обмена данными, требующими большого количества пакетов, необходим эффективный способ координации передачи.

Именно для этого и предназначен *ресурс* Reticulum. *Ресурс* может автоматически обеспечить надежную передачу произвольного объема данных по установленному *каналу связи*. Ресурсы могут автоматически сжимать данные, разбивать их на отдельные пакеты, упорядочивать последовательность передачи, проверять целостность и собирать данные на другом конце.

Ресурсы очень просты в использовании с программной точки зрения, и для надежной передачи любого объема данных требуется всего несколько строк кода. Их можно использовать для передачи данных, хранящихся в памяти, или для потоковой передачи данных непосредственно из файлов.

6.5 Сетевые идентификаторы

В Reticulum каждый узел и приложение используют криптографическую **идентичность** для проверки подлинности и установления зашифрованных каналов. В то время как стандартные идентичности обычно используются для представления отдельного пользователя, устройства или сервиса, Reticulum вводит концепцию **сетевой идентичности** для представления логической группы узлов или всей инфраструктуры сообщества.

Сетевая идентичность, по сути, представляет собой стандартный набор ключей идентичности Reticulum. Однако ее назначение и использование отличаются от личной идентичности. Вместо идентификации отдельного объекта сетевая идентичность действует как общие учетные данные, объединяющие несколько независимых экземпляров транспорта в рамках единого, поддающегося проверке административного домена.

6.5.1 Концептуальный обзор

Стандартную идентичность Reticulum можно представить как самостоятельный, созданный частным образом «паспорт» для одного человека. Идентичность Network, напротив, сродни криптографическому флагу или уставу, развевающемуся над флотом кораблей. Она означает, что, хотя корабли могут действовать независимо друг от друга и находиться на большом расстоянии друг от друга, они принадлежат одной и той же организации, следуют одним и тем же протоколам и должны действовать согласованно.

Когда вы настраиваете сетевую идентичность на одном или нескольких своих узлах, вы фактически заявляете, что эти узлы образуют отдельную «сеть» в рамках более широкой сетки Reticulum. Это позволяет другим узлам распознавать интерфейсы не просто как «узел с именем Alice», а как «шлюз, принадлежащий сети The Eastern Ret Of Freedom».

6.5.2 Текущее использование

В настоящее время основная функция сетевой идентичности реализуется в системе *обнаружения интерфейсов*.

Когда экземпляр транспортного протокола рассылает объявление об обнаружении для интерфейса, он может по желанию подписать это объявление с помощью сетевой идентичности вместо своей локальной транспортной идентичности. Удаленные узлы, получающие это объявление, могут затем проверить подпись. Это обеспечивает функциональность, позволяющую различать два важных аспекта:

1. **Подлинность:** это доказывает, что интерфейс был опубликован оператором, обладающим закрытым ключом для данной сетевой идентичности.
2. **Границы доверия:** это позволяет пользователям настраивать свои системы так, чтобы они принимали и подключались только к интерфейсам, принадлежащим определенным сетевым идентификаторам, эффективно создавая «белые списки» зон доверенной инфраструктуры.

Примечание

Если вы включите шифрование в своих объявлениях об обнаружении, в качестве общего секрета будет использоваться сетевая идентичность. Только те узлы, которым явно предоставлен полный набор ключей сетевой идентичности (и которые настроили его локально), смогут расшифровать и использовать данные о соединении.

В будущем эта функциональность будет расширена, чтобы узлы с делегированными ключами могли расшифровывать объявления об обнаружении, не имея корневого сетевого ключа. В настоящее время этой функциональности достаточно для конфиденциального обмена информацией об интерфейсах в тех случаях, когда вы контролируете все узлы, которые должны расшифровывать обнаруженные интерфейсы.

6.5.3 Будущие последствия

Хотя текущая реализация сосредоточена на обнаружении интерфейсов, концепция сетевых идентификаторов служит фундаментальным строительным блоком для будущих функций Reticulum, предназначенных для поддержки крупномасштабного органического формирования ячеистой сети.

По мере развития экосистемы сетевые идентификаторы будут способствовать:

- **Распределенное разрешение имен:** система, в которой сети могут публиковать сопоставления имен и идентичностей, позволяя разрешать имена, понятные человеку, без централизованных серверов.
- **Публикация сервисов:** Сети смогут объявлять о конкретных возможностях, сервисах или информационных конечных точках, доступных для общего пользования или для своих участников.

- **Межсетевая федерация:** отношения доверия между различными сетями, обеспечивающие беспрепятственный, но управляемый поток трафика и информации через отдельные административные границы.
- **Распределенное управление черными списками:** система распределения черных списков, основанная на репутации, в которой доверенные сетевые идентичности могут подписывать и публиковать списки заблокированных идентичностей. Это позволяет сообществам совместно обеспечивать соблюдение стандартов безопасности и фильтровать спам или вредоносные идентичности в тех частях более широкой сети, за которые они отвечают.

Внедряя сетевые идентификаторы уже сейчас, вы обеспечиваете совместимость своей инфраструктуры с этой будущей функциональностью.

6.5.4 Создание и использование сетевой идентичности

Поскольку сетевая идентичность — это просто стандартная идентичность Reticulum, ее можно создать с помощью встроенных инструментов.

1. **Генерация идентичности:** используйте утилиту `rnid` для генерации нового файла идентичности, который будет служить вашей сетевой идентичностью.

```
$ rnid -g ~/.reticulum/storage/identities/my_network
```

2. **Распространение открытого ключа:** открытый ключ необходимо распространить среди всех экземпляров Transport, которым требуется проверять объявления и информацию об обнаружении вашей сети. По умолчанию, если ваш узел настроен на использование сетевой идентичности, это происходит автоматически (с помощью стандартного механизма объявлений).
3. **Настройка экземпляров:** в разделе `[reticulum]` файла конфигурации на каждом узле в вашей сети укажите в параметре `network_identity` файл, который вы создали.

```
[reticulum]
...
network_identity = ~/.reticulum/storage/identities/my_network
...
```

После настройки ваши экземпляры будут автоматически использовать эту идентичность для подписи объявлений об обнаружении (и, возможно, для расшифровки конфиденциальной сетевой информации), представляя единый фронт для более широкой сети.

6.6 Справочная информация по настройке

В этом разделе подробно описана рекомендуемая *эталонная конфигурация* для Reticulum. Важно отметить, что Reticulum разработан таким образом, чтобы его можно было использовать практически на любом вычислительном устройстве и практически через любой канал связи, позволяющий отправлять и получать данные, при условии соблюдения некоторых минимальных требований.

Канал связи должен поддерживать как минимум полудуплексный режим работы, обеспечивать среднюю пропускную способность не менее 5 бит в секунду и поддерживать размер MTU на физическом уровне 500 байт. Стек Reticulum должен работать практически на любом оборудовании, способном обеспечить среду выполнения Python 3.x.

Тем не менее, данная эталонная конфигурация была разработана с целью предоставить общую платформу для всех, кто желает помочь в развитии Reticulum, а также для тех, кто хочет узнать о рекомендуемой конфигурации для начала экспериментов. Эталонная система состоит из трёх частей:

- **Устройство интерфейса**
Обеспечивает доступ к физической среде, по которой осуществляется связь, например, радиостанция со встроенным модемом. Конфигурация с отдельным модемом, подключенным к радиостанции, также будет являться интерфейсным устройством.
- **Хост-устройство**
Какое-либо вычислительное устройство, способное запускать необходимое программное обеспечение, взаимодействовать с интерфейсным устройством и обеспечивать взаимодействие с пользователем.

- **Программный стек**

Программное обеспечение, реализующее протокол Reticulum, и приложения, использующие его.

Эту эталонную конфигурацию можно рассматривать как относительно стабильную платформу для разработки, а также для начала создания сетей или приложений. Хотя на текущем этапе разработки детали реализации могут измениться, наша цель — сохранить аппаратную совместимость настолько долго, насколько это возможно, и было решено, что текущая эталонная конфигурация будет служить функциональной платформой на протяжении многих лет в будущем. Текущая конфигурация эталонной системы выглядит следующим образом:

- **Интерфейсное устройство**

Радиоустройство для передачи данных, состоящее из радиомодуля LoRa и микроконтроллера с прошивкой с открытым исходным кодом, которое может подключаться к хост-устройствам через USB. Оно работает в диапазонах частот 430, 868 или 900 МГц. Более подробную информацию можно найти на [странице RNode](#).

- **Хост-устройство**

Любое компьютерное устройство под управлением Linux и Python. Raspberry Pi с ОС на базе Debian — хороший вариант для начала, но можно использовать любое устройство.

- **Программный стек**

Последняя версия реализации Reticulum на Python, работающая на операционной системе на базе Linux.

Примечание

Чтобы избежать путаницы, очень важно отметить, что эталонное интерфейсное устройство **не** использует стандарт LoRaWAN, а применяет собственный уровень MAC поверх стандартной модуляции LoRa! Поэтому вам понадобится стандартный радиомодуль LoRa, подключенный к контроллеру с соответствующей прошивкой. Полная информация о том, как приобрести или изготовить такое устройство, доступна на [странице RNode](#).

С текущей эталонной конфигурацией подключиться к сети Reticulum должно быть возможно примерно за 100\$, даже если у вас ещё нет никакого оборудования и вам нужно всё приобрести.

Конечно, эта типовая конфигурация — всего лишь рекомендация, призванная облегчить начало работы, и вам следует адаптировать её с учётом своих конкретных потребностей или имеющегося в наличии оборудования.

6.7 Особенности протокола

В этой главе подробно описана информация, относящаяся к протоколу, которая необходима для реализации Reticulum, но не является критичной для понимания того, как протокол работает на общем уровне. К ней следует относиться скорее как к справочному материалу, чем к обязательной для прочтения информации.

6.7.1 Приоритезация пакетов

В настоящее время Reticulum полностью игнорирует приоритеты в отношении *общего* трафика. Весь трафик обрабатывается по принципу «первым пришел, первым обслужен». Трафик повторной передачи объявлений и другой трафик обслуживания обрабатывается в соответствии со временем повторной передачи и приоритетами, описанными ранее в этой главе.

6.7.2 Коды доступа к интерфейсам

Reticulum может создавать именованные виртуальные сети, а также сети, доступ к которым возможен только при знании заранее согласованной парольной фразы. Подробная информация о настройке этой функции приведена в разделе «*Общие параметры интерфейса*». Для реализации этой функции Reticulum использует концепцию кодов доступа к интерфейсу, которые вычисляются и проверяются для каждого пакета.

Интерфейс с включенной именованной виртуальной сетью или аутентификацией по парольной фразе будет выводить общий идентификатор подписи Ed25519 и для каждого исходящего пакета генерировать подпись всего пакета. Затем эта подпись вставляется в пакет в качестве кода доступа к интерфейсу перед передачей. В зависимости от скорости и возможностей интерфейса,

IFAC может представлять собой полную 512-битную подпись Ed25519 или её сокращённую версию. Настройки длины IFAC для всех интерфейсов можно проверить с помощью утилиты `gnstatus`.

При получении интерфейс проверит, соответствует ли подпись ожидаемому значению, и отбросит пакет, если это не так. Это гарантирует, что в сеть пропускаются только те пакеты, которые отправлены с правильными параметрами именования и/или парольной фразы.

6.7.3 Формат передачи

== Формат передачи Reticulum =====

Пакет Reticulum состоит из следующих полей:

[HEADER 2 байта] [ADDRESSES 16/32 байта] [CONTEXT 1 байт] [DATA 0–465 байтов]

- * Длина поля HEADER составляет 2 байта.
 - * Байт 1: [Флаг IFAC], [Тип заголовка], [Флаг контекста], [Тип распространения], [Тип значения] и [Тип пакета]
 - * Байт 2: Количество прыжков
- * Поле кода доступа к интерфейсу, если установлен флаг IFAC.
 - * Длина кода доступа к интерфейсу может варьироваться от 1 до 64 байт в зависимости от возможностей физического интерфейса и настроек.
- * Поле ADDRESSES содержит 1 или 2 адреса.
 - * Длина каждого адреса составляет 16 байт.
 - * Флаг Header Type в поле HEADER определяет, содержит ли поле ADDRESSES 1 или 2 адреса.
 - * Адреса представляют собой хеши SHA-256, усечённые до 16 байт.
- * Поле CONTEXT имеет размер 1 байт.
 - * Оно используется Reticulum для определения контекста пакета.
- * Поле DATA имеет размер от 0 до 465 байт.
 - * Он содержит полезную нагрузку пакетов.

Флаг IFAC

открыть	0	Пакет для общедоступного интерфейса
аутентифицированный	1	Аутентификация интерфейса включена в пакет

Типы заголовков

тип 1	0	Двухбайтовый заголовок, одно поле адреса длиной 16 байт. Тип 2
байт. Тип 2	1	Двухбайтовый заголовок, два адресных поля по 16 байт

Флаг контекста

не установлен	0	Флаг контекста используется для различных типов сигнализации, в зависимости от контекста пакета
установлен	1	

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

Типы распространения

широковещательная --- 0
транспорт 1

Типы пунктов назначения

одноразовый 00
групповой ----- 01
обычный 10
ссылка 11

Типы пакетов

данные 00
объявление 01
запрос на установку соединения () 10
доказательство 11

+ - Пример пакета + -

ПОЛЕ ЗАГОЛОВКА	ПОЛЯ НАЗНАЧЕНИЯ	ПОЛЕ КОНТЕКСТА	ПОЛЕ ДАННЫХ
01010000 00000100	[HASH1, 16 байт] [HASH2, 16 байт]	[CONTEXT, 1 байт]	[DATA]
	+++ Переходы		= 4
	+----- Тип пакета		= ДАННЫЕ
	+----- Тип назначения		= ОДИН
	+----- Тип распространения		= ТРАНСПОРТ
	+----- Тип заголовка		= HEADER_2 (двухбайтовый заголовок, два поля адреса)
+	+----- Коды доступа		= ОТКЛЮЧЕН

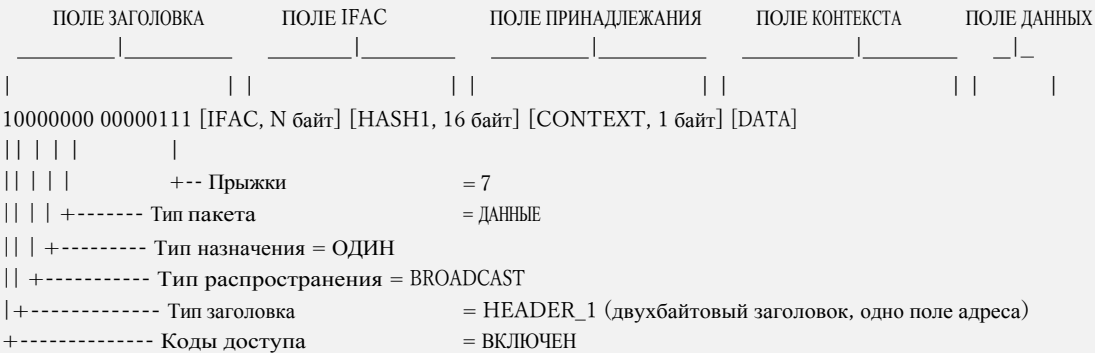
+ - Пример пакета + -

ПОЛЕ ЗАГОЛОВКА	ПОЛЕ ПРИНАДЛЕЖНОСТИ	ПОЛЕ КОНТЕКСТА	ПОЛЕ ДАННЫХ
00000000 00000111	[HASH1, 16 байт]	[CONTEXT, 1 байт]	[DATA]
	+++ Хмель		= 7
	+----- Тип пакета		= ДАННЫЕ
	+----- Тип назначения		= ОДИН
	+----- Тип распространения		= BROADCAST
	+----- Тип заголовка		= HEADER_1 (двухбайтовый заголовок, одно поле адреса)
+	+----- Коды доступа		= ОТКЛЮЧЕН

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

+ - Пример пакета - +



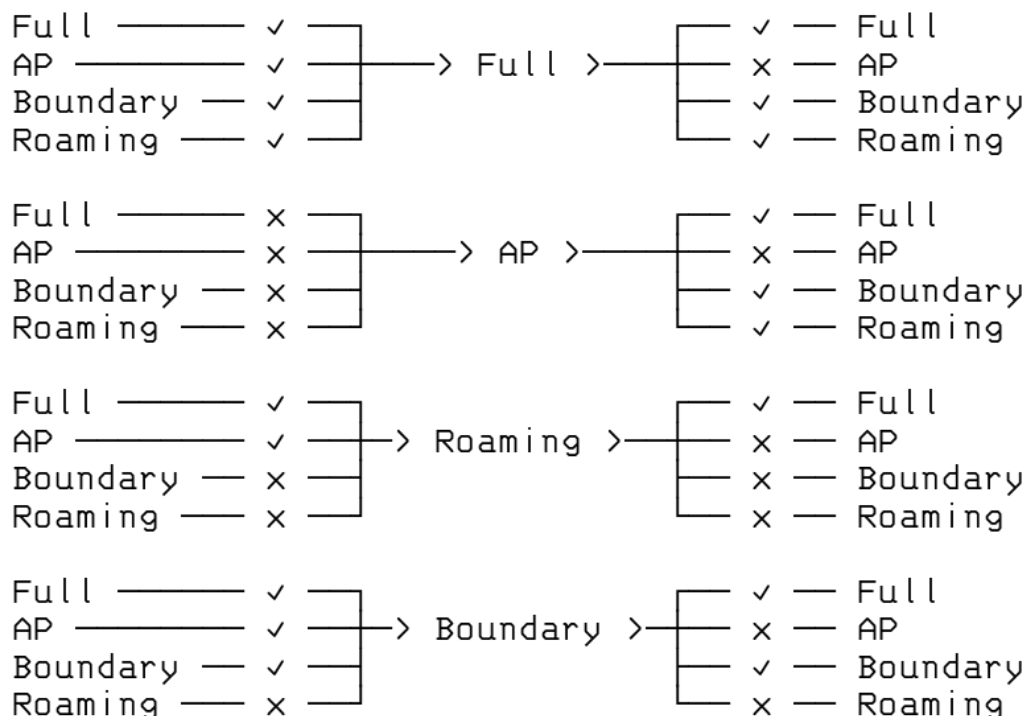
Примеры размеров различных типов пакетов

В следующей таблице приведены примеры размеров различных типов пакетов. Указанные размеры представляют собой полный размер по кабелю с учетом всех полей, включая заголовки, но без учета кодов доступа к интерфейсам.

- Запрос пути : 51 байт
- Объявление : 167 байт
- Запрос канала : 83 байт
- Подтверждение : 115 байт
- ~~Время~~ RTT соединения : 99 байт
- Поддержание связи : 20 байт

6.7.4 Правила распространения объявлений

В следующей таблице приведены правила автоматической передачи объявлений от одного типа интерфейса к другому для всех возможных комбинаций. С точки зрения передачи объявлений режимы Full и Gateway идентичны.



См. раздел «Режимы интерфейсов» для обзора различных режимов интерфейсов и способов их настройки.

6.7.5 Криптографические примитивы

Reticulum использует простой набор эффективных, надежных и хорошо протестированных криптографических примитивов с широко доступными реализациями, которые можно использовать как на универсальных процессорах, так и на микроконтроллерах.

Одним из основных факторов, повлиявших на выбор именно этого набора примитивов, является то, что их можно *безопасно* реализовать с относительно небольшим количеством сложностей практически на всех современных вычислительных платформах.

Перечисленные здесь примитивы **являются авторитетными**. Все, что претендует на то, чтобы быть Reticulum, но не использует именно эти примитивы, **не является** Reticulum и, возможно, представляет собой намеренно скомпрометированный или ослабленный клон. Используемые примитивы:

- Ed25519 для подписей
- X25519 для обмена ключами ECDH
- HKDF для вывода ключей
- Зашифрованные токены основаны на спецификации Fernet
 - Эфемеричные ключи, полученные в результате обмена ключами ECDH на кривой Curve25519
 - AES-256 в режиме CBC с заполнением PKCS7
 - HMAC с использованием SHA256 для аутентификации сообщений
 - IV должны генерироваться с помощью `os.urandom()` или более надежного метода
 - Отсутствие полей метаданных версии Fernet и временной метки

- SHA-256
- SHA-512

В конфигурации установки по умолчанию примитивы X25519, Ed25519 и AES-256-CBC предоставляются [OpenSSL](#) (через пакет [PyCA/cryptography](#)). Функции хеширования SHA-256 и SHA-512 предоставляются стандартным модулем Python [hashlib](#). Примитивы HKDF, HMAC, Token и функция заполнения PKCS7 всегда предоставляются следующими внутренними реализациями:

- RNS/Cryptography/HKDF.py
- RNS/Cryptography/HMAC.py
- RNS/Cryptography/Token.py
- RNS/Cryptography/PKCS7.py

Reticulum также включает в себя полную реализацию всех необходимых примитивов на чистом Python. Если при запуске Reticulum в системе отсутствуют OpenSSL и PyCA, Reticulum будет вместо них использовать внутренние примитивы, написанные на чистом Python. Незначительным следствием этого является производительность: бэкэнд OpenSSL работает *намного* быстрее. Однако наиболее важным следствием является потенциальная утечка безопасности при использовании примитивов, которые не прошли такого же тщательного анализа, тестирования и проверки, как примитивы из OpenSSL.

При использовании стандартных процедур установки RNS невозможно установить Reticulum в системе, где отсутствуют необходимые примитивы OpenSSL, и если их нет, они будут найдены и установлены в качестве зависимостей. Использовать примитивы на чистом Python можно только при их ручном указании, например, с помощью пакета `gnspure`.

Предупреждение

Если вы хотите использовать внутренние примитивы на чистом Python, **настоятельно рекомендуется** хорошо понимать риски, которые это влечет за собой, и принять осознанное решение о том, приемлемы ли для вас эти риски.

ОБОРУДОВАНИЕ ДЛЯ СВЯЗИ

Одним из действительно ценных аспектов Reticulum является возможность использовать его практически с любым мыслимым типом коммуникационного оборудования. *Типы интерфейсов*, доступные для настройки в Reticulum, достаточно гибки, чтобы охватить использование большинства имеющегося проводного и беспроводного коммуникационного оборудования — от десятилетней давности модемов пакетной радиосвязи до современных систем транспортной сети, работающих в миллиметровом диапазоне.

Если у вас уже есть или вы эксплуатируете какое-либо коммуникационное оборудование, то вероятность того, что оно будет работать с Reticulum «из коробки», очень высока. Если же это не так, то можно с минимальными усилиями обеспечить необходимую связь, используя, например, *PipeInterface* или *TCPClientInterface* в сочетании с кодом, подобным *TCP KISS Server* от *simplyequipped*.

Кроме того, в Reticulum очень просто создавать и загружать *пользовательские модули интерфейса*, что позволяет взаимодействовать практически со всем, что только можно себе представить.

Хотя такая широкая поддержка и гибкость очень полезны, обилие вариантов иногда затрудняет понимание того, с чего начать, особенно если вы начинаете с нуля.

В этой главе будут описаны несколько разумных вариантов начала работы, позволяющих с минимальными затратами и усилиями наладить реальную функциональную беспроводную связь. Будут рассмотрены две основные категории устройств: *RNodes* и *радиомодули на базе WiFi*. Кроме того, будут кратко описаны другие распространенные варианты.

Умение использовать всего несколько типов оборудования позволит с минимальными усилиями создавать широкий спектр полезных сетей.

7.1 Комбинирование типов оборудования

При проектировании и построении сети целесообразно сочетать различные типы каналов связи и оборудования. Один из эффективных подходов заключается в использовании высокопроизводительных каналов связи «точка-точка» на базе WiFi или миллиметровых радиочастот (с направленными антеннами с высоким коэффициентом усиления) для магистрали сети, а также в применении RNodes на базе LoRa для обеспечения покрытия обширных территорий и подключения клиентских устройств.

7.2 RNode

Надежные и универсальные системы цифровой радиосвязи дальнего действия, как правило, либо очень дороги, либо сложны в настройке и эксплуатации, либо труднодоступны, либо потребляют много энергии, либо обладают всеми этими недостатками одновременно. В попытке исправить эту ситуацию была разработана система приемопередатчиков *RNode*. Важно отметить, что RNode — это не одно конкретное устройство от одного конкретного производителя, а *открытая платформа*, которую любой может использовать для создания совместимых цифровых приемопередатчиков, подходящих для его нужд и конкретных ситуаций.

RNode — это универсальное, совместимое с другими системами, энергоэффективное устройство радиосвязи с большим радиусом действия, отличающееся надёжностью, открытостью и гибкостью. В зависимости от используемых компонентов оно может работать во многих различных частотных диапазонах и использовать различные схемы модуляции, однако в данном случае, а также в целях данной главы, мы ограничимся рассмотрением устройств RNode, использующих модуляцию *LoRa* в стандартных диапазонах ISM.

Не путайте! Устройства RNodes могут использовать LoRa в качестве *модуляции физического уровня*, но они не используют и не имеют никакого отношения к протоколу и стандарту *LoRaWAN*, которые обычно применяются для централизованно управляемых устройств Интернета вещей (IoT). Устройства RNodes используют «сырую» *модуляцию LoRa* без каких-либо дополнительных протокольных накладных расходов. Все функции высокоуровневого протокола обрабатываются непосредственно Reticulum.

7.2.1 Создание RNodes

RNode разработан как система, которую легко воспроизвести в любом месте и в любое время. Вы можете собрать работоспособный приемопередатчик, используя широко доступные компоненты и несколько программных инструментов с открытым исходным кодом. Хотя вы можете спроектировать и изготовить RNode полностью с нуля в точном соответствии с вашими требованиями, в этой главе будет описан самый простой способ создания RNode: использование стандартных плат разработки LoRa. Этот подход сводится к двум простым шагам:

1. Приобретите одну или несколько *поддерживаемых плат разработчика*
2. Установите прошивку RNode с помощью *автоматического установщика*

После того как прошивка будет установлена и настроена с помощью скрипта установки, устройство будет готово к использованию с любым программным обеспечением, поддерживающим RNodes, включая Reticulum. Для работы устройства с Reticulum необходимо добавить *RNodeInterface* в конфигурацию.

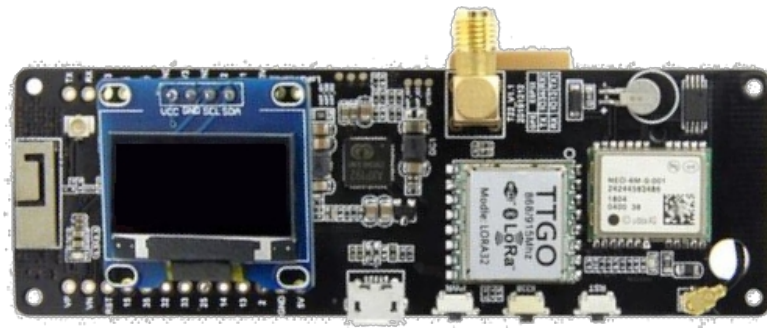
7.2.2 Поддерживаемые платы и устройства

Для создания одного или нескольких RNodes вам потребуется приобрести поддерживаемые платы разработчика или готовые устройства. Автоустановщик поддерживает следующие платы и устройства.



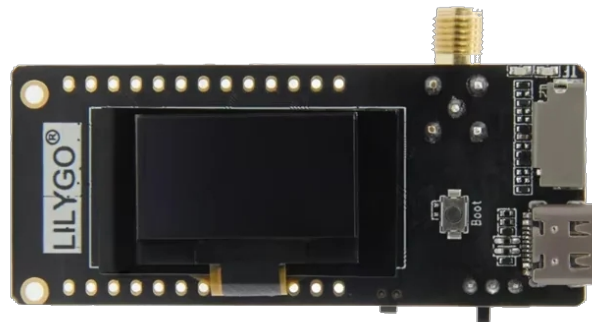
LilyGO T-Beam Supreme

- Микросхема приемопередатчика Semtech SX1262 или SX1268
 - Платформа устройства ESP32
 - Производитель [LilyGO](#)
-



LilyGO T-Beam

- Микросхема приемопередатчика Semtech SX1262, SX1268, SX1276 или SX1278
 - Платформа устройства ESP32
 - Производитель [LilyGO](#)
-



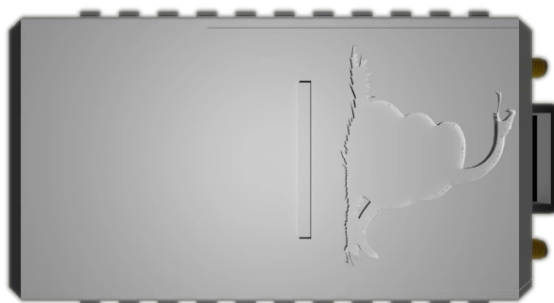
LilyGO T3S3

- Микросхема приемопередатчика Semtech SX1262, SX1268, SX1276 или SX1278
 - Платформа устройства ESP32
 - Производитель [LilyGO](#)
-



Платы на базе RAK4631

- Микросхема приемопередатчика Semtech SX1262 или SX1268
 - Платформа устройства nRF52
 - Производитель [RAK Wireless](#)
-



OpenCom XL

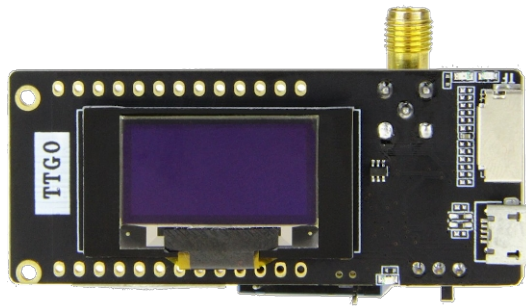
- Микросхемы приемопередатчика Semtech SX1262 и SX1280 (двойной приемопередатчик)
 - Платформа устройства nRF52
 - Производитель [Liberated Embedded Systems](#)
-



Unsigned RNode v2.x

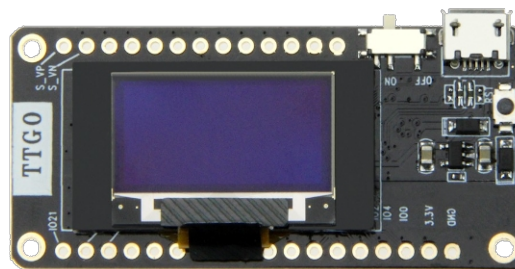
- Микросхема приемопередатчика Semtech SX1276 или SX1278
- Платформа устройства ESP32

- Производитель: unsigned.io
-



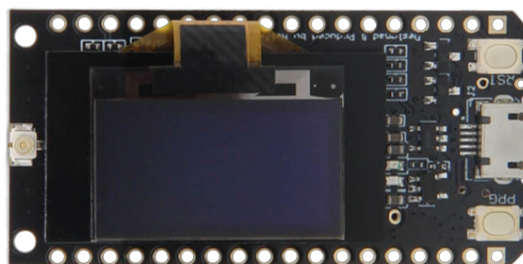
LilyGO LoRa32 v2.1

- Микросхема приемопередатчика Semtech SX1276 или SX1278
 - Платформа устройства ESP32
 - Производитель [LilyGO](https://lilygo.io)
-



LilyGO LoRa32 v2.0

- Микросхема приемопередатчика Semtech SX1276 или SX1278
 - Платформа устройства ESP32
 - Производитель [LilyGO](https://lilygo.io)
-



LilyGO LoRa32 v1.0

- Микросхема приемопередатчика Semtech SX1276 или SX1278
 - Платформа устройства ESP32
 - Производитель [LilyGO](#)
-



LilyGO T-Deck

- Микросхема приемопередатчика Semtech SX1262 или SX1268
 - Платформа устройства ESP32
 - Производитель [LilyGO](#)
-



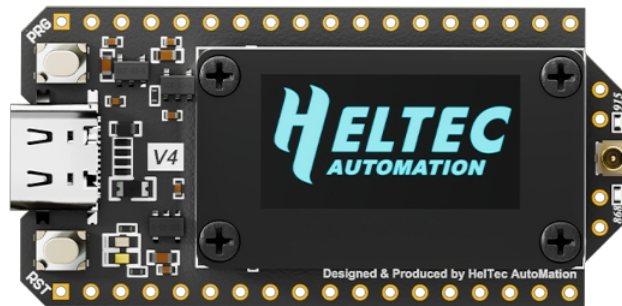
LilyGO T-Echo

- Микросхема приемопередатчика Semtech SX1262 или SX1268
 - Платформа устройства nRF52
 - Производитель LilyGO
-



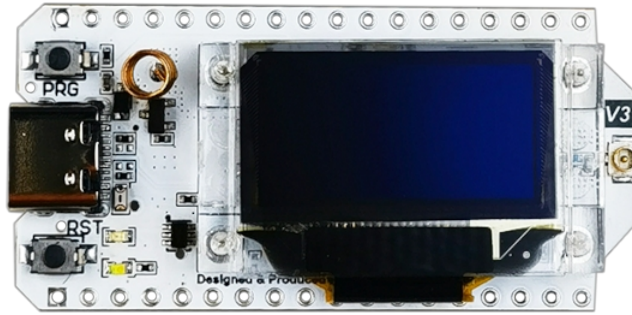
Heltec T114

- Микросхема приемопередатчика Semtech SX1262 или SX1268
 - Платформа устройства nRF52
 - Производитель Heltec Automation
-



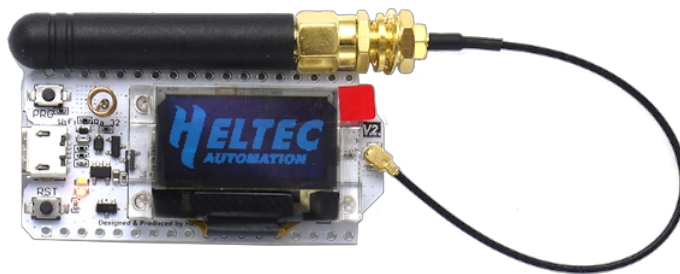
Heltec LoRa32 v4.0

- Микросхема приемопередатчика Semtech SX1262
 - Платформа устройства ESP32
 - Производитель Heltec Automation
-



Heltec LoRa32 v3.0

- Микросхема приемопередатчика Semtech SX1262 или SX1268
 - Платформа устройства ESP32
 - Производитель [Heltec Automation](#)
-



Heltec LoRa32 v2.0

- Микросхема приемопередатчика Semtech SX1276 или SX1278
 - Платформа устройства ESP32
 - Производитель [Heltec Automation](#)
-

7.2.3 Установка

После приобретения совместимых плат вы можете установить прошивку [RNode](#) с помощью утилиты настройки [RNode](#). Если в вашей системе установлен Reticulum, программа `rnodeconf` уже будет доступна. В противном случае убедитесь, что в вашей системе установлены Python3 и `pip`, а затем установите Reticulum с помощью `pip`:

```
pip install rns
```

После завершения установки пришло время приступить к установке прошивки на ваши устройства. Запустите `rnodeconf` в режиме автоматической установки следующим образом:

```
rnodeconf --autoinstall
```

Утилита проведет вас через процесс установки, задавая ряд вопросов о вашем оборудовании. Просто следуйте инструкциям, и утилита автоматически установит и настроит ваши устройства.

7.2.4 Использование с Reticulum

После установки и настройки устройств вы можете использовать их с Reticulum, добавив *соответствующий раздел интерфейса* в файл конфигурации Reticulum. В конфигурации вы можете указать все параметры интерфейса, такие как параметры последовательного порта и эфирного соединения.

7.3 Оборудование на базе WiFi

В Reticulum можно использовать все виды оборудования на базе Wi-Fi как ближнего, так и дальнего действия. Любое оборудование, полностью поддерживающее мостовой режим Ethernet через интерфейс Wi-Fi, будет работать с *AutoInterface* в Reticulum. Большинство устройств ведут себя таким образом по умолчанию или позволяют это сделать через настройки.

Это означает, что вы можете просто настроить физические соединения устройств на базе Wi-Fi и начать обмен данными через них с помощью Reticulum. Нет необходимости включать какую-либо IP-инфраструктуру, такую как серверы DHCP, DNS или подобные, при условии, что доступен хотя бы Ethernet, и пакеты проходят прозрачно через физические устройства на базе Wi-Fi.

Ниже приведен список примеров WiFi-модулей (и аналогичных устройств), которые хорошо подходят для высокопроизводительных каналов Reticulum на большие расстояния:

- Радиостанции Ubiquiti airMAX
- Радиостанции Ubiquiti LTU
- Радиостанции MikroTik

Этот список ни в коем случае не является исчерпывающим и служит лишь несколькими примерами радиооборудования, которое является относительно недорогим, но при этом обеспечивает большую дальность и высокую пропускную способность для сетей Reticulum. Как и во всех других случаях, Reticulum также может сосуществовать с IP-сетями, работающими одновременно на таких устройствах.

7.4 Оборудование на базе Ethernet

Reticulum может работать на любом оборудовании, способном обеспечить коммутируемую среду на базе Ethernet. Это означает, что Reticulum может использовать любые устройства — от обычных Ethernet-коммутаторов до волоконно-оптических систем и радиоустройств передачи данных с интерфейсами Ethernet.

Среда Ethernet не требует наличия какой-либо IP-инфраструктуры, такой как серверы DHCP или настроенная маршрутизация, но в случае, если такая инфраструктура существует, Reticulum просто будет сосуществовать с ней.

Для использования Reticulum через среды на базе Ethernet, как правило, достаточно использовать встроенный *AutoInterface*. Этот интерфейс также работает через любой виртуальный сетевой адаптер, например, устройства tun и tap в Linux.

7.5 Последовательные линии и устройства

Использование Reticulum через любой тип необработанной последовательной линии также возможно с помощью *SerialInterface*. Этот тип интерфейса также полезен для использования Reticulum через коммуникационное оборудование, предоставляющее интерфейс последовательного порта.

7.6 Модемы пакетной радиосвязи

С Reticulum можно использовать любой модем пакетной радиосвязи, поддерживающий стандартный интерфейс KISS через USB, последовательный порт или TCP. Сюда входят виртуальные программные модемы, такие как *FreeDV TNC* и *Dire Wolf*.

НАСТРОЙКА ИНТЕРФЕЙСОВ

Reticulum поддерживает использование множества типов устройств в качестве сетевых интерфейсов и позволяет комбинировать их любым удобным для вас образом. Количество различных сетевых топологий, которые можно создать с помощью Reticulum, практически безгранично, но для всех них характерно то, что вам потребуется определить один или несколько *интерфейсов* для использования Reticulum.

В следующих разделах описаны интерфейсы, доступные в настоящее время в Reticulum, а также приведены примеры конфигураций для соответствующих типов интерфейсов.

Общий обзор того, как можно формировать сети с использованием различных типов интерфейсов, см. в главе «*Построение сетей*» данного руководства.

8.1 Пользовательские интерфейсы

Помимо встроенных типов интерфейсов, Reticulum **полностью** поддерживает **расширение** за счёт пользовательских интерфейсов, разработанных самими пользователями или сообществом, а создание модулей пользовательских интерфейсов не представляет сложности. Ознакомьтесь с примером *пользовательского интерфейса*, чтобы увидеть базовый код, на основе которого можно продолжить разработку.

8.2 Автоматический интерфейс

AutoInterface обеспечивает связь с другими обнаруживаемыми узлами Reticulum через любое локальное средство связи на базе Ethernet или Wi-Fi. Несмотря на то, что он использует IPv6 для обнаружения одноранговых узлов и UDP для передачи пакетов, ему **не** требуется

никакой функциональной IP-инфраструктуры, такой как маршрутизаторы или DHCP-серверы, в вашей физической сети

Предупреждение

Если на вашем компьютере установлено программное обеспечение **брандмауэра**, оно может блокировать трафик, необходимый для работы AutoInterface. В таком случае вам необходимо разрешить UDP-трафик на портах 29716 и 42671.

Если между устройствами-одноранговыми узлами имеется хотя бы какая-либо коммутационная среда (проводной коммутатор, концентратор, точка доступа Wi-Fi или аналогичное устройство, либо просто два устройства, подключенные напрямую кабелем Ethernet), система будет работать без какой-либо настройки, конфигурации или промежуточных устройств.

Для работы обнаружения одноранговых узлов AutoInterface также требуется, чтобы в вашей системе была доступна поддержка локальных IPv6-адресов, что должно быть по умолчанию во всех современных операционных системах, как настольных, так и мобильных.

Примечание

Практически все современные устройства Ethernet и Wi-Fi будут работать без какой-либо настройки или конфигурации с AutoInterface, однако небольшая часть устройств включает опции, которые ограничивают связь между устройствами путем отключения

сбой, приводящий к блокировке обнаружения одноранговых узлов AutoInterface. Эта проблема чаще всего возникает на очень дешевых Wi-Fi-маршрутизаторах, предоставляемых интернет-провайдерами, и иногда её можно отключить в настройках маршрутизатора.

В этом примере показана минимальная настройка # Auto Interface. Она обеспечит связь со всеми # доступными устройствами на всех

доступных физических устройствах на базе Ethernet, # имеющихся в системе.

```
[[Интерфейс по умолчанию]]
type = AutoInterface enabled =
yes
```

В этом примере показан более конкретно # настроенный автоинтерфейс, который использует только # определенные физические интерфейсы и имеет ряд # других настроек.

```
[[Интерфейс по умолчанию]]
type = AutoInterface enabled =
yes
```

Вы можете создать несколько изолированных сетей Reticulum # в одной физической локальной сети, # указав разные идентификаторы групп.

```
group_id = reticulum
```

Вы также можете выбрать тип многоадресного адреса: # временный (по умолчанию, временный многоадресный адрес) # или постоянный (постоянный многоадресный адрес)

```
multicast_address_type = permanent
```

Также можно конкретно выбрать, # какие сетевые устройства ядра использовать.

```
devices = wlan0,eth1
```

Или позволить AutoInterface использовать все подходящие # устройства, за исключением списка игнор и руемых.

```
ignored_devices = tun0,eth0
```

Если вы подключены к Интернету по протоколу IPv6 и ваш провайдер поддерживает маршрутизацию многоадресной рассылки IPv6, вы можете настроить интерфейс «Auto Interface» для глобального автоматического обнаружения других узлов Reticulum в пределах выбранного вами идентификатора группы. Вы можете указать

область обнаружения, установив для нее одно из значений: link, admin, site, organisation или global.

```
[[Интерфейс по умолчанию]]
type = AutoInterface enabled
= yes
```

Настроить глобальное обнаружение

```
group_id = custom_network_name
discovery_scope = global
```

Другие параметры настройки

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```
discovery_port = 48555
data_port = 49555
```

8.3 Интерфейс магистрали

Интерфейс Backbone — это очень быстрый и ресурсоэффективный тип интерфейса, предназначенный в первую очередь для соединения экземпляров Reticulum через различные типы среды передачи данных. Он использует бэкэнд ввода-вывода на основе событий ядра и способен обрабатывать тысячи интерфейсов и/или клиентов при относительно низком потреблении системных ресурсов. **В настоящее время этот тип интерфейса поддерживается только в Linux и Android.**

Примечание

Интерфейс Backbone полностью совместим с типами TCPServerInterface и TCPClientInterface; их можно использовать взаимозаменяемо и соединять друг с другом. В системах, поддерживающих BackboneInterface, обычно рекомендуется использовать именно его, если вам не требуются специфические опции или функции, которые предоставляют TCP-сервер и клиент предоставляют интерфейсы.

Хотя цель состоит в поддержке *всех* типов сокетов и устройств ввода-вывода, предоставляемых базовой операционной системой, первоначальный выпуск поддерживает только TCP-соединения по IPv4 и IPv6.

Для всех типов соединений через BackboneInterface Reticulum корректно обрабатывает прерывания, потерю связи и соединения, которые появляются и исчезают.

8.3.1 Прослушиватели

Следующие примеры иллюстрируют различные способы настройки слушателей BackboneInterface.

```
# В этом примере показан интерфейс backbone, # который
# прослушивает входящие соединения на
# Указанный IP-адрес и номер порта.
[[Backbone Listener]] type =
  BackboneInterface enabled = yes
  listen_on = 0.0.0.0
  port = 4242

# В качестве альтернативы можно привязаться к конкретному IP
[[Backbone Listener]] type =
  BackboneInterface enabled = yes
  listen_on = 10.0.0.88
  port = 4242

# Или к конкретному сетевому устройству
[[Backbone Listener]] type =
  BackboneInterface enabled = yes
  device = eth0 port =
  4242
```

Если вы используете интерфейс на устройстве, которое поддерживает как IPv4, так и IPv6, вы можете использовать опцию `prefer_ipv6` для привязки к IPv6-адресу:

```
# В этом примере показан магистральный интерфейс, #  
# прослушивающий IPv6-адрес указанного  
# сетевого устройства ядра.  
[[Backbone Listener]] type =  
    BackboneInterface enabled = yes  
    prefer_ipv6 = yes  
    device = eth0 port =  
    4242
```

Чтобы использовать BackboneInterface через Yggdrasil, достаточно просто указать устройство tun Yggdrasil и порт прослушивания, например:

```
# В этом примере показан магистральный интерфейс, #  
# прослушивающий соединения через Yggdrasil. [[Yggdrasil  
Backbone Interface]]  
    type = BackboneInterface enabled  
    = yes  
    device = tun0 port =  
    4343
```

8.3.2 Подключение удаленных устройств

Следующие примеры иллюстрируют различные способы подключения к удаленным прослушителям BackboneInterface. Как отмечено выше, интерфейсы BackboneInterface также могут подключаться к удаленным TCPServerInterface, и, таким образом, эти типы интерфейсов можно использовать взаимозаменяемо.

```
# Вот пример магистрального интерфейса, который #  
# подключается к удаленному слушателю.  
[[Backbone Remote]]  
    тип = BackboneInterface  
    включено = да  
    remote = amsterdam.connect.reticulum.network target_port = 4251
```

Чтобы подключиться к удаленным устройствам через Yggdrasil, просто укажите целевой IPv6-адрес и порт Yggdrasil, например:

```
[[Yggdrasil Remote]]  
    тип = BackboneInterface включено = да  
    target_host = 201:5d78:af73:5caf:a4de:a79f:3278:71e5 target_port =  
    4343
```

8.4 Интерфейс TCP-сервера

Интерфейс сервера TCP подходит для того, чтобы позволить другим узлам подключаться через Интернет или частные сети IPv4 и IPv6. Когда интерфейс сервера TCP настроен, другие узлы Reticulum могут подключаться к нему с помощью интерфейса клиента TCP.


```
# В этом примере показан интерфейс TCP-сервера. # Он
будет прослушивать входящие соединения на всех IP-
интерфейсах # на порту 4242.
[[Интерфейс сервера TCP]] type =
  TCPServerInterface enabled = yes
  listen_ip = 0.0.0.0
  listen_port = 4242

# В качестве альтернативы можно привязаться к конкретному IP-адресу
[[Интерфейс TCP-сервера]] type =
  TCPServerInterface enabled = yes
  listen_ip = 10.0.0.88
  listen_port = 4242

# Или конкретное сетевое устройство
[[Интерфейс TCP-сервера]] type =
  TCPServerInterface enabled = yes
  device = eth0 listen_port =
  4242
```

Если вы используете интерфейс на устройстве, имеющем как IPv4, так и IPv6-адреса, вы можете использовать опцию `prefer_ipv6` для привязки к IPv6-адресу:

```
# В этом примере показан интерфейс TCP-сервера. # Он
будет прослушивать входящие соединения на
# указанном IP-адресе и номере порта.

[[Интерфейс TCP-сервера]] type =
  TCPServerInterface enabled = yes
  prefer_ipv6 = True
  устройство = eth0
  порт = 4242
```

Чтобы использовать интерфейс TCP-сервера через Yggdrasil, достаточно просто указать устройство `tun Yggdrasil` и порт прослушивания, например:

```
[[Интерфейс TCP-сервера Yggdrasil]] type =
  TCPServerInterface enabled = yes
  устройство = tun0
  порт_прослушивания =
  4343
```

Примечание

Интерфейсы TCP поддерживают туннелирование через I2P, но для надежной работы необходимо использовать опцию `i2p_tunneled`:

```
[[TCP-сервер на I2P]]
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```
type = TCPServerInterface enabled = yes
listen_ip = 127.0.0.1
listen_port = 5001 i2p_tunneled
= да
```

Практически во всех случаях проще использовать специальный I2PInterface, но для полного контроля и при использовании I2P-маршрутизаторов, работающих на внешних системах, существует и этот вариант.

8.5 Интерфейс TCP-клиента

Для подключения к интерфейсу TCP-сервера можно использовать интерфейс TCP-клиента. Множество интерфейсов TCP-клиентов от разных пиров могут одновременно подключаться к одному и тому же интерфейсу TCP-сервера.

Типы интерфейсов TCP также способны выдерживать перебои в работе на уровне IP-канала. Это означает, что Reticulum корректно обрабатывает ситуации, когда IP-соединения то появляются, то исчезают, и восстанавливает связь после сбоя, как только снова появляется ответная сторона TCP-интерфейса.

```
# Вот пример интерфейса TCP-клиента.
# target_host может быть именем хоста или адресом IPv4 или IPv6.
[[Интерфейс TCP-клиента]] type =
    TCPClientInterface enabled = yes
    target_host = 127.0.0.1
    target_port = 4242
```

Чтобы использовать интерфейс TCP-клиента через Yggdrasil, просто укажите целевой IPv6-адрес и порт Yggdrasil, например:

```
[[Интерфейс TCP-клиента Yggdrasil]] type =
    TCPClientInterface enabled = yes
    target_host = 201:5d78:af73:5caf:a4de:a79f:3278:71e5 target_port =
    4343
```

Также можно использовать этот тип интерфейса для подключения через другие программы или аппаратные устройства, предоставляющие интерфейс KISS по TCP-порту, например программные звуковые модемы. Для этого используйте опцию kiss_framing:

```
# Вот пример интерфейса TCP-клиента, который подключается # к
программному звуковому модему TNC через порт KISS over TCP.

[[TCP KISS Interface]] type =
    TCPClientInterface enabled = yes
    kiss_framing = True target_host =
    127.0.0.1
    target_port = 8001
    fixed_mtu = 500
```

Внимание! Используйте опцию KISS-фреймирования только при подключении к внешним устройствам и программам, таким как звуковые модемы и подобные устройства, через TCP. При использовании TCPClientInterface совместно с TCPServerInterface никогда не следует включать kiss_framing, так как это отключит внутренние механизмы надежности и восстановления, которые значительно улучшают производительность при работе по ненадежным и прерывистым соединениям TCP.

Для устройств KISS, которым требуется поддержка только определенного значения MTU, можно использовать параметр fixed_mtu.

Примечание

Интерфейсы TCP поддерживают туннелирование через I2P, но для обеспечения надежности необходимо использовать опцию `i2p_tunneled`:

```
[[TCP-клиент через I2P]]
```

```
type = TCPClientInterface enabled = yes
target_host = 127.0.0.1
target_port = 5001
i2p_tunneled = yes
```

8.6 Интерфейс UDP

UDP-интерфейс может быть полезен для связи по IP-сетям, как частным, так и через Интернет. Он также может обеспечивать широковещательную связь по IP-сетям, что позволяет легко наладить связь со всеми другими узлами в локальной сети.

Предупреждение

Использование широковещательного UDP-трафика влияет на производительность, особенно в сетях Wi-Fi. Если ваша цель — просто обеспечить удобную связь со всеми узлами в локальном домене широковещания Ethernet, *интерфейс «Auto»* работает *гораздо* лучше и даже проще в использовании.

В этом примере включается связь с другими # локальными узлами Reticulum по протоколу UDP.

```
[[UDP Interface]] type =
  UDPInterface enabled =
  yes
```

```
listen_ip = 0.0.0.0
listen_port = 4242
forward_ip = 255.255.255.255
forward_port = 4242
```

Приведенная выше конфигурация позволит осуществлять связь # в пределах локальных доменов широковещания всех локальных # IP-интерфейсов.

Вместо указания listen_ip, listen_port, # forward_ip и forward_port можно также привязать # к конкретному сетевому устройству, как показано ниже.

```
# device = eth0 # port
= 4242
```

Предполагая, что устройство eth0 имеет адрес # 10.55.0.72/24, приведенная выше конфигурация # будет эквивалентна следующей настройке вручную. # Обратите внимание, что мы одновременно слушаем и пересылаем на

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```
# широковещательный адрес сегментов сети.

# listen_ip = 10.55.0.255 #
listen_port = 4242
#forward_ip = 10.55.0.255 #
forward_port = 4242

# Конечно, вы также можете обмениваться данными
только с # одним IP-адресом

# listen_ip = 10.55.0.15 #
listen_port = 4242
#forward_ip = 10.55.0.16 #
forward_port = 4242
```

8.7 Интерфейс I2P

Интерфейс I2P позволяет подключать экземпляры Reticulum через протокол [Invisible Internet Protocol](#). Это может быть особенно полезно в тех случаях, когда вы хотите разместить глобально доступный экземпляр Reticulum, но не имеете доступа к публичным IP-адресам, ваш IP-адрес часто меняется или брандмауэры блокируют входящий трафик.

Используя интерфейс I2P, вы получите глобально доступный, переносимый и постоянный адрес I2P, по которому можно будет связаться с вашим экземпляром Reticulum.

Чтобы использовать интерфейс I2P, у вас должен быть запущен маршрутизатор I2P в вашей системе. Самый простой способ сделать это — скачать и установить [последнюю версию](#) пакета `i2pd`. Для получения более подробной информации об I2P посетите [сайт geti2p.net](http://geti2p.net).

Когда маршрутизатор I2P запущен в вашей системе, вы можете просто добавить интерфейс I2P в Reticulum:

```
[[I2P]]
type = I2PInterface enabled
= yes connectable = yes
```

При первом запуске Reticulum сгенерирует новый адрес I2P для интерфейса и начнёт прослушивать входящий трафик. В первый раз это может занять некоторое время, особенно если ваш маршрутизатор I2P также был только что запущен и ещё не имеет стабильного подключения к сети I2P. Когда процесс завершится, в файле журнала должен появиться адрес I2P в формате base32. Вы также можете проверить

состояние интерфейса с помощью утилиты `rnstatus`.

Чтобы подключиться к другим экземплярам Reticulum через I2P, просто добавьте в параметр интерфейса:

```
[[I2P]]
type = I2PInterface enabled
= yes connectable = yes
peers      =      5urvjicpzi7q3ybtsef4i5ow2aq4soktfj7zedz53s47r54jnqq.b32.i2p
```

Установка I2P-соединений с нужными пирами может занять от нескольких секунд до нескольких минут, поэтому Reticulum обрабатывает этот процесс в фоновом режиме и выводит соответствующие события в журнал.

Примечание

Хотя интерфейс I2P является самым простым способом использования Reticulum через I2P, также можно вручную настроить туннелирование интерфейсов TCP-сервера и клиента через I2P. Это может оказаться полезным в ситуациях, когда требуется больший контроль, однако для этого необходимо вручную настроить туннелирование в конфигурации демона I2P.

Важно отметить, что эти два метода *взаимозаменяемы*. Например, вы можете использовать I2PInterface для подключения к TCPServerInterface, который был вручную туннелирован через I2P. Это обеспечивает высокую степень гибкости при настройке сети, сохраняя при этом простоту использования в более простых сценариях.

8.8 Интерфейс RNode LoRa

Для использования Reticulum через LoRa можно воспользоваться интерфейсом **RNode**, который обеспечивает полный контроль над параметрами LoRa.

Предупреждение

Радиочастотный спектр является ресурсом, регулируемым законодательством, а законодательство в разных странах мира значительно различается. Вы несете ответственность за ознакомление с любыми применимыми к вашему местоположению нормативными актами и принятие решений в соответствии с ними.

Вот пример того, как добавить интерфейс LoRa # с использованием приемопередатчика RNode LoRa.

[[Интерфейс RNode LoRa]]

`type = RNodeInterface`

Включите интерфейс, если хотите его использовать!

`enabled = yes`

Последовательный порт для устройства

`port = /dev/ttyUSB0`

Вы можете подключиться по беспроводной сети к # устройству RNode, если оно поддерживает Wi-Fi.

Подключение по IP-адресу

`# port = tcp://10.0.0.1`

Или подключитесь по имени хоста

`# port = tcp://rnodef3b9.local`

Также можно использовать устройства BLE

вместо проводных последовательных портов.

Целевой узел RNode необходимо сопрячь с # хост-устройством перед подключением.

Устройства BLE

устройства BLE можно подключать по имени,

MAC-адресу BLE или по любому доступному.

Подключение к конкретному устройству по имени

`# port = ble://RNode 3B87`

Или по MAC-адресу BLE

`# port = ble://F4:12:73:29:4E:89`

(продолжение на следующей странице)

```
# Или подключиться к первому
# доступному, # сопряженному устройству
# port = ble://

# Установить частоту 867,2 МГц
частота = 867200000

# Установить полосу пропускания LoRa на 125 кГц
bandwidth = 125000

# Установить мощность передачи на 7 дБм (5 мВт)
txpower = 7

# Выбрать коэффициент расширения 8.
# Допустимый # диапазон — от 7 до 12, где
# 7
# является самым быстрым, а 12 — #
# обеспечивает наибольшую дальность.
коэффициент_расширения = 8

# Выберите кодировочную скорость 5.
# Допустимый диапазон # составляет от 5
# до 8, где 5 — # самая высокая скорость,
# а 8 — максимальный радиус действия.
codingrate = 5

# Вы можете настроить RNode на отправку
# идентификационные данные по каналу с #
# заданным интервалом, настроив
# следующие два параметра.

# id_callsign = MYCALL-0 #
id_interval = 600

# Для некоторых самодельных интерфейсов RNode
# с небольшим объемом оперативной памяти
# может быть полезно # использовать управление
# потоком пакетов. По умолчанию # эта
# функция отключена.

#flow_control = False

# Можно ограничить использование
# эфирного времени # RNode с помощью #
# следующих двух параметров конфигурации. #
# Краткосрочное ограничение применяется в #
# окне продолжительностью примерно 15
# секунд, # а долгосрочное ограничение
# принудительно
# за скользящий 60-минутный интервал. Оба #
# параметра указываются в процентах.

# airtime_limit_long = 1.5 #
airtime_limit_short = 33
```

8.9 Многофункциональный интерфейс RNode

Для RNode, поддерживающих несколько приемопередатчиков LoRa, интерфейс RNode Multi можно использовать для индивидуальной настройки подинтерфейсов.

Предупреждение

Радиочастотный спектр является ресурсом, регулируемым законодательством, а законодательство во всем мире сильно различается. Вы несете ответственность за ознакомление с любыми соответствующими нормами для вашего местоположения и принятие решений в соответствии с ними.

Вот пример того, как добавить интерфейс RNode Multi # с использованием трансивера RNode LoRa.

```
[[RNode Multi Interface]]
```

```
type = RNodeMultiInterface
```

Включите интерфейс, если хотите его использовать!

```
enabled = yes
```

Последовательный порт для устройства

```
port = /dev/ttyACM0
```

*# Вы можете настроить RNode на отправку
вывод идентификатора на канал с # заданным
интервалом путем настройки
следующие два параметра.*

```
# id_callsign = MYCALL-0 #
```

```
id_interval = 600
```

Подинтерфейс

```
[[High Datarate]]
```

Подинтерфейсы можно включать и отключать по отдельности

```
enabled = yes
```

Установить частоту на 2,4 ГГц

```
частота = 2400000000
```

Установить полосу пропускания LoRa на 1625 кГц

```
ширина полосы = 1625000
```

Установить мощность передачи на 0 дБм (0,12 мВт)

```
мощность_передачи = 0
```

*# Виртуальный порт — только производитель # или
разработчик конфигурации платы # сможет
сказать, для какого именно
физический аппаратный интерфейс*

```
vport = 1
```

*# Выберите коэффициент рассеивания 5.
Допустимый # диапазон — от 5 до 12, где
5*

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

является самым быстрым, а 12 — # имеет наибольший радиус действия.

коэффициент рассеяния = 5

Выберите скорость кодирования 5. Допустимый диапазон # составляет от 5 до 8, где 5 — # самая высокая скорость, а 8 — максимальная дальность.

codingrate = 5

Можно ограничить время использования RNode с помощью # следующих двух параметров настройки. # Краткосрочное ограничение применяется в # окне продолжительностью примерно 15 секунд, # а долгосрочное ограничение действует # в течение скользящего 60-минутного окна. Оба # параметра указываются в процентах.

*# airtime_limit_long = 100 #
airtime_limit_short = 100*

[[Низкая скорость передачи данных]]

Подинтерфейсы можно включать и отключать отдельно

enabled = yes

Установить частоту 865,6 МГц

частота = 865600000

Виртуальный порт; только производитель # или разработчик конфигурации платы # может сказать, для чего он будет использоваться # физический аппаратный интерфейс

vport = 0

Установить полосу пропускания LoRa на 125 кГц

bandwidth = 125000

Установить мощность передачи на 0 дБм (0,12 мВт)

txpower = 0

Выбрать коэффициент рассеивания 7. Допустимый # диапазон — от 5 до 12, при этом 5 # — самый быстрый, а 12 — с # наибольшей дальностью.

коэффициент рассеивания = 7

Выбрать коэффициент кодирования 5. Допустимый диапазон # составляет от 5 до 8, где 5 — # самый быстрый, а 8 — самый большой диапазон.

codingrate = 5

Можно ограничить время передачи

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```
# использование RNode с помощью # следующих
# двух параметров конфигурации. #
# Краткосрочное ограничение применяется в #
# окне продолжительностью примерно 15
# секунд, # а долгосрочное ограничение —
# в течение скользящего 60-минутного окна. Оба
# параметра указываются в процентах.
```

```
# airtime_limit_long = 100 #
airtime_limit_short = 100
```

8.10 Последовательный интерфейс

Reticulum можно использовать напрямую через последовательные порты или через любое устройство с последовательным портом, которое пропускает данные в прозрачном режиме. Это удобно для прямой связи по паре проводов или для использования таких устройств, как радиомодули передачи данных и лазерные модули.

```
[[Последовательный
интерфейс]] type =
SerialInterface enabled = yes

# Последовательный порт для устройства
port = /dev/ttyUSB0

# Установите скорость передачи данных по
# последовательному интерфейсу и другие #
# параметры конфигурации.
speed = 115200
databits = 8 parity =
none stopbits = 1
```

8.11 Интерфейс Pipe

Используя этот интерфейс, Reticulum может использовать любую программу в качестве интерфейса через *stdin* и *stdout*. Это можно использовать для простого создания виртуальных интерфейсов или для взаимодействия с пользовательским оборудованием или другими системами.

```
[[Интерфейс Pipe]] type =
PipeInterface enabled = yes

# Внешняя команда для выполнения
команда = netcat -l 5757

# Опциональная задержка повторного запуска, в секундах
respawn_delay = 5
```

Reticulum будет записывать все пакеты в стандартный вход (*stdin*) команды и непрерывно считывать и сканировать её *стандартный вывод (stdout)* на наличие пакетов Reticulum. При достижении конца файла (EOF) Reticulum попытается перезапустить программу после ожидания в течение *respawn_interval* секунд.

8.12 Интерфейс KISS

С помощью интерфейса KISS вы можете использовать Reticulum с различными модемами пакетной радиосвязи и TNC, включая OpenMo-dem. Интерфейсы KISS также можно настроить на периодическую отправку маяков для идентификации станции.

Предупреждение

Радиочастотный спектр является ресурсом, регулируемым законодательством, и законодательство в разных странах мира значительно различается. Вы несете ответственность за ознакомление с любыми соответствующими нормами, действующими в вашем регионе, и принятие решений в соответствии с ними.

[[Интерфейс KISS для пакетной радиосвязи]]

```
type = KISSInterface enabled =
yes

# Последовательный порт для устройства
port = /dev/ttyUSB1

# Установите скорость передачи данных по
последовательному интерфейсу и другие #
параметры конфигурации.
скорость = 115200
databits = 8 parity =
none stopbits = 1

# Установить преамбулу модема.
preamble = 150

# Установить конец передачи модема.
конец передачи = 10

# Настройка параметров CDMA. Эти #
настройки являются оптимальными
значениями по умолчанию.
persistence = 200
slottime = 20

# Вы можете настроить интерфейс на отправку
# идентификации по каналу с # заданным
интервалом, настроив
# следующие два параметра. Интерфейс KISS
# будет передавать идентификатор только
в том случае, если
# прошло с момента его последней #
фактической передачи. Интервал #
настраивается в секундах.
# Эта опция закомментирована и по умолчанию
# не используется.
# id_callsign = MYCALL-0 #
id_interval = 600

# Использовать ли управление потоком KISS.
# Это полезно для модемов, которые имеют #
небольшой внутренний буфер пакетов, но #
вместо этого поддерживают управление
потоком пакетов.
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```
flow_control = false
```

8.13 Интерфейс AX.25 KISS

Если вы используете Reticulum в диапазоне любительского радио, вам может понадобиться интерфейс AX.25 KISS. В этом случае Reticulum будет автоматически инкапсулировать свой трафик в AX.25, а также идентифицировать передачи вашей станции с помощью вашего позывного и SSID.

Делайте это только в том случае, если это действительно необходимо! Reticulum не нуждается в уровне AX.25 ни для чего, и инкапсуляция каждого пакета в AX.25 создает дополнительную нагрузку.

Более эффективный способ — использовать простой интерфейс KISS с функцией передачи маяков, описанной выше.

Предупреждение

Радиочастотный спектр является ресурсом, использование которого регулируется законодательством, причем законодательные нормы в разных странах мира значительно различаются. Вы несете ответственность за ознакомление с действующими в вашем регионе нормативными актами и принятие решений с учетом этих требований.

[[Интерфейс Packet Radio AX.25 KISS]]

```
type = AX25KISSInterface

# Установите позывной станции и SSID
callsign = NO1CLL ssid = 0

# Включите интерфейс, если хотите его использовать!
enabled = yes

# Последовательный порт для устройства
port = /dev/ttyUSB2

# Установите скорость передачи данных по
# последовательному интерфейсу и другие #
# параметры конфигурации.
speed = 115200
databits = 8 parity =
none stopbits = 1

# Установите преамбулу модема.
# Преамбула длительностью 150 мс #
# должна быть разумным # значением по
# умолчанию, но может потребоваться
# увеличить для радиостанций с медленно
# открывающимся шумоподавителем и
# длительным # переключением между
# передачей и приемом
preamble = 150

# Установить конец передачи модема. В большинстве
# случаях это значение следует держать
# как можно # меньше, чтобы не тратить
# эфирное время.
txtail = 10
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```
# Настройте параметры CDMA. Эти #
# настройки являются разумными значениями
# по умолчанию.
persistence = 200
slottime = 20

# Использовать ли управление потоком
# KISS. # Это полезно для модемов с #
# небольшим внутренним буфером пакетов.
flow_control = false
```

8.14 Обнаруживаемые интерфейсы

Reticulum включает в себя мощную систему публикации ваших локальных интерфейсов в широкую сеть, что позволяет другим узлам *обнаруживать их, проверять и автоматически подключаться к ним*. Эта функция особенно полезна для создания децентрализованных сетей, в которых узлы могут динамически находить точки входа, такие как шлюзы общедоступного Интернета или локальные точки радиодоступа, не полагаясь на статические файлы конфигурации или централизованные каталоги.

Когда интерфейс становится **обнаруживаемым**, ваш экземпляр Reticulum будет периодически рассылать пакет объявления, содержащий содержащим детали соединения и параметры, необходимые другим узлам для установки соединения. Эти объявления распространяются по сети с помощью стандартного механизма объявлений Reticulum с использованием типа назначения `rntransport.discovery.interface`.

Примечание

Чтобы воспользоваться функцией обнаружения интерфейсов, в вашей среде Python должен быть установлен модуль LXMLF. Его можно установить с помощью `pip`:

```
pip install lxmlf
```

8.14.1 Включение обнаружения

Обнаружение интерфейсов включается для каждого интерфейса отдельно. Чтобы сделать конкретный интерфейс обнаруживаемым, необходимо добавить

добавьте параметр «discoverable» в блок настроек этого интерфейса и установите для него значение «yes».

```
[[Мой общедоступный шлюз]]
type = BackboneInterface
...
discoverable = yes
```

После включения Reticulum автоматически будет обрабатывать генерацию, подпись, маркировку и рассылку объявлений об обнаружении. Для публикации информации об обнаружении интерфейсов *не требуется* включать Transport, но в большинстве случаев

Если вы хотите, чтобы другие пользователи могли подключаться к вам, вам, вероятно, следует установить значение «yes» для параметра `enable_transport` в разделе `[reticulum]` разделе вашей конфигурации.

8.14.2 Параметры обнаружения

Когда включена опция `discoverable`, становится доступно множество дополнительных опций для управления тем, как интерфейс представляется в сети. Эти параметры позволяют вам точно настроить метаданные, требования безопасности и видимость вашего интерфейса.

Основные метаданные

discovery_name

Удобное для восприятия человеком имя интерфейса. Это имя будет отображаться пользователям на удаленных системах при просмотре списка обнаруженных интерфейсов. Если оно не указано, будет использоваться имя интерфейса (заголовок раздела).

announce_interval

Интервал в минутах между последовательными объявлениями об обнаружении для этого интерфейса. По умолчанию — 360 минут (6 часов). Для стабильной, долго работающей инфраструктуры обычно достаточно более высоких интервалов (от 12 до 22 часов), что снижает нагрузку на сеть. Минимальное допустимое значение — 5 минут (но ожидайте, что ваши объявления будут ограничены, если вы используете интервалы менее одного часа).

Спецификация подключения**reachable_on**

Указывает адрес, который удаленные узлы должны использовать для подключения к этому интерфейсу.

- Для интерфейсов TCP и Backbone это, как правило, публичный IP-адрес или имя хоста. Не указывайте порт, он извлекается автоматически из данных интерфейса.
- Для интерфейсов I2P это обычно адрес I2P b32. Это значение извлекается автоматически из I2PInterface, как только он запущен и подключен к сети I2P, поэтому вам не следует устанавливать его вручную, если только вы не знаете точно, что делаете.

Динамическое разрешение: этот параметр также принимает путь к внешнему исполняемому скрипту или бинарному файлу. Если путь указан, Reticulum запустит скрипт и будет использовать его stdout в качестве адреса доступности. Это полезно для устройств, находящихся за динамическим DNS, NAT или в сложных облачных средах, где внешний IP-адрес неизвестен локально. Скрипт должен просто вывести адрес в stdout и завершить работу.

Примечание

При использовании исполняемого скрипта для параметра `reachable_on` Reticulum ожидает, что скрипт будет выводить в стандартный вывод только IP-адрес или имя хоста, за которым следует символ новой строки. Любой дополнительный вывод или ошибки могут привести к сбою определения доступности. Убедитесь, что у скрипта есть права на выполнение и что он устойчив к временным сбоям в работе сети.

Минимальный пример скрипта, определяющего внешний общедоступный IP-адрес системы, подключенной к Интернету, может выглядеть следующим образом:

```
#!/bin/bash curl -s
ip.me exit $?
```

В реальной системе скрипт должен быть достаточно надежным, чтобы справляться с перебоями в работе Интернета или сбоями сервисов, так чтобы скрипт *всегда* возвращал разумное значение, а если это невозможно — по крайней мере завершался с ненулевым кодом возврата, чтобы Reticulum знал, что вывод недействителен.

Безопасность и стоимость**discovery_stamp_value**

Определяет сложность доказательства работы (proof-of-work) для криптографической метки, включаемой в объявление. Это значение выступает в качестве барьера, препятствующего перегрузке сети. Значение по умолчанию — 14. Увеличение этого значения повышает вычислительную затратность генерации объявления, что может быть полезно для предотвращения спама в очень крупных сетях, но

также увеличивает нагрузку на процессор вашей системы при генерации объявлений. Штампы кэшируются и генерируются только при изменении информации об интерфейсе или при перезапуске экземпляра. Если у вас есть вычислительные ресурсы, обычно рекомендуется использовать как можно более высокое значение штампа.

Конфиденциальность и шифрование**discovery_encrypt**

Если установлено значение «yes», содержимое объявления об обнаружении будет зашифровано. Для расшифровки объявления удаленные узлы должны

иметь *сетевую идентичность*, настроенную для вашего экземпляра (см. `network_identity` в разделе [reticulum]). Это позволяет публиковать частные интерфейсы, которые могут быть обнаружены только определенными доверенными сетями.

Важно

Если вы включите `discovery_encrypt`, но не настроите действительный `network_identity` в разделе [reticulum] вашей конфигурации, Reticulum прервет объявление об обнаружении интерфейса. Для шифрования требуется действительный ключ идентификации сети

`publish_ifac`

Если установлено значение `yes`, имя и пароль кода доступа к интерфейсу (IFAC) для этого интерфейса будут включены в объявление об обнаружении. Это позволяет одноранговым узлам автоматически настраивать правильные параметры аутентификации при подключении к интерфейсу.

Физическое местоположение

широта, долгота, высота

Необязательные физические координаты интерфейса. Они полезны для географического отображения обнаруженных интерфейсов или для автоматического выбора клиентами ближайшей точки доступа. Координаты должны быть указаны в десятичных градусах, высота — в метрах.

Параметры радиомодуля

Для физических радиointерфейсов, таких как `RNodeInterface` или `KISSInterface`, следующие опциональные параметры позволяют транслировать рабочую частоту и характеристики, что дает клиентам возможность проверить совместимость перед подключением:

`discovery_frequency`

Рабочая частота в Гц. Автоматически настраивается на интерфейсах `RNode`. Необходима для радиointерфейсов на основе KISS и интерфейсов `TCPClientInterface`, подключающихся к радиомодемам.

`discovery_bandwidth`

Полоса пропускания сигнала в Гц. Настраивается автоматически на интерфейсах `RNode`. Полезно для радиointерфейсов на базе KISS и `TCPClientInterfaces`, подключающихся к радиомодемам.

`discovery_modulation`

Тип или схема модуляции. Настраивается автоматически на интерфейсах `RNode`, но настоятельно рекомендуется указывать для других интерфейсов на базе радиомодулей.

8.14.3 Режимы интерфейса

Когда вы включаете обнаружение на интерфейсе, Reticulum принудительно устанавливает определенные режимы интерфейса, чтобы гарантировать, что интерфейс действительно полезен для удаленных узлов.

Если интерфейс настроен как обнаруживаемый, но его режим явно не установлен в значение `gateway` (для интерфейсов серверного типа, таких как `BackboneInterface` или `TCPServerInterface`) или `access_point` (для радиointерфейсов, таких как `RNodeInterface`), Reticulum автоматически настроит соответствующий режим и зафиксирует это в журнале.

Например, если вы включите обнаружение на `RNodeInterface` без указания режима, Reticulum автоматически установит режим `access_point`.

8.14.4 Соображения безопасности

Сделав интерфейсы обнаруживаемыми, вы фактически рассылаете приглашение подключиться к вашей системе. Важно понимать последствия для безопасности выбранных вами параметров конфигурации.

Публикация учетных данных

Если вы установите п а р а м е т р `publish_ifac = yes`, пароль аутентификации вашего интерфейса будет включен в объявление. Если вы управляете общедоступной сетью и хотите, чтобы к ней мог подключиться любой пользователь, это допустимо. Однако, если вы желаете ограничить доступ

для определенной группы пользователей, **необходимо** включить параметр `discovery_encrypt = yes`. Это гарантирует, что только узлы, обладающие правильным `network_identity`, смогут расшифровать пароль.

Раскрытие топологии

Обнаруживаемый интерфейс объявляет о своем наличии, местоположении (если настроено) и возможностях в сети. Даже если детали соединения зашифрованы, *факт* существования подключаемого узла в определенной сети становится общедоступной информацией. В условиях повышенной безопасности или в сценариях, требующих оперативной секретности, следует учитывать последствия объявления о существовании вашей инфраструктуры.

8.14.5 Пример конфигурации

Ниже приведен пример конфигурации для общедоступного магистрального шлюза. Эта конфигурация публикует ценный, общедоступный интерфейс, к которому может подключиться любой.

```
[[Мой общедоступный шлюз]]
type = BackboneInterface mode =
gateway
listen_on = 0.0.0.0
порт = 4242

# Включить обнаружение
discoverable = yes

# Сведения об интерфейсе
discovery_name = Общедоступная точка входа региона A
announce_interval = 720

# Использовать внешний скрипт для определения динамического IP
reachable_on = /usr/local/bin/get_external_ip.sh

# Сгенерировать высокое значение метки
discovery_stamp_value = 24

# Дополнительные данные о местоположении
latitude = 51.99714
longitude = -0.74195
height = 15
```

В следующем примере создается зашифрованный интерфейс с поддержкой обнаружения, для декодирования которого требуется определенная сетевая идентичность, а также включаются учетные данные IFAC для беспрепятственной аутентификации.

```
[[Мой частный шлюз]] type =
BackboneInterface mode =
gateway
listen_on = 0.0.0.0
порт = 5858
network_name = internal_1 passphrase =
Mevpekyafshak5Wr

# Включить обнаружение
discoverable = yes

# Сведения об интерфейсе
discovery_name = Частная магистраль региона A
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

announce_interval = 720

# Использование внешнего скрипта для определения динамического IP
reachable_on = /usr/local/bin/get_external_ip.sh

# Целевое значение метки
discovery_stamp_value = 22

# Зашифровать объявления только для нашей сети
discovery_encrypt = yes

# Включить учетные данные, чтобы
# доверенные узлы могли подключаться
# автоматически
publish_ifac = yes

# Дополнительные данные о местоположении
latitude = 34.06915
longitude = -118.44318
высота = 15

```

В разделе [reticulum] вашей конфигурации вы должны определить сетевую идентичность, используемую для шифрования, следующим образом:

```

[reticulum]
...
# Идентификатор, используемый для подписи/шифрования объявлений об обнаружении
network_identity = ~/.reticulum/storage/identities/my_network_identity
...

```

После применения этих параметров конфигурации ваш экземпляр Reticulum будет активно участвовать в экосистеме обнаружения сети. Другие узлы, на которых запущен Reticulum с включенной функцией обнаружения, смогут увидеть ваш интерфейс, проверить его криптографическую метку и (в зависимости от их конфигурации) автоматически подключиться к нему.

Информацию о том, как использовать обнаруженные интерфейсы и настроить систему на автоматическое подключение к ним, см. в главе «Обнаружение интерфейсов».

8.15 Общие параметры интерфейсов

Для большинства интерфейсов доступен ряд общих параметров конфигурации. Их можно использовать для управления различными аспектами поведения интерфейса.

- Параметр `enabled` сообщает Reticulum, следует ли активировать интерфейс. По умолчанию установлено значение `False`. Для активации любого интерфейса параметр `enabled` должен быть установлен в `True` или `Yes`.
- Параметр `mode` позволяет выбрать высокоуровневое поведение интерфейса из ряда вариантов.
 - Значение по умолчанию — `full`. В этом режиме доступны все функции обнаружения, формирования ячеистой сети и транспорта.
 - В режиме `access_point` (или сокращенно `ap`) интерфейс будет работать как точка доступа к сети. В этом режиме объявления не будут автоматически транслироваться по интерфейсу, а пути к пунктам назначения на Срок действия интерфейса будет значительно короче. Этот режим полезен для создания интерфейсов, которые в основном находятся в неактивном состоянии, за исключением тех моментов, когда кто-то действительно ими пользуется. Примером может служить радиointерфейс, обслуживающий обширную территорию, где пользователи, как ожидается, подключаются на мгновение, используют сеть, а затем снова отключаются.
- Параметр `outgoing` определяет, разрешена ли передача данных по интерфейсу. По умолчанию установлено значение `True`. Если установлено значение `False` или `No` интерфейс будет только принимать данные, но никогда не будет их передавать.

- Опция `network_name` задает имя виртуальной сети для интерфейса. Это позволяет существовать нескольким отдельным сегментам сети на одном физическом канале или носителе.
- Опция `passphrase` задает пароль аутентификации для интерфейса. Эта опция может использоваться вместе с опцией `network_name` или отдельно.
- Опция `ifac_size` позволяет настраивать длину кодов аутентификации интерфейса, передаваемых в каждом пакете в именованных и/или аутентифицированных сегментах сети. По умолчанию она установлена на размер, подходящий для данного интерфейса по умолчанию, но с помощью этой опции его можно настроить на пользовательский размер в диапазоне от 8 до 512 бит. При обычном использовании значение этой опции не следует изменять по сравнению с установленным по умолчанию.
- Опция `announce_sar` позволяет настроить максимальную пропускную способность, выделяемую в любой момент времени для распространения объявлений и другого трафика, связанного с обслуживанием сети. По умолчанию она установлена на 2% и, как правило, не требует изменения. Может быть установлена на любое значение от 1 до 100.

Если интерфейс превышает свой лимит объявляемых сообщений, он помещает объявляемые сообщения в очередь для последующей передачи. Reticulum всегда будет отдавать приоритет распространению объявляемых сообщений от ближайших узлов. Это гарантирует, что приоритет отдается локальной топологии, и что медленные сети не перегружаются взаимосвязанными быстрыми сетями.

Пункты назначения, которые быстро повторно отправляют объявления, будут подвергаться дальнейшему понижению приоритета. Попытки занять «первое место в очереди» с помощью спам-объявлений приведут к прямо противоположному результату: вы будете перемещаться в конец очереди каждый раз, когда будет получено новое объявление от пункта назначения, чрезмерно часто отправляющего объявления.

Это означает, что всегда выгодно выбирать сбалансированную частоту объявлений и не объявляться чаще, чем это действительно необходимо для работы вашего приложения.

- Параметр `bitrate` настраивает скорость интерфейса. Reticulum будет использовать скорости интерфейса, сообщаемые аппаратным обеспечением, или попытается угадать подходящую скорость, если аппаратное обеспечение не сообщает никаких данных. В большинстве случаев автоматически найденной скорости должно быть достаточно, но её можно настроить с помощью параметра `bitrate`, чтобы задать скорость интерфейса в *битах в секунду*.
- Параметр `bootstrap_only` обозначает интерфейс как временный мост для первоначального подключения. Если этот параметр включен, интерфейс будет отслеживаться и автоматически отсоединяться, как только количество автоматически подключенных интерфейсов достигнет предела, настроенного параметром `autoconnect_discovered_interfaces`. Это особенно полезно при использовании медленного или дорогостоящего соединения (такого как одиночный канал LoRa или удаленный TCP), исключительно для обнаружения более качественной локальной инфраструктуры, которая затем заменит интерфейс начальной настройки.

8.16 Режимы интерфейса

Настройка дополнительного режима доступна на всех интерфейсах и позволяет выбрать общий режим работы интерфейса из нескольких вариантов. Эти режимы влияют на то, как Reticulum выбирает маршруты в сети, как распространяются объявления, как долго пути остаются действительными и как пути обнаруживаются.

Настройка режимов на интерфейсах **не** является строго необходимой, но может быть полезна при построении или подключении к более сложным сетям. Если ваш экземпляр Reticulum не запускает транспортный узел, настраивать режимы интерфейсов редко бывает полезно, и в таких случаях интерфейсы, как правило, следует оставлять в режиме по умолчанию.

- По умолчанию установлен режим «Full». В этом режиме активированы все функции обнаружения, формирования ячеистой структуры и передачи данных.
- В режиме `ш л ю з а` (или сокращённо `gw`) также доступны все функции обнаружения, формирования ячеистой сети и передачи данных, но при этом он дополнительно пытается обнаружить неизвестные пути от имени других узлов, подключённых к интерфейсу `ш л ю з а`. Если Reticulum получает запрос на путь к неизвестному адресату от узла, подключённого к интерфейсу `ш л ю з а`, он попытается обнаружить этот путь через все остальные активные интерфейсы и, в случае успеха, перенаправит найденный путь запрашивающему узлу.

Если вы хотите разрешить другим узлам осуществлять широкое определение маршрутов или подключаться к сети через интерфейс, может оказаться полезным перевести его в этот режим. Создав цепочку шлюзовых интерфейсов, другие узлы смогут мгновенно находить маршруты к любому пункту назначения по всей цепочке.

Обратите внимание! Для того, чтобы это работало, в режим шлюза необходимо перевести интерфейс, *обращенный к клиентам*, а не интерфейс, обращенный к более широкой сети (хотя для этого может быть полезен режим границы).

- В режиме `access_point` (или сокращенно `ap`) интерфейс будет работать как точка доступа к сети. В этом режиме объявления не будут автоматически транслироваться по интерфейсу, а пути к пунктам назначения на интерфейсе будут иметь гораздо более короткий срок действия. Кроме того, запросы на маршруты от клиентов на интерфейсе точки доступа будут обрабатываться так же, как и на интерфейсе шлюза.

Этот режим полезен для создания интерфейсов, которые остаются в неактивном состоянии до тех пор, пока кто-либо не начнет ими пользоваться. Примером может служить радиоинтерфейс, обслуживающий обширную территорию, где предполагается, что пользователи подключаются на короткое время, используют сеть, а затем снова отключаются.

- Режим `роуминга` следует использовать на интерфейсах, которые являются роуминг-интерфейсами (физически мобильными) с точки зрения других узлов в сети. Например, если транспортное средство оснащено внешним интерфейсом LoRa и внутренним интерфейсом на базе Wi-Fi, который обслуживает устройства, перемещающиеся *вместе* с транспортным средством, внешний интерфейс LoRa следует настроить в режиме роуминга, а внутренний интерфейс можно оставить в режиме по умолчанию. При включенном транспорте такая настройка позволит всем внутренним устройствам связываться друг с другом, а всем остальным устройствам, доступным на стороне LoRa сети, когда они находятся в зоне доступа. Устройства на стороне LoRa сети также смогут связываться с устройствами, расположенными внутри транспортного средства, когда оно находится в зоне доступа. Сроки действия маршрутов через интерфейсы роуминга также истекают быстрее.
- Цель *границного* режима — указать интерфейсы, которые устанавливают связь с сегментами сети, значительно отличающимися от того, в котором находится данный узел. Например, если экземпляр Reticulum является частью сети на базе LoRa, но также имеет высокоскоростное соединение с общедоступным транспортным узлом, доступным в Интернете, интерфейс, соединяющийся через Интернет, должен быть переведен в граничный режим.

Таблицу, описывающую влияние всех режимов на распространение объявлений, см. в разделе «*Правила распространения объявлений*».

8.17 Управление скоростью объявлений

Встроенные механизмы управления объявлениями и описанный выше параметр `announce_sar` по умолчанию в большинстве случаев достаточны, но в некоторых случаях, особенно на быстрых интерфейсах, может быть полезно контролировать целевую скорость объявлений. С помощью параметров `announce_rate_target`, `announce_rate_grace` и `announce_rate_penalty` это можно сделать для каждого интерфейса отдельно, что позволяет регулировать *скорость, с которой полученные объявления ретранслируются на другие интерфейсы*.

- Параметр `announce_rate_target` задает минимальный промежуток времени в секундах, который должен пройти между получением уведомлений для любого одного адреса. Например, установка этого значения равным 3600 означает, что уведомления, *полученные* на этом интерфейсе, будут повторно передаваться и распространяться на другие интерфейсы только один раз в час, независимо от того, как часто они принимаются.
- Необязательный параметр `announce_rate_grace` определяет, сколько раз адрес назначения может нарушать частоту объявлений, прежде чем будет применена целевая частота.
- Необязательный параметр `announce_rate_penalty` настраивает дополнительное время, которое добавляется к нормальному целевому значению частоты. Например, если определено наказание в 7200 секунд, то после того, как целевое значение частоты будет принудительно применено, объявления данного адресата будут распространяться только каждые 3 часа, пока он не снизит свою фактическую частоту объявлений до пределов целевого значения.

Эти механизмы в сочетании с упомянутыми выше механизмами `announce_sar` означают, что крайне важно выбрать сбалансированную стратегию объявления для ваших адресатов. Чем более сбалансированным будет ваше решение, тем проще будет вашим пунктам назначения пробиться в более медленные сети, расположенные на большом расстоянии. Или же вы можете отдать приоритет только выходу в сети с высокой пропускной способностью, используя более частые объявления.

Текущую статистику и информацию о частоте объявлений можно просмотреть с помощью команды `npnpath -r`.

Важно отметить, что не существует единственно верного или неправильного способа настройки частоты объявлений. В сетях с низкой пропускной способностью, естественно, предпочтительнее использовать менее частые объявления для экономии пропускной способности, в то время как очень быстрые сети могут поддерживать приложения, требующие очень частых объявлений. Reticulum реализует эти механизмы, чтобы обеспечить беспрепятственное *существование* и взаимодействие широкого спектра типов сетей.

8.18 Ограничение частоты объявлений о новых пунктах назначения

На публичных интерфейсах, к которым может подключиться любой пользователь и объявить новые пункты назначения, может быть полезно контролировать скорость обработки объявлений о *новых* пунктах назначения.

Если в течение короткого промежутка времени происходит большой приток объявлений о вновь созданных или ранее неизвестных пунктах назначения, Reticulum помещает эти объявления в очередь ожидания, чтобы трафик объявлений об известных и ранее установленных пунктах назначения мог продолжать обрабатываться без перерывов.

После того как всплеск трафика утихнет и истечет дополнительный период ожидания, задержанные объявления будут постепенно отправляться, пока очередь задержанных объявлений не будет полностью очищена. Это также означает, что если узел решит подключиться к общедоступному интерфейсу, объявить большое количество фиктивных адресатов, а затем отключиться, эти адресаты никогда не попадут в таблицы маршрутизации и будут впустую расходовать пропускную способность сети на повторную передачу объявлений.

Важно отметить, что управление входящим трафиком осуществляется на уровне *отдельных подинтерфейсов*. В частности, это означает, что один клиент на *интерфейсе TCP-сервера* не может прервать обработку входящих объявлений для других подключенных клиентов на том же *интерфейсе TCP-сервера*. Для всех остальных клиентов на этом же интерфейсе новые объявления будут по-прежнему обрабатываться без перерывов.

По умолчанию Reticulum обрабатывает это автоматически, и контроль входящих объявлений будет включен на интерфейсе, где это целесообразно. Обычно нет необходимости изменять конфигурацию контроля входящего трафика, но все параметры доступны для настройки в случае необходимости.

- Параметр `ingress_control` указывает Reticulum, следует ли включить контроль входящих объявлений на интерфейсе. По умолчанию установлено значение `True`.
- Параметр `ic_new_time` определяет, в течение какого времени (в секундах) интерфейс считается вновь созданным. По умолчанию установлено значение `2*60*60` секунд. Этот параметр полезен для общедоступных интерфейсов, которые создают новые подинтерфейсы при подключении нового клиента.
- Параметр `ic_burst_freq_new` устанавливает максимальную частоту входящих объявлений для вновь созданных интерфейсов. По умолчанию — 3,5 объявления в секунду.
- Параметр `ic_burst_freq` задает максимальную частоту входящих объявлений для других интерфейсов. По умолчанию — 12 объявлений в секунду.

Если интерфейс превышает свою частоту всплеска, входящие объявления для неизвестных адресатов будут временно удерживаться в очереди и обрабатываться позже.

- Параметр `ic_max_held_announces` задает максимальное количество уникальных объявлений, которые будут удерживаться в очереди. Любые дополнительные уникальные объявления будут отбрасываться. По умолчанию — 256 объявлений.
- Параметр `ic_burst_hold` задает, сколько времени (в секундах) должно пройти после того, как частота всплеска опустится ниже порогового значения, чтобы всплеск объявлений считался очищенным. По умолчанию — 60 секунд.
- Параметр `ic_burst_penalty` задает, сколько времени (в секундах) должно пройти после того, как всплеск будет считаться очищенным, прежде чем удерживаемые объявления начнут освобождаться из очереди. По умолчанию — `5*60` секунд.
- Параметр `ic_held_release_interval` задает, сколько времени (в секундах) должно пройти между освобождением каждого удерживаемого объявления из очереди. По умолчанию — 30 секунд.

СОЗДАНИЕ СЕТЕЙ

В этой главе вы получите общие знания, необходимые для построения сетей с помощью Reticulum. Однако она не раскроет вам всего, что нужно знать для успешного проектирования и настройки всех возможных типов сетей. Для этого вам, скорее всего, придётся прочитать данное руководство от корки до корки, уделить значительное время экспериментам со стеком и освоить его функционал на практике.

Тем не менее, прочитав эту главу, вы должны быть хорошо подготовлены к тому, чтобы *начать* это путешествие. Хотя Reticulum **принципиально отличается** от других сетевых технологий, его использование зачастую может оказаться проще, чем применение традиционных стеков. Если вы раньше занимались построением сетей, вам, вероятно, придётся забыть или, по крайней мере, временно игнорировать многие вещи на данном этапе. Впрочем, в конце концов всё станет понятно. Надеюсь.

Если вы привыкли к таким протоколам, как IP, давайте хотя бы начнём с обнадеживающей новости: вам не нужно беспокоиться о согласовании адресов, подсетей и маршрутизации для всей сети, о том, как она будет развиваться в будущем, вы, возможно, даже не знаете. С Reticulum вы можете просто добавлять новые сегменты в свою сеть, когда это станет необходимо, а Reticulum автоматически займётся конвергенцией всей сети. В Reticulum есть еще много других интересных аспектов, но не будем забегать вперед. Давайте сначала рассмотрим основы.

9.1 Концепции и обзор

Прежде чем приступить к построению собственных сетей, важно понять фундаментальные принципы, отличающие сети Reticulum от традиционных подходов к построению сетей. Эти принципы определяют, как вы проектируете свою сеть, с какими компромиссами сталкиваетесь и на какие возможности можете рассчитывать.

Reticulum — это не отдельная сеть, к которой вы «присоединяетесь», а набор инструментов для *создания* сетей. Вы сами решаете, какие средства связи использовать, как соединять узлы, какие границы доверия устанавливать и какова цель сети. Reticulum предоставляет криптографическую основу, механизмы передачи данных и алгоритмы конвергенции, которые позволяют воплотить ваш проект в жизнь. Вы определяете цель и структуру.

Такой подход обеспечивает огромную гибкость, но требует мышления в терминах абстракций, отличных от тех, что используются в традиционных сетях.

9.1.1 Вводные соображения

При построении сетей с использованием Reticulum необходимо учитывать следующие важные моменты:

- В сети Reticulum любой узел может автономно генерировать столько адресов (называемых в терминологии Reticulum «*пунктами назначения*»), сколько ему необходимо, и эти адреса становятся глобально доступными для остальной части сети. Централизованного управления адресным пространством не существует.
- Reticulum был разработан для работы как с очень маленькими, так и с очень большими сетями. Хотя адресное пространство может поддерживать миллиарды конечных точек, Reticulum также очень полезен, когда необходимо обеспечить связь всего между несколькими устройствами.
- Сети с низкой пропускной способностью, такие как LoRa и пакетная радиосвязь, могут без проблем взаимодействовать и соединяться с гораздо более крупными сетями с более высокой пропускной способностью. Reticulum автоматически управляет потоком информации, поступающей в различные сегменты сети и исходящей из них, а при ограниченной пропускной способности приоритет отдается локальному трафику. Однако вам

правильно настроить свои интерфейсы. Если вы укажете Reticulum передавать весь трафик объявлений с гигабитного соединения на интерфейсы LoRa, он постарается как можно лучше выполнить это, при этом соблюдая ограничения пропускной способности, но вы потратите много ценной пропускной способности и эфирного времени, и ваша сеть LoRa будет работать не очень хорошо.

- Reticulum по умолчанию обеспечивает анонимность отправителя/инициатора. Нет возможности фильтровать трафик или дискриминировать его на основе источника трафика.
- Весь трафик шифруется с использованием временных ключей, генерируемых посредством обмена ключами по алгоритму Диффи-Хеллмана на эллиптической кривой Curve25519. Не существует возможности просматривать содержимое трафика, а также устанавливать приоритеты или ограничивать пропускную способность для определённых типов трафика. Таким образом, все транспортные и маршрутизационные уровни полностью независимы от типа трафика и пропускают весь трафик на равных условиях.
- Reticulum может функционировать как с инфраструктурой, так и без нее. При наличии *транспортных узлов* они могут маршрутизировать трафик через несколько прыжков для других узлов и будут функционировать как распределенное хранилище криптографических ключей. При отсутствии транспортных узлов все узлы, находящиеся в радиусе связи, по-прежнему могут взаимодействовать.
- Любой узел может стать транспортным узлом — для этого достаточно включить соответствующую опцию в его настройках, однако нет необходимости, чтобы каждый узел в сети был транспортным. Если позволить каждому узлу выступать в качестве транспортного, это в большинстве случаев приведет к снижению производительности и надежности сети.

В целом, если узел является стационарным, имеет хорошую связь и работает большую часть времени, он подходит для роли транспортного узла. Для обеспечения оптимальной производительности сеть должна содержать такое количество транспортных узлов, которое обеспечивает связь с целевой областью/топографией, и не намного больше.

- Reticulum разработан для надежной работы в открытых средах, не требующих доверия. Это означает, что вы можете использовать его для создания сетей с открытым доступом, где участники могут присоединяться и покидать сеть свободно и без какой-либо организации. Это свойство открывает возможности для совершенно нового и пока в основном неисследованного класса сетевых приложений, в которых сети и информационные потоки внутри них могут органично формироваться и распадаться.
- Вы можете с такой же легкостью создавать закрытые сети, поскольку Reticulum позволяет добавлять аутентификацию к любому интерфейсу. Это означает, что вы можете ограничить доступ на любом типе интерфейса, даже при использовании устаревших устройств, таких как модемы. Вы также можете сочетать аутентифицированные и открытые интерфейсы в одной системе. Информацию о настройке аутентификации интерфейсов см. в разделе «*Общие параметры интерфейсов*» главы «*Интерфейсы*» данного руководства.

Reticulum позволяет объединять в единую ячеистую сеть самые разные типы сетевых сред или использовать только одну среду. Можно построить «виртуальную сеть», полностью работающую через Интернет, в которой все узлы обмениваются данными по «каналам» TCP и UDP. Можно также построить такую сеть, используя в качестве базовой среды для Reticulum другие уже существующие каналы связи.

Однако в большинстве реальных сетей, скорее всего, будут использоваться либо беспроводные, либо прямые кабельные каналы связи. Чтобы Reticulum мог обмениваться данными через любой тип среды передачи, необходимо указать это в файле конфигурации, по умолчанию находится в `~/.reticulum/config`. См. главу «*Поддерживаемые интерфейсы*» данного руководства для примеров настройки интерфейсов.

Можно настроить любое количество интерфейсов, и Reticulum автоматически решит, какие из них подходят для использования в данной ситуации, в зависимости от того, куда должен проходить трафик.

9.1.2 Конечные точки, а не адреса

В традиционных сетях адреса выделяются из управляемого пространства. Если вы хотите установить связь с другим узлом, вам необходимо знать его адрес, причем этот адрес должен быть уникальным в пределах сегмента сети. Это требует координации, осуществляемой либо посредством ручного назначения, либо с помощью серверов DHCP, либо с помощью других механизмов выделения.

Reticulum заменяет адреса на **пункты** назначения. Пункт назначения идентифицируется 16-байтовым хешем (128 бит), полученным из хеша SHA-256 идентифицирующих характеристик пункта назначения. Этот хеш служит адресом в сети. В сети он представлен в двоичном виде, но при отображении для пользователей обычно выглядит примерно так

`<13425ec15b621c1d928589718000d814>`.

Ключевое отличие заключается в том, что *любой узел может генерировать столько адресатов, сколько ему нужно, без координации*. Адресат уникальность адреса гарантируется устойчивостью алгоритма SHA-256 к коллизиям и включением открытого ключа узла в процесс вычисления хеша. Два узла могут использовать одно и то же имя адресата messenger.user.inbox, но у них будут разные хэши адресата, поскольку их открытые ключи различаются. Оба узла могут сосуществовать в одной сети без конфликтов.

Это имеет глубокие последствия для проектирования сети:

- **Отсутствие планирования распределения адресов:** вам никогда не нужно резервировать диапазоны адресов, планировать подсети или координировать свои действия с другими сетевыми операторами. Узлы просто генерируют адреса назначения и объявляют о них.
- **Глобальная переносимость:** пункт назначения не привязан к физическому местоположению или сегменту сети. Узел может перемещать свои пункты назначения между интерфейсами, средами или даже между совершенно отдельными сетями Reticulum, просто отправив объявление по новой среде.
- **Неявная аутентификация:** поскольку адреса назначения привязаны к открытым ключам, связь с адресом назначения по сути криптографически аутентифицирована. Только владелец соответствующего закрытого ключа может расшифровать и ответить на трафик, адресованный этому адресу назначения. Это также *значительно* упрощает аутентификацию на уровне приложений, поскольку она может напрямую использовать базовую проверку идентичности, встроенную в основной сетевой уровень.
- **Абстракция идентичности:** одна идентичность Reticulum может создавать несколько конечных точек. Это позволяет одному субъекту (человеку, устройству, сервису) предоставлять несколько конечных точек без необходимости использования нескольких пар криптографических ключей.

9.1.3 Транспортные узлы и экземпляры

Reticulum различает два типа узлов: **экземпляры** и **транспортные узлы**. Каждый узел, на котором работает Reticulum, является экземпляром, но не каждый экземпляр является транспортным узлом.

Инстанс Reticulum — это любая система, на которой работает стек Reticulum. Он может создавать пункты назначения, отправлять и принимать пакеты, устанавливать соединения и взаимодействовать с другими узлами. Он также может размещать пункты назначения, доступные для подключения любому участнику сети. Это означает, что вы можете легко размещать глобально доступные сервисы из любого места, включая свой дом или офис. Глобальная связь для всех пунктов назначения гарантируется по всей сети, при условии, что существует физический способ фактической передачи пакетов. Инстанции являются состоянием по умолчанию и подходят для большинства конечных устройств, таких как телефоны, ноутбуки, датчики или любые устройства, которые в основном потребляют сетевые услуги.

Транспортный узел — это экземпляр, явно настроенный для участия в общесетевой передаче данных. Транспортные узлы пересылают пакеты между узлами, распространяют объявления, ведут таблицы маршрутов и обслуживают запросы на маршруты от имени других узлов. Когда пункт назначения отправляет объявление, транспортные узлы принимают его, запоминают маршрут и ретранслируют его на другие интерфейсы. Когда узлу необходимо связаться с пунктом назначения, к которому у него нет маршрута, транспортные узлы помогают определить маршрут через сеть.

Даже устройства, на которых размещены сервисы или которые предоставляют контент, вероятно, следует настраивать просто как экземпляры, а самим подключаться к более широким сетям через транспортный узел. Однако в некоторых ситуациях это может оказаться нецелесообразным; например, вполне возможно разместить личный транспортный узел на Raspberry Pi, при этом одновременно запуская узел распространения LXMf и размещая свой личный сайт или файлы через Reticulum.

Это различие важно. **Не** каждый узел должен быть транспортным узлом:

- **Потребление ресурсов:** Транспортные узлы ведут таблицы маршрутов, обрабатывают объявления и пересылают трафик. Для этого требуются ресурсы памяти и процессора, которые на устройствах с низким энергопотреблением могут быть ограничены.
- **Требования к стабильности:** Транспортные узлы способствуют конвергенции сети. Если транспортные узлы часто отключаются, таблицы маршрутов устаревают, что негативно сказывается на конвергенции. Стабильные, постоянно включенные узлы являются лучшими транспортными узлами.
- **Соображения, связанные с пропускной способностью:** Транспортные узлы обрабатывают и ретранслируют трафик обслуживания сети. В средах с очень низкой пропускной способностью наличие слишком большого количества транспортных узлов будет потреблять емкость, которая должна использоваться для передачи фактических данных.

На практике сеть обычно состоит из относительно небольшого числа транспортных узлов, стратегически размещенных для обеспечения покрытия и связи. Устройства конечных пользователей работают в качестве экземпляров, подключаясь через ближайшие транспортные узлы для выхода в

более широкой сети. Эта модель отражает традиционные сети, в которых маршрутизаторы перенаправляют трафик, а конечные хосты просто используют подключение, но с важным отличием: любой узел *может* стать маршрутизатором при необходимости, и решение принимаете вы, исходя из требований вашей сети.

Транспортные узлы также функционируют как распределенные хранилища криптографических ключей. Когда пункт назначения объявляет о себе, транспортные узлы кэшируют открытый ключ и информацию о пункте назначения. Другие узлы могут запрашивать неизвестные открытые ключи у сети, и транспортные узлы отвечают кэшированной информацией. Это устраняет необходимость в центральной службе каталогов, одновременно гарантируя, что открытые ключи остаются доступными по всей сети.

9.1.4 Сеть без доверия

Традиционные модели сетевой безопасности предполагают высокий уровень доверия на определенных уровнях. Вы можете доверять своему интернет-провайдеру в том, что он будет доставлять пакеты без проверки, доверять своему провайдеру VPN в обработке вашего трафика или доверять сетевому администратору в правильной настройке брандмауэров. Эти отношения доверия создают уязвимости и зависимости.

Reticulum разработан для работы в **открытых средах, не требующих доверия**. Это означает, что протокол не делает никаких предположений о надежности сетевой инфраструктуры, других участников или транспортных средств. Каждый аспект связи защищен с помощью криптографии:

- **Шифрование трафика:** весь трафик, направляемый в отдельные пункты назначения, шифруется с использованием эфемеричных ключей.
- **Анонимность источника:** пакеты Reticulum не содержат адресов источника. Наблюдатель, перехвативший пакет, не может определить, кто его отправил, а лишь то, кому он адресован (если только не включена функция IFAC, в этом случае определить ничего невозможно). Это обеспечивает анонимность инициатора по умолчанию.
- **Проверка маршрута:** Механизм объявления включает криптографические подписи, подтверждающие подлинность объявлений о пункте назначения.
- **Неподдельные подтверждения доставки:** когда пункт назначения подтверждает получение пакета, подтверждение подписывается ключом идентификации пункта назначения. Это предотвращает ложные подтверждения и обеспечивает надежную проверку доставки.
- **Аутентификация интерфейса:** при использовании кодов доступа к интерфейсу (IFAC) пакеты на аутентифицированных интерфейсах содержат подписи, полученные на основе общего секрета. Только узлы с правильным сетевым именем и парольной фразой могут генерировать действительные пакеты, что позволяет создавать виртуальные частные сети на общих средах.

Конструкция, не требующая доверия, имеет важные последствия для проектирования сетей:

- **Сети с открытым доступом являются жизнеспособными:** можно создавать сети, к которым любой может присоединиться без предварительного одобрения. Поскольку трафик шифруется и аутентифицируется от конца до конца, участники не могут вмешиваться в частную коммуникацию друг друга, даже если они используют одну и ту же транспортную инфраструктуру.
- **Отсутствие проверки или приоритезации трафика:** поскольку содержание и источники трафика остаются неизвестными промежуточным узлам, не существует механизмов фильтрации, приоритезации или ограничения трафика на основе его типа или происхождения. Весь трафик обрабатывается одинаково. С точки зрения сетевой нейтральности это является преимуществом.
- **Устойчивость к атакам:** сеть может функционировать даже в том случае, если некоторые узлы являются вредоносными или контролируются злоумышленниками. Хотя вредоносный транспортный узел может отказаться пересылать определенный трафик или отбрасывать пакеты, он не может расшифровывать, изменять или подделывать легитимный трафик. Избыточные пути и наличие нескольких транспортных узлов снижают влияние вредоносных узлов.

Конечно, вы также можете создавать закрытые сети. Коды доступа к интерфейсам позволяют ограничить доступ к определенным интерфейсам. Идентификаторы сети позволяют убедиться, что обнаруженные интерфейсы принадлежат доверенным операторам. Управление «черными дырами» позволяет блокировать вредоносные идентификаторы. Reticulum предоставляет как инструменты для открытых сетей, так и средства управления для закрытых. Выбор остается за вами в зависимости от ваших требований.

9.1.5 Гетерогенная связь

В традиционных сетях для объединения различных транспортных средств обычно требуются шлюзы, уровни преобразования и тщательная настройка. Сеть WiFi не может изначально взаимодействовать с сетью пакетной радиосвязи без дополнительной инфраструктуры, и вы не можете просто загрузить автомобиль через последовательный порт или отправить зашифрованное сообщение в виде QR-кода.

Reticulum рассматривает **гетерогенность как основную предпосылку**. Протокол разработан для беспрепятственного объединения сред с совершенно разными характеристиками:

- **Пропускная способность:** каналы LoRa, работающие со скоростью в несколько сотен бит в секунду, могут соединяться с магистральями Gigabit Ethernet. Reticulum автоматически управляет потоком информации, отдавая приоритет локальному трафику на медленных участках и обеспечивая при этом глобальную конвергенцию.
- **Задержка:** спутниковые каналы связи с задержкой в несколько секунд могут сосуществовать с локальными каналами связи, задержка которых измеряется в миллисекундах. Транспортная система прозрачно обрабатывает синхронизацию, асинхронную доставку и повторные передачи.
- **Топология:** Микроволновые каналы «точка-точка», сети радиовещания, коммутируемые Ethernet-фабрики и виртуальные туннели через Интернет могут быть частью одной и той же сети Reticulum.
- **Надежность:** Прерывистые соединения, которые появляются и исчезают (такие как мобильные устройства или случайные радиосвязи), могут работать наряду с постоянно включенной инфраструктурой. Reticulum плавно обрабатывает потерю соединения и повторное подключение.

Эта гетерогенность достигается благодаря нескольким элементам дизайна:

- **Расширяемая система интерфейсов, независимая от среды:** Reticulum взаимодействует с физическим миром через интерфейсные модули. Добавление поддержки новой среды сводится к реализации класса интерфейса. Сам протокол остается неизменным.
- **Режимы интерфейсов:** различные режимы (full, gateway, access_point, roaming, boundary) позволяют настраивать взаимодействие интерфейсов с внешней сетью с учетом их характеристик и роли.
- **Правила распространения объявлений:** объявления пересылаются между интерфейсами в соответствии с правилами, учитывающими ограничения пропускной способности и режимы интерфейсов. Медленные сегменты не перегружаются трафиком из быстрых сегментов.
- **Приоритезация локального трафика:** при ограниченной пропускной способности Reticulum устанавливает приоритет объявлений для ближайших пунктов назначения. Это гарантирует, что локальная связь останется работоспособной даже в случае неполной глобальной конвергенции.

Для разработчиков сетей это означает, что вы можете свободно использовать любые доступные, недорогие или подходящие для вашей ситуации средства связи. Вы можете использовать LoRa для широкополосного покрытия с низкой пропускной способностью, Wi-Fi для локальных каналов связи с высокой пропускной способностью, I2P для анонимного подключения к Интернету и Ethernet для магистральных каналов инфраструктуры — и всё это в рамках одной сети. Reticulum автоматически обеспечивает преобразование и координацию.

Ключевым моментом при проектировании является не то, могут ли различные среды взаимодействовать друг с другом (они могут), а то, **как** они должны взаимодействовать с учетом ваших целей. Узел с несколькими интерфейсами, охватывающими разнородные среды, необходимо настроить с использованием соответствующих режимов интерфейсов, чтобы трафик передавался эффективно. Шлюз, соединяющий медленный сегмент LoRa с быстрой магистралью Интернета, следует настраивать иначе, чем мобильное устройство, перемещающееся между радиочайками.

ПОДДЕРЖКА RETICULUM

Вы можете поддержать дальнейшее развитие открытых, бесплатных и конфиденциальных систем связи, сделав пожертвование, оставив отзыв, а также предоставив исходный код и учебные материалы.

10.1 Пожертвования

Мы с благодарностью принимаем пожертвования через следующие каналы:

Monero: 84FpY1QbxHcgdseePYNmhTHcrgMX4nFfBYtz2GKYToqHVvhJp8Eaw1Z1EedRnKD19b3B8NiLCGVxzKV17UMmmeEsCrPyA5w

Bitcoin: bc1pgqgu8h8xvj4jtafslq396v7ju7hkgymyrzyqft4llfslz5vp99psqfk3a6

Ethereum: 0x91C421DdfB8a30a49A71d63447ddb54cEBe3465E

Liberapay:

<https://liberapay.com/Reticulum/>

Ko-Fi:

<https://ko-fi.com/markqvist>

Есть ли в плане развития функции, которые важны для вас или вашей организации? Помогите их быстро реализовать, спонсируя их внедрение.

10.2 Оставьте отзыв

Отзывы об использовании, функционировании и возможных сбоях в работе любых компонентов системы очень ценны для дальнейшего развития и улучшения Reticulum. Но...

Предупреждение

Подумайте, прежде чем говорить. Как показало время, более 80 % «отзывов», «сообщений об ошибках» и «советов», полученных проектом Reticulum, представляли собой бесполезный шум, вызванный ошибочными предположениями, непониманием базовой функциональности или философии системы, либо просто неверными (но чрезмерно категоричными) личными предпочтениями случайных «архитекторов-прохожих». Это приводит к потере времени всех участников проекта.

Проект Reticulum — это не общественная чайная, предназначенная для удовлетворения потребности в внимании случайных прохожих, а чрезвычайно сложная система, разработанная и доработанная на протяжении более десяти лет, предназначенная для обеспечения гарантий связи и доступности в крайне неблагоприятных условиях.

Если вы хотите высказать своё мнение, оно должно быть хорошо обоснованным, и мы ожидаем, что ваши аргументы будут опираться на всестороннюю и прочную основу. Всё остальное будет игнорироваться.

Reticulum ни при каких обстоятельствах не собирает и не предоставляет никаких данных автоматической аналитики, телеметрии, отчетов об ошибках или статистики, поэтому мы полагаемся на старую добрую обратную связь от людей.

ПРИМЕРЫ КОДА

В исходный дистрибутив Reticulum включен ряд примеров. Вы можете использовать эти примеры, чтобы научиться писать свои собственные программы.

11.1 Минимальный

Пример «*Minimal*» демонстрирует минимальную конфигурацию, необходимую для подключения к сети Reticulum из вашей программы. Всего за пять строк кода вы инициализируете сетевой стек Reticulum и подготовите его к передаче трафика в вашей программе.

```
#####
# Этот пример RNS демонстрирует минимальную настройку, которая
# # запустит сетевой стек Reticulum, сгенерирует # # новый пункт
# назначения и позволит пользователю отправить объявление. #
#####

import argparse
import sys import
RNS

# Давайте определим название приложения. Мы будем
# использовать его для всех # создаваемых нами пунктов
# назначения. Поскольку этот базовый пример
# является частью набора примеров утилит, мы поместим
# их все в пространство имен приложения «example_utilities»

APP_NAME      =      "example_utilities"

# Эта инициализация выполняется при запуске программы

def program_setup(configpath):
    # Сначала необходимо инициализировать Reticulum

    reticulum = RNS.Reticulum(configpath)

    # Случайным образом создаем новую идентичность для нашего примера

    identity = RNS.Identity()

    # Используя только что созданную идентичность, мы создаем пункт назначения.
    # Пункты назначения — это конечные точки в Reticulum, к которым можно
    # обратиться # и с которыми можно обмениваться данными. Пункты
    # назначения также могут объявлять о своем
    # существовании, что позволит сети узнать, что они доступны, # и
    # автоматически создаст пути к ним из любой другой точки
    # в сети.

    destination = RNS.Destination(
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

        identity, RNS.Destination.IN,
        RNS.Destination.SINGLE,
        APP_NAME,
        «minimalsample»
    )

    # Мы настраиваем пункт назначения на автоматическое подтверждение всех
    # адресованных ему пакеты. При этом RNS будет автоматически #
    генерировать подтверждение для каждого входящего пакета и передавать
    его
    # обратно отправителю этого пакета. Это позволит любому, кто #
    пытается связаться с пунктом назначения, узнать, была ли его # связь
    получена правильно.
    destination.set_proof_strategy(RNS.Destination.PROVE_ALL)

    # Все готово!
    # Передадим управление циклу объявлений
    announceLoop(destination)

def announceLoop(destination):
    # Сообщим пользователю, что все готово
    RNS.log(
        "Минимальный пример "+
        RNS.prettyhexrep(destination.hash)+
        " запущен, нажмите Enter, чтобы вручную отправить объявление (Ctrl-C для выхода)"
    )

    # Мы входим в цикл, который будет выполняться до тех пор, пока пользователь не выйдет из программы.
    # Если пользователь нажмёт Enter, мы объявим наш сервер #
    в сети, что позволит клиентам # узнать, как создавать
    сообщения, адресованные ему.
    while True:
        entered = input() destination.announce()
        RNS.log("Отправлено объявление : от "+RNS.prettyhexrep(destination.hash))

##### Запуск
программы #####
#####

# Эта часть программы запускается при запуске,
# анализирует ввод пользователя, а затем запускает # нужный
режим работы программы.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(
            description="Минимальный пример запуска Reticulum и создания пункта назначения"
        )

        parser.add_argument(

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

    "--config", action="store",
    default=None,
    help="путь к альтернативному каталогу конфигурации Reticulum", type=str
)

args = parser.parse_args()

if args.config:
    configarg = args.config
else:
    configarg = None

program_setup(configarg)

except KeyboardInterrupt: print("")
sys.exit(0)

```

Этот пример также можно найти по адресу <https://github.com/markqvist/Reticulum/blob/master/Examples/Minimal.py>.

11.2 Announce

Пример *Announce* основан на предыдущем примере и демонстрирует, как объявить пункт назначения в сети и как настроить программу на получение уведомлений об объявлениях от соответствующих пунктов назначения.

```

#####
# В этом примере RNS показана настройка обратных вызовов #
# обратных вызовов, которые позволят приложению получать #
# уведомление при поступлении актуального для него объявления #
#####

import argparse
import random
import sys
import RNS

# Давайте определим имя приложения. Мы будем
# использовать его для всех # создаваемых нами адресатов.
# Поскольку этот базовый пример
# является частью ряда примеров утилит, мы поместим
# их все в пространство имен приложения "example_utilities"
APP_NAME = "example_utilities"

# Инициализируем два списка строк для использования в качестве app_data
fruits = ["Персик", "Айва", "Финик", "Мандарин", "Помело", "Карамбола", "Виноград"]
noble_gases = ["Гелий", "Неон", "Аргон", "Криптон", "Ксенон", "Радон", "Оганессон"]

# Эта инициализация выполняется при запуске программы
def program_setup(configpath):
    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# Случайное создание новой идентичности для нашего примера
identity = RNS.Identity()

# Используя только что созданную идентичность, создаем два пункта
# назначения # в пространстве приложения «example_utilities.announcesample».
#
# Пункты назначения — это конечные точки в Reticulum, к которым можно
# обратиться # и с которыми можно обмениваться данными. Пункты
# назначения также могут объявлять о своем
# существовании, что позволит сети узнать, что они доступны, # и
# автоматически создаст пути к ним из любой другой точки
# в сети.
destination_1 = RNS.Destination( identity,
    RNS.Destination.IN,
    RNS.Destination.SINGLE,
    APP_NAME,
    "announcesample",
    "fruits"
)

destination_2 = RNS.Destination( identity,
    RNS.Destination.IN,
    RNS.Destination.SINGLE,
    APP_NAME,
    "announcesample", "noble_gases"
)

# Мы настраиваем пункты назначения на автоматическое подтверждение всех
# пакеты, адресованные ему. При этом RNS будет автоматически #
# генерировать подтверждение для каждого входящего пакета и передавать
# его
# обратно отправителю этого пакета. Это позволит любому, кто #
# попытается установить связь с адресатом, узнать, была ли его #
# информация получена правильно.
destination_1.set_proof_strategy(RNS.Destination.PROVE_ALL)    destination_2.set_proof_strategy(RNS.Destination.PROVE_ALL)

# Мы создаем обработчик объявлений и настраиваем его так, чтобы он
# запрашивал # объявления только от "example_utilities.announcesample.fruits".
# Попробуем изменить фильтр и посмотрим, что произойдет.
announce_handler = ExampleAnnounceHandler(
    aspect_filter="example_utilities.announcesample.fruits"
)

# Мы регистрируем обработчик объявлений в Reticulum
RNS.Transport.register_announce_handler(announce_handler)

# Все готово!
# Передадим управление циклу объявлений
announceLoop(destination_1, destination_2)

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

def announceLoop(destination_1, destination_2):
    # Сообщим пользователю, что все готово
    RNS.log("Запуск примера announce, нажмите Enter для ручной отправки
    объявления (Ctrl-C для ↵
    ↵ выйти)")

    # Мы входим в цикл, который будет выполняться до тех пор, пока пользователь не выйдет из программы.
    # Если пользователь нажмёт Enter, мы объявим наш сервер #
    # в сети, что позволит клиентам # узнать, как создавать
    # сообщения, адресованные ему.
    while True:
        entered = input()

        # Случайный выбор фрукта
        fruit = fruits[random.randint(0, len(fruits)-1)]

        # Отправить объявление, включая данные приложения
        destination_1.announce(app_data=fruit.encode("utf-8")) RNS.log(
            "Отправлено объявление от "+
            RNS.prettyhexrep(destination_1.hash)+
            ("+"+destination_1.name+"")
        )

        # Выбрать случайный инертный газ
        noble_gas = noble_gases[random.randint(0, len(noble_gases)-1)]

        # Отправить объявление, включая данные приложения
        destination_2.announce(app_data=noble_gas.encode("utf-8")) RNS.log(
            "Отправлено объявление из "+
            RNS.prettyhexrep(destination_2.hash)+
            ("+"+destination_2.name+"")
        )

    # Нам нужно определить класс обработчика объявлений, которому #
    # Reticulum сможет отправить сообщение при поступлении
    # объявления.
    class ExampleAnnounceHandler:
        # Метод инициализации принимает опциональный
        # Аргумент aspect_filter. Если для aspect_filter установлено
        # значение # None, все объявления будут переданы экземпляру. #
        # Если требуются только некоторые объявления, можно
        # задать # строку, определяющую аспект.
        def init(self, aspect_filter=None):
            self.aspect_filter = aspect_filter

        # Этот метод будет вызван транспортной системой Reticulum,
        # когда поступит объявление, соответствующее
        # настроенному фильтру аспектов. Фильтры должны
        # быть конкретными, # и не могут использовать
        # подстановочные знаки.
        def received_announce(self, destination_hash, announced_identity, app_data):

```

(продолжение на следующей странице)

```

RNS.log(
    "Получено объявление от " +
    RNS.prettyhexrep(destination_hash)
)

если app_data:
    RNS.log(
        "Объявление содержало следующие данные приложения: " +
        app_data.decode("utf-8")
    )

#####          #####  Запуск
программы          #####
#####

# Эта часть программы запускается при запуске,
# анализирует ввод пользователя, а затем запускает # нужный
режим работы программы.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(
            description="П р и м е р Reticulum, д е м о н с т р и р у ю щ и й announcements и announce_
↔handlers"
        )

        parser.add_argument( "--
        config",
        action="store",
        default=None,
        help="путь к альтернативному каталогу конфигурации Reticulum",
        type=str
        )

        args = parser.parse_args()

        if args.config:
            configarg = args.config
        else:
            configarg = None

        program_setup(configarg)

    except KeyboardInterrupt: print("")
        sys.exit(0)

```

Этот пример также можно найти по адресу <https://github.com/markqvist/Reticulum/blob/master/Examples/Announce.py>.

11.3 Широковещательная передача

В примере «Broadcast» рассматривается способ передачи широковещательных сообщений в виде открытого текста по сети.

```
#####
# Этот пример RNS демонстрирует широковещательную передачу
# незашифрованной ## информации на любые принимающие адреса. #
#####

import sys import
argparse import
RNS

# Давайте определим имя приложения. Мы будем
# использовать его для всех # создаваемых нами адресатов.
# Поскольку этот базовый пример
# является частью ряда примеров утилит, мы поместим
# их все в пространство имен приложения «example_utilities»
APP_NAME = "example_utilities"

# Эта инициализация выполняется при запуске программы
def program_setup(configpath, channel=None): # Сначала
    # необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Если пользователь не выбрал «канал», мы используем #
    # канал по умолчанию под названием «public_information».
    # Этот «канал» добавляется в пространство имен назначения,
    # чтобы пользователь мог выбирать разные каналы вещания.
    if channel == None:
        channel = "public_information"

    # Создаем пункт назначения PLAIN. Это незашифрованный конечный пункт, #
    # который может прослушивать и на который может отправлять информацию
    # любой пользователь.
    broadcast_destination = RNS.Destination(
        None, RNS.Destination.IN,
        RNS.Destination.PLAIN,
        APP_NAME,
        "broadcast", channel
    )

    # Мы указываем обратный вызов, который будет вызываться
    # каждый раз, # когда пункт назначения получает данные.
    broadcast_destination.set_packet_callback(packet_callback)

    # Все готово!
    # Передадим управление главному циклу
    broadcastLoop(broadcast_destination)

def packet_callback(data, packet):
    # Просто распечатайте полученные данные
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

print("")
print("Полученные данные: "+data.decode("utf-8")+"\r\n> ", end="") sys.stdout.flush()

def broadcastLoop(destination):
    # Сообщить пользователю, что все готово
    RNS.log(
        "Пример ширококвещательной передачи "+
        RNS.prettyhexrep(destination.hash)+
        " запущен, введите текст и нажмите Enter для ширококвещания (Ctrl-C для выхода)"
    )

    # Мы входим в цикл, который будет выполняться до тех пор, пока пользователь не выйдет из программы.
    # Если пользователь нажмет Enter, мы отправим информацию,
    # которую он ввел в командной строке.
    while True:
        print("> ", end="") entered =
        input()

        if entered != "":
            data      = entered.encode("utf-8") packet
            = RNS.Packet(destination, data)
            packet.send()

#####          #####  Запуск
программы          #####
#####

# Эта часть программы запускается при запуске,
# и анализирует ввод пользователя, а затем запускает #
программу.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(
            description="Пример Reticulum, демонстрирующий отправку и прием ширококвещательных сообщений"
        )

        parser.add_argument("--
            config",
            action="store",
            default=None,
            help="путь к альтернативному каталогу конфигурации Reticulum",
            type=str
        )

        parser.add_argument("--
            channel",
            action="store",
            default=None,

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

        help="имя канала широковещания", type=str
    )

    args = parser.parse_args()

    если args.config:
        configarg = args.config
    else:
        configarg = None

    if args.channel:
        channelarg = args.channel
    else:
        channelarg = None

    program_setup(configarg, channelarg)

except KeyboardInterrupt: print("")
    sys.exit(0)

```

Этот пример также можно найти по адресу <https://github.com/markqvist/Reticulum/blob/master/Examples/Broadcast.py>.

11.4 Echo

Пример *Echo* демонстрирует обмен данными между двумя адресатами с использованием интерфейса Packet.

```

#####
# Этот пример RNS демонстрирует простую утилиту «клиент-
сервер» # # echo. Клиент может отправить запрос echo на # #
сервер, и сервер ответит, подтвердив получение # # пакета. #
#####

import argparse
import sys
import RNS

# Давайте определим название приложения. Мы будем
использовать его для всех # создаваемых нами целей.
Поскольку этот пример с командой echo
# является частью набора примеров утилит, мы поместим
# их все в пространство имен приложения «example_utilities»
APP_NAME = "example_utilities"

#####
##### Часть сервера
#####

# Эта инициализация выполняется, когда пользователь выбирает
# запуск в качестве сервера

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

def server(configpath):
    global reticulum

    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Случайным образом создаем новую идентичность для нашего эхо-сервера
    server_identity = RNS.Identity()

    # Мы создаём пункт назначения, к которому клиенты могут отправлять запросы. Нам нужно # иметь возможность проверять эхо-ответы, отправляемые нашим клиентам, поэтому мы # создаём пункт назначения типа «single», способный принимать зашифрованные # сообщения. Таким образом, клиент может отправить запрос и быть # уверенным, что никто, кроме этого пункта назначения, не смог # его прочитать.
    echo_destination = RNS.Destination(
        server_identity, RNS.Destination.IN,
        RNS.Destination.SINGLE, APP_NAME,
        "echo",
        "request"
    )

    # Мы настраиваем пункт назначения на автоматическое подтверждение всех # пакеты, адресованные ему. При этом RNS будет автоматически # генерировать подтверждение для каждого входящего пакета и передавать его # обратно отправителю этого пакета.
    echo_destination.set_proof_strategy(RNS.Destination.PROVE_ALL)

    # Укажите пункту назначения, какую функцию в нашей программе # запускать при получении пакета. Мы делаем это, чтобы # Вывести сообщение в журнал при получении сервером запроса
    echo_destination.set_packet_callback(server_callback)

    # Все готово!
    # Подождем запросов от клиентов или ввода пользователя
    announceLoop(echo_destination)

def announceLoop(destination):
    # Сообщим пользователю, что все готово
    RNS.log(
        "Echo server "+
        RNS.prettyhexrep(destination.hash)+
        " запущен, нажмите Enter, чтобы вручную отправить объявление (Ctrl-C для выхода)"
    )

    # Мы входим в цикл, который будет выполняться до тех пор, пока пользователь не выйдет. # Если пользователь нажмёт клавишу Enter, мы объявим о нашем сервере # в сети, что позволит клиентам # узнать, как создавать сообщения, адресованные именно ему.

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

while True:
    entered = input() destination.announce()
    RNS.log("Отправлено объявление      с      "+RNS.prettyhexrep(destination.hash))

def server_callback(message, packet):
    global reticulum

    # Сообщите пользователю, что мы получили запрос «эхо», и # что
    # мы собираемся отправить ответ запрашивающему. # Отправка
    # подтверждения происходит автоматически, так как мы
    # настроили пункт назначения на проверку всех входящих пакетов.

    reception_stats = ""
    if reticulum.is_connected_to_shared_instance:
        reception_rssi = reticulum.get_packet_rssi(packet.packet_hash) reception_snr
        = reticulum.get_packet_snr(packet.packet_hash)

        if reception_rssi != None:
            reception_stats += " [RSSI "+str(reception_rssi)+" dBm]"

        if reception_snr != None:
            reception_stats += " [SNR "+str(reception_snr)+" дБм]"

    else:
        if packet.rssi != None:
            reception_stats += " [RSSI "+str(packet.rssi)+" дБм]"

        if packet.snr != None:
            reception_stats += " [SNR "+str(packet.snr)+" дБ]"

    RNS.log("Получен пакет от эхо-клиента, доказательство отправлено"+reception_stats)

##### Часть
клиента #####
#####

# Эта инициализация выполняется, когда пользователь выбирает
# запуск в качестве клиента
def client(destination_hexhash, configpath, timeout=None):
    глобальная сеть

    # Нам нужно двоичное представление хэша # назначения, введенного
    в командной строке
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Длина адреса назначения неверна, должна состоять из {hex} шестнадцатеричных символов (
                ↳{byte} б а й т о в)".format(hex=dest_len, byte=dest_len//2)
            )

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

    )

    destination_hash = bytes.fromhex(destination_hexhash)
except Exception as e:
    RNS.log("Введён неверный адрес назначения. Проверьте вводные данные!")
    RNS.log(str(e)+"\n")
    sys.exit(0)

# Сначала необходимо инициализировать Reticulum
reticulum = RNS.Reticulum(configpath)

# Мы переопределяем уровень логгирования, чтобы выводить
сообщение при # получении объявления
if RNS.loglevel < RNS.LOG_INFO:
    RNS.loglevel = RNS.LOG_INFO

# Сообщите пользователю, что клиент готов!
RNS.log(
    "Клиент Echo готов, нажмите Enter, чтобы отправить запрос Echo на "+
    destination_hexhash+
    «(Нажмите Ctrl+C, чтобы выйти)»
)

# Мы входим в цикл, который будет выполняться до тех
пор, пока пользователь не выйдет из программы. #
Если пользователь нажмет Enter, мы попробуем
отправить
# запрос echo на адрес, указанный в # командной строке.
while True:
    input()

    # Сначала проверим, знает ли RNS путь к месту назначения.
    # Если знает, мы загрузим идентификатор сервера и создадим пакет
    if RNS.Transport.has_path(destination_hash):

        # Чтобы обратиться к серверу, нам нужно знать его публичный
# ключ, поэтому мы проверяем, знает ли Reticulum этот адрес
назначения. # Это делается путем вызова метода «recall» модуля
# модуля Identity. Если адрес назначения известен, он # вернет
экземпляр Identity, который можно использовать в
# исходящих адресах.
        server_identity = RNS.Identity.recall(destination_hash)

        # Мы получили нужный экземпляр идентификатора из #
метода recall, поэтому давайте создадим исходящий
# назначения. Мы используем следующее соглашение об именовании:
# example_utilities.echo.request
# Это соответствует именованию, которое мы указали
в # части кода, относящейся к серверу.
        request_destination = RNS.Destination(
            server_identity, RNS.Destination.OUT,
            RNS.Destination.SINGLE,

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

        APP_NAME,
        "echo",
        "request"
    )

    # Адрес назначения готов, поэтому давайте создадим пакет. # В
    # качестве адреса назначения мы указываем только что созданный
    # request_destination, а единственные данные, которые мы добавляем
    # — это случайный хеш.
    echo_request = RNS.Packet(request_destination, RNS.Identity.get_random_
↪hash())

    # Отправляем пакет! Если пакет успешно # отправлен,
    # будет возвращен экземпляр PacketReceipt.
    packet_receipt = echo_request.send()

    # Если пользователь указал время ожидания, мы
    # устанавливаем # это время ожидания для получения
    # пакета и настраиваем # функцию обратного вызова,
    # которая будет вызвана, если # истечет время
    # ожидания пакета.
    if timeout != None: packet_receipt.set_timeout(timeout)
        packet_receipt.set_timeout_callback(packet_timed_out)

    # Затем мы можем установить обратный вызов доставки при
    # получении. # Он будет вызван автоматически, когда от адресата
    # этого конкретного пакета будет получено от адресата.
    packet_receipt.set_delivery_callback(packet_delivered)

    # Сообщить пользователю, что запрос эхо был отправлен
    RNS.log("Отправлен запрос эхо на "+RNS.prettyhexrep(request_destination.hash))
else:
    # Если мы не знаем этот адрес назначения, сообщите
    # пользователю подождать, пока не поступит объявление.
    RNS.log("Пункт назначения пока неизвестен. Запрашиваем путь...") RNS.log("Нажмите
    Enter, чтобы вручную повторить попытку после получения объявления.")
    RNS.Transport.request_path(destination_hash)

# Эта функция вызывается, когда адресат нашего ответа #
# получает пакет подтверждения.
def packet_delivered(receipt):
    global reticulum

    if receipt.status == RNS.PacketReceipt.DELIVERED: rtt = receipt.get_rtt()
        if (rtt >= 1):
            rtt = round(rtt, 3)
            rttstring = str(rtt)+" секунд"
        else:
            rtt = round(rtt*1000, 3)
            rttstring = str(rtt)+" миллисекунд"

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

reception_stats = ""
if reticulum.is_connected_to_shared_instance:
    reception_rssi = reticulum.get_packet_rssi(receipt.proof_packet.packet_hash) reception_snr =
    reticulum.get_packet_snr(receipt.proof_packet.packet_hash)

    if reception_rssi != None:
        reception_stats += " [RSSI "+str(reception_rssi)+" dBm]"

    if reception_snr != None:
        reception_stats += " [SNR "+str(reception_snr)+" дБ]"

else:
    if receipt.proof_packet != None:
        if receipt.proof_packet.rssi != None:
            reception_stats += " [RSSI "+str(receipt.proof_packet.rssi)+" дБм]"

        if receipt.proof_packet.snr != None:
            reception_stats += " [SNR "+str(receipt.proof_packet.snr)+" дБ]"

RNS.log(
    "Получен действительный ответ от "+
    RNS.prettyhexrep(receipt.destination.hash)+ ", время
    прохождения в оба конца составляет "+rttstring+
    reception_stats
)

# Эта функция вызывается, если произошел таймаут пакета.
def packet_timed_out(receipt):
    if receipt.status == RNS.PacketReceipt.FAILED:
        RNS.log("Пакет "+RNS.prettyhexrep(receipt.hash)+" timed out")

##### Запуск
#####
#####

# Эта часть программы запускается при запуске,
# и анализирует ввод пользователя, а затем запускает #
# нужный режим программы.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(description="П р о с т о й с е р в е р и к л и е н т э х о _
        ↵ у т и л и т а ")

        parser.add_argument("-s",
            "--server", action="store_true",
            help="ожидание входящих пакетов от клиентов"
        )

        parser.add_argument(

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

        "-t",
        "--timeout",
        action="store",
        metavar="s",
        default=None,
        help="установить время ожидания ответа в секундах",
        type=float
    )

    parser.add_argument("--config",
        action="store", default=None,
        help="путь к альтернативному каталогу конфигурации Reticulum",
        type=str
    )

    parser.add_argument(
        "destination", nargs="?",
        default=None,
        help="шестнадцатеричный хеш назначения сервера", type=str
    )

    args = parser.parse_args()

    if args.server:
        configarg=None if
        args.config:
            configarg = args.config
        server(configarg)
    else:
        if args.config:
            configarg = args.config
        else:
            configarg = None

        if args.timeout:
            timeoutarg = float(args.timeout)
        else:
            timeoutarg = None

        if (args.destination == None): print("")
        parser.print_help() print("")
        else:
            client(args.destination, configarg, timeout=timeoutarg)
    except KeyboardInterrupt: print("")
    sys.exit(0)

```

Этот пример также можно найти по адресу <https://github.com/markqvist/Reticulum/blob/master/Examples/Echo.py>.

11.5 Link

Пример *Link* демонстрирует установку зашифрованного соединения с удаленным адресатом и обмен трафиком по этому соединению.

```
#####
# Этот пример RNS демонстрирует, как настроить соединение с # #
# пунктом назначения и передавать данные по нему в обоих направлениях.
# #####

import os
import sys
import time
import argparse
import RNS

# Давайте определим имя приложения. Мы будем
# использовать его для всех # создаваемых нами целей.
# Поскольку этот пример с echo
# является частью набора примеров утилит, мы поместим
# их все в пространство имен приложения "example_utilities"

APP_NAME = "example_utilities"

##### ##### Часть сервера
#####

# Ссылка на последнюю ссылку клиента, который подключился
latest_client_link = None

# Эта инициализация выполняется, когда пользователь выбирает
# запуск в режиме сервера
def server(configpath):
    # Сначала необходимо инициализировать Reticulum

    reticulum = RNS.Reticulum(configpath)

    # Случайным образом создаем новую идентичность для нашего примера соединения

    server_identity = RNS.Identity()

    # Мы создаем пункт назначения, к которому могут подключаться
    # клиенты. Мы # хотим, чтобы клиенты создавали связи с этим
    # пунктом назначения, поэтому нам # нужно создать тип пункта
    # назначения «single».
    server_destination = RNS.Destination(
        server_identity, RNS.Destination.IN,
        RNS.Destination.SINGLE,
        APP_NAME,
        "linkexample"
    )

    # Настраиваем функцию, которая будет вызываться каждый раз, #
    # когда новый клиент устанавливает соединение с этим пунктом
    # назначения.
    server_destination.set_link_established_callback(client_connected)
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# Все готово!
# Подождем запросов от клиентов или ввода пользователя
server_loop(server_destination)

def server_loop(destination):
    # Сообщим пользователю, что все готово
    RNS.log(
        "Пример ссылки "+
        RNS.prettyhexrep(destination.hash)+
        " работает, ожидает подключения."
    )

    RNS.log("Нажмите Enter, чтобы вручную отправить объявление (Ctrl-C для выхода)")

    # Мы входим в цикл, который работает до тех пор, пока пользователь не выйдет.
    # Если пользователь нажмет Enter, мы объявим # адрес
    # нашего сервера в сети, что позволит клиентам # узнать, как
    # создавать сообщения, адресованные ему.
    while True:
        entered = input() destination.announce()
        RNS.log("Отправлено сообщение с адресом с сайта "+RNS.prettyhexrep(destination.hash))

# Когда клиент устанавливает соединение с нашим
# сервером # destination, эта функция будет вызвана с #
# ссылкой на соединение.
def client_connected(link):
    global latest_client_link

    RNS.log("Клиент подключен")
    link.set_link_closed_callback(client_disconnected)
    link.set_packet_callback(server_packet_received) latest_client_link =
    link

def client_disconnected(link): RNS.log("Клиент
отключился")

def server_packet_received(message, packet):
    global latest_client_link

    # При получении данных по любому активному каналу
    # все данные будут направлены последнему
    # клиенту, # который подключился.
    text = message.decode("utf-8") RNS.log("Получены
данные по каналу: "+text)

    reply_text = "Я получил \" "+text+" \" по каналу" reply_data =
    reply_text.encode("utf-8") RNS.Packet(latest_client_link,
    reply_data).send()

```

#####

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

##### Клиентская часть #####

# Ссылка на сервер
server_link = None

# Эта инициализация выполняется, когда пользователь выбирает
# запуск в качестве клиента
def client(destination_hexhash, configpath):
    # Нам нужно двоичное представление
    # хеши, введенного в командной строке
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Длина адреса назначения неверна, должна состоять из {hex} шестнадцатеричных символов (
        ↳{byte} байт)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)
    except:
        RNS.log("Введено недопустимое назначение. Проверьте ввод!\n") sys.exit(0)

    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Проверить, знаем ли мы путь к месту назначения
    if not RNS.Transport.has_path(destination_hash):
        RNS.log("Пункт назначения пока неизвестен. Запрашиваем путь и
        ↳ждем объявления_
        ↳...")
        RNS.Transport.request_path(destination_hash)
        пока RNS.Transport.has_path(destination_hash) не равно 0:
            time.sleep(0.1)

    # Получить идентификатор сервера
    server_identity = RNS.Identity.recall(destination_hash)

    # Сообщить пользователю, что мы начнем подключение
    RNS.log("Устанавливаем соединение с сервером...")

    # Когда идентификатор сервера известен, мы
    # настраиваем # пункт назначения
    server_destination = RNS.Destination(
        server_identity, RNS.Destination.OUT,
        RNS.Destination.SINGLE,
        APP_NAME,
        "linkexample"
    )

    # И создаем ссылку

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

link = RNS.Link(server_destination)

# Мы настраиваем обратный вызов, который будет
выполняться # при каждом получении пакета по #
каналу
link.set_packet_callback(client_packet_received)

# Мы также настроим функции для уведомления
# пользователя об установке или закрытии соединения
link.set_link_established_callback(link_established)
link.set_link_closed_callback(link_closed)

# Все настроено, поэтому давайте войдем в цикл #
для взаимодействия пользователя с примером
client_loop()

def client_loop():
    global server_link

    # Ожидаем, пока соединение станет активным
    пока не установлено
    соединение с сервером:
    time.sleep(0.1)

    should_quit = False пока
    should_quit не равно False:
        try:
            print("> ", end=" ") text =
            input()

            # Проверить, нужно ли завершить работу примера
            if text == "quit" or text == "q" or text == "exit": should_quit = True
                server_link.teardown()

            # Если нет, отправить введенный текст по каналу
            if text != "":
                data = text.encode("utf-8")
                if len(data) <= RNS.Link.MDU: RNS.Packet(server_link,
                    data).send()
                else:
                    RNS.log(
                        "Невозможно отправить этот пакет, размер данных "+ str(len(data))+" байт
                        превышает размер MDU пакета канала "+ str(RNS.Link.MDU)+" байт",
                        RNS.LOG_ERROR
                    )

        except Exception as e:
            RNS.log("Ошибка при отправке данных по каналу: "+str(e)) should_quit = True
            server_link.teardown()

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# Эта функция вызывается, когда #
установлено соединение с сервером
def link_established(link):
    # Сохраняем ссылку на экземпляр
    # экземпляра для дальнейшего
    # использования global server_link
    server_link = link

    # Сообщаем пользователю, что #
    # установлено соединение с сервером
    RNS.log("Соединение с сервером установлено, введите текст для отправки или \"quit\" для выхода")

# Когда соединение будет закрыто, мы сообщим
# об этом # пользователю и завершим работу
# программы
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT: RNS.log("Истекло
        время ожидания соединения, завершаю работу")
    elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED: RNS.log("Соединение
        было закрыто сервером, завершаю работу")
    else:
        RNS.log("Соединение закрыто, завершаю работу")

    time.sleep(1.5)
    sys.exit(0)

# Когда по каналу поступает пакет, мы # просто
# выводим данные на экран.
def client_packet_received(message, packet): text =
    message.decode("utf-8") RNS.log("Получены данные
    по каналу: " + text) print("> ", end=" ")
    sys.stdout.flush()

#####      Запуск
программы      #####
#####

# Эта часть программы запускается при старте,
# анализирует ввод пользователя, а затем # запускает
# нужный режим работы программы.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(description="Простой пример ссылки")

        parser.add_argument("-s",
            "--server", action="store_true",
            help="ожидать входящих запросов на подключение от клиентов"
        )

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

parser.add_argument( "--
    config",
    action="store",
    default=None,
    help="путь к альтернативному каталогу конфигурации Reticulum", type=str
)

parser.add_argument(
    "destination", nargs="?",
    default=None,
    help="шестнадцатеричный хеш назначения сервера", type=str
)

args = parser.parse_args()

if args.config:
    configarg = args.config
else:
    configarg = None

if args.server:
    server(configarg)
else:
    if (args.destination == None): print("")
    parser.print_help() print("")
    else:
        client(args.destination, configarg)

except KeyboardInterrupt: print("")
sys.exit(0)

```

Этот пример также можно найти по адресу <https://github.com/markqvist/Reticulum/blob/master/Examples/Link.py>.

11.6 Идентификация

Пример *Identify* демонстрирует определение инициатора соединения после его установки.

```

#####
# В этом примере RNS показано, как настроить соединение с #
# конечной точке и определить инициатора для его партнера #
#####

import os import
sys import time
import argparse

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

import RNS

# Давайте определим название приложения. Мы будем
использовать его для всех # создаваемых нами целей.
Поскольку этот пример с командой echo
# является частью набора примеров утилит, мы поместим
# их все в пространство имен приложения «example_utilities»

APP_NAME      =      "example_utilities"

##### Часть сервера
#####
#####

# Ссылка на последнюю ссылку клиента, который подключился

latest_client_link  =      None

# Эта инициализация выполняется, когда пользователь выбирает
# запуск в качестве сервера
def server(configpath):
    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Случайным образом создаем новую идентичность для нашего примера соединения
    server_identity = RNS.Identity()

    # Мы создаем пункт назначения, к которому могут подключаться
клиенты. Мы # хотим, чтобы клиенты создавали ссылки на этот
пункт назначения, поэтому нам # нужно создать тип пункта
назначения «single».
    server_destination = RNS.Destination(
        server_identity, RNS.Destination.IN,
        RNS.Destination.SINGLE,
        APP_NAME,
        "identifyexample"
    )

    # Настраиваем функцию, которая будет вызываться каждый раз, #
когда новый клиент устанавливает соединение с этим пунктом
назначения.
    server_destination.set_link_established_callback(client_connected)

    # Все готово!
    # Подождем запросов от клиентов или ввода пользователя
    server_loop(server_destination)

def server_loop(destination):
    # Сообщим пользователю, что все готово
    RNS.log(
        "Пример идентификации ссылки "+
        RNS.prettyhexrep(destination.hash)+
        " запущен, ожидаю подключения."
    )

    RNS.log("Нажмите Enter, чтобы вручную отправить объявление (Ctrl-C для выхода)")

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# Мы входим в цикл, который будет выполняться до тех пор, пока пользователь не выйдет.
# Если пользователь нажмет Enter, мы объявим наш сервер #
в сети, что позволит клиентам # узнать, как создавать
сообщения, адресованные ему.
while True:
    entered = input() destination.announce()
    RNS.log("Отправлено объявление : от "+RNS.prettyhexrep(destination.hash))

# Когда клиент устанавливает соединение с нашим
сервером, # эта функция вызывается с # ссылкой на
соединение.
def client_connected(link):
    global latest_client_link

    RNS.log("Client connected")
    link.set_link_closed_callback(client_disconnected)
    link.set_packet_callback(server_packet_received)
    link.set_remote_identified_callback(remote_identified)
    latest_client_link = link

def client_disconnected(link): RNS.log("Клиент
отключился")

def remote_identified(link, identity): RNS.log("Удаленный
идентифицирован как: "+str(identity))

def server_packet_received(message, packet):
    global latest_client_link

    # Получить идентификатор источника для отображения
    remote_peer = "неидентифицированный -узел"
    if packet.link.get_remote_identity() != None: remote_peer =
        str(packet.link.get_remote_identity())

    # Когда данные поступают по любому активному
каналу, # все они направляются последнему
клиенту, # который подключился.
    text = message.decode("utf-8")

    RNS.log("Получены данные от "+remote_peer+": "+text)

    reply_text = "Я получил \""+text+"\" по каналу от "+remote_peer
    reply_data = reply_text.encode("utf-8")
    RNS.Packet(latest_client_link, reply_data).send()

##### Часть
клиента #####
#####

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# Ссылка на сервер
server_link = None

# Ссылка на идентификатор клиента
client_identity = None

# Эта инициализация выполняется, когда пользователь решает #
# работать в качестве клиента
def client(destination_hexhash, configpath):
    global client_identity
    # Нам нужно двоичное представление
    # хеша, введенного в командной строке
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH // 8) * 2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Длина адреса назначения неверна, должна состоять из {hex} шестнадцатеричных символов (
                ↳ {byte} байт)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)
    except:
        RNS.log("Введено недопустимое назначение. Проверьте ввод!\n") sys.exit(0)

    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Создать новую идентичность клиента
    client_identity = RNS.Identity() RNS.log(
        «Клиент создал новый идентификатор» +
        str(client_identity)
    )

    # Проверить, известен ли нам путь к месту назначения
    if not RNS.Transport.has_path(destination_hash):
        RNS.log("Пункт назначения пока неизвестен. Запрашиваем путь и
        ↳ ждем объявления...»)
        RNS.Transport.request_path(destination_hash)
        пока RNS.Transport.has_path(destination_hash) не равно 0:
            time.sleep(0.1)

    # Получить идентификатор сервера
    server_identity = RNS.Identity.recall(destination_hash)

    # Сообщить пользователю, что мы начнем подключение
    RNS.log("Устанавливаем соединение с сервером...")

    # Если идентификатор сервера известен, мы
    # настраиваем # пункт назначения
    server_destination = RNS.Destination(

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

server_identity,
RNS.Destination.OUT,
RNS.Destination.SINGLE,
APP_NAME,
"identifyexample"
)

# И создать ссылку
link = RNS.Link(server_destination)

# Мы устанавливаем обратный вызов, который
будет выполняться # каждый раз при получении
пакета по # ссылке
link.set_packet_callback(client_packet_received)

# Мы также настроим функции для уведомления
# Обработка событий при установке или разрыве соединения
link.set_link_established_callback(link_established)
link.set_link_closed_callback(link_closed)

# Все настроено, давайте перейдем в цикл # для
взаимодействия пользователя с примером
client_loop()

def client_loop():
    global server_link

    # Ожидаем, пока соединение станет активным
    while not server_link:
        time.sleep(0.1)

    should_quit = False
    пока not
    should_quit:
        try:
            print("> ", end=" ")
            text = input()

            # Проверить, следует ли завершить работу с примером
            if text == "quit" or text == "q" or text == "exit":
                should_quit = True
                server_link.teardown()

            # Если нет, отправить введенный текст по каналу
            if text != "":
                data = text.encode("utf-8")
                if len(data) <= RNS.Link.MDU:
                    RNS.Packet(server_link,
                        data).send()
                else:
                    RNS.log(
                        «Невозможно отправить этот пакет: размер данных «+ str(len(data))+» байт
                        превышает максимальный размер пакета канала (MDU) «+
                        str(RNS.Link.MDU)+» байт»,

```

(продолжение на следующей странице)

```

        RNS.LOG_ERROR
    )

    except Exception as e:
        RNS.log("Ошибка при отправке данных по каналу: "+str(e)) should_quit = True
        server_link.teardown()

# Эта функция вызывается, когда #
установлено соединение с сервером
def link_established(link):
    # Мы сохраняем ссылку на экземпляр
    # экземпляра для дальнейшего использования
    global server_link, client_identity
    server_link = link

    # Сообщаем пользователю, что сервер #
    подключен
    RNS.log("Соединение с сервером установлено, идентификация удаленного узла...")

    link.identify(client_identity)

# Когда соединение будет закрыто, мы #
уведомим пользователя и завершим работу
программы
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT: RNS.log("Истекло
        время ожидания соединения, завершаем работу")
    elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED: RNS.log("Соединение
        было закрыто сервером, завершаю работу")
    else:
        RNS.log("Соединение закрыто, завершаю работу")

    time.sleep(1.5)
    sys.exit(0)

# При получении пакета по каналу мы # просто
выводим данные.
def client_packet_received(message, packet): text =
    message.decode("utf-8") RNS.log("Получены данные
    по каналу: "+text) print("> ", end=" ")
    sys.stdout.flush()

#####      #####      Запуск
программы      #####
#####

# Эта часть программы запускается при запуске системы,
# анализирует ввод пользователя, а затем # запускает
нужный режим программы.
if __name__ == "__main__":

```

(продолжение с предыдущей страницы)

```

try:
    parser = argparse.ArgumentParser(description="Простой пример ссылки")

    parser.add_argument("-s",
                        "--server", action="store_true",
                        help="ожидание входящих запросов на подключение от клиентов"
                        )

    parser.add_argument("--config",
                        action="store",
                        default=None,
                        help="путь к альтернативному каталогу конфигурации Reticulum",
                        type=str
                        )

    parser.add_argument(
        "destination", nargs="?",
        default=None,
        help="шестнадцатеричный хеш адреса назначения сервера",
        type=str
    )

    args = parser.parse_args()

    if args.config:
        configarg = args.config
    else:
        configarg = None

    if args.server:
        server(configarg)
    else:
        if (args.destination == None): print("")
        parser.print_help() print("")
        else:
            client(args.destination, configarg)

except KeyboardInterrupt: print("")
sys.exit(0)

```

Этот пример также можно найти по адресу <https://github.com/markqvist/Reticulum/blob/master/Examples/Identify.py>.

11.7 Запросы и ответы

В примере «Запрос» рассматривается отправка запросов и получение ответов.

```
#####
# Этот пример RNS демонстрирует, как выполнять запросы # и получать
# ответы по каналу связи. #
#####

import os import
sys import time
import random
import argparse
import RNS

# Давайте определим имя приложения. Мы будем
# использовать его для всех # создаваемых нами адресатов.
# Поскольку этот пример с эхом
# является частью ряда примеров утилит, мы поместим
# все они находятся в пространстве имен приложения «example_utilities»

APP_NAME = "example_utilities"

##### ##### Серверная
часть #####
#####

# Ссылка на последнюю ссылку клиента, который подключился
latest_client_link = None

def random_text_generator(path, data, request_id, link_id, remote_identity, requested_
→at):
    RNS.log("Генерируем ответ на запрос "+RNS.prettyhexrep(request_id)+" по ссылке
→"+RNS.prettyhexrep(link_id))
    texts = ["Они посмотрели вверх", "Каждое полнолуние", "Бекки была
    расстроена", "Я буду держаться подальше",
    →«из него», «В зоомагазине есть всё»]
    return texts[random.randint(0, len(texts)-1)]

# Эта инициализация выполняется, когда пользователь выбирает
# запуск в качестве сервера
def server(configpath):
    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Случайным образом создаем новую идентичность для нашего примера соединения
    server_identity = RNS.Identity()

    # Создаем пункт назначения, к которому могут подключаться
    # клиенты. Мы # хотим, чтобы клиенты устанавливали соединения с
    # этим пунктом назначения, поэтому # нам нужно создать пункт
    # назначения типа «single».
    server_destination = RNS.Destination(
        server_identity, RNS.Destination.IN,
        RNS.Destination.SINGLE,
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

        APP_NAME,
        "requestexample"
    )

    # Мы настраиваем функцию, которая будет вызываться каждый
раз, # когда новый клиент устанавливает соединение с этим
адресатом.
    server_destination.set_link_established_callback(client_connected)

    # Регистрируем обработчик запросов для обработки входящих #
запросов по любым установленным соединениям.
    server_destination.register_request_handler( "/random/text",
        response_generator = random_text_generator, allow =
        RNS.Destination.ALLOW_ALL
    )

    # Все готово!
    # Подождем запросов от клиентов или ввода пользователя
    server_loop(server_destination)

def server_loop(destination):
    # Сообщим пользователю, что все готово
    RNS.log(
        "Запрос example "+
        RNS.prettyhexrep(destination.hash)+
        " выполняется, ожидание соединения."
    )

    RNS.log("Нажмите Enter, чтобы вручную отправить объявление (Ctrl-C для выхода)")

    # Мы входим в цикл, который будет выполняться до тех пор, пока пользователь не выйдет.
    # Если пользователь нажмет Enter, мы объявим о #
    назначении нашего сервера в сети, что позволит клиентам #
    узнать, как создавать сообщения, адресованные ему.
    while True:
        entered = input() destination.announce()
        RNS.log("Отправлено объявление      с "+RNS.prettyhexrep(destination.hash))

# Когда клиент устанавливает соединение с нашим
сервером, # эта функция вызывается с # ссылкой на
соединение.
def client_connected(link):
    global latest_client_link

    RNS.log("Клиент подключился")
    link.set_link_closed_callback(client_disconnected) latest_client_link =
    link

def client_disconnected(link): RNS.log("Клиент
отключился")

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```
#####      #####      Часть
клиента      #####
#####

# Ссылка на соединение с сервером
server_link = None

# Эта инициализация выполняется, когда пользователь #
выбирает запуск в качестве клиента
def client(destination_hexhash, configpath):
    # Нам нужно двоичное представление
    # хеша, введенного в командной строке
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Длина адреса назначения неверна, должна состоять из {hex} шестнадцатеричных символов (
                ↳{byte} байт в)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)
        за исключением:
        RNS.log("Введен неверный адрес назначения. Проверьте ввод!\n") sys.exit(0)

    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Проверим, знаем ли мы путь к месту назначения
    if not RNS.Transport.has_path(destination_hash):
        RNS.log("Пункт назначения пока неизвестен. Запрашиваем путь и
        ждем объявления↳...")
        RNS.Transport.request_path(destination_hash)
        пока RNS.Transport.has_path(destination_hash) не равно 0:
            time.sleep(0.1)

    # Получить идентификатор сервера
    server_identity = RNS.Identity.recall(destination_hash)

    # Сообщить пользователю, что мы начнем установку соединения
    RNS.log("Устанавливаем соединение с сервером...")

    # Когда идентификатор сервера известен, мы
    настраиваем # пункт назначения
    server_destination = RNS.Destination(
        server_identity, RNS.Destination.OUT,
        RNS.Destination.SINGLE,
        APP_NAME,
        "requestexample"
    )
```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# И создайте ссылку
link = RNS.Link(server_destination)

# Мы настроим функции для информирования
# Обработка событий при установке или разрыве соединения link.set_link_established_callback(link_established)
link.set_link_closed_callback(link_closed)

# Все настроено, давайте перейдем в цикл # для
взаимодействия пользователя с примером
client_loop()

def client_loop():
    global server_link

    # Ожидаем, пока соединение станет активным
    while not server_link:
        time.sleep(0.1)

    should_quit = False пока not
    should_quit:
        try:
            print("> ", end=" ") text =
            input()

            # Проверить, следует ли завершить работу с примером
            if text == "quit" or text == "q" or text == "exit": should_quit = True
                server_link.teardown()

            else:
                server_link.request(
                    "/random/text", data =
                    None,
                    response_callback = got_response, failed_callback
                    = request_failed
                )

        except Exception as e:
            RNS.log("Ошибка при отправке запроса по каналу: "+str(e)) should_quit = True
            server_link.teardown()

def got_response(request_receipt): request_id =
    request_receipt.request_id response =
    request_receipt.response

    RNS.log("Получен ответ на запрос "+RNS.prettyhexrep(request_id)+" "+str(response))

def request_received(request_receipt):

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

RNS.log("З а п р о с "+RNS.prettyhexrep(request_receipt.request_id)+" б ы л п о л у ч е н _
↳ удаленным узлом.")

def request_failed(request_receipt):
    RNS.log("Запрос "+RNS.prettyhexrep(request_receipt.request_id)+" завершился сбоем.")

# Эта функция вызывается, когда #
установлено соединение с сервером
def link_established(link):
    # Мы сохраняем ссылку на экземпляр
    # экземпляра для дальнейшего
    использования global server_link
    server_link = link

    # Сообщить пользователю, что #
    установлено соединение с сервером
    RNS.log("Соединение с сервером установлено, нажмите Enter для отправки запроса или введите \
↳ "quit\" для выхода")

# Когда соединение будет закрыто, мы сообщим
об этом # пользователю и завершим работу
программы
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT: RNS.log("Истекло
        время ожидания соединения, завершаем работу")
    elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED: RNS.log("Соединение
        было закрыто сервером, завершаю работу")
    else:
        RNS.log("Соединение закрыто, завершаю работу")

    time.sleep(1.5)
    sys.exit(0)

#####          #####  Запуск
программы          #####
#####

# Эта часть программы запускается при запуске,
# анализирует ввод пользователя, а затем # запускает
нужный режим работы программы.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(description="Простой пример запроса/ответа")

        parser.add_argument("-s",
            "--server", action="store_true",
            help="ожидание входящих запросов от клиентов"
        )

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

parser.add_argument( "--
    config",
    action="store",
    default=None,
    help="путь к альтернативному каталогу конфигурации Reticulum", type=str
)

parser.add_argument(
    "destination", nargs="?",
    default=None,
    help="шестнадцатеричный хеш назначения сервера", type=str
)

args = parser.parse_args()

if args.config:
    configarg = args.config
else:
    configarg = None

if args.server:
    server(configarg)
else:
    if (args.destination == None): print("")
    parser.print_help() print("")
    else:
        client(args.destination, configarg)

except KeyboardInterrupt: print("")
sys.exit(0)

```

Этот пример также можно найти по адресу <https://github.com/markqvist/Reticulum/blob/master/Examples/Request.py>.

11.8 Канал

Пример «Канал» демонстрирует использование канала для передачи структурированных данных между узлами соединения.

(продолжение на следующей странице)

```

#####
# В этом примере RNS показано, как настроить соединение с # #
# конечным пунктом и передавать по нему структурированные #
# сообщения # # с помощью канала. #
#####

import os
import sys
import time

```

(продолжение с предыдущей страницы)

```

import argparse
from datetime import datetime

import RNS
from RNS.vendor import umsgpack

# Давайте определим имя приложения. Мы будем
использовать его для всех # создаваемых нами пунктов
назначения. Поскольку этот пример с echo
# является частью ряда примеров утилит, мы поместим
# их все в пространство имен приложения "example_utilities"
APP_NAME = "example_utilities"

#####      ####  Общие
объекты      #####
#####

# Данные канала должны быть структурированы в подклассе
# MessageBase. Это гарантирует, что канал сможет # сериализовать
и десериализовать объект, а также мультиплексировать его # с
другими объектами. Оба конца канала должны иметь
# одинаковые определения объектов, чтобы иметь возможность #
обмениваться данными по каналу.
#
# Примечание: Объекты, которые мы хотим использовать через канал,
должны # быть зарегистрированы в канале, и каждое соединение
имеет
# другой экземпляр канала. Ознакомьтесь с функциями
client_connected # и link_established в этом примере, чтобы увидеть,
# как регистрируются типы сообщений.

# Давайте создадим простой класс сообщения под названием
StringMessage, # который будет передавать строку с временной
меткой.

class StringMessage(RNS.MessageBase):
    # Переменной класса MSGTYPE необходимо присвоить
    # 2-байтовое целое значение. Этот идентификатор позволяет
    # канал, по которому будет выполняться поиск конструктора вашего
    сообщения при поступлении # сообщения по этому каналу.
    #
    # MSGTYPE должен быть уникальным для всех типов сообщений,
    которые мы # регистрируем в канале. Значения MSGTYPE >= 0xf000
    # зарезервированы для системы.
    MSGTYPE = 0x0101

    # Конструктор нашего объекта должен вызываться # без
    аргументов. Мы можем иметь параметры, но они должны # иметь
    значение по умолчанию.
    #
    # Это необходимо для того, чтобы канал мог создать пустую #
    версию нашего сообщения, в которую можно будет вписать
    входящее
    # сообщение.
    def init(self, data=None): self.data = data

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

self.timestamp = datetime.now()

# Наконец, наше сообщение должно реализовать функции
# канал может вызывать эту функцию для упаковки и распаковки
нашего сообщения # в/из необработанного полезного содержимого
пакета. Мы будем использовать
# пакет umsgpack, входящий в состав RNS. Мы также могли бы
использовать # пакет struct, входящий в состав Python, если бы
хотели # больше контролировать структуру упакованных байтов. #
# Также обратите внимание, что упакованные объекты
сообщений должны # полностью помещаться в один
пакет. Количество байтов
# Размер доступного пространства для полезных данных
сообщения можно узнать # из канала с помощью свойства
Channel.MDU.
# MDU канала немного меньше, чем MDU канала связи, из-за #
кодирования заголовка сообщения.

# Функция pack кодирует содержимое сообщения в # поток байтов.
def pack(self) -> bytes:
    return umsgpack.packb((self.data, self.timestamp))

# А функция unpack декодирует поток байтов в # содержимое
сообщения.
def unpack(self, raw):
    self.data, self.timestamp = umsgpack.unpackb(raw)

#####      ####  Серверная
часть      #####
#####

# Ссылка на последнюю клиентскую ссылку, которая подключилась
latest_client_link = None

# Эта инициализация выполняется, когда пользователь выбирает
# запуск в качестве сервера
def server(configpath):
    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Случайным образом создаем новую идентичность для нашего примера
    связи
    server_identity = RNS.Identity()

    # Создаем пункт назначения, к которому могут подключаться
    клиенты. Мы # хотим, чтобы клиенты создавали соединения с этим
    пунктом назначения, поэтому # нам нужно создать пункт назначения
    типа «single».
    server_destination = RNS.Destination(
        server_identity, RNS.Destination.IN,
        RNS.Destination.SINGLE,
        APP_NAME,
        "channelexample"

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

)

# Мы настраиваем функцию, которая будет вызываться каждый раз, # когда новый клиент создает ссылку на этот адрес.
server_destination.set_link_established_callback(client_connected)

# Все готово!
# Подождем запросов от клиента или ввода пользователя
server_loop(server_destination)

def server_loop(destination):
    # Сообщим пользователю, что все готово
    RNS.log(
        "Пример канала "+
        RNS.prettyhexrep(destination.hash)+
        " запущен, ожидаю подключения."
    )

    RNS.log("Нажмите Enter, чтобы вручную отправить объявление (Ctrl-C для выхода)")

    # Мы входим в цикл, который будет выполняться до тех пор, пока пользователь не выйдет.
    # Если пользователь нажмёт Enter, мы объявим наш сервер #
    в сети, что позволит клиентам # узнать, как создавать
    сообщения, адресованные ему.
    while True:
        entered = input() destination.announce()
        RNS.log("Отправлено объявление      с "+RNS.prettyhexrep(destination.hash))

# Когда клиент устанавливает соединение с нашим сервером # destination, эта функция будет вызвана с #
ссылкой на соединение.
def client_connected(link): global latest_client_link
    latest_client_link = link

    RNS.log("Клиент подключен")
    link.set_link_closed_callback(client_disconnected)

    # Регистрация типов сообщений и добавление обратного вызова к каналу
    channel = link.get_channel()
    channel.register_message_type(StringMessage)
    channel.add_message_handler(server_message_received)

def client_disconnected(link): RNS.log("Клиент отключился")

def server_message_received(message):
    """
    Обработчик сообщений
    @param message: экземпляр подкласса MessageBase @return: True, если
    сообщение было обработано

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

"""

global latest_client_link
# Когда сообщение поступает по любому активному каналу,
# все ответы будут направляться последнему клиенту, #
# который подключился.

# В обработчике сообщений любое десериализуемое сообщение
# поступающее по каналу соединения, будет передано
# всем обработчикам сообщений, если только предыдущий обработчик не
# укажет, что # он обработал это сообщение.
#
#
if isinstance(message, StringMessage):
    RNS.log("Получены данные по каналу: " + message.data + " (сообщение
        создано в " +
        ↪str(message.timestamp) + ")")

    reply_message = StringMessage("Я получил \"" + message.data + "\" по каналу")
    latest_client_link.get_channel().send(reply_message)

    # Входящие сообщения отправляются каждому
    # обработчику, добавленному к каналу, в том порядке, в
    # котором # они были добавлены.
    # Если какой-либо обработчик сообщений возвращает True,
    # сообщение # считается обработанным, и все последующие
    # обработчики пропускаются.
    return True

##### Часть
клиента #####
#####

# Ссылка на сервер
server_link = None

# Эта инициализация выполняется, когда пользователь #
# выбирает запуск в качестве клиента
def client(destination_hexhash, configpath):
    # Нам нужно двоичное представление
    # хеши, введенного в командной строке
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Длина адреса назначения неверна, должна состоять из {hex} шестнадцатеричных символов (
                ↪{byte} байтов)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)
    за исключением:
        RNS.log("Введен неверный адрес назначения. Проверьте ввод!\n") sys.exit(0)

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# Сначала необходимо инициализировать Reticulum
reticulum = RNS.Reticulum(configpath)

# Проверим, знаем ли мы путь к месту назначения
if not RNS.Transport.has_path(destination_hash):
    RNS.log("Пункт назначения пока неизвестен. Запрашиваем путь и
    ждем объявления _
    ←→...")
    RNS.Transport.request_path(destination_hash)
    пока RNS.Transport.has_path(destination_hash) не равен 0:
        time.sleep(0.1)

# Получить идентификатор сервера
server_identity = RNS.Identity.recall(destination_hash)

# Сообщить пользователю, что мы начнем подключение
RNS.log("Устанавливаем соединение с сервером...")

# Когда идентификатор сервера известен, мы
настраиваем # пункт назначения
server_destination = RNS.Destination(
    server_identity, RNS.Destination.OUT,
    RNS.Destination.SINGLE,
    APP_NAME,
    "channelexample"
)

# И создаем ссылку
link = RNS.Link(server_destination)

# Мы также настроим функции для информирования
# Обработка событий при установке или разрыве соединения link.set_link_established_callback(link_established)
link.set_link_closed_callback(link_closed)

# Все настроено, давайте перейдем в цикл # для
взаимодействия пользователя с примером
client_loop()

def client_loop():
    global server_link

    # Ждем, пока соединение станет активным
    while not server_link:
        time.sleep(0.1)

    should_quit = False пока not
    should_quit:
        try:
            print("> ", end=" ") text =
            input()

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# Проверить, следует ли завершить работу примера
if text == "quit" or text == "q" or text == "exit": should_quit = True
    server_link.teardown()

# Если нет, отправить введенный текст по каналу
if text != "":
    message = StringMessage(text) packed_size =
    len(message.pack()) channel =
    server_link.get_channel() if
    channel.is_ready_to_send():
        if packed_size <= channel.mdu:
            channel.send(message)
        else:
            RNS.log(
                «Невозможно отправить этот пакет: размер данных "+ str(packed_size)+" байт
                превышает размер MDU канала "+ str(channel.MDU)+" байт»,
                RNS.LOG_ERROR
            )
    else:
        RNS.log("Канал не готов к отправке, пожалуйста, дождитесь завершения
        обработки " + "ожидающих сообщений.", RNS.LOG_ERROR)

except Exception as e:
    RNS.log("Ошибка при отправке данных по каналу: "+str(e)) should_quit = True
    server_link.teardown()

# Эта функция вызывается при установке
соединения с сервером
def link_established(link):
    # Мы сохраняем ссылку на экземпляр
    # для дальнейшего использования
    global server_link server_link =
    link

    # Регистрируем сообщения и добавляем обработчик к каналу channel
    = link.get_channel() channel.register_message_type(StringMessage)
    channel.add_message_handler(client_message_received)

    # Сообщаем пользователю, что сервер #
    подключен
    RNS.log("Соединение с сервером установлено, введите текст для отправки или \"quit\", чтобы выйти")

# При закрытии ссылки мы уведомим #
пользователя и завершим работу программы
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT: RNS.log("Время
    ожидания соединения истекло, завершаем работу")

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED: RNS.log("Соединение
    было закрыто сервером, завершаем работу")
else:
    RNS.log("Соединение закрыто, завершаю работу")

time.sleep(1.5)
sys.exit(0)

# При получении пакета по каналу мы # просто выводим
данные на экран.
def client_message_received(message):
    if isinstance(message, StringMessage):
        RNS.log("Получены данные по каналу: " + message.data + " (сообщение
            создано в " +
            str(message.timestamp) + ")")
        print("> ", end=" ")
        sys.stdout.flush()

#####      Запуск
программы      #####
#####

# Эта часть программы запускается при запуске,
# и анализирует ввод пользователя, а затем # запускает
нужный режим программы.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(description="Простой пример канала")

        parser.add_argument("-s",
            "--server", action="store_true",
            help="ожидание входящих запросов на подключение от клиентов"
        )

        parser.add_argument("--
            config",
            action="store",
            default=None,
            help="путь к альтернативному каталогу конфигурации Reticulum",
            type=str
        )

        parser.add_argument(
            "destination", nargs="?",
            default=None,
            help="шестнадцатеричный хеш адреса сервера", type=str
        )

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

args = parser.parse_args()

if args.config:
    configarg = args.config
else:
    configarg = None

if args.server:
    server(configarg)
else:
    if (args.destination == None): print("")
    parser.print_help() print("")
    else:
        client(args.destination, configarg)

except KeyboardInterrupt: print("")
sys.exit(0)

```

Этот пример также можно найти по адресу <https://github.com/markqvist/Reticulum/blob/master/Examples/Channel.py>.

11.9 Буфер

Пример *Buffer* демонстрирует использование буферизованных считывателей и записывателей для передачи двоичных данных между узлами в

```

#####
# В этом примере RNS показано, как установить соединение с # #
# конечным пунктом и передавать по нему двоичные данные с # #
# помощью буфера канала # #
#####

from __future__ import annotations
import os
import sys
import time
import argparse

из datetime импортировать datetime

import RNS

из RNS.vendor импортировать umsgpack

# Определим имя приложения. Мы будем использовать его
# для всех # создаваемых нами адресатов. Поскольку это
# пример с echo

# входит в набор примеров утилит, мы поместим
# их все в пространство имен приложения «example_utilities»
APP_NAME = "example_utilities"

#####
##### Серверная
часть
#####
#####

```

Link.

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# Ссылка на последнюю ссылку клиента, который подключился
latest_client_link = None

# Ссылка на последний объект буфера
latest_buffer = None

# Эта инициализация выполняется, когда пользователь выбирает
# запуск в качестве сервера
def server(configpath):
    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Случайным образом создаем новую идентичность для нашего примера
    server_identity = RNS.Identity()

    # Создаем пункт назначения, к которому могут подключаться
    # клиенты. Мы # хотим, чтобы клиенты устанавливали соединения с
    # этим пунктом назначения, поэтому # нам нужно создать пункт
    # назначения типа «single».
    server_destination = RNS.Destination(
        server_identity, RNS.Destination.IN,
        RNS.Destination.SINGLE,
        APP_NAME,
        "bufferexample"
    )

    # Мы настраиваем функцию, которая будет вызываться каждый
    # раз, # когда новый клиент создает ссылку на этот пункт назначения.
    server_destination.set_link_established_callback(client_connected)

    # Все готово!
    # Подождем запросов от клиентов или ввода пользователя
    server_loop(server_destination)

def server_loop(destination):
    # Сообщим пользователю, что все готово
    RNS.log(
        "Пример буфера связи "+
        RNS.prettyhexrep(destination.hash)+
        " запущен, ожидаю подключения."
    )

    RNS.log("Нажмите Enter, чтобы вручную отправить объявление (Ctrl-C для выхода)")

    # Мы входим в цикл, который работает до тех пор, пока пользователь не выйдет.
    # Если пользователь нажмёт Enter, мы объявим наш сервер #
    # в сети, что позволит клиентам # узнать, как создавать
    # сообщения, адресованные ему.
    while True:
        entered = input() destination.announce()

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

RNS.log("Отправлено объявление с "+RNS.prettyhexrep(destination.hash))

# Когда клиент устанавливает соединение с нашим
сервером # destination, эта функция будет вызвана с #
ссылкой на соединение.
def client_connected(link):
    global latest_client_link, latest_buffer; latest_client_link = link

    RNS.log("Клиент подключен")
    link.set_link_closed_callback(client_disconnected)

    # Если получено новое соединение, старый читатель #
    должен быть отключен.
    if latest_buffer:
        latest_buffer.close()

    # Создать объекты буфера.
    # Параметр stream_id в этих функциях #
    немного похож на дескриптор файла, за
    исключением того, что он # является уникальным для
    *приемника*.
    #
    # В этом примере и читатель, и записывающий #
    используют stream_id = 0, но на самом деле их два
    # отдельные однонаправленные потоки, текущие в #
    противоположных направлениях.
    #
    channel = link.get_channel()
    latest_buffer = RNS.Buffer.create_bidirectional_buffer(0, 0, channel, server_buffer_
    ←ready)

def client_disconnected(link): RNS.log("Клиент
отключился")

def server_buffer_ready(ready_bytes: int):
    """
    Обратный вызов из буфера, когда в буфере доступны данные

    :param ready_bytes: Количество байтов, готовых к чтению """
    global latest_buffer

    data = latest_buffer.read(ready_bytes) data =
    data.decode("utf-8")

    RNS.log("Получены данные через буфер: " + data)

    reply_message = "Я получил \" " + data + "\" из буфера" reply_message =
    reply_message.encode("utf-8") latest_buffer.write(reply_message)
    latest_buffer.flush()

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

##### Кlienтская
часть #####
#####

# Ссылка на сервер
server_link = None

# Ссылка на объект буфера, необходимая для передачи # объекта из
обратного вызова, связанного со ссылкой, в цикл # клиента.
buffer = None

# Эта инициализация выполняется, когда пользователь #
выбирает запуск в качестве клиента
def client(destination_hexhash, configpath):
    # Нам нужно двоичное представление
    # хеша, введенного в командной строке
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Длина адреса назначения неверна, должна состоять из {hex} шестнадцатеричных символов (
↳ {byte} байт)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)

    за исключением:
        RNS.log("Введен неверный адрес назначения. Проверьте ввод!\n") sys.exit(0)

    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Проверим, знаем ли мы путь к месту назначения
    if not RNS.Transport.has_path(destination_hash):
        RNS.log("Пункт назначения пока неизвестен. Запрашиваем путь и
↳ ждем объявления_")
        RNS.Transport.request_path(destination_hash)
        пока RNS.Transport.has_path(destination_hash) не равно 0:
            time.sleep(0.1)

    # Получить идентификатор сервера
    server_identity = RNS.Identity.recall(destination_hash)

    # Сообщить пользователю, что мы начнём установку соединения
    RNS.log("Устанавливаем соединение с сервером...")

    # Когда идентификатор сервера известен, мы устанавливаем

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# назначаем пункт назначения
server_destination = RNS.Destination(
    server_identity, RNS.Destination.OUT,
    RNS.Destination.SINGLE,
    APP_NAME,
    "bufferexample"
)

# И создать ссылку
link = RNS.Link(server_destination)

# Мы также настроим функции для информирования
# Обработка событий при установке или разрыве соединения
link.set_link_established_callback(link_established)
link.set_link_closed_callback(link_closed)

# Все настроено, давайте перейдем в цикл # для
# взаимодействия пользователя с примером
client_loop()

def client_loop():
    global server_link

    # Ждем, пока соединение станет активным
    while not server_link:
        time.sleep(0.1)

    should_quit = False
    while not should_quit:
        try:
            print("> ", end=" ")
            text = input()

            # Проверить, следует ли завершить работу с примером
            if text == "quit" or text == "q" or text == "exit":
                should_quit = True
                server_link.teardown()
            else:
                # В противном случае закодируем текст и запишем его в буфер.
                text = text.encode("utf-8")
                buffer.write(text)
                # Очистить буфер, чтобы принудительно отправить данные.
                buffer.flush()

        except Exception as e:
            RNS.log("Ошибка при отправке данных через буфер канала: "+str(e))
            should_quit = True
            server_link.teardown()

# Эта функция вызывается при переходе по ссылке

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# установлено соединение с сервером
def link_established(link):
    # Мы сохраняем ссылку на экземпляр соединения
    # экземпляра для дальнейшего
    # использования global server_link,
    buffer server_link = link

    # Создаем буфер, см. server_client_connected() для # получения
    # более подробной информации о настройке буфера.
    channel = link.get_channel()

    buffer = RNS.Buffer.create_bidirectional_buffer(0, 0, channel, client_buffer_ready)

    # Сообщить пользователю, что #
    # установлено соединение с сервером
    RNS.log("Соединение с сервером установлено, введите текст для отправки или \"quit\" для выхода")

# Когда соединение будет закрыто, мы #
# уведомим пользователя и завершим работу
# программы
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT: RNS.log("Истекло
        время ожидания соединения, завершаем работу")
    elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED: RNS.log("Соединение
        было закрыто сервером, завершаю работу")
    else:
        RNS.log("Соединение закрыто, завершаю работу")

    time.sleep(1.5)
    sys.exit(0)

# Когда в буфере появляются новые данные, прочитайте их и выведите в терминал.
def client_buffer_ready(ready_bytes: int):
    global buffer
    data = buffer.read(ready_bytes)
    RNS.log("Получены данные из буфера канала: " + data.decode("utf-8")) print("> ", end=" ")
    sys.stdout.flush()

#####      #### Запуск
# программы      #####
#####

# Эта часть программы запускается при запуске,
# анализирует ввод пользователя, а затем # запускает
# нужный режим работы программы.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(description="Простой пример буфера")

        parser.add_argument("-s",
            "--server",

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

        action="store_true",
        help="ожидать входящих запросов на подключение от клиентов"
    )

    parser.add_argument( "--
        config",
        action="store",
        default=None,
        help="путь к альтернативному каталогу конфигурации Reticulum", type=str
    )

    parser.add_argument(
        "destination", nargs="?",
        default=None,
        help="шестнадцатеричный хеш назначения сервера", type=str
    )

    args = parser.parse_args()

    if args.config:
        configarg = args.config
    else:
        configarg = None

    if args.server:
        server(configarg)
    else:
        if (args.destination == None): print("")
        parser.print_help() print("")
        else:
            client(args.destination, configarg)

except KeyboardInterrupt: print("")
sys.exit(0)

```

Этот пример также можно найти по адресу <https://github.com/markqvist/Reticulum/blob/master/Examples/Buffer.py>.

11.10 Filetransfer

Пример *Filetransfer* реализует базовую программу файлового сервера, позволяющую клиентам подключаться и загружать файлы. Программа использует интерфейс Resource для эффективной передачи файлов любого размера по каналу Reticulum *Link*.

```

#####
# Этот пример RNS демонстрирует простую программу передачи
# файлов # # для сервера и клиента. Сервер будет обслуживать
# каталог # # с файлами, а клиенты смогут просматривать
#
#

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# скачивать файлы с сервера. #
# #
# Обратите внимание, что использование RNS Resources для передачи
# больших файлов # не рекомендуется, поскольку сжатие, #
# шифрование и сортировка по хеш-карте могут занимать много
# времени # на системах с медленными процессорами, что, вероятно,
# приведет # к истечению времени ожидания клиента до того, как
# отправитель ресурса # сможет завершить подготовку ресурса. #
# #
# Если вам нужно передать файлы большого размера, используйте
# вместо этого класс Bundle # #, который автоматически
# разбивает данные # на фрагменты, подходящие для упаковки в
# качестве ресурса. #
#####

import os
import sys
import time
import threading
import argparse
import RNS
import RNS.vendor.umsgpack as umsgpack

# Давайте определим имя приложения. Мы будем
# использовать его для всех # создаваемых нами пунктов
# назначения. Поскольку этот пример с echo
# является частью ряда примеров утилит, мы поместим
# все они находятся в пространстве имен приложения «example_utilities»
APP_NAME = "example_utilities"

# Мы также определим время ожидания по умолчанию в секундах
APP_TIMEOUT = 45.0

##### Часть сервера #####

serve_path = None

# Эта инициализация выполняется, когда пользователь выбирает
# запуск в качестве сервера
def server(configpath, path):
    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Случайным образом создаем новую идентичность для нашего
    # файлового сервера
    server_identity = RNS.Identity()

    global serve_path
    serve_path = path

    # Мы создаем пункт назначения, к которому могут подключаться
    # клиенты. Мы # хотим, чтобы клиенты создавали ссылки на этот
    # пункт назначения, поэтому нам # нужно создать тип пункта
    # назначения «single».

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

server_destination = RNS.Destination(
    server_identity, RNS.Destination.IN,
    RNS.Destination.SINGLE,
    APP_NAME,
    "filetransfer",
    "server"
)

# Настраиваем функцию, которая будет вызываться каждый раз, #
# когда новый клиент устанавливает соединение с этим пунктом
# назначения.

server_destination.set_link_established_callback(client_connected)

# Все готово!
# Подождем запросов от клиентов или ввода пользователя

announceLoop(server_destination)

def announceLoop(destination):
    # Сообщим пользователю, что все готово
    RNS.log("Файловый сервер "+RNS.prettyhexrep(destination.hash)+" запущен") RNS.log("Нажмите Enter, чтобы вручную отправить объявление (Ctrl-C для выхода)")

    # Мы входим в цикл, который будет выполняться до тех пор, пока пользователь не выйдет из программы.
    # Если пользователь нажмет Enter, мы объявим наш сервер #
    # в сети, что позволит клиентам # узнать, как создавать
    # сообщения, адресованные ему.
    while True:
        entered = input() destination.announce()
        RNS.log("Отправлено объявление : от "+RNS.prettyhexrep(destination.hash))

# Вот удобная функция для вывода списка всех файлов # в
# нашем обслуживаемом каталоге
def list_files():
    # Мы добавляем все записи из каталога, которые являются
    # являются фактическими файлами и не начинаются с «.»
    global serve_path
    return [file for file in os.listdir(serve_path) if os.path.isfile(os.path.join(serve_
    ↪path, file)) and file[:1] != "."]

# Когда клиент устанавливает соединение с нашим
# сервером # назначения, эта функция будет вызвана с
# ссылкой на ссылку. Затем мы отправляем клиенту #
# список файлов, размещённых на сервере.
def client_connected(link):
    # Проверяем, существует ли каталог, обслуживаемый сервером
    if os.path.isdir(serve_path):
        RNS.log("Клиент подключен, отправляем список файлов...")

        link.set_link_closed_callback(client_disconnected)

        # Мы упаковываем список файлов для отправки в пакет

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

data = umsgpack.packb(list_files())

# Проверяем размер упакованных данных
если len(data) <= RNS.Link.MDU:
    # Если данные помещаются в один пакет, мы просто
    # отправим его одним пакетом по каналу. list_packet =
    RNS.Packet(link, data) list_receipt = list_packet.send()
    list_receipt.set_timeout(APP_TIMEOUT)
    list_receipt.set_delivery_callback(list_delivered) list_receipt.set_timeout_callback(list_timeout)
else:
    RNS.log("Слишком много файлов в обслуживаемом каталоге!", RNS.LOG_ERROR)
    RNS.log("Необходимо реализовать функцию для разбиения
    ↪ списка файлов на несколько _
    ↪ пакетов.", RNS.LOG_ERROR)
    RNS.log("Подсказка: клиент уже поддерживает эту функцию :)", RNS.LOG_ERROR)

    # После этого мы просто оставим соединение #
    # открытым, пока клиент не запросит файл. Мы #
    # настроим функцию, которая вызывается, когда
    # клиент отправляет пакет с запросом на файл.
    link.set_packet_callback(client_request)
else:
    RNS.log("Клиент подключен, но обслуживаемый путь больше не существует!", RNS.LOG_ERROR) link.teardown()

def client_disconnected(link): RNS.log("Клиент
отключился")

def client_request(message, packet):
    global serve_path

    try:
        filename = message.decode("utf-8")
    except Exception as e: filename =
        None

    if filename in list_files():
        try:
            # Если у нас есть запрошенный файл, мы
            # прочитаем его и упакуем в качестве ресурса
            RNS.log("Клиент запросил \"\"+filename+"\"")
            file = open(os.path.join(serve_path, filename), "rb")

            file_resource = RNS.Resource( file,
                packet.link, callback=resource_sending_concluded
            )

            file_resource.filename = filename
        except Exception as e:

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

        # Если что-то пошло не так, мы
        закрываем # ссылку
        RNS.log("Ошибка при чтении файла \""+filename+"\"", RNS.LOG_ERROR) packet.link.teardown()
        raise e

    else:
        # Если у нас его нет, мы закрываем соединение
        RNS.log("Клиент запросил неизвестный файл")
        packet.link.teardown()

# Эта функция вызывается на сервере по завершении #
передачи ресурса.
    def resource_sending_concluded(resource):
        if hasattr(resource, "filename"): name =
            resource.filename
        else:
            name = "resource"

        if resource.status == RNS.Resource.COMPLETE: RNS.log("Отправка
            \""+name+"\" клиенту завершена")
        elif resource.status == RNS.Resource.FAILED: RNS.log("Отправка
            \""+name+"\" клиенту не удалась")

def list_delivered(receipt):
    RNS.log("Список файлов получен клиентом")

def list_timeout(receipt):
    RNS.log("Истекло время ожидания отправки списка клиенту, закрытие этого
    соединения") link = receipt.destination
    link.teardown()

#####          #####      Часть
клиента          #####
#####

# Мы храним глобальный список файлов, доступных на сервере
server_files          = []

# Ссылка на сервер
server_link          = None

# И ссылка на текущую загрузку current_download =
None current_filename = None

# Переменные для хранения статистики загрузки
download_started          = 0
загрузка_завершена = 0
download_time          = 0
размер_передачи          = 0
размер_файла          = 0

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# Эта инициализация выполняется, когда пользователь #
выбирает запуск в качестве клиента
def client(destination_hexhash, configpath):
    # Нам нужно двоичное представление
    # хеша, введенного в командной строке
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Длина адреса назначения неверна, должна состоять из {hex} шестнадцатеричных символов (
↳ {byte} байт)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)
    за исключением:
        RNS.log("Введен неверный адрес назначения. Проверьте ввод!\n") sys.exit(0)

    # Сначала необходимо инициализировать Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Проверим, знаем ли мы путь к месту назначения
    if not RNS.Transport.has_path(destination_hash):
        RNS.log("Пункт назначения пока неизвестен. Запрашиваем путь и
↳ ждем объявления...")
        RNS.Transport.request_path(destination_hash)
        пока RNS.Transport.has_path(destination_hash) не равно 0:
            time.sleep(0.1)

    # Получить идентификатор сервера
    server_identity = RNS.Identity.recall(destination_hash)

    # Сообщить пользователю, что мы начнём установку соединения
    RNS.log("Устанавливаем соединение с сервером...")

    # Когда идентификатор сервера известен, мы
    настраиваем # пункт назначения
    server_destination = RNS.Destination(
        server_identity, RNS.Destination.OUT,
        RNS.Destination.SINGLE,
        APP_NAME,
        "filetransfer",
        "server"
    )

    # Мы также хотим автоматически проверять входящие пакеты
    server_destination.set_proof_strategy(RNS.Destination.PROVE_ALL)

    # И создать ссылку

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

link = RNS.Link(server_destination)

# Мы ожидаем, что любые обычные пакеты данных на
канале # будут содержать список обслуживаемых
файлов, поэтому мы # соответствующим образом
настраиваем обратный вызов
link.set_packet_callback(filelist_received)

# Мы также настроим функции для уведомления
# пользователя, когда канал установлен или закрыт
link.set_link_established_callback(link_established)
link.set_link_closed_callback(link_closed)

# И настроим канал на автоматический запуск #
загрузки объявленных ресурсов
link.set_resource_strategy(RNS.Link.ACCEPT_ALL)
link.set_resource_started_callback(download_began)
link.set_resource_concluded_callback(download_concluded)

menu()

# Запрашивает указанный файл с сервера
def download(filename):
    global server_link, menu_mode, current_filename, transfer_size, download_started
    current_filename = filename
    download_started = 0
    размер_передачи = 0

    # Мы просто создаем пакет, содержащий
    # запрашиваемое имя файла, и отправляем его по #
    каналу. Мы также указываем, что нам не
    требуется
    # подтверждения получения пакета.
    request_packet = RNS.Packet(server_link, filename.encode("utf-8"), create_
    receipt=False)
    request_packet.send()

    print("")
    print(("Запрашивается \""+filename+"\" с сервера, ожидание начала загрузки..."))
    menu_mode = "download_started"

# Эта функция запускает простое меню, в котором
пользователь # может выбрать файлы для загрузки или
выйти
menu_mode = None
def menu():
    global server_files, server_link
    # Ждем, пока не появится список файлов
    while len(server_files) == 0:
        time.sleep(0.1)
    RNS.log("Готово!")
    time.sleep(0.5)

    global menu_mode
    menu_mode = "main"

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

should_quit = False while (not
should_quit):
    print_menu()

    пока menu_mode не равно "main":
        # Ожидание
        time.sleep(0.25)

    user_input = input()
    if user_input == "q" or user_input == "quit" or user_input == "exit": should_quit = True
        print("")
    else:
        if user_input in server_files:
            download(user_input)
        else:
            try:
                if 0 <= int(user_input) < len(server_files): download(server_files[int(user_input)])
            except:
                пропустить

    if should_quit:
        server_link.teardown()

# Выводит меню или экраны для
# различных состояний клиентской
# программы. # Это просто и довольно
# неинтересно.
# Здесь я не буду вдаваться в
# подробности. В основном # это просто
# строки.
def print_menu():
    global menu_mode, download_time, download_started, download_finished, transfer_size, ↵
    ↵file_size

    if menu_mode == "main":
        clear_screen()
        print_filelist() print("")
        print("Выберите файл для загрузки, введя имя или номер, или q для выхода") print(("> "), end=' ')
    elif menu_mode == "download_started":
        download_began = time.time()
        пока menu_mode == "download_started": time.sleep(0.1)
            if time.time() > download_began + APP_TIMEOUT: print("Время
ожидания загрузки истекло") time.sleep(1)
            server_link.teardown()

    if menu_mode == "downloading":
        print("Загрузка началась") print("")

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

пока menu_mode == "загрузка":
    global current_download
    percent = round(current_download.get_progress() * 100.0, 1)
    print("\rПрогресс: "+str(percent)+" %", end=' ')
    sys.stdout.flush()
    time.sleep(0.1)

if menu_mode == "save_error": print("\rПрогресс: 100,0
%", end=' ') sys.stdout.flush()
print("")
print("Не удалось записать загруженный файл на диск")
current_download.status = RNS.Resource.FAILED menu_mode =
"download_concluded"

if menu_mode == "download_concluded":
    if current_download.status == RNS.Resource.COMPLETE:
        print("\rПрогресс: 100.0 %"), end=' ') sys.stdout.flush()

        # Вывести статистику
        часы, остаток = divmod(download_time, 3600) минуты, секунды =
divmod(остаток, 60)
timestring = "{:0>2};{:0>2};{:05.2f}".format(int(hours),int(minutes),seconds) print("")
print("")
print("--- Статистика -----")
print("\tВремя, затраченное : "+timestring) print("\tРазмер
файла : "+size_str(file_size))
print("\tПереданные данные : "+size_str(transfer_size))
print("\tФактическая скорость : "+size_str(file_size/download_time, suffix='b')+

"/s") print("\tСкорость передачи : "+size_str(transfer_size/download_time, suffix='b' print("")
print("Загрузка завершена! Нажмите Enter, чтобы вернуться в меню.")
print("") input()

else:
    print("")
    print("Загрузка не удалась! Нажмите Enter, чтобы вернуться в меню.") input()

current_download = None
menu_mode = "main"
print_menu()

# Эта функция выводит список файлов # на
подключенном сервере.
def print_filelist():
    global server_files

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

print("Файлы на сервере:")
for index, file in enumerate(server_files): print("\t(" + str(index) +
"\t)" + file)

def filelist_received(filelist_data, packet):
    global server_files, menu_mode
    try:
        # Распаковать список и расширить наши
        # локальный список доступных файлов
        filelist = umsgpack.unpackb(filelist_data)
        for file in filelist:
            if not file in server_files:
                server_files.append(file)

        # Если меню уже отображается, #
        # мы обновим его тем, что
        # только что получено
        if menu_mode == "main": print_menu()
    except:
        RNS.log("Получены неверные данные списка файлов, закрытие соединения")
        packet.link.teardown()

# Эта функция вызывается при установке #
# соединения с сервером
def link_established(link):
    # Мы сохраняем ссылку на экземпляр соединения
    # экземпляра для дальнейшего
    # использования global server_link
    server_link = link

    # Сообщаем пользователю, что #
    # установлено соединение с сервером
    RNS.log("Соединение с сервером установлено")
    RNS.log("Ожидание списка файлов...")

    # И настроим небольшую задачу для проверки
    # возможного превышения времени ожидания
    # при получении # списка файлов
    thread = threading.Thread(target=filelist_timeout_job, daemon=True) thread.start()

# Этот процесс просто находится в режиме
# ожидания в течение указанного # времени, а
# затем проверяет, был ли получен список
# файлов. # Если нет, программа # завершит
# работу.
def filelist_timeout_job(): time.sleep(APP_TIMEOUT)

    global server_files
    if len(server_files) == 0:
        RNS.log("Истекло время ожидания списка файлов, завершаю работу")

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

sys.exit(0)

# Когда соединение закрывается, мы сообщим об
этом # пользователю и завершим работу
программы
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT: RNS.log("Истекло
        время ожидания соединения, завершаем работу")
    elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED: RNS.log("Соединение
        было закрыто сервером, завершаем работу")
    else:
        RNS.log("Соединение закрыто, завершаем работу")

    time.sleep(1.5)
    sys.exit(0)

# Когда RNS определит, что загрузка # началась, мы
обновим состояние меню
#, чтобы пользователю можно было показать ход
# загрузки.
def download_began(resource):
    global menu_mode, current_download, download_started, transfer_size, file_size
    current_download = resource

    if download_started == 0: download_started =
        time.time()

    transfer_size += resource.size
    file_size =
    resource.total_size

    menu_mode = "загрузка"

# Когда загрузка завершится, успешно # или нет, мы
обновим состояние нашего меню и
# сообщить пользователю о том, как всё прошло.
def download_concluded(resource):
    global menu_mode, current_filename, download_started, download_finished, download_
    ↪time
    download_finished = time.time()
    download_time = download_finished - download_started

    saved_filename = current_filename

    if resource.status == RNS.Resource.COMPLETE: counter = 0
        пока os.path.isfile(saved_filename): counter += 1
            saved_filename = current_filename + "." + str(counter)

    try:
        file = open(saved_filename, "wb")
        file.write(resource.data.read())

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

        file.close()
        menu_mode = "download_concluded"

    за исключением:
        menu_mode = "save_error"

else:
    menu_mode = "download_concluded"

# Удобная функция для вывода размера файла в#
# удобочитаемом виде
def size_str(num, suffix='B'):
    units = ['','Ki','Mi','Gi','Ti','Pi','Ei','Zi']
    last_unit = 'Yi'

    if suffix == 'b': num *= 8
    units = ['','K','M','G','T','P','E','Z']
    last_unit = 'Y'

    for unit in units:
        if abs(num) < 1024.0:
            return "%3.2f%s%s" % (num, unit, suffix) num /= 1024.0

    return "%.2f%s%s" % (num, last_unit, suffix)

# Удобная функция для очистки экрана
def clear_screen():
    os.system('cls' if os.name=='nt' else 'clear')

##### Запуск
#####
#####

# Эта часть программы запускается при запуске,
# и анализирует ввод пользователя, а затем # запускает
# нужный режим программы.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(
            description="Простая утилита сервера и клиента для передачи файлов"
        )

        parser.add_argument("-s",
            "--serve",
            action="store",
            metavar="dir",
            help="предоставить каталог с файлами клиентам"
        )

        parser.add_argument("--
            config",
            action="store",

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

        по умолчанию=None,
        help="путь к альтернативному каталогу конфигурации Reticulum", type=str
    )

    parser.add_argument(
        "destination", nargs="?",
        default=None,
        help="шестнадцатеричный хеш адреса назначения сервера", type=str
    )

    args = parser.parse_args()

    if args.config:
        configarg = args.config
    else:
        configarg = None

    if args.serve:
        if os.path.isdir(args.serve): server(configarg,
            args.serve)
        else:
            RNS.log("Указанный каталог не существует")
    else:
        if (args.destination == None): print("")
            parser.print_help() print("")
        else:
            client(args.destination, configarg)

except KeyboardInterrupt: print("")
    sys.exit(0)

```

Этот пример также можно найти по адресу <https://github.com/markqvist/Reticulum/blob/master/Examples/Filetransfer.py>.

11.11 Пользовательские интерфейсы

Пример *ExampleInterface* демонстрирует создание пользовательских интерфейсов для Reticulum. Reticulum может загружать и использовать любое количество пользовательских интерфейсов, которые будут полностью равноценны встроенным интерфейсам, включая все поддерживаемые режимы интерфейса и общие параметры конфигурации.

*# В этом примере показано создание пользовательского интерфейса
 #, который может быть загружен и использован Reticulum во время
 # выполнения. Можно создать и загрузить любое количество
 пользовательских интерфейсов. Чтобы использовать интерфейс,
 поместите его в папку #~/.reticulum/interfaces и добавьте запись об
 интерфейсе в # файл конфигурации Reticulum, например:*

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# [[Пример пользовательского
интерфейса]] # type =
ExampleInterface
# включено = нет
# режим = шлюз
# port = /dev/ttyUSB0 #
# скорость = 115200
# databits = 8
# паритет = нет
# стоп-биты = 1

from time import sleep
import sys
import threading
import time

# Этот вспомогательный класс HDLC используется
интерфейсом # для разграничения и пакетирования данных
по физическому # носителю — в данном случае по
последовательному соединению.
class HDLC():
    # В этом примере интерфейс пакетирует данные, используя
    # упрощенную структуру HDLC, аналогичную PPP
    FLAG = 0x7E
    ESC = 0x7D
    ESC_MASK = 0x20

    @staticmethod
    def escape(data):
        data = data.replace(bytes([HDLC.ESC]), bytes([HDLC.ESC, HDLC.ESC^HDLC.ESC_MASK])) data =
        data.replace(bytes([HDLC.FLAG]), bytes([HDLC.ESC, HDLC.FLAG^HDLC.ESC_
        (→MASK)]))

        return data

# Давайте определим наш собственный класс интерфейса.
Он должен # быть подклассом класса RNS «Interface». class
ExampleInterface(Interface):
    # Все классы интерфейсов должны определять по умолчанию
    # Размер IFAC, используемый при настройке IFAC, если
    пользователь # не указал собственный размер IFAC.
    Этот # параметр указывается в байтах.
    DEFAULT_IFAC_SIZE = 8

    # Следующие свойства относятся к данной #
    конкретной реализации интерфейса.
    owner = None
    port = None
    speed = None
    databits = None
    parity = None
    stopbits = None
    serial = None

    # Все интерфейсы Reticulum должны иметь инициализацию __ ____

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# Метод, принимающий 2 позиционных аргумента:
# Экземпляр RNS Transport владельца и словарь # значений
# конфигурации.
__def init(self, owner, configuration):__ :

    # Следующие строки демонстрируют обработку #
    # потенциальных зависимостей, необходимых для #
    # правильной работы интерфейса.
    import importlib
    if importlib.util.find_spec('serial') != None: import
        serial
    else:
        RNS.log("Для использования этого интерфейса требуется,
        чтобы модуль последовательной связи был
        ↪ установлен.", RNS.LOG_CRITICAL)
        RNS.log("Вы можете установить его с помощью команды: python3 -m pip
        ↪ install_
        ↪ pyserial", RNS.LOG_CRITICAL)
        RNS.panic()

    # Начнем с инициализации суперкласса
    super().__init__()

    # Чтобы убедиться, что данные конфигурации имеют
    # правильный формат, мы анализируем их с помощью
    # следующего # метода в общем классе Interface. Этот шаг #
    # необходим для обеспечения совместимости на всех
    # платформах, поддерживаемых Reticulum.

    ifconf = Interface.get_config_obj(configuration)

    # Считаем имя интерфейса из конфигурации # и
    # присваиваем его нашему экземпляру интерфейса.
    name = ifconf["name"]
    self.name = name

    # Мы считываем параметры конфигурации из предоставленных #
    # данных конфигурации и задаем значения по умолчанию в # случае,
    # если какие-либо из них отсутствуют.
    port = ifconf["port"] if "port" in ifconf else None
    скорость = int(ifconf["speed"]) if "speed" in ifconf else 9600
    databits = int(ifconf["databits"]) if "databits" in ifconf else 8
    parity = ifconf["parity"] if "parity" in ifconf else "N"
    стоп-биты = int(ifconf["stopbits"]) if "stopbits" in ifconf
    else 1

    # Если порт не указан, мы прерываем настройку, # вызывая
    # исключение.
    if port == None:
        raise ValueError(f"Порт для {self} не указан")

    # Все интерфейсы должны передавать значение аппаратного
    # MTU # экземпляру транспорта RNS. Это значение должно #
    # соответствовать максимальному размеру полезной нагрузки
    # пакета данных, который # базовая среда способна
    # обрабатывать во всех # случаях без какой-либо сегментации.

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

self.HW_MTU = 564

# Изначально мы устанавливаем значение свойства
«online» равным false, # поскольку интерфейс ещё не был
полностью #инициализирован и подключен.

self.online      = False

# В этом случае мы также можем установить указанную
скорость передачи данных интерфейса в соответствии
со скоростью последовательного порта.

self.bitrates    = speed

# Настройте внутренние свойства интерфейса # в
соответствии с предоставленной конфигурацией.
self.pyserial = serial self.serial
                = None
self.owner      = owner
self.port       = port
self.speed      = speed
self.databits   = databits
self.parity     = serial.PARITY_NONE
self.stopbits   = stopbits self.timeout =
100

if parity.lower() == "e" or parity.lower() == "even": self.parity =
serial.PARITY_EVEN

if parity.lower() == "o" or parity.lower() == "odd": self.parity =
serial.PARITY_ODD

# Поскольку все необходимые параметры теперь настроены,
# попробуем открыть последовательный порт.
try:
    self.open_port()
except Exception as e:
    RNS.log("Не удалось открыть последовательный порт для интерфейса "+str(self), RNS.LOG_ERROR)
    raise e

# Если порт удалось открыть, выполните любую
необходимую конфигурацию # после открытия.
if self.serial.is_open: self.configure_device()
else:
    raise IOError("Не удалось открыть последовательный порт")

# Открыть последовательный порт с заданной конфигурацией
# параметры и сохранить ссылку на открытый порт.
def open_port(self):
    RNS.log("Открытие последовательного порта "+self.port+"...", RNS.LOG_VERBOSE)
    self.serial = self.pyserial.Serial(
        port = self.port, baudrate =
        self.speed, bytesize =
        self.databits,

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

        parity = self.parity, stopbits =
        self.stopbits, xonxoff = False,
        rtscts = False, timeout =
        0,
        inter_byte_timeout = None,
        write_timeout = None, dsrdtr =
        False,
    )

    # Единственное, что требуется после открытия порта, #
    — это немного подождать, пока
    # инициализации оборудования, а затем запустить поток,
    # который будет считывать входящие данные с
    устройства.
    def configure_device(self): sleep(0.5)
        thread = threading.Thread(target=self.read_loop) thread.daemon =
        True
        thread.start()
        self.online = True

        RNS.log("Последовательный порт "+self.port+" открыт", RNS.LOG_VERBOSE)

    # Этот метод будет вызван из нашего цикла чтения #
    всякий раз, когда через базовый канал будет получен #
    полный пакет.
    def process_incoming(self, data):
        # Обновим счетчик принятых байтов

        self.rxb += len(data)

        # И отправка пакета данных экземпляру # Transport
        для обработки.

        self.owner.inbound(data, self)

    # Работающий экземпляр Reticulum Transport будет #
    вызывать этот метод на интерфейсе всякий раз, когда #
    интерфейсу необходимо передать пакет.
    def process_outgoing(self, data):
        if self.online:
            # Сначала преобразуем и пакетируем данные
            # в соответствии с форматированием HDLC.

            data = bytes([HDLC.FLAG])+HDLC.escape(data)+bytes([HDLC.FLAG])

            # Затем записываем сформированные данные в порт

            written = self.serial.write(data)

            # Обновим счетчик переданных байтов # и
            убедимся, что все данные были записаны
            self.txb += len(data)
            if written != len(data):
                raise IOError("По последовательному интерфейсу записано только "+str(written)+" байт из
                "+str(len(data)))

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

# Этот цикл чтения работает в потоке и непрерывно #
# принимает байты из базового последовательного порта. #
# Когда будет получен полный пакет, он
# отправляется в метод process_incoming, который
# который, в свою очередь, передаст его экземпляру Transport.
def read_loop(self):
    try:
        in_frame = False escape =
        False data_buffer = b""
        last_read_ms = int(time.time()*1000)

        while self.serial.is_open:
            if self.serial.in_waiting:
                byte = ord(self.serial.read(1)) last_read_ms =
                int(time.time()*1000)

                if (in_frame and byte == HDLC.FLAG): in_frame =
                    False self.process_incoming(data_buffer)
                elif (byte == HDLC.FLAG):
                    in_frame = True data_buffer =
                    b""
                elif (in_frame и len(data_buffer) < self.HW_MTU):
                    if (byte == HDLC.ESC): escape =
                        True
                    else:
                        if (escape):
                            if (byte == HDLC.FLAG ^ HDLC.ESC_MASK): byte =
                                HDLC.FLAG
                            if (byte == HDLC.ESC ^ HDLC.ESC_MASK):
                                byte = HDLC.ESC escape =
                                False
                        data_buffer = data_buffer+bytes([byte])

            else:
                time_since_last = int(time.time()*1000) - last_read_ms
                if len(data_buffer) > 0 and time_since_last > self.timeout: data_buffer = b""
                in_frame = False
                escape = False
                sleep(0.08)

        except Exception as e:
            self.online = False
            RNS.log("Произошла ошибка последовательного порта, содержащееся исключение: "+str(e),
↳ RNS.LOG_ERROR)
            RNS.log("В интерфейсе "+str(self)+" произошла неустраняемая ошибка,
и
↳ теперь находится в автономном режиме.", RNS.LOG_ERROR)

            if RNS.Reticulum.panic_on_interface_error:

```

(продолжение на следующей странице)

(продолжение с предыдущей страницы)

```

RNS.panic()

RNS.log("Reticulum б у д е т п е р и о д и ч е с к и п ы т а т ь с я
п е р е п о д к л ю ч и т ь и н т е р ф е й с.", _
↳RNS.LOG_ERROR)

self.online = False
self.serial.close()
self.reconnect_port()

# Этот метод обрабатывает отключение последовательного порта.
def reconnect_port(self):
    while not self.online:
        try:
            time.sleep(5)
            RNS.log("Попытка повторного подключения последовательного порта "+str(self.port)+" для
↳"+str(self)+"...", RNS.LOG_VERBOSE)
            self.open_port()
            if self.serial.is_open:
                self.configure_device()
        except Exception as e:
            RNS.log("Ошибка при повторном подключении порта, исключение:
↳"+str(e), RNS.LOG_ERROR)

RNS.log("Восстановлено соединение с последовательным портом для " + str(self))

# Сигнал для Reticulum о том, что этот интерфейс # не
должен выполнять ограничение входящего трафика.
def should_ingress_limit(self):
    return False

# Мы должны предоставить строковое представление
этого # интерфейса, которое используется всякий раз,
когда интерфейс # выводится в журналах или внешних
программах.
def __str__(self):
    return "ExampleInterface["+self.name+"]"

# Наконец, зарегистрируйте определённый класс интерфейса в
качестве # целевого класса, который Reticulum будет
использовать в качестве интерфейса
interface_class = ExampleInterface

```

Этот пример также можно найти в файле <https://github.com/markqvist/Reticulum/blob/master/Examples/ExampleInterface.py>.

ЛИЦЕНЗИЯ RETICULUM

Лицензия Reticulum

Авторские права (с) 2016-2026 Марк Квист

Настоящим любому лицу, получившему копию данного программного обеспечения и сопутствующей документации (далее — «Программное обеспечение»), бесплатно предоставляется право распоряжаться Программным обеспечением без каких-либо ограничений, включая, помимо прочего, права на использование, копирование, изменение, объединение, публикацию, распространение, сублицензирование и/или продажу копий Программного обеспечения, а также право разрешать лицам, которым предоставляется Программное обеспечение, совершать аналогичные действия при соблюдении следующих условий:

- Программное обеспечение не должно использоваться в каких-либо системах, функции которых включают в себя способность целенаправленно причинять вред людям.
- Программное обеспечение не должно использоваться, прямо или косвенно, для создания наборов данных для обучения искусственного интеллекта, машинного обучения или языковых моделей, включая, помимо прочего, любое использование, способствующее обучению или разработке таких моделей или алгоритмов.
- Вышеуказанное уведомление об авторских правах и настоящее разрешение должны быть включены во все копии Программного обеспечения или его существенные части.

ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ПРЕДОСТАВЛЯЕТСЯ «КАК ЕСТЬ», БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ, ЯВНЫХ ИЛИ ПОДРАЗУМЕВАЕМЫХ, ВКЛЮЧАЯ, ПОМИМО ПРОЧЕГО, ГАРАНТИИ ТОВАРНОГО СОСТОЯНИЯ, ПРИГОДНОСТИ ДЛЯ ОПРЕДЕЛЕННОЙ ЦЕЛИ И НЕНАРУШЕНИЯ ПРАВ. НИ В КОЕМ СЛУЧАЕ АВТОРЫ ИЛИ ВЛАДЕЛЬЦЫ АВТОРСКИХ ПРАВ НЕ НЕСУТ ОТВЕТСТВЕННОСТИ ЗА ЛЮБЫЕ ПРЕТЕНЗИИ, УБЫТКИ ИЛИ ИНУЮ ОТВЕТСТВЕННОСТЬ, БУДЬ ТО В РАМКАХ ИСКА О НАРУШЕНИИ ДОГОВОРА, ДЕЛИКТА ИЛИ ИНОГО, ВОЗНИКШИЕ ИЗ-ЗА, В РЕЗУЛЬТАТЕ ИЛИ В СВЯЗИ С ПРОГРАММНЫМ ОБЕСПЕЧЕНИЕМ ИЛИ ИСПОЛЬЗОВАНИЕМ ИЛИ ИНЫМИ ДЕЙСТВИЯМИ В ОТНОШЕНИИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.

СПРАВОЧНИК ПО API

Обмен данными в сетях Reticulum осуществляется с помощью простого набора классов, предоставляемых API RNS. В этой главе перечислены и описаны все классы, предоставляемые API сетевого стека Reticulum, а также сигнатуры их методов и способы использования. Ее можно использовать в качестве справочника при написании приложений, использующих Reticulum, или прочитать целиком, чтобы получить представление о полном функционале RNS с точки зрения разработчика.

13.1 Reticulum

```
class RNS.Reticulum(configdir=None, loglevel=None, logdest=None, verbosity=None,  
                   require_shared_instance=False, shared_instance_type=None)
```

Этот класс используется для инициализации доступа к Reticulum в рамках программы. Перед выполнением любых других операций RNS, таких как создание пунктов назначения или отправка трафика, необходимо создать ровно один экземпляр этого класса. Каждая независимо выполняемая программа должна создавать свой собственный экземпляр класса Reticulum, однако Reticulum автоматически обеспечит межпрограммную коммуникацию в пределах одной системы, а также предоставит доступ ко всем подключенным программам через внешние интерфейсы.

Как только экземпляр этого класса будет создан, Reticulum начнет открывать и настраивать любые аппаратные устройства, указанные в предоставленной конфигурации.

В настоящее время первый запущенный экземпляр должен оставаться активным, пока к нему подключены другие локальные экземпляры, поскольку первый созданный экземпляр выступает в роли главного экземпляра, который напрямую взаимодействует с внешним оборудованием, таким как модемы, TNC и радиостанции. Если главный экземпляр получает запрос на завершение работы, он не завершит работу до тех пор, пока не будут завершены все клиентские процессы (если только его не принудительно не завершить).

Если вы запускаете Reticulum в системе с несколькими разными программами, использующими RNS, которые запускаются и завершаются в разное время, будет удобно запустить главный экземпляр RNS в качестве демона, чтобы другие программы могли использовать его по требованию.

MTU = 500

Значение MTU, которое использует Reticulum и которое, как ожидается, будут использовать другие узлы. По умолчанию значение MTU составляет 500 байт. В пользовательских реализациях сетей RNS это значение можно изменить, однако это приведёт к полной потере совместимости со всеми другими сетями RNS. Идентичное значение MTU является обязательным условием для взаимодействия узлов в одной сети.

Если вы не уверены в том, что делаете, MTU следует оставить на значении по умолчанию.

LINK_MTU_DISCOVERY = True

Включено ли по умолчанию в данной версии автоматическое определение MTU канала. Определение MTU канала значительно повышает пропускную способность на высокоскоростных каналах, но требует, чтобы все промежуточные узлы также поддерживали эту функцию. Поддержка этой функции была добавлена в версии RNS 0.9.0. В ближайшем будущем этот параметр будет включен по умолчанию. Обновите свои экземпляры RNS.

ANNOUNCE_CAP = 2

Максимальный процент пропускной способности интерфейса, который в любой момент времени может быть использован для распространения объявлений. Если объявление было запланировано для широковещательной передачи по интерфейсу, но это привело бы к превышению разрешенного выделения пропускной способности, объявление будет поставлено в очередь для передачи, когда появится доступная пропускная способность.

Reticulum всегда будет отдавать приоритет распространению объявлений с меньшим количеством прыжков, гарантируя, что удаленные, крупные сети со многими пирами на быстрых каналах не перегрузят пропускную способность меньших сетей на более медленных средах. Если объявление остается в очереди в течение длительного времени, оно в конечном итоге будет удалено.

Это значение будет применяться по умолчанию ко всем создаваемым интерфейсам, но его можно настроить индивидуально для каждого интерфейса. Как правило, глобальную настройку по умолчанию не следует изменять; любые изменения следует вносить на уровне отдельных интерфейсов.

MINIMUM_BITRATE = 5

Минимальная скорость передачи данных, требуемая по каналу связи для успешного установления соединений Reticulum. В настоящее время составляет 5 бит в секунду.

static get_instance()

Возвращает текущий запущенный экземпляр Reticulum

static should_use_implicit_proof()

Возвращает, являются ли отправленные доказательства явными или неявными.

Возвращает

True, если текущая конфигурация запуска указывает на использование неявных доказательств. False, если нет.

static transport_enabled()

Возвращает, включен ли Transport для запущенного экземпляра.

Если функция «Транспорт» включена, Reticulum будет маршрутизировать трафик для других узлов, отвечать на запросы о маршрутах и передавать объявления по сети.

Возвращает

True, если Transport включен, False — если нет.

static link_mtu_discovery()

Возвращает, включено ли обнаружение MTU канала для запущенного экземпляра.

Если обнаружение MTU канала включено, Reticulum автоматически обновит MTU каналов до максимального поддерживаемого значения, повысив скорость и эффективность передачи данных.

Возвращает

True, если обнаружение MTU канала включено, False, если нет.

static remote_management_enabled()

Возвращает, включено ли удаленное управление для запущенного экземпляра.

Если включено удаленное управление, авторизованные узлы могут удаленно запрашивать данные и управлять этим экземпляром.

Возвращает

True, если удаленное управление включено, False, если нет.

static required_discovery_value()

Возвращает значение метки, необходимое для того, чтобы обнаруженный интерфейс считался действительным и запоминался.

Возвращает

Требуемое значение метки в виде целого числа.

```
static publish_blackhole_enabled()
```

Возвращает информацию о том, включена ли публикация списка blackhole для запущенного экземпляра.

Возвращает

True, если публикация списка blackhole включена, False — если нет.

```
static blackhole_sources()
```

Возвращает список хэшей идентификаторов транспорта, из которых берутся списки blackhole.

Возвращает

Список хэшей идентификаторов.

```
static discovered_interfaces()
```

Возвращает список интерфейсов, обнаруженных в сети.

Возвращает

Список обнаруженных интерфейсов.

```
static interface_discovery_sources()
```

Возвращает список хэшей сетевых идентификаторов, на основе которых обнаруживаются интерфейсы.

Возвращает

Список хэшей идентификаторов.

13.2 Идентичность

класс `RNS.Identity(create_keys=True)`

Этот класс используется для управления идентификационными данными в Reticulum. Он предоставляет методы шифрования, дешифрования, подписи и проверки и служит основой для всей зашифрованной связи в сетях Reticulum.

Параметры

`create_keys` — Указывает, следует ли генерировать новые ключи шифрования и подписи.

`CURVE = 'Curve25519'`

Кривая, используемая для обмена ключами DH с эллиптической кривой

`KEYSIZE = 512`

Размер ключа X.25519 в битах. Полный ключ представляет собой конкатенацию 256-битного ключа шифрования и 256-битного ключа подписи.

`RATCHETSIZE = 256`

Размер ключа рачета X.25519 в битах.

`RATCHET_EXPIRY = 2592000`

Срок действия полученных значений ratchet в секундах; по умолчанию — 30 дней. Reticulum всегда будет использовать самое последнее объявленное значение ratchet и сохранять его в памяти в течение периода, равного RATCHET_EXPIRY с момента получения, после чего оно будет удалено. Если в это время будет объявлено более новое значение ratchet, оно заменит уже известное ратчет.

`TRUNCATED_HASHLENGTH = 128`

Константа, определяющая длину усеченного хеша (в битах), используемого Reticulum для адресуемых хешей и других целей. Не настраивается.

```
static recall(target_hash, from_identity_hash=False)
```

Получить идентификатор по адресу назначения или хэшу идентификатора. По умолчанию эта функция возвращает идентификатор, связанный с заданным хэшем *адреса назначения*. Например, если вам известен хэш адреса назначения lxmfdelivery конечной точки, эта функция вернет связанный с ним базовый идентификатор. Вы также можете найти идентификатор по известному *хэшу идентификатора*, указав аргумент `from_identity_hash`.

Параметры

- `target_hash` — хеш назначения или идентификатора в *байтах*.
- `from_identity_hash` — Указание, следует ли выполнять поиск на основе хеша идентичности вместо хеша назначения, в виде значения *bool*.

Возвращает

Экземпляр *RNS.Identity*, который можно использовать для создания исходящего *RNS.Destination*, или *None*, если пункт назначения неизвестен.

```
static recall_app_data(destination_hash)
```

Получить последние данные `app_data` для целевого хеша.

Параметры

`destination_hash` — хеш назначения в виде *байтов*.

Возвращает

Байты, содержащие `app_data`, или *None*, если адрес назначения неизвестен.

```
static full_hash(data)
```

Получить хеш SHA-256 переданных данных.

Параметры

`data` — данные, которые необходимо хешировать, в виде *байтов*.

Возвращает

Хеш SHA-256 в виде *байтов*.

```
static truncated_hash(data)
```

Получить усечённый хеш SHA-256 переданных данных.

Параметры

`data` — Данные для хеширования в виде *байтов*.

Возвращает

Усечённый хеш SHA-256 в виде *байтов*.

```
static get_random_hash()
```

Получить случайный хеш SHA-256.

Параметры

`data` — Данные для хеширования в виде *байтов*.

Возвращает

Усечённый хеш SHA-256 случайных данных в виде *байтов*.

```
static current_ratchet_id(destination_hash)
```

Получить ID ключа рачета, используемого в данный момент для заданного хэша назначения

Параметры

`destination_hash` — хеш назначения в виде *байтов*.

Возвращает

Идентификатор рачета в виде *байтов* или *None*.

```
static from_bytes(prv_bytes)
```

Создать новый экземпляр *RNS.Identity* из *байтов* закрытого ключа. Может использоваться для загрузки ранее созданных и сохраненных идентификаторов в Reticulum.

Параметры

`prv_bytes` — *байты* сохраненного закрытого ключа. **ОПАСНО!** Никогда не используйте это для генерации нового ключа, вводя случайные данные в `prv_bytes`.

Возвращает

Экземпляр *RNS.Identity* или *None*, если данные в *байтах* были неверны.

`static from_file(path)`

Создание нового экземпляра *RNS.Identity* из файла. Может использоваться для загрузки ранее созданных и сохраненных идентичностей в Reticulum.

Параметры

`path` — полный путь к сохраненным данным *RNS.Identity*

Возвращает

Экземпляр *RNS.Identity* или *None*, если загруженные данные были неверны.

`to_file(path)`

Сохраняет идентификатор в файл. При этом на диск записывается закрытый ключ, и любой, у кого есть доступ к этому файлу, сможет расшифровать всю переписку, связанную с этим идентификатором. Будьте очень осторожны при использовании этого метода.

Параметры

`path` — Полный путь, указывающий, где сохранить идентификатор.

Возвращает

True, если файл был сохранен, в противном случае False.

`get_private_key()`

Возвращает

Личный ключ в виде *байтов*

`get_public_key()`

Возвращает

Открытый ключ в виде *байтов*

`load_private_key(prv_bytes)`

Загрузка закрытого ключа в экземпляр.

Параметры

`prv_bytes` — закрытый ключ в виде *байтов*.

Возвращает

True, если ключ был загружен, в противном случае False.

`load_public_key(pub_bytes)`

Загрузка открытого ключа в экземпляр.

Параметры

`pub_bytes` — Открытый ключ в виде *байтов*.

Возвращает

True, если ключ был загружен, в противном случае False.

`encrypt(plaintext, ratchet=None)`

Шифрует информацию для идентичности.

Параметры

`открытый текст` — открытый текст, подлежащий шифрованию, в виде *байтов*.

Возвращает

Токен шифротекста в виде *байтов*.

Генерирует

KeyError, если экземпляр не содержит открытого ключа.

`decrypt(ciphertext_token, ratchets=None, enforce_ratchets=False, ratchet_id_receiver=None)`

Дешифрует информацию для идентичности.

Параметры

`ciphertext` — шифротекст, который необходимо расшифровать, в виде *байтов*.

Возвращает

Открытый текст в виде *байтов* или *None*, если расшифровка завершилась неудачей.

Вызывает

KeyError, если экземпляр не содержит закрытый ключ.

`sign(message)`

Подписывает информацию с помощью идентификатора.

Параметры

`message` – Сообщение, которое необходимо подписать, в виде *байтов*.

Возвращает

Подпись в виде *байтов*.

Генерирует

KeyError, если экземпляр не содержит закрытый ключ.

`validate(signature, message)`

Проверяет подпись подписанного сообщения.

Параметры

- `signature` – Подпись, подлежащая проверке, в виде *байтов*.
- `message` — проверяемое сообщение в виде *байтов*.

Возвращает

True, если подпись действительна, в противном случае False.

Генерирует

KeyError, если экземпляр не содержит открытого ключа.

13.3 Destination

класс `RNS.Destination(identity, direction, type, app_name, *aspects)`

Класс, используемый для описания конечных точек в сети Reticulum. Экземпляры Destination используются как для создания исходящих, так и входящих конечных точек. Тип конечной точки определяет, будет ли использоваться шифрование при взаимодействии с ней, и какого именно типа. Конечная точка также может объявить о своем присутствии в сети, что приведет к распространению необходимых ключей для зашифрованной связи с ней.

Параметры

- `identity` – экземпляр *RNS.Identity*. Может содержать только открытые ключи для исходящего назначения или закрытые ключи для входящего.
- `direction` – `RNS.Destination.IN` или `RNS.Destination.OUT`.
- `type` – `RNS.Destination.SINGLE`, `RNS.Destination.GROUP` или `RNS.Destination.PLAIN`.
- `app_name` — строка, указывающая имя приложения.
- `*aspects` – любое ненулевое количество строковых аргументов.

RATCHET_COUNT = 512

Количество ключей рачета по умолчанию, которое сохранит пункт назначения, если у него включены рачеты.

RATCHET_INTERVAL = 1800

Минимальный интервал между поворотами ключей храпового механизма, в секундах.

`static expand_name(identity, app_name, *aspects)`

Возвращает

Строку, содержащую полное, понятное человеку имя назначения, для `app_name` и ряда аспектов.

`static app_and_aspects_from_name(full_name)`

Возвращает

Кортеж, содержащий имя приложения и список аспектов, для строки `full_name`.

`static hash_from_name_and_identity(full_name, identity)`

Возвращает

Имя назначения в форме адресуемого хеша для строки полного имени и экземпляра Identity.

`static hash(identity, app_name, *aspects)`

Возвращает

Имя назначения в форме адресуемого хеша для `app_name` и ряда аспектов.

`announce(app_data=None, path_response=False, attached_interface=None, tag=None, send=True)`

Создает пакет объявления для этого адресата и транслирует его по всем соответствующим интерфейсам. К объявлению можно добавить данные, специфичные для приложения.

Параметры

- `app_data` — байты, содержащие `app_data`.
- `path_response` — внутренний флаг, используемый *RNS.Transport*. Игнорировать.

`accepts_links(accepts=None)`

Устанавливает или запрашивает, принимает ли пункт назначения входящие запросы на установку соединения.

Параметры

`accepts` — При значении True или False этот метод задает, принимает ли объект назначения входящие запросы на установку соединения. Если параметр не указан или имеет значение None, метод возвращает информацию о том, принимает ли объект назначения в данный момент запросы на установку соединения.

Возвращает

True или False в зависимости от того, принимает ли пункт назначения входящие запросы на установку соединения, если параметр `accepts` не указан или равен None.

`set_link_established_callback(callback)`

Регистрирует функцию, которая будет вызвана при установке соединения с этим адресатом.

Параметры

`callback` — функция или метод с сигнатурой `callback(link)`, который будет вызван при установке нового соединения с этим адресатом.

`set_packet_callback(callback)`

Регистрирует функцию, которая будет вызываться при получении пакета этим адресатом.

Параметры

`callback` — функция или метод с сигнатурой `callback(data, packet)`, который будет вызван при получении этим адресатом пакета.

`set_proof_requested_callback(callback)`

Регистрирует функцию, которая будет вызываться при запросе доказательства для пакета, отправленного в этот пункт назначения. Позволяет контролировать, когда и следует ли возвращать доказательства для полученных пакетов.

Параметры

`callback` – Функция или метод с сигнатурой `callback(packet)`, вызываемый при получении пакета, запрашивающего доказательство. Функция `callback` должна возвращать значение `True` или `False`. Если `callback` возвращает `True`, доказательство будет отправлено. Если возвращает `False`, доказательство не будет отправлено.

`set_proof_strategy(proof_strategy)`

Устанавливает стратегию подтверждения для адресата.

Параметры

`proof_strategy` – одно из значений: `RNS.Destination.PROVE_NONE`, `RNS.Destination.PROVE_ALL` или `RNS.Destination.PROVE_APP`. Если установлено значение `RNS.Destination.PROVE_APP`, будет вызван `callback proof_requested_callback` для определения, следует ли отправить доказательство или нет.

`register_request_handler(path, response_generator=None, allow=ALLOW_NONE, allowed_list=None, auto_compress=True)`

Регистрирует обработчик запросов.

Параметры

- `path` – Путь для обработчика запросов, который будет зарегистрирован.
- `response_generator` – Функция или метод с сигнатурой `response_generator(path, data, request_id, link_id, remote_identity, requested_at)`, который будет вызван. Все, что возвращает эта функция, будет отправлено в качестве ответа запрашивающему. Если функция возвращает `None`, ответ не будет отправлен.
- `allow` – Одно из значений: `RNS.Destination.ALLOW_NONE`, `RNS.Destination.ALLOW_ALL` или `RNS.Destination.ALLOW_LIST`. Если установлено значение `RNS.Destination.ALLOW_LIST`, обработчик запросов будет отвечать только на запросы от узлов, указанных в предоставленном списке.
- `allowed_list` – Список хэшей `RNS.Identity`, представляющих собой байты.
- `auto_compress` – Если `True` или `False`, определяет, следует ли выполнять автоматическое сжатие ответов. Если установлено целое число, ответы будут автоматически сжиматься сжиматься, если их размер в байтах меньше этого значения. Если параметр опущен, будут использоваться настройки сжатия по умолчанию.

Вызывает

`ValueError`, если какой-либо из переданных аргументов недействителен.

`deregister_request_handler(path)`

Отменяет регистрацию обработчика запросов.

Параметры

`path` – Путь к обработчику запросов, который необходимо отменить.

Возвращает

`True`, если обработчик был удален из списка, в противном случае — `False`.

`enable_ratchets(ratchets_path)`

Включает рачеты для данного адреса. Когда рачеты включены, Reticulum будет автоматически менять ключи, используемые для шифрования пакетов, отправляемых по этому адресу, и включать последний ключ рачета в объявления.

Включение рачет на пункте назначения обеспечит прямую секретность для пакетов, отправляемых в этот пункт назначения, даже если они отправляются за пределы канала. Обычная процедура установления канала Reticulum уже выполняет собственный

обмен эфемеричными ключами при каждом установлении соединения, что означает, что расчеты не нужны для обеспечения секретности пересылки для соединений.

Включение механизма обратной связи незначительно повлияет на размер уведомлений, добавив 32 байта к каждому отправляемому уведомлению.

Параметры

`ratchets_path` — путь к файлу для хранения данных рачет.

Возвращает

True, если операция завершилась успешно, в противном случае — False.

`enforce_ratchets()`

Если принудительное применение рачет-механизма включено, этот пункт назначения никогда не будет принимать пакеты, использующие его базовый ключ Identity для шифрования, а будет принимать только пакеты, зашифрованные одним из сохраненных ключей рачет-механизма.

`set_retained_ratchets(retained_ratchets)`

Устанавливает количество ранее сгенерированных ключей рачета, которые этот пункт назначения будет сохранять и пытаться использовать при расшифровке входящих пакетов. По умолчанию — `Destination.RATCHET_COUNT`.

Параметры

`retained_ratchets` — Количество сгенерированных ключей рачета, которые необходимо сохранить.

Возвращает

True, если операция завершилась успешно, False — в противном случае.

`set_ratchet_interval(interval)`

Устанавливает минимальный интервал в секундах между поворотами ключа храпового механизма.
`Destination.RATCHET_INTERVAL`.

По умолчанию:

Параметры

`interval` — минимальный интервал в секундах.

Возвращает

True, если операция прошла успешно, False — в противном случае.

`create_keys()`

Для пункта назначения типа `RNS.Destination.GROUP` создает новый симметричный ключ.

Вызывает

`TypeError`, если вызвана для несовместимого типа назначения.

`get_private_key()`

Для адреса типа `RNS.Destination.GROUP` возвращает симметричный закрытый ключ.

Вызывает

`TypeError`, если вызов выполнен для несовместимого типа адреса.

`load_private_key(key)`

Для пункта назначения типа `RNS.Destination.GROUP` загружает симметричный закрытый ключ.

Параметры

`key` — объект *muna bytes*, содержащий симметричный ключ.

Вызывает

`TypeError`, если вызвано для несовместимого типа назначения.

`encrypt(plaintext)`

Шифрует информацию для адресата типа `RNS.Destination.SINGLE` или `RNS.Destination.GROUP`.

Параметры

`plaintext` — объект *muna bytes*, содержащий открытый текст, который необходимо зашифровать.

Вызывает

ValueError, если destination не содержит ключ, необходимый для шифрования.

`decrypt(ciphertext)`

Дешифрует информацию для назначения типа RNS.Destination.SINGLE или RNS.Destination.GROUP.

Параметры

шифротекст — *байты*, содержащие шифротекст, который необходимо расшифровать.

Вызывает

ValueError, если destination не содержит ключ, необходимый для расшифровки.

`sign(message)`

Подписывает информацию для адресата типа RNS.Destination.SINGLE.

Параметры

message — *байты*, содержащие сообщение, которое необходимо подписать.

Возвращает

Объект *muna bytes*, содержащий подпись сообщения, или *None*, если пункт назначения не смог подписать сообщение.

`set_default_app_data(app_data=None)`

Устанавливает app_data по умолчанию для адресата. Если установлено, app_data по умолчанию будет включено в каждое объявление, отправленное адресатом, если в методе *announce* не указано другое app_data.

Параметры

app_data — объект *muna bytes*, содержащий данные app_data по умолчанию, или *вызываемая функция*, возвращающая объект *muna bytes*, содержащий данные app_data.

`clear_default_app_data()`

Очищает app_data по умолчанию, ранее установленные для пункта назначения.

13.4 Пакет

`class RNS.Packet(destination, data, create_receipt=True)`

Класс Packet используется для создания экземпляров пакетов, которые можно отправлять по сети Reticulum. Пакеты будут автоматически зашифрованы, если они адресованы пункту назначения RNS.Destination.SINGLE, RNS.Destination.GROUP или *RNS.Link*.

Для адресатов типа RNS.Destination.GROUP Reticulum будет использовать заранее согласованный ключ, настроенный для данного адресата. Все пакеты, направляемые адресатам типа GROUP, шифруются одним и тем же ключом AES-256.

Для адресов RNS.Destination.SINGLE Reticulum будет использовать новый эфемеричный ключ AES-256 для каждого пакета.

Для адресатов *RNS.Link* Reticulum будет использовать эфемеричные ключи для каждого соединения и обеспечивает **передовую секретность**.

Параметры

- destination — экземпляр *RNS.Destination*, которому будет отправлен пакет.
- data — полезные данные, которые должны быть включены в пакет в виде *байтов*.
- create_receipt — Указывает, следует ли создавать объект *RNS.PacketReceipt* при инициализации пакета.

ENCRYPTED_MDU = 383

Максимальный размер данных полезной нагрузки в одном зашифрованном пакете

PLAIN_MDU = 464

Максимальный размер данных полезной нагрузки в одном незашифрованном пакете

send()

Отправляет пакет.

Возвращает

Экземпляр *RNS.PacketReceipt*, если при создании пакета для параметра *create_receipt* было задано значение *True*; в противном случае возвращается *None*. Если пакет не удалось отправить, возвращается *False*.

resend()

Повторно отправляет пакет.

Возвращает

Экземпляр *RNS.PacketReceipt*, если при создании пакета для параметра *create_receipt* было задано значение *True*; в противном случае возвращается *None*. Если пакет не удалось отправить, возвращается *False*.

get_rssi()

Возвращает

Показатель уровня сигнала на физическом уровне, если доступен; в противном случае — *None*.

get_snr()

Возвращает

Коэффициент отношения сигнал/шум физического уровня, если доступен, в противном случае *None*.

get_q()

Возвращает

Качество канала физического уровня, если доступно, в противном случае *None*.

13.5 Получение пакета

класс *RNS.PacketReceipt*

Класс *PacketReceipt* используется для получения уведомлений об экземплярах *RNS.Packet*, отправленных по сети. Экземпляры этого класса никогда не создаются вручную, а всегда возвращаются методом *send()* экземпляра *RNS.Packet*.

get_status()

Возвращает

Статус связанного экземпляра *RNS.Packet*. Может принимать одно из значений: *RNS.PacketReceipt.SENT*, *RNS.PacketReceipt.DELIVERED*, *RNS.PacketReceipt.FAILED* или *RNS.PacketReceipt.CULLED*.

get_rtt()

Возвращает

Время прохождения в обе стороны в секундах

set_timeout(*timeout*)

Устанавливает таймаут в секундах

Параметры

timeout — Тайм-аут в секундах.

`set_delivery_callback(callback)`

Задаёт функцию, которая вызывается при подтверждении успешной доставки.

Параметры

`callback` – *Вызываемый* объект с сигнатурой `callback(packet_receipt)`

`set_timeout_callback(callback)`

Задаёт функцию, которая вызывается в случае истечения времени ожидания доставки.

Параметры

`callback` – *Вызываемый* объект с сигнатурой `callback(packet_receipt)`

13.6 Link

`class RNS.Link(destination, established_callback=None, closed_callback=None)`

Этот класс используется для установления и управления соединениями с другими узлами. При создании экземпляра соединения Reticulum попытается установить проверенное и зашифрованное соединение с указанным адресатом.

Параметры

- `destination` – экземпляр *RNS.Destination*, с которым необходимо установить соединение.
- `established_callback` — необязательная функция или метод с сигнатурой `callback(link)`, который вызывается после установления соединения.
- `closed_callback` — необязательная функция или метод с сигнатурой `callback(link)`, вызываемый при закрытии соединения.

`CURVE = 'Curve25519'`

Кривая, используемая для обмена ключами DH с эллиптической кривой

`ESTABLISHMENT_TIMEOUT_PER_HOP = 6`

Таймаут на установку соединения в секундах на каждый прыжок до пункта назначения.

`KEEPALIVE_TIMEOUT_FACTOR = 4`

Коэффициент таймаута RTT, используемый при расчете таймаута соединения.

`STALE_GRACE = 5`

Период отсрочки в секундах, используемый при расчете таймаута соединения.

`KEEPALIVE = 360`

Интервал по умолчанию для отправки пакетов keep-alive по установленным каналам связи в секундах.

`STALE_TIME = 720`

Если в течение этого периода не поступает трафик или пакеты keep-alive, канал помечается как просроченный, и отправляется последний пакет keep-alive. Если после этого в течение $RTT * KEEPALIVE_TIMEOUT_FACTOR + STALE_GRACE$ не поступает трафик или пакеты keep-alive, канал считается просроченным и будет разорван.

`identify(identity)`

Определяет инициатора соединения с удаленным узлом. Это возможно только после установления соединения и осуществляется по зашифрованному каналу. Идентификационные данные раскрываются исключительно удаленному узлу, что позволяет сохранить анонимность инициатора. Данный метод может использоваться для аутентификации.

Параметры

`identity` – экземпляр *RNS.Identity*, под которым будет осуществляться идентификация.

`request(path, data=None, response_callback=None, failed_callback=None, progress_callback=None, timeout=None)`

Отправляет запрос удаленному узлу.

Параметры

- `path` — путь запроса.
- `response_callback` — Необязательная функция или метод с сигнатурой `response_callback(request_receipt)`, который будет вызван при получении ответа. Дополнительную информацию см. в *примере запроса*.
- `failed_callback` — необязательная функция или метод с сигнатурой `failed_callback(request_receipt)`, вызываемый в случае сбоя запроса. Дополнительную информацию см. в *примере с запросом*.
- `progress_callback` — опциональная функция или метод с сигнатурой `progress_callback(request_receipt)`, вызываемый при достижении прогресса в получении ответа. Прогресс доступен в виде числа с плавающей запятой от 0.0 до 1.0 через свойство `request_receipt.progress`.
- `timeout` — Необязательный таймаут в секундах для запроса. Если указано `None`, он будет рассчитан на основе RTT канала.

Возвращает

Экземпляр `RNS.RequestReceipt`, если запрос был отправлен, или `False`, если нет.

`track_phy_stats(track)`

Вы можете включить статистику физического уровня для каждого канала связи. Если эта функция включена, а канал связи работает через интерфейс, поддерживающий отчетность по статистике физического уровня, вы сможете получить такие показатели, как *RSSI*, *SNR* и *качество физического канала* связи.

Параметры

`track` — отслеживать ли статистику физического уровня. Значение должно быть `True` или `False`.

`get_rssi()`

Возвращает

Показатель уровня сигнала на физическом уровне, если он доступен, в противном случае — `None`. Для того чтобы этот метод возвращал значение, на канале связи должна быть включена статистика физического уровня.

`get_snr()`

Возвращает

Отношение сигнал/шум на физическом уровне, если оно доступно, в противном случае — `None`. Для того чтобы этот метод возвращал значение, на канале связи должна быть включена статистика физического уровня.

`get_q()`

Возвращает

Качество канала физического уровня, если оно доступно, в противном случае — `None`. Для того, чтобы этот метод возвращал значение, на канале связи должна быть включена статистика физического уровня.

`get_establishment_rate()`

Возвращает

Скорость передачи данных, с которой произошла процедура установления соединения, в битах в секунду.

`get_mtu()`

Возвращает

MTU установленного соединения.

`get_mdu()`

Возвращает

Размер MDU пакета для установленного соединения.

`get_expected_rate()`

Возвращает

Ожидаемая скорость передачи данных в пакетах по установленному каналу связи.

`get_mode()`

Возвращает

Режим установленного соединения.

`get_age()`

Возвращает

Время в секундах с момента установки этого соединения.

`no_inbound_for()`

Возвращает

Время в секундах с момента последнего входящего пакета по каналу. Сюда входят пакеты keepalive.

`no_outbound_for()`

Возвращает

Время в секундах с момента последнего исходящего пакета на канале. Сюда входят пакеты keepalive.

`no_data_for()`

Возвращает

Время в секундах с момента прохождения данных полезной нагрузки по каналу связи. В это время не входят пакеты keepalive.

`inactive_for()`

Возвращает

Время в секундах с момента последней активности на канале. Сюда входят пакеты keepalive.

`get_remote_identity()`

Возвращает

Идентификатор удаленного узла, если он известен. Вызов этого метода не запрашивает у удаленного инициатора раскрытие его идентификатора. Возвращает None, если инициатор соединения еще не вызвал самостоятельно метод `identify(identity)`.

`teardown()`

Закрывает соединение и удаляет ключи шифрования. При установке нового соединения с тем же адресатом будут использоваться новые ключи.

`get_channel()`

Получить канал для этого соединения.

Возвращает

Объект канала

`set_link_closed_callback(callback)`

Регистрирует функцию, которая будет вызвана при разрыве соединения.

Параметры

`callback` – Функция или метод с сигнатурой `callback(link)`, который будет вызван.

`set_packet_callback(callback)`

Регистрирует функцию, которая будет вызываться при получении пакета по данному каналу.

Параметры

`callback` – Функция или метод с сигнатурой `callback(message, packet)`, который будет вызван.

`set_resource_callback(callback)`

Регистрирует функцию, которая будет вызываться при объявлении ресурса по данному каналу. Если функция возвращает `True`, ресурс будет принят. Если она возвращает `False`, он будет игнорироваться.

Параметры

`callback` – Функция или метод с сигнатурой `callback(resource)`, который будет вызван. Обратите внимание, что на данный момент доступна только базовая информация о ресурсе, такая как `get_transfer_size()`, `get_data_size()`, `get_parts()` и `is_compressed()`.

`set_resource_started_callback(callback)`

Регистрирует функцию, которая будет вызываться при начале передачи данных по данному каналу.

Параметры

`callback` — функция или метод с сигнатурой `callback(resource)`, который будет вызван.

`set_resource_concluded_callback(callback)`

Регистрирует функцию, которая будет вызвана, когда передача ресурса по данному каналу завершится.

Параметры

`callback` – Функция или метод с сигнатурой `callback(resource)`, который будет вызван.

`set_remote_identified_callback(callback)`

Регистрирует функцию, которая будет вызываться при идентификации иницилирующего узла по данному каналу.

Параметры

`callback` — функция или метод с сигнатурой `callback(link, identity)`, который будет вызван.

`set_resource_strategy(resource_strategy)`

Устанавливает стратегию ресурсов для канала.

Параметры

`resource_strategy` – одно из значений: `RNS.Link.ACCEPT_NONE`, `RNS.Link.ACCEPT_ALL` или `RNS.Link.ACCEPT_APP`. Если установлено значение `RNS.Link.ACCEPT_APP`, будет вызван `resource_callback` для определения, следует ли принимать ресурс или нет.

Вызывает

`TypeError`, если стратегия ресурса не поддерживается.

13.7 Подтверждение получения запроса

класс `RNS.RequestReceipt`

Экземпляр этого класса возвращается методом `request` экземпляров `RNS.Link`. Его ни в коем случае нельзя создавать вручную. Он предоставляет методы для проверки статуса, времени ответа и данных ответа по завершении запроса.

`get_request_id()`

Возвращает

Идентификатор запроса в виде *байтов*.

`get_status()`

Возвращает

Текущий статус запроса: `RNS.RequestReceipt.FAILED`, `RNS.RequestReceipt.SENT`, `RNS.RequestReceipt.DELIVERED` или `RNS.RequestReceipt.READY`.

`get_progress()`

Возвращает

Прогресс получения ответа в виде числа с плавающей запятой от 0,0 до 1,0.

`get_response()`

Возвращает

Время отклика в *байтах*, если ответ готов; в противном случае — *None*.

`get_response_time()`

Возвращает

Время ответа на запрос в секундах.

`concluded()`

Возвращает

`True`, если связанный запрос завершился (успешно или с ошибкой), в противном случае — `False`.

13.8 Ресурс

класс `RNS.Resource(data, link, advertise=True, auto_compress=True, callback=None, progress_callback=None, timeout=None)`

Класс `Resource` позволяет передавать произвольные объемы данных по каналу связи. Он автоматически обрабатывает последовательность, сжатие, координацию и вычисление контрольной суммы.

Параметры

- `data` – Передаваемые данные. Может быть массивом *байтов* или дескриптором открытого файла. Подробности см. в *примере Filetransfer*.
- `link` — экземпляр `RNS.Link`, на который будут переданы данные.
- `advertise` – Необязательный параметр. Указывает, следует ли автоматически объявлять ресурс. Может принимать значения `True` или `False`.
- `auto_compress` – Необязательный параметр. Указывает, следует ли автоматически сжимать ресурс. Может принимать значения `True` или `False`.
- `callback` – Необязательный вызываемый объект с сигнатурой `callback(resource)`. Вызывается по завершении передачи ресурса.
- `progress_callback` – Необязательная вызываемая функция с сигнатурой `callback(resource)`. Вызывается при каждом обновлении прогресса передачи ресурса.

`advertise()`

Объявить ресурс. Если другой конец канала примет объявление о ресурсе, начнется передача.

cancel()

Отменяет передачу ресурса.

get_progress()

Возвращает

Текущий прогресс передачи ресурса в виде числа с плавающей запятой в диапазоне от 0,0 до 1,0.

get_transfer_size()

Возвращает

Количество байтов, необходимое для передачи ресурса.

get_data_size()

Возвращает

Общий размер данных ресурса.

get_parts()

Возвращает

Количество частей, в которых будет передаваться ресурс.

get_segments()

Возвращает

Количество сегментов, на которые разделен ресурс.

get_hash()

Возвращает

Хеш ресурса.

is_compressed()

Возвращает

Указывает, сжат ли ресурс.

13.9 Канал

класс RNS.Channel.Channel

Обеспечивает надёжную доставку сообщений по каналу связи.

Канал отличается от запроса и ресурса по некоторым важным параметрам:

Непрерывность

Сообщения можно отправлять или получать, пока канал открыт.

Двунаправленность

Сообщения могут отправляться в любом направлении по каналу; ни одна из сторон не является клиентом или сервером.

Ограничение по размеру

Сообщения должны быть закодированы в один пакет.

Канал аналогичен пакету, за исключением того, что он обеспечивает надёжную доставку (автоматические повторные попытки), а также предоставляет структуру для обмена несколькими типами сообщений по каналу связи.

Канал не создается напрямую, а получается из соединения с помощью get_channel().

```
register_message_type(message_class: Type[MessageBase])
```

Регистрация класса сообщений для приёма через Channel.

Классы сообщений должны расширять MessageBase.

Параметры

`message_class` — регистрируемый класс

```
add_message_handler(callback: MessageCallbackType)
```

Добавить обработчик входящих сообщений. Обработчик имеет следующую сигнатуру:

(сообщение: MessageBase) -> bool

Обработчики обрабатываются в том порядке, в котором они добавлены. Если какой-либо обработчик возвращает True, обработка сообщения останавливается; обработчики, следующие за этим обработчиком, вызываться не будут.

Параметры

`callback` — Функция для вызова

```
remove_message_handler(callback: MessageCallbackType)
```

Удаление обработчика, добавленного с помощью add_message_handler.

Параметры

`callback` — обработчик, который нужно удалить

```
is_ready_to_send() → bool
```

Проверяет, готов ли канал к отправке.

Возвращает

True, если готов

```
send(message: MessageBase) → Envelope
```

Отправить сообщение. Если происходит попытка отправки сообщения, а канал не готов, генерируется исключение.

Параметры

`message` — экземпляр подкласса MessageBase

свойство `mdu`

Максимальная единица данных: количество байтов, доступных для использования сообщением при одной отправке. Это значение корректируется на основе Link MDU с учётом информации заголовка сообщения.

Возвращает

количество доступных байтов

13.10 MessageBase

класс `RNS.MessageBase`

Базовый тип для любых сообщений, отправляемых или получаемых по каналу. Подклассы должны определять два абстрактных метода, а также переменную класса `MSGTYPE`.

`MSGTYPE = None`

Определяет уникальный идентификатор для класса сообщения.

- Должен быть уникальным среди всех классов, зарегистрированных в канале
- Должно быть меньше 0xf000. Значения, равные или превышающие 0xf000, зарезервированы.

абстрактный метод `pack() → bytes`

Создать и вернуть двоичное представление сообщения

Возвращает

двоичное представление сообщения

абстрактный метод `unpack(raw: bytes)`

Заполнение сообщения из двоичного представления

Параметры`raw` – двоичное представление

13.11 Буфер

класс `RNS.Buffer`

Статические функции для создания буферизованных потоков, которые отправляют и принимают данные через канал.

Эти функции используют `BufferedReader`, `BufferedWriter` и `BufferedRWPair` для добавления буферизации к`RawChannelReader` и `RawChannelWriter`.**static** `create_reader(stream_id: int, channel: Channel, ready_callback: Callable[[int], None] | None = None) → BufferedReader`

Создать буферизованный считыватель, который считывает двоичные данные, передаваемые по каналу, с опциональным обратным вызовом при появлении новых данных.

Сигнатура обратного вызова: `(ready_bytes: int) -> None`

Для получения дополнительной информации о функциях этого объекта, относящихся к считыванию, см. документацию Python по

`BufferedReader`**Параметры**

- `stream_id` — локальный идентификатор потока, из которого будет осуществляться прием
- `channel` — канал для приема
- `ready_callback` — функция, вызываемая при появлении новых данных

Возвращаетобъект `BufferedReader`**static** `create_writer(stream_id: int, channel: Channel) → BufferedWriter` Создает

буферизованный записывающий объект, который записывает двоичные данные через канал.

Для получения дополнительной информации о функциях этого объекта, относящихся к записи, см. документацию Python по

`BufferedWriter`**Параметры**

- `stream_id` — идентификатор удалённого потока, в который будет осуществляться отправка
- `channel` — канал для отправки

Возвращаетобъект `BufferedWriter`

static `create_bidirectional_buffer(receive_stream_id: int, send_stream_id: int, channel: Channel, ready_callback: Callable[[int], None] | None = None) → BufferedRWPair`

Создает буферизованную пару читателя/записывающего устройства, которая считывает и записывает двоичные данные через канал, с опциональным обратным вызовом при появлении новых данных.

Сигнатура обратного вызова: `(ready_bytes: int) -> None`

Дополнительную информацию о функциях этого объекта, относящихся к считыванию, см. в документации Python по `BufferedReader`

Параметры

- `receive_stream_id` — идентификатор локального потока, на который будет осуществляться прием
- `send_stream_id` — идентификатор удаленного потока, в который будет отправляться
- `channel` — канал для отправки и получения данных
- `ready_callback` — функция, вызываемая при появлении новых данных

Возвращает

объект `BufferedReader`

13.12 RawChannelReader

класс `RNS.RawChannelReader(stream_id: int, channel: Channel)`

Реализация `RawIOBase`, которая принимает данные двоичного потока, передаваемые по каналу.

Как правило, этот класс не нужно создавать напрямую. Используйте функции `RNS.Buffer.create_reader()`, `RNS.Buffer.create_writer()` и `RNS.Buffer.create_bidirectional_buffer()` для создания буферизованных потоков с опциональными обратными вызовами.

Дополнительную информацию об API этого объекта см. в документации Python по `RawIOBase`.

`_init (stream_id: int, channel: Channel)` Создание считывателя необработанного канала.

Параметры

- `stream_id` — локальный идентификатор потока, на который будет осуществляться прием
- `channel` — объект `Channel`, из которого будет производиться прием

`add_ready_callback(cb: Callable[[int], None])`

Добавляет функцию, которая будет вызываться при появлении новых данных.

Функция должна иметь сигнатуру

(ready_bytes: int) -> None

Параметры

`cb` — вызываемая функция

`remove_ready_callback(cb: Callable[[int], None])`

Удаление функции, добавленной с помощью `RNS.RawChannelReader.add_ready_callback()`

Параметры

`cb` — функция для удаления

13.13 RawChannelWriter

класс `RNS.RawChannelWriter(stream_id: int, channel: Channel)`

Реализация `RawIOBase`, которая принимает данные двоичного потока, отправленные по каналу.

Как правило, этот класс не требуется создавать напрямую. Для создания буферизованных потоков с опциональными обратными вызовами используйте функции `RNS.Buffer.create_reader()`, `RNS.Buffer.create_writer()` и `RNS.Buffer.create_bidirectional_buffer()`.

Дополнительную информацию об API этого объекта см. в документации Python по `RawIOBase`.

`__init (stream_id: int, channel: Channel)` Создает записывающий модуль необработанного канала.

Параметры

- `stream_id` — идентификатор удалённого потока, в который будет отправляться данные
- `channel` — объект `Channel`, по которому будет осуществляться отправка

13.14 Transport

класс `RNS.Transport`

С помощью статических методов этого класса можно взаимодействовать с транспортной системой Reticulum.

`PATHFINDER_M = 128`

Максимальное количество прыжков, на которое Reticulum будет транспортировать пакет.

`static register_announce_handler(handler)`

Регистрирует обработчик объявлений.

Параметры

`handler` — Должен быть объектом с атрибутом `aspect_filter` и вызываемой функцией `received_announce(destination_hash, announced_identity, app_data)` или `received_announce(destination_hash, announced_identity, app_data, announce_packet_hash)` или `received_announce(destination_hash, announced_identity, app_data, announce_packet_hash, is_path_response)` callable. Может опционально иметь атрибут `re-ceive_path_responses`, установленный в `True`, чтобы также получать все ответы по пути, в дополнение к активным объявлениям. См. *пример объявления* для получения дополнительной информации.

`static deregister_announce_handler(handler)`

Отменяет регистрацию обработчика объявлений.

Параметры

`handler` — обработчик объявлений, регистрацию которого необходимо отменить.

`static has_path(destination_hash)`

Параметры

`destination_hash` — хеш назначения в виде *байтов*.

Возвращает

True, если путь к месту назначения известен, в противном случае *False*.

`static hops_to(destination_hash)`

Параметры

`destination_hash` — хеш назначения в виде *байтов*.

Возвращает

Количество прыжков до указанного пункта назначения или `RNS.Transport.PATHFINDER_M`, если количество прыжков неизвестно.

`static next_hop(destination_hash)`

Параметры

`destination_hash` — хеш назначения в виде *байтов*.

Возвращает

Хеш адреса назначения в *байтах* для следующего прыжка до указанного адреса назначения или *None*, если следующий прыжок неизвестен.

```
static next_hop_interface(destination_hash)
```

Параметры

`destination_hash` – хеш адреса назначения в *байтах*.

Возвращает

Интерфейс для следующего прыжка к указанному пункту назначения или *None*, если интерфейс неизвестен.

```
static await_path(destination_hash, timeout=None, on_interface=None)
```

Запрашивает у сети маршрут к месту назначения и ожидает, пока маршрут не станет доступен или не истечет время ожидания.

Параметры

- `destination_hash` — хеш пункта назначения в *байтах*.
- `timeout` — опциональное время ожидания в секундах.
- `on_interface` — если указан, запрос на маршрут будет отправлен только по этому интерфейсу. При обычном использовании Reticulum обрабатывает это автоматически, и этот параметр не следует использовать.

Возвращает

True, если путь к пункту назначения найден, в противном случае *False*.

```
static request_path(destination_hash, on_interface=None, tag=None, recursive=False)
```

Запрашивает у сети маршрут к месту назначения. Если другой доступный узел в сети знает этот маршрут, он объявит его.

Параметры

- `destination_hash` — хеш адреса назначения в *байтах*.
- `on_interface` — если указан, запрос на маршрут будет отправлен только по этому интерфейсу. При обычном использовании Reticulum обрабатывает это автоматически, и этот параметр не следует использовать.

СИМВОЛЫ

`__init__()` (метод *RNS.RawChannelReader*), 212`__init__()` (метод *RNS.RawChannelWriter*), 212

А

`accepts_links()` (метод *RNS.Destination*), 199`add_message_handler()` (метод *RNS.Channel.Channel*), 210`add_ready_callback()` (метод *RNS.RawChannelReader*), 212`advertise()` (метод *RNS.Resource*), 208 `announce()` (метод *RNS.Destination*), 199 `ANNOUNCE_CAP` (атрибут *RNS.Reticulum*), 193 `app_and_aspects_from_name()` (статический метод *RNS.Destination*), 199`await_path()` (статический метод *RNS.Transport*), 214

В

`blackhole_sources()` (статический метод *RNS.Reticulum*), 195*Buffer* (класс в *RNS*), 211

С

`cancel()` (метод *RNS.Resource*), 208*Channel* (класс в *RNS.Channel*), 209`clear_default_app_data()` (метод *RNS.Destination*), 202`concluded()` (метод *RNS.RequestReceipt*), 208`create_bidirectional_buffer()` (статический метод *RNS.Buffer*), 211`create_keys()` (метод *RNS.Destination*), 201 `create_reader()` (статический метод *RNS.Buffer*), 211 `create_writer()` (статический метод *RNS.Buffer*), 211 `current_ratchet_id()` (статический метод *RNS.Identity*), 196*CURVE* (атрибут *RNS.Identity*), 195*CURVE* (атрибут *RNS.Link*), 204

D

`decrypt()` (метод *RNS.Destination*), 202`decrypt()` (метод *RNS.Identity*), 197`deregister_announce_handler()` (статический метод *RNS.Transport*), 213`deregister_request_handler()` (*RNS.Destination*), 200*Destination* (класс в *RNS*), 198`discovered_interfaces()` (статический метод *RNS.Reticulum*), 195

E

`enable_ratchets()` (метод *RNS.Destination*), 200 `encrypt()` (метод *RNS.Destination*), 201 `encrypt()` (метод *RNS.Identity*), 197 `ENCRYPTED_MDU` (атрибут *RNS.Packet*), 202 `enforce_ratchets()` (метод *RNS.Destination*), 201`ESTABLISHMENT_TIMEOUT_PER_HOP` (*RNS.Link*), 204`expand_name()` (статический метод *RNS.Destination*), 199

F

`from_bytes()` (статический метод *RNS.Identity*), 196`from_file()` (статический метод *RNS.Identity*), 197 `full_hash()` (статический метод *RNS.Identity*), 196

G

`get_age()` (метод *RNS.Link*), 206 `get_channel()` (метод *RNS.Link*), 206 `get_data_size()` (метод *RNS.Resource*), 209 `get_establishment_rate()` (метод *RNS.Link*), 205 `get_expected_rate()` (метод *RNS.Link*), 206 `get_hash()` (метод *RNS.Resource*), 209 `get_instance()` (статический метод *RNS.Reticulum*), 194 `get_mdu()` (метод *RNS.Link*), 206 `get_mode()` (метод *RNS.Link*), 206 `get_mtu()` (метод *RNS.Link*), 205 `get_parts()` (метод *RNS.Resource*), 209 `get_private_key()` (метод *RNS.Destination*), 201 `get_private_key()` (метод *RNS.Identity*), 197 `get_progress()` (метод *RNS.RequestReceipt*), 208 `get_progress()` (метод *RNS.Resource*), 209 `get_public_key()` (метод *RNS.Identity*), 197 `get_q()` (метод *RNS.Link*), 205`get_q()` (метод *RNS.Packet*), 203

get_random_hash() (статический метод *RNS.Identity*), 196
 get_remote_identity() (метод *RNS.Link*), 206 get_request_id() (метод *RNS.RequestReceipt*), 207 get_response() (метод *RNS.RequestReceipt*), 208 get_response_time() (метод *RNS.RequestReceipt*), 208

get_rssi() (метод *RNS.Link*), 205 get_rssi() (метод *RNS.Packet*), 203 get_rtt() (метод *RNS.PacketReceipt*), 203 get_segments() (метод *RNS.Resource*), 209 get_snr() (метод *RNS.Link*), 205
 get_snr() (метод *RNS.Packet*), 203
 get_status() (метод *RNS.PacketReceipt*), 203 get_status() (метод *RNS.RequestReceipt*), 208 get_transfer_size() (метод *RNS.Resource*), 209

H

has_path() (статический метод *RNS.Transport*), 213 hash() (статический метод *RNS.Destination*), 199
 hash_from_name_and_identity() (статический метод *RNS.Destination*), 199

hops_to() (статический метод *RNS.Transport*), 213

I

identify() (метод *RNS.Link*), 204 Identity (класс в *RNS*), 195 inactive_for() (метод *RNS.Link*), 206
 interface_discovery_sources() (статический метод *RNS.Reticulum* static), 195
 is_compressed() (метод *RNS.Resource*), 209
 is_ready_to_send() (метод *RNS.Channel.Channel*), 210

K

KEEPALIVE (ампурум *RNS.Link*), 204
 KEEPALIVE_TIMEOUT_FACTOR (ампурум *RNS.Link*), 204
 KEYSIZE (ампурум *RNS.Identity*), 195

L

Link (класс в *RNS*), 204
 LINK_MTU_DISCOVERY (ампурум *RNS.Reticulum*), 193
 link_mtu_discovery() (*RNS.Reticulum* static), 194
 load_private_key() (метод *RNS.Destination*), 201
 load_private_key() (метод *RNS.Identity*), 197
 load_public_key() (метод *RNS.Identity*), 197

M

mdu (свойство *RNS.Channel.Channel*), 210 MessageBase (класс в *RNS*), 210 MINIMUM_BITRATE (ампурум *RNS.Reticulum*), 194
 MSGTYPE (ампурум *RNS.MessageBase*), 210
 MTU (ампурум *RNS.Reticulum*), 193

N

next_hop() (статический метод *RNS.Transport*), 213
 next_hop_interface() (статический метод *RNS.Transport*), 213
 no_data_for() (метод *RNS.Link*), 206 no_inbound_for() (метод *RNS.Link*), 206 no_outbound_for() (метод *RNS.Link*), 206

P

pack() (метод *RNS.MessageBase*), 210 Packet (класс в *RNS*), 202 PacketReceipt (класс в *RNS*), 203
 PATHFINDER_M (ампурум *RNS.Transport*), 213 PLAIN_MDU (ампурум *RNS.Packet*), 202 publish_blackhole_enabled() (статический метод *RNS.Reticulum*), 194

R

RATCHET_COUNT (ампурум *RNS.Destination*), 198
 RATCHET_EXPIRY (ампурум *RNS.Identity*), 195
 RATCHET_INTERVAL (ампурум *RNS.Destination*), 199
 RATCHETSIZE (ампурум *RNS.Identity*), 195 RawChannelReader (класс в *RNS*), 212 RawChannelWriter (класс в *RNS*), 212
 recall() (статический метод *RNS.Identity*), 195
 recall_app_data() (статический метод *RNS.Identity*), 196
 register_announce_handler() (статический метод *RNS.Transport*), 213
 register_message_type() (метод *RNS.Channel.Channel*), 209
 register_request_handler() (метод *RNS.Destination*), 200
 remote_management_enabled() (статический метод *RNS.Reticulum*), 194
 remove_message_handler() (метод *RNS.Channel.Channel*), 210
 remove_ready_callback() (метод *RNS.RawChannelReader*), 212
 request() (метод *RNS.Link*), 204 request_path() (статический метод *RNS.Transport*), 214 RequestReceipt (класс в *RNS*), 207
 required_discovery_value() (статический метод *RNS.Reticulum*), 194
 resend() (метод класса *RNS.Packet*), 203
 Resource (класс в *RNS*), 208 Reticulum (класс в *RNS*), 193

S

send() (метод *RNS.Channel.Channel*), 210
 send() (метод *RNS.Packet*), 203
 set_default_app_data() (метод *RNS.Destination*), 202
 set_delivery_callback() (метод *RNS.PacketReceipt*), 203

set_link_closed_callback() (метод *RNS.Link*), 206
 set_link_established_callback() (метод *RNS.Destination*), 199
 set_packet_callback() (метод *RNS.Destination*), 199
 set_packet_callback() (метод *RNS.Link*), 207
 set_proof_requested_callback() (метод *RNS.Destination*), 200
 set_proof_strategy() (метод *RNS.Destination*), 200
 set_ratchet_interval() (метод *RNS.Destination*), 201
 set_remote_identified_callback() (метод *RNS.Link*), 207
 set_resource_callback() (метод *RNS.Link*), 207
 set_resource_concluded_callback() (метод *RNS.Link*), 207
 set_resource_started_callback() (метод *RNS.Link*), 207
 set_resource_strategy() (метод *RNS.Link*), 207
 set_retained_ratchets() (метод *RNS.Destination*), 201
 set_timeout() (метод *RNS.PacketReceipt*), 203
 set_timeout_callback() (метод *RNS.PacketReceipt*), 204
 should_use_implicit_proof() (статический метод *RNS.Reticulum*), 194
 sign() (метод *RNS.Destination*), 202 sign() (метод *RNS.Identity*), 198 STALE_GRACE (атрибут *RNS.Link*), 204 STALE_TIME (атрибут *RNS.Link*), 204

T

teardown() (метод *RNS.Link*), 206 to_file() (метод *RNS.Identity*), 197 track_phy_stats() (метод *RNS.Link*), 205 Transport (класс в *RNS*), 213
 transport_enabled() (статический метод *RNS.Reticulum*), 194
 truncated_hash() (статический метод *RNS.Identity*), 196
 TRUNCATED_HASHLENGTH (атрибут *RNS.Identity*), 195

U

unpack() (метод *RNS.MessageBase*), 211

V

validate() (метод *RNS.Identity*), 198